



User Guide

Version 5.3.0

Published September 1, 2025

Table of Contents

About this guide	ix
Conventions	x
Intended Audience	xi
Feedback	xi
Copyrights	xi
Trademarks	xi
Chapter 1: Introducing the PX5 RTOS.....	1
How the PX5 RTOS benefits you	1
Why use an RTOS?	2
The PX5 RTOS is here to help!.....	5
Chapter 2: Brief history of POSIX pthreads	6
Why pthreads for Embedded?.....	7
Chapter 3: PX5 RTOS Installation and Use	8
3 Simple Steps.....	8
PX5_RTOS_Binding_User_Guide.pdf	9
Defined PX5 Symbol	9
Configuration Options for the PX5 RTOS.....	9
Using Configuration Options	14
Troubleshooting	15
Version Information	16
Long Term Support (LTS)	17
Issue Notification.....	17
Contact Us	17
Chapter 4: PX5 RTOS Safety and Security	18
Functional Safety Certification.....	18
Hardware Safety & Security Features	18
Software Safety & Security Features	20
PX5 RTOS Pointer/Data Verification (PDV)	20
Run-Time Stack Checking & Verification	20
Application Best Practices.....	21
Chapter 5: PX5 RTOS Pointer/Data Verification.....	23

Default Verification Code	23
Important Information Verified.....	24
Enabling PDV	24
PDV Overhead	25
Chapter 6: PX5 RTOS Functional Overview	27
Composition	28
Namespace	28
Native pthreads Implementation.....	28
pthreads Extensions.....	28
Internal Memory Management	28
Multicore Support	30
Pointer/Data Verification (PDV).....	30
Run-time Stack Checking/Verification	30
Central Error Handling	31
Scheduling Policy	31
Thread Priorities.....	32
Thread States.....	32
System Objects	33
Threads.....	34
Condition Variables	35
Message Queues	36
Mutexes	37
Semaphores.....	38
Signals	39
Fastqueues	40
Event Flags	41
Memory Pools	42
Partition Memory Pools	42
Ticktimer Services.....	43
Chapter 7: PX5 RTOS API definitions.....	44
clock_getres	45
clock_gettime	47
clock_settime	49
mq_close.....	51
mq_getattr	53
mq_open	55
mq_receive.....	58
mq_send	61
mq_setattr	64
mq_timedreceive.....	67
mq_timedsend.....	70

nanosleep	73
pthread_attr_destroy	76
pthread_attr_getdetachstate	78
pthread_attr_getstackaddr	80
pthread_attr_getstacksize	82
pthread_attr_init	84
pthread_attr_setdetachstate	86
pthread_attr_setstackaddr	88
pthread_attr_setstacksize	90
pthread_cancel.....	92
pthread_cleanup_pop	94
pthread_cleanup_push.....	96
pthread_cond_broadcast	98
pthread_cond_destroy	100
pthread_cond_init.....	102
pthread_cond_signal.....	104
pthread_cond_timedwait.....	106
pthread_cond_wait.....	109
pthread_condattr_destroy	112
pthread_condattr_getpshared	114
pthread_condattr_init	116
pthread_condattr_setpshared	118
pthread_create	120
pthread_detach	124
pthread_equal	126
pthread_exit	128
pthread_getaffinity_np.....	130
pthread_getspecific.....	132
pthread_join	134
pthread_key_create	137
pthread_key_delete.....	139
pthread_kill.....	141
pthread_mutex_destroy	143
pthread_mutex_init.....	145
pthread_mutex_lock.....	147
pthread_mutex_trylock.....	149
pthread_mutex_unlock.....	151
pthread_mutexattr_destroy	153
pthread_mutexattr_getprotocol	155
pthread_mutexattr_getpshared	157
pthread_mutexattr_gettype	159

pthread_mutexattr_init	161
pthread_mutexattr_setprotocol.....	163
pthread_mutexattr_setpshared	165
pthread_mutexattr_settype	167
pthread_once	169
pthread_self	171
pthread_setaffinity_np.....	173
pthread_setcancelstate	175
pthread_setcanceltype	177
pthread_setspecific	179
pthread_sigmask.....	181
pthread_testcancel.....	184
px5_errno_get.....	186
px5_errno_set.....	188
px5_mq_extend_open.....	190
px5_mq_extendattr_destroy.....	193
px5_mq_extendattr_getcontroladdr.....	195
px5_mq_extendattr_getcontrolsiz	197
px5_mq_extendattr_getqueueaddr	199
px5_mq_extendattr_getqueuesize	201
px5_mq_extendattr_init.....	203
px5_mq_extendattr_setcontroladdr.....	205
px5_mq_extendattr_setqueueaddr.....	208
px5_mq_information_get.....	210
px5_mq_send_notify.....	213
px5_pthread_attr_getaffinity.....	215
px5_pthread_attr_getautostart.....	217
px5_pthread_attr_getcontroladdr	219
px5_pthread_attr_getcontrolsiz	221
px5_pthread_attr_getname	223
px5_pthread_attr_getpriority	225
px5_pthread_attr_gettimeslice	227
px5_pthread_attr_setaffinity	229
px5_pthread_attr_setautostart	231
px5_pthread_attr_setcontroladdr	233
px5_pthread_attr_setname	235
px5_pthread_attr_setpriority.....	237
px5_pthread_attr_settimeslice	239
px5_pthread_cond_information_get.....	241
px5_pthread_condattr_getcontroladdr	244
px5_pthread_condattr_getcontrolsiz	246

px5_thread_condattr_getname	248
px5_thread_condattr_setcontroladdr.....	250
px5_thread_condattr_setname.....	253
px5_thread_event_flags_clear	255
px5_thread_event_flags_clearspecific	257
px5_thread_event_flags_create	259
px5_thread_event_flags_destroy	261
px5_thread_event_flags_information_get	263
px5_thread_event_flags_set.....	266
px5_thread_event_flags_set_notify.....	268
px5_thread_event_flags_timedwait.....	270
px5_thread_event_flags_trywait.....	273
px5_thread_event_flags_wait.....	275
px5_thread_event_flagsattr_destroy	278
px5_thread_event_flagsattr_getautoconsumemask.....	280
px5_thread_event_flagsattr_getcontroladdr	282
px5_thread_event_flagsattr_getcontrolsiz.....	284
px5_thread_event_flagsattr_getname.....	286
px5_thread_event_flagsattr_init	288
px5_thread_event_flagsattr_setautoconsumemask.....	290
px5_thread_event_flagsattr_setcontroladdr	293
px5_thread_event_flagsattr_setname	296
px5_thread_fastqueue_create.....	298
px5_thread_fastqueue_destroy.....	300
px5_thread_fastqueue_information_get.....	302
px5_thread_fastqueue_receive	305
px5_thread_fastqueue_send.....	307
px5_thread_fastqueue_send_notify	309
px5_thread_fastqueue_timedreceive	311
px5_thread_fastqueue_timedsend.....	314
px5_thread_fastqueue_tryreceive	317
px5_thread_fastqueue_trysend.....	319
px5_thread_fastqueueattr_destroy.....	321
px5_thread_fastqueueattr_getcontroladdr	323
px5_thread_fastqueueattr_getcontrolsiz.....	325
px5_thread_fastqueueattr_getname	327
px5_thread_fastqueueattr_getqueueaddr	329
px5_thread_fastqueueattr_getqueuesiz.....	331
px5_thread_fastqueueattr_init.....	333
px5_thread_fastqueueattr_setcontroladdr.....	335
px5_thread_fastqueueattr_setname.....	338

px5_thread_fastqueueattr_setqueueaddr	340
px5_thread_information_get.....	342
px5_thread_memory_manager_enable	345
px5_thread_memory_manager_get	347
px5_thread_memory_manager_set	349
px5_thread_memorypool_allocate	352
px5_thread_memorypool_create.....	355
px5_thread_memorypool_destroy.....	357
px5_thread_memorypool_free	359
px5_thread_memorypool_information_get.....	361
px5_thread_memorypool_timedallocate	364
px5_thread_memorypool_tryallocate	367
px5_thread_memorypoolattr_destroy.....	370
px5_thread_memorypoolattr_getcontroladdr.....	372
px5_thread_memorypoolattr_getcontrolsiz	374
px5_thread_memorypoolattr_getname	376
px5_thread_memorypoolattr_init.....	378
px5_thread_memorypoolattr_setcontroladdr.....	380
px5_thread_memorypoolattr_setname.....	383
px5_thread_mutex_information_get.....	385
px5_thread_mutexattr_getcontroladdr	388
px5_thread_mutexattr_getcontrolsiz	390
px5_thread_mutexattr_getname	392
px5_thread_mutexattr_setcontroladdr.....	394
px5_thread_mutexattr_setname.....	397
px5_thread_partitionpool_allocate	399
px5_thread_partitionpool_create.....	402
px5_thread_partitionpool_destroy	405
px5_thread_partitionpool_free.....	407
px5_thread_partitionpool_information_get.....	409
px5_thread_partitionpool_timedallocate.....	412
px5_thread_partitionpool_tryallocate	415
px5_thread_partitionpoolattr_destroy.....	418
px5_thread_partitionpoolattr_getcontroladdr.....	420
px5_thread_partitionpoolattr_getcontrolsiz	422
px5_thread_partitionpoolattr_getname.....	424
px5_thread_partitionpoolattr_init.....	426
px5_thread_partitionpoolattr_setcontroladdr.....	428
px5_thread_partitionpoolattr_setname.....	431
px5_thread_priority_change.....	433
px5_thread_resume	435

px5_pthread_stack_check	437
px5_pthread_start	439
px5_pthread_suspend	442
px5_pthread_tick_sleep	444
px5_pthread_ticks_get	446
px5_pthread_ticktimer_create	448
px5_pthread_ticktimer_destroy	451
px5_pthread_ticktimer_information_get	453
px5_pthread_ticktimer_start	456
px5_pthread_ticktimer_stop	458
px5_pthread_ticktimer_update	460
px5_pthread_ticktimerattr_destroy	462
px5_pthread_ticktimerattr_getcontroladdr	464
px5_pthread_ticktimerattr_getcontrolsize	466
px5_pthread_ticktimerattr_getname	468
px5_pthread_ticktimerattr_init	470
px5_pthread_ticktimerattr_setcontroladdr	472
px5_pthread_ticktimerattr_setname	475
px5_pthread_timeslice_change	477
px5_sem_extend_init	479
px5_sem_information_get	481
px5_sem_post_notify	484
px5_sem_tickwait	486
px5_semattr_destroy	488
px5_semattr_getcontroladdr	490
px5_semattr_getcontrolsize	492
px5_semattr_getname	494
px5_semattr_init	496
px5_semattr_setcontroladdr	498
px5_semattr_setname	501
sched_yield	503
sem_destroy	506
sem_init	508
sem_post	510
sem_timedwait	512
sem_trywait	515
sem_wait	517
sigaction	519
sigaddset	522
sigdelset	524
sigemptyset	526

sigfillset	528
sigismember	530
sigpending	532
sigtimedwait	534
sigwait	536
sigwaitinfo	539
sleep	542
time	544
usleep	546
Index	548



About this guide

This guide provides embedded developers with the information necessary to utilize the PX5 Real-Time Operating System (RTOS). The guide is organized into the following chapters:

Chapter 1: Introducing the PX5 RTOS. This chapter briefly introduces multithreading, pthreads, and the PX5 RTOS with pthreads+ extensions.

Chapter 2: Brief history of POSIX pthreads. This chapter provides a brief history of POSIX pthreads.

Chapter 3: PX5 RTOS Installation and Use. This chapter provides a high-level overview of how to install and use the PX5 RTOS.

Chapter 4: PX5 RTOS Safety and Security. This chapter provides an overview of safety and security in PX5 RTOS applications.

Chapter 5: PX5 RTOS Pointer/Data Verification. This chapter provides an overview of the unique Pointer/Data Verification capabilities of the PX5 RTOS.

Chapter 6: PX5 RTOS Functional Overview. This chapter provides a functional overview of the PX5 RTOS.

Chapter 7: PX5 RTOS API Definitions. This chapter describes the APIs—both standard pthread APIs and pthreads+ extensions—of the PX5 RTOS.

Conventions

Conventions used in this guide help to identify types of information quickly. C source code function prototypes and examples are written in the `Courier` font. Supplemental documentation, API, and parameter names are italicized when discussed. Several symbols are used to highlight important features or topics, as follows:

Symbol	Meaning
	This is a general information symbol.
	The caution symbol indicates that the developer should know important usage scenarios associated with a specific topic or API.
	The danger symbol indicates that the developer should be aware of the potentially serious consequences of a topic or API.
	This symbol indicates the associated API scenario does not result in preemption.
	This symbol indicates the associated API scenario results in preemption.
	This symbol indicates the associated API scenario, which results in suspension.
	This symbol indicates that there is video content for the current topic.
	This symbol indicates the stack usage estimate for the specific API.

Intended Audience

The reader and the PX5 RTOS integrator are expected to be proficient in C programming and multithreading in embedded systems.

Feedback

All feedback is greatly appreciated. Please send e-mail feedback to support@px5rtos.com with "*PX5 RTOS User Guide Feedback*" in the subject line.

Copyrights

This guide is copyrighted by and is the sole property of PX5. Without prior written consent from PX5, all reproduction of this guide in any form is strictly forbidden. PX5 reserves all rights pertaining to this guide.

Trademarks

PX5 and PX5 RTOS are trademarks of PX5. All other product and company names are trademarks of their respective holders.

Chapter 1: Introducing the PX5 RTOS

The industrial-grade PX5 RTOS is an advanced, 5th-generation RTOS designed for the most demanding embedded applications. Its ultra-small size (~1KB for minimal use) allows PX5 RTOS to fit into some of the most memory-constrained devices. Its ultra-high performance (sub-microsecond context-switching and API calls on most microprocessors) and rich determinism make it ideal for the most demanding real-time needs. PX5 RTOS also boasts best-in-class safety and security. The entire PX5 RTOS code base is rigorously tested (100% C statement and branch decision coverage testing for every release).

In addition, the PX5 RTOS offers *Pointer/Data Verification (PDV)*, a unique technology for unprecedented run-time function pointer, system object, buffer, and stack verification. Perhaps most importantly, PX5 RTOS is simple. The API consists of a native implementation of the well-known POSIX pthreads standard, making applications written for the PX5 RTOS highly portable to any POSIX pthread implementation—whether that is Linux or another RTOS supporting the pthread API. The PX5 RTOS also offers optional POSIX pthreads extensions explicitly designed for deeply embedded, real-time applications.

How the PX5 RTOS benefits you

The PX5 RTOS features benefit embedded development in many ways, including the following:

- Accelerated time-to-market
- Enhanced product quality
- Safer and more secure products – PX5 RTOS is certified for use in safety-critical devices requiring IEC 61508 SIL 4, IEC 62304 Class C, ISO 26262 ASIL D, and EN 50128 SW-SIL 4 level of certification.
- More portable/reusable application code via industry-standard pthreads API

- Reduced engineering training via pthreads API
- Professional support

Why use an RTOS?



Backing up a bit, let's briefly discuss why an RTOS is important in the first place. Not all embedded applications require an RTOS. Some elementary applications can perform all of their required processing in a control loop within the C *main* function, as the example illustrates below:

```
int  main(void)
{

    while(1)
    {

        /* Process primary task.  */
        process_primary_task();

        /* Process secondary task.  */
        process_secondary_task();

    }

}
```

Typically, applications with a total memory size of less than 64KB, and no network connectivity or physical device I/O, may use and even benefit from a simple control loop. The simple control loop technique eliminates memory and processing cycles required by a more robust RTOS platform. However, it places all responsibility for allocating processor cycles and meeting real-time requirements on the application code itself. Take, for example, the simple control loop illustrated above. If the *process_primary_task* function has a real-time requirement, the application developer must ensure that the worst-case processing time of the rest of the control loop is sufficiently small to meet that requirement.

This may not be an easy task. To accomplish it, the developer must thoroughly analyze and test all code paths before and after the call to *process_primary_task*. Of course, this becomes exponentially more difficult as the code size and complexity increase, or if new real-time requirements are added. One might think, "Well, I'll just add more calls to *process_primary_task* in the control loop so I don't have to

calculate the entire worst-case processing of the entire loop.” This can temporarily help; however, now there is extra processing overhead (Extra calls to *process_primary_task* equates to additional overhead). At some point, this additional overhead may become more significant than the context-switching overhead of an RTOS.

Waiting for peripheral device I/O (blocking) is another complex issue to overcome with a simple control loop design. Suppose *process_secondary_task* controls a peripheral I/O device and waits for the peripheral I/O device to complete its operation. Indeed, the real-time requirements in *process_primary_task* could be adversely affected, and this could be even worse if the wait time is not deterministic. In this situation, the application developer would likely be forced to create a state machine within *process_secondary_task*, such that it would never wait, instead setting up a state machine to allow it to "find its way back" to the peripheral I/O device operation to check for completion.

This state machine processing necessarily adds complexity overhead. In contrast, the thread implementing *process_secondary_task* can suspend (block) in-line on the peripheral I/O device operation if an RTOS is used. When the I/O is complete, the thread resumes immediately where it last executed with no complicated state machine to find its way back, i.e., less complexity and overhead.

Finally, development without an RTOS requires every developer to have precise knowledge of the real-time processing of each component. Not only does this become exponentially more difficult as the complexity and real-time requirements increase, but it also doesn't scale well as more developers are added to the project. In contrast, an RTOS enables encapsulating application components into threads of varying priorities. For example, the *process_primary_task* functionality could be encapsulated inside the highest priority thread, and thus, nothing else in the system would interfere with meeting its real-time requirements. Therefore, developers need not know (and can avoid impacting) the system's real-time needs, which lets them focus solely on their area of interest.

The following is an example of the control loop converted to operate using the pthread API:

```
pthread_t primary_thread_handle;

void * process_primary_task(void * argument)
{
    struct sched_param param;

    /* Assuming priority 31 is highest. */
    param.sched_priority = 31;

    /* Raise the priority of the real-time thread. */
    pthread_setschedparam(pthread_self(), SCHED_OTHER, &param);

    while(1)
    {
        /* Perform third real-time duties and suspend until more
           work needs to be done. */
    }
}

int main(void)
{
    /* Create a thread for the real-time requirements associated with the
       primary task. */
    pthread_create(&primary_thread_handle, NULL, process_primary_task, NULL);

    /* Relinquish to the primary thread. */
    sched_yield();

    while(1)
    {
        /* Perform the secondary task from the main thread. */
        process_secondary_task();
    }
}
```

The example shows how easily one can convert processing a control loop implementation into two threads under an RTOS. The main loop becomes a lower-priority thread, and the processing in *process_primary_task* is contained within a higher-priority thread that preempts whenever it has real-time processing to accomplish. The beauty of an RTOS-based design is that boundless new functionality can be added without adversely affecting the real-time processing in *process_primary_task* – assuming that it remains the highest priority thread.

In summary, an RTOS dramatically reduces the complexity of an application—especially for meeting real-time requirements and eliminating state machines necessitated by a lack of in-line suspension. An RTOS can even reduce overhead when the control loop deficiencies have necessitated significant logic to manage the

real-time requirements of the application. Using an RTOS improves development in many ways:

- Enhances application real-time responsiveness
- Reduces complexity and makes development easier
- Simplifies dividing an application into more manageable pieces
- Enables more features and project developers
- Can reduce overhead via elimination of polling and state machines
- Achieves concurrency by enabling other processing while waiting for blocking I/O
- Provides true parallel processing in symmetric multiprocessing RTOS environments

The PX5 RTOS is here to help!

As mentioned at the beginning of this chapter, the PX5 RTOS combines best-of-class RTOS technology with the industry-standard pthread API. In addition, the PX5 RTOS provides optional pthreads+ extensions that complement the POSIX pthreads API to address embedded multithreading requirements better. In summary, the PX5 RTOS provides embedded developers the best of both worlds: best-in-class RTOS technology united with the industry standard pthreads API!

Chapter 2: Brief history of POSIX pthreads

The Portable Operating System Interface (POSIX) is a set of standards IEEE defines and maintains. Launched in UNIX, POSIX offers application portability across compatible hardware and operating systems. Many of today's operating systems support POSIX, including, most significantly, all embedded Linux distributions.

The first POSIX standard was the IEEE Std 1003.1-1988 specification, released in 1988. The latest was released in 2017 (POSIX.1-2017 IEEE Std 1003.1-2017). Initially, all multitasking in POSIX was process-based. Communicating and switching between processes required significant overhead and highly capable hardware (virtual memory support, large amounts of memory, super-fast processors, etc.). To reduce overhead, the POSIX pthreads specification was introduced in the IEEE Std 1003.1c-1995 specification of 1995. The specification's lightweight multithreading—including thread synchronization and communication primitives—provided a valuable new paradigm to application developers. With POSIX pthreads, developers could share global variables and data structures between threads using low-overhead mutual exclusion to coordinate the access.

Lightweight POSIX pthreads APIs ideally suit resource-constrained embedded devices. These APIs also make development easier and more intuitive. Take, for example, the *pthread_create* API to create a new thread with the entry function of "my_thread_entry":

```
pthread_create(&my_thread_handle, NULL, my_thread_entry, NULL);
```

The *pthread_create* API requires only two elements from the application developer: a thread handle and an entry function for the thread. Notably, the *pthread_create* API is much simpler than most RTOS thread/task creation APIs, many of which require more parameters and, therefore, more complexity.

The robust POSIX pthread standard also specifies many services, including mutual exclusion, synchronization, and communication primitives. Additional POSIX (not pthreads per se) APIs apply to resource-constrained embedded systems, including semaphores, message queues, signals, and time-related services.



Why pthreads for Embedded?

Recent surveys show that embedded Linux accounts for nearly 70% of embedded development. As mentioned, embedded Linux is based on the POSIX pthread API, making pthreads the most well-known and used API in the embedded industry. While there have long been many good proprietary RTOS APIs in the embedded space, they all require significant developer training, limiting the underlying RTOS's use (and adoption). Proprietary RTOS APIs make application code less portable. Furthermore, many device makers have Linux and RTOS-based devices servicing different target markets and price points. An RTOS based on the industry-standard POSIX pthreads API solves these problems, reducing developer training and making application code portable across various platforms.

Chapter 3: PX5 RTOS Installation and Use

It's easy to Install and use the PX5 RTOS. Starting from a basic C *main* program, there are just three steps:

3 Simple Steps

1. Place the PX5 RTOS distribution into your C *main* project source directory.
2. The PX5 RTOS distribution contains two main source files, namely ***px5.c*** and ***px5_binding.s***, as well as supporting C header files. Add both of them to your project (IDE or makefile).
3. Modify your C main program to include ***pthread.h*** and call ***px5_pthread_start*** in your main program. Next, create a *while(1)* loop, since ***px5_pthread_start*** upscales the *main* function into your system's first thread!

Your C main program should now look something like the following (PX5 RTOS-specific additions in red):

```
#include "pthread.h"

int main(void)
{
    /* Call PX5 RTOS initialization. */
    px5_pthread_start(1, NULL, NULL);

    /* We are now in the context of a thread. */
    while(1)
    {
        /* All other PX5 RTOS API calls are now available! */
    }
}
```

Of course, the exact process varies depending on the processor or tool. Please review the binding layer documentation for more details.

After these three steps, you should be up and running with the PX5 RTOS! To enable timer-related services, add a call to ***px5_timer_interrupt_process*** from

within the periodic timer interrupt handler (note that this call requires approximately 160 bytes of stack on most 32-bit architectures).

No linker control file changes are typically required to install and use the PX5 RTOS. There also shouldn't be any project setting changes if the PX5 RTOS source code is in the same directory as the C *main* function source. However, if the PX5 RTOS source is placed in another location, you will need to update the C include paths of your project to point to the PX5 RTOS header files.

Please review the *px5_pthread_start* API defined later in this guide. This API provides parameters for run-time identification and memory for object creation. It also performs essential error checking, including verification of the binding between the *px5.c* C source and the *px5_binding.s* assembly code. The return code of this API should always be checked. For clarity, we omitted checking for API return code errors in this simple example.

PX5_RTOS_Binding_User_Guide.pdf

As mentioned previously, please review the *PX5_RTOS_Binding_User_Guide.pdf* for more detailed information about the operation of the PX5 RTOS with the processor and development tool you are using – including binding-specific configuration options. This guide also includes ready-to-run example(s) specific to your processor and development tool and guidance on enabling periodic timer-related services.

Defined PX5 Symbol

If you include any of the PX5 RTOS include files, the symbol *PX5* is defined. This is useful for cross-platform applications so that conditionals can be placed around PX5 RTOS pthread+ extensions.

Configuration Options for the PX5 RTOS

There are multiple compile-time configuration options for building and using the PX5 RTOS. Generally, the options are independent of each other. The following describes each option in detail (Note that processor/tool-specific configuration options are defined in the *PX5_RTOS_Binding_User_Guide.pdf* guide.):

Build Option	Meaning
<i>PTHREAD_KEYS_MAX</i>	Defines the maximum <i>Thread Local Storage (TLS)</i> keys. The default value is one, which means each application thread may have one TLS pointer.

PX5_CANCELLATION_POINTS_ENABLE

When defined, the cancellation points in the blocking APIs are enabled. This option is not defined by default. Note that when this option is used, the stack requirements for all APIs that can suspend or terminate a thread increase by approximately 100 bytes (or 280 bytes if internal memory management is enabled).

PX5_DEFAULT_CORE_AFFINITY

When *PX5_MAXIMUM_CORES* is greater than one, this value defines the default core affinity of each created thread (each bit position set represents allowed execution on the corresponding core). The default value enables execution on all cores. Please see the *px5_binding.h* file for the specific value. This option is not relevant in single core operation.

PX5_DEFAULT_MAIN_CORE_AFFINITY

When *PX5_MAXIMUM_CORES* is greater than one, this value defines the default core affinity of the main thread (each bit position set represents allowed execution on the corresponding core). The default value enables execution on all cores. Core 0 must always be allowed. Please see the *px5_binding.h* file for the specific value. This option is not relevant in single core operation.

PX5_DEFAULT_PRIORITY

When defined, this value overrides the default thread priority for thread creation (priority 16 is the default thread create priority).

PX5_DEFAULT_SIGNAL_MASK

When defined, this value overrides the default signal bit mask of the

	main thread (all signals masked - 0xFFFFFFFF).
<i>PX5_DEFAULT_STACK_SIZE</i>	When defined, this value overrides the default stack size, which is processor-specific (typically on the order of 1K bytes).
<i>PX5_EVENT_FLAG_CONSUMPTION_MASK_ENABLE</i>	When defined, the automatic event flag consumption can be selectively overridden by setting the automatic consumption mask in the attributes when the event flags group is created. This option is not defined by default.
<i>PX5_FUNCTION_POINTER_VERIFY_ENABLE</i>	When defined, all function pointers used internally in the PX5 RTOS are evaluated against the verification code established when they were set up (using PDV). This mechanism facilitates the early detection of memory corruption, whether intentional or unintentional. This option is not defined by default.
<i>PX5_MEMORYPOOL_VERIFY_ENABLE</i>	When defined, all internal PX5 RTOS memory pool linked-lists are evaluated against the established verification code when each memory fragment is created. Verification occurs when memory from the pool is allocated or released. This mechanism helps early detection of memory corruption, most usually associated with the application writing past the allocated memory. This option is not enabled by default.
<i>PX5_MISRA_ENABLE</i>	When defined, PX5 RTOS is configured to conform to MISRA

	coding standards, with very few exceptions.
<i>PX5_NOTIFICATION_ENABLE</i>	When defined, application notification via callbacks for message queues, event flags, fastqueues, and semaphores is enabled. Please review the <i>px5_*_notify</i> APIs for registering the notification callback functions. This option is not defined by default.
<i>PX5_OBJECT_VERIFY_ENABLE</i>	When defined, all internal PX5 RTOS objects (including the global PX5 RTOS data) are evaluated against the verification code established when they were created (using PDV). This mechanism helps early detection of memory corruption—both intentional and non-intentional. This option is not enabled by default.
<i>PX5_PARAMETER_CHECKING_DISABLE</i>	When defined, basic parameter checking is disabled. (By default, parameter checking is enabled.) This can be used on a per-file basis, i.e., parameter checking can be enabled (default) for some application files and disabled via this option in other files. This option is not defined by default.
<i>PX5_PARTITIONPOOL_VERIFY_ENABLE</i>	When defined, all internal PX5 RTOS partition pool linked-lists are evaluated against the verification code established when the partition pool was created. The verification occurs as partitions from the pool are allocated or released. This mechanism helps early detection of memory corruption, most usually

	associated with the application writing past the allocated memory partition. This option is not enabled by default.
<i>PX5_SMP_SAME_PRIORITY_RESTRICT</i>	When defined, the PX5 RTOS only allows threads of the same priority to execute simultaneously on multiple cores (SMP only). This option is not relevant in single core operation.
<i>PX5_SPECIFIC_ERRNO</i>	When defined, the PX5 RTOS does not redefine <i>errno</i> in the application code. Instead, the <i>px5_errno</i> symbol is remapped to retrieve the thread-specific API error information. The application may also use the <i>px5_errno_get</i> API directly. This define is useful in situations where multiple software entities have their own definition of <i>errno</i> . This option is not defined by default.
<i>PX5_SPECIFIC_SIGACTION</i>	When defined, the PX5 RTOS does not redefine <i>sigaction</i> in the application code. Instead, the <i>px5_sigaction</i> API and <i>px5_sigaction_t</i> and <i>px5_sinfo_t</i> types must be used. This define is useful in situations where multiple software entities have their own definition of <i>sigaction</i> . This option is not defined by default.
<i>PX5_STACK_CHECK_ENABLE</i>	When defined, the PX5 RTOS performs stack size checking, including updating the minimal available stack and determining if the stack has overflowed or is in imminent danger of an overflow.

	This option is not enabled by default.
<code>PX5_STACK_VERIFY_ENABLE</code>	When defined, the PX5 RTOS performs stack integrity checking (using PDV), including verification of the function call return address when possible. This option is not enabled by default.
<code>PX5_TICKS_PER_SECOND</code>	By default, this option is defined as 1000, representing a 1ms timer interrupt frequency. However, the application may use a different timer interrupt frequency if this define is adjusted accordingly.
<code>PX5_TIME_REMAPPING_DISABLE</code>	When defined, the PX5 RTOS does not remap application time types (e.g., <i>time_t</i> and <i>timespec</i>) to the PX5 RTOS equivalents. If this is defined, the application code must prepend <i>px5_</i> to any time type referenced, e.g., <i>px5_time_t</i> and <i>px5_timespec</i> . This define is useful when multiple software entities have their own definition of time types. This option is not defined by default.

Using Configuration Options

The PX5 RTOS configuration options mentioned previously may be defined via project setting or via -D compiler command line options. Alternatively, you may use the *px5_user_config.h* in the PX5 RTOS distribution as dedicated for application use. It is a safe place to define configuration options for the PX5 RTOS since it does not change with each new version of the PX5 RTOS.



It is highly recommended to compile the application code with the same configuration options used to compile the PX5 RTOS source files (px5.c and px5_binding.s).

Troubleshooting

The PX5 RTOS is designed for ease of use and reliability, so you will not likely experience issues. However, if you do experience issues, here are some basic troubleshooting suggestions that may help:

1. Ensure that the processor/tool-specific examples mentioned in the *PX5_RTOS_Binding_User_Guide.pdf* (or supplemental documentation) are executing normally in your environment.
2. Your application code should always check the return status on all API calls.
3. Be suspicious of any recent change(s).
4. Please ensure each thread has a large enough stack allocated. Each thread stack must be large enough to hold the worst-case C function call depth with memory sufficient for all local variables. Please review the stack-checking APIs and functionality described later in this guide for assistance. Thread stack corruption is often the cause of unusual run-time problems in any RTOS-based application.
5. If a ready thread isn't running, ensure that all higher-priority and same-priority threads are eventually suspended (or relinquished) so the thread in question can execute. Remember to use as few priorities as possible. This will reduce context-switching overhead as well as the potential for thread starvation.
6. If time-related services are not working, verify that the periodic timer interrupt is occurring and that you are calling *px5_timer_interrupt_process* from within the interrupt handler.
7. Set a breakpoint on the PX5 RTOS central error handling function *px5_internal_central_error_process*. If the breakpoint is hit, the type of error, executing thread, current interrupt level, and return address should provide valuable debug information.

Version Information

The PX5 RTOS version identification is comprised of four numbers separated by periods in the general format of V.M.U.P, with the following meaning:

Version Number	Meaning
V	Major version
M	Minor version
U	Update version
P	Patch version

The specific version of PX5 you are using can be found near the top of any source file, as shown below:

[illegible]

The following constants in *px5.h* also contain the version:

```
#define PX5_MAJOR_VERSION 5U
#define PX5_MINOR_VERSION 3U
#define PX5_UPDATE_VERSION 0U
#define PX5_PATCH_VERSION 0U
```

Long Term Support (LTS)

We support major versions (the first number in the version ID) for five years from release. Minor versions (represented by the second number) are supported for two years from release.

Issue Notification

Our policy is to alert customers via e-mail when an issue is detected. At the same time, we will supply instructions for downloading an update that resolves the problem.

Contact Us

Please feel free to contact us with any problem. We are glad to help! To make the request as efficient as possible, please provide the following information:

1. Your name and company name
2. Your *px5_binding.h* and *px5.h* header files
3. Brief description of the problem
4. Screen shots and any data collection (sometimes a memory display of the *px5_globals* data structure is helpful)

The best place to initiate a support ticket is on the PX5 website:

www.px5rtos.com/support

You may also send an e-mail to:

support@px5rtos.com

You may also contact us by mail and telephone:

PX5

11440 West Bernardo Court, Suite 300
San Diego, CA 92127
Phone: +1 (858) 753-1715

Chapter 4: PX5 RTOS Safety and Security

Today, safety and security for embedded devices are paramount. Although safety and security are distinct objectives, they overlap in the context of the embedded PX5 RTOS. Memory corruption—intentional or unintentional—is the most common source of safety and security issues in embedded devices. In addition to helping avoid memory corruption, the PX5 RTOS safety and security features help form a defense-in-depth solution that includes application software, device hardware, and other network/cloud entities and their configuration/settings.

Safety and security requirements vary depending on the application. They are ultimately defined by the attack surface and what is practical from a technological and business standpoint. Stated another way, embedded safety and security doesn't call for a one-size-fits-all approach but rather a deliberate risk-benefit analysis based on each use case.

Functional Safety Certification

The PX5 RTOS has achieved functional safety certification to the highest levels of the IEC 61508, IEC 62304, ISO 26262, and EN 50128 functional safety standards, specifically IEC 61508 SIL 4, IEC 62304 Class C, ISO 26262 ASIL D, and EN 50128 SW-SIL 4. The PX5 RTOS certifications were performed by SGS-TÜV Saar, a leading accredited and independent company for testing, auditing, verifying, and certifying embedded software for safety-related systems.

Hardware Safety & Security Features

The wide range of embedded processors offers an even greater variety of hardware safety and security features, the most common of which are:

1. **Anti-tampering.** This hardware feature protects the firmware IP of the device from unauthorized access. It is generally outside the scope of the PX5 RTOS or application firmware but an important consideration when selecting hardware.
2. **Lock-step execution.** This hardware feature employs multiple processors executing the same code with the same data, thus guaranteeing exact code execution. Such hardware is primarily found in safety-critical

applications. This is invisible to the PX5 RTOS and most application firmware.

3. Anti-glitch. This hardware feature employs circuitry to prevent an attacker from causing abnormal program execution via manipulating power or other system signals. This, too, is generally outside the scope of the PX5 RTOS or application firmware but may be an important consideration when selecting hardware.
4. Execute only from flash. Most microcontrollers (MCUs) execute instructions from flash. Some of these MCUs can prohibit execution from RAM, which is recommended to help prevent the dynamic insertion of malicious code in remote execution attacks. This sometimes requires support within the application firmware to enable, but otherwise, it is invisible to the PX5 RTOS and the application.
5. Hardware stack limit. Some processors have a stack limit register that guards against memory corruption caused by stack overflow. This is generally implemented as an additional register and is set up by the PX5 RTOS on each thread context-switch.
6. Hardware watchdog timer. Many processors have a non-maskable hardware watchdog timer. This feature acts as a fail-safe. Under regular operation, the application code resets the watchdog regularly and consistently before expiration. During abnormal execution, the watchdog is likely not reset, thus leading to a non-maskable interrupt that halts the abnormal execution. Typically, applications will reset after a watchdog expiration.
7. True Random Number Generator (TRNG). A TRNG or even more basic Random Number Generator (RNG) in hardware is very beneficial. This is most important for networked devices, but it's also useful for the PX5 RTOS—especially for the unique PX5 RTOS Pointer/Data Verification (PDV) feature.
8. Memory Management Unit (MMU). Typically, available only on larger, more powerful processors, this hardware feature can restrict access to various memory regions. This is often set up once by the application firmware after reset to map and protect various memory regions.
9. Memory Protection Unit (MPU). This hardware feature is similar to the MMU but is found in smaller, more resource-constrained devices. Again, this is typically set up by the application firmware after reset to protect various memory regions. In cases with no stack limit registers, the MPU

can set up a protected block at the top of each thread's stack to prevent overflow. This functionality must be accomplished inside the PX5 RTOS thread context-switching logic.

10. Secure Element (SE) or Trusted Platform Module (TPM). An SE or TPM can securely isolate credentials or other secrets from the main application for networked devices. This can significantly increase the network security of the device and is therefore highly recommended.

Of course, each hardware safety and security feature mentioned has an associated cost—in circuitry, power consumption, size, etc. These costs must go through the risk-benefit analysis discussed previously.

Software Safety & Security Features

Hardware safety and security features require software support, such as setting up the hardware stack limit feature or using the MPU to protect certain memory areas. The PX5 RTOS binding layer is where such support is typically located. The application firmware may also have logic to support the various hardware safety and security features.

PX5 RTOS Pointer/Data Verification (PDV)

To further enhance safety and security, the PX5 RTOS provides Pointer/Data Verification (PDV), a unique software-only technique to help detect corruption of important data like function pointers, function return addresses, internal system objects, allocated memory, etc. PDV utilizes the pointer or data value, the storage location of the verification code, and the unique run-time identification provided to *px5_pthread_start* to create a verification code (fingerprint) for each important data element during its initialization. Before the important data is used, it is authenticated against the verification code. If corruption is detected, the application is alerted via the PX5 RTOS central error handling, at which point the application can take the necessary measures to respond to the memory corruption. PDV helps detect memory corruption early and dramatically reduces the chance of unwanted execution associated with function pointer corruption.



Run-Time Stack Checking & Verification

The PX5 RTOS offers additional software-only stack safety and security measures. When enabled, run-time stack checking examines the current stack pointer upon function entry to check for overflow or imminent overflow and keeps track of the minimal available amount of stack memory. Simply build the PX5 RTOS source with *PX5_STACK_CHECKING_ENABLE*, and each thread's stack is checked throughout PX5 RTOS execution. The application may also utilize the *px5_pthread_stack_check*

API to perform stack checking from the application C code. Stack verification uses PDV to help verify stack integrity—specifically the caller return address on the stack when supported by the compiler—before returning to the caller of a function. To enable stack verification, build the PX5 RTOS source with `PX5_STACK_VERIFY_ENABLE` defined.

If a stack corruption or overflow is detected, the central error handling function is called with a fatal error. If stack checking detects a stack at risk of overflow, the central error handling is called with the appropriate advisory error.

Application Best Practices

For enhanced safety and security, an application should follow *best practices* that include:

1. Harden the device firmware. The PX5 RTOS was implemented in a Test Driven Development (TDD) manner, which means the tests are written before the actual code. Furthermore, by design, the PX5 RTOS code base must achieve 100% statement and 100% branch/decision coverage testing before each minor release. We recommend the application firmware take a similar approach: There is never enough testing. The better-vetted software is, the safer and more secure it is.
2. Leverage static analysis and related tools. In addition to the hardening mentioned previously, it's a good idea to leverage static analysis tools, penetration testing, and fuzzing tools. These tools help find subtle issues in advance, which is often much easier than debugging fielded devices.
3. Use PX5 RTOS Pointer/Data Verification (PDV). The PX5 RTOS optionally (as determined by `px5.c` build options) uses PDV to verify function pointers, stack integrity, internal system objects, and allocated memory. Through API extensions, the application can also use PDV to verify its important function pointers and data. By using PVD, memory corruption can be detected early, and the risk of unwanted program execution, including malicious remote execution, can be significantly reduced.
4. Use an adequate (or larger) stack size. Stack overflow is the number one cause of memory corruption in embedded systems. Each thread stack must have enough memory for its worst-case function call nesting – including all local variables in each function. If not, the stack may overflow into the memory directly preceding the stack. This problem can be mitigated by using hardware stack limit features or the MPU/MMU to guard the area directly above the stack. The PX5 RTOS stack-checking features can also help prevent stack overflow issues.

5. Use the PX5 RTOS run-time stack checking. The PX5 RTOS run-time stack checking is an easy way to detect (and correct) stack overflows.
6. Explicitly specify and check buffer sizes. In all functions where a buffer is supplied, it is essential for the caller to explicitly provide the size of the buffer and the callee to explicitly check the size to avoid overrun. The PX5 RTOS does this internally, and the application firmware should also do it.
7. Be mindful of more likely areas of memory corruption. When a thread stack overflows, it generally corrupts the memory immediately *preceding* the thread stack memory (in most architectures, thread stack grows toward lower addresses). In contrast, buffer overflows are more likely to corrupt memory immediately *following* the buffer. Therefore, avoiding placing critical data before stacks or immediately following buffers might be safer.
8. Use PDV to place markers before stacks and after data buffers to help detect memory corruption caused by stack and buffer overflows. Please see the `px5_pthread_pdv_*` APIs for more details.
9. Be careful with function pointers. Function pointers provide an easy path to unwanted program execution – both unintentional and intentional. For example, it's not good practice to place function pointers inside buffers since a buffer overflow could overwrite the function pointer. This is the easiest way for an attacker to initiate unwanted remote execution. Of course, the PDV feature of the PX5 RTOS can be used to verify application function pointers before they are called, which helps mitigate this issue.

Chapter 5: PX5 RTOS Pointer/Data Verification



Pointer/Data Verification is a software-only technique to help detect and mitigate both intentional and accidental memory corruption. For important information, a verification code is created and stored in memory. Before the important information is used, the verification code is generated and compared with what was stored. If the codes don't match, memory corruption has occurred, and the PX5 RTOS immediately alerts the application by calling the central error handling function. The application can define precisely what happens in the central error handling.

Default Verification Code

The application can define the default formula for generating the verification code. However, by default, the verification code combines a run-time identification (secret) passed to the PX5 RTOS in the `px5_thread_start` API, along with the value of the important information and the address to store the generated code. The default formula looks something like this:

$$\text{Verification Code} = ((\text{Data Value}) + (\text{Address to Store Code}) + (\text{Secret})) ^ (\text{Secret})$$

We recommend using a True Random Number Generator (TRNG) if available in hardware for verification codes that are run-time unique. With a TRNG, the verification code for each important data element has a temporal property, i.e., it will be unique for each application execution running on top of the PX5 RTOS. This makes it much harder for hackers to insert malicious information, such as function pointers for remote execution attacks. The verification code storage address provides a particular property to the verification code. It's unlikely that any two images will have the same memory layout, making it more difficult for hackers to change important information without detection.



Note that the default verification code can be changed by overriding the PX5 RTOS macro `PX5_POINTER_DATA_VERIFY_CODE_COMPUTE` to any formula the application desires.

Important Information Verified

The PX5 RTOS provides optional PDV protection over a series of important internal information, including the following:

- All function pointers used in the PX5 RTOS
- Global data of the PX5 RTOS
- Internal system structures with the PX5 RTOS (threads, queues, etc.)
- Return addresses on internal PX5 RTOS functions (with compiler support)
- Metadata pointers used for memory management
- APIs for application-specific use of PDV

Enabling PDV

As mentioned, the use of PDV is optional and is not enabled by default. In addition, it can be selectively enabled for specific PX5 RTOS areas. To enable PDV, the PX5 RTOS source should be built with the following defines (depending on the exact verification requested):

<i>PX5_FUNCTION_POINTER_VERIFY_ENABLE</i>	When defined, all function pointers used internally in the PX5 RTOS are evaluated against the verification code established when they were set up. With this enabled, it's much harder for hackers to insert rogue function pointers in remote execution attacks.
<i>PX5_OBJECT_VERIFY_ENABLE</i>	When defined, all internal PX5 RTOS objects (including the global PX5 RTOS data) are evaluated against the verification code that was established when they were created. This mechanism facilitates early detection of memory corruption – both intentional and non-intentional.
<i>PX5_MEMORYPOOL_VERIFY_ENABLE</i>	When defined, all internal PX5 RTOS memory pool linked-lists are

evaluated against the verification code established when the memory pool is created. The verification occurs when memory from the pool is allocated or released. This mechanism helps early detection of memory corruption, most usually associated with the application writing past the allocated memory.

PX5_PARTITIONPOOL_VERIFY_ENABLE

When defined, all internal PX5 RTOS partition pool linked-lists are evaluated against the verification code established when the partition pool is created. The verification occurs when a memory block from the pool is allocated or released. This mechanism helps early detection of memory corruption, most usually associated with the application writing past the allocated memory block.

PX5_STACK_VERIFY_ENABLE

When defined, the PX5 RTOS performs stack integrity checking, including verification of the function call return address when possible (when supported by the compiler).

PDV Overhead

The overhead associated with PDV depends on CPU architecture and the compiler but is generally minimal. Assuming the default verification code generation described previously, the assembly code to generate the verification code is only a couple of instructions on a typical Arm Cortex-M architecture as follows:

```

        ADDS  R3, R2, R0
        EORS  R3, R3, R4

```

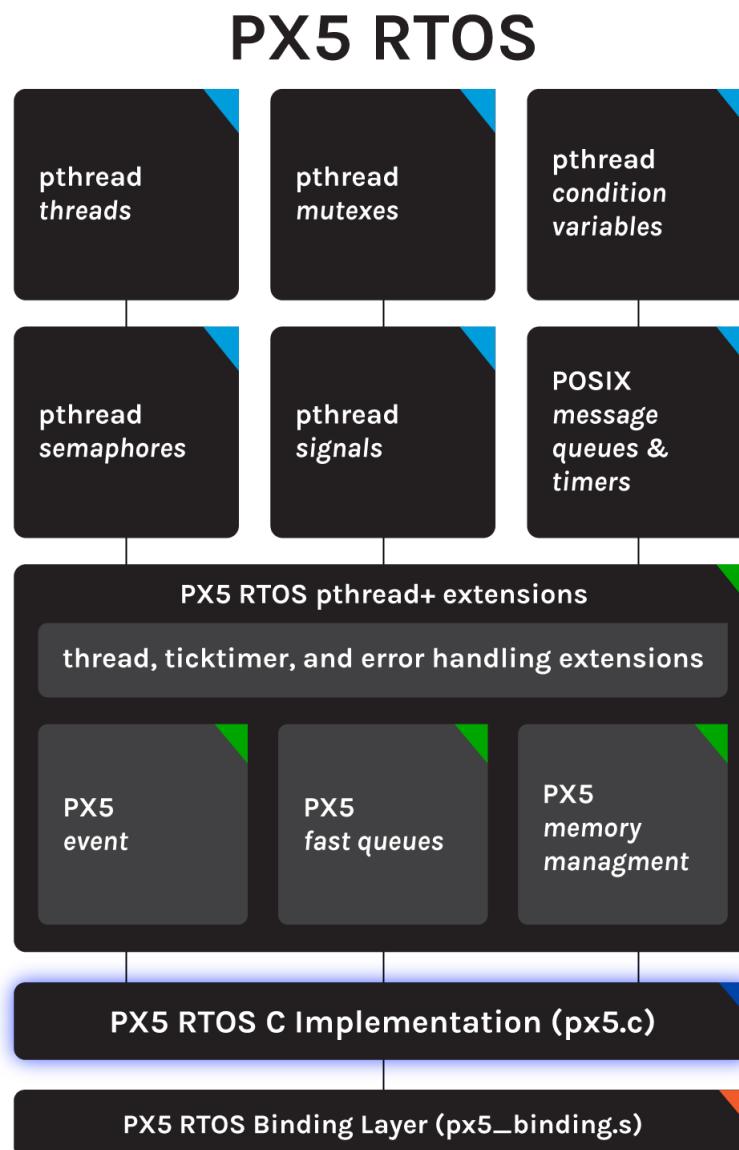
This code assumes that R0 contains the important data value, R2 contains the address to store the verification code, and R3 contains the run-time secret. It's reasonable to assume that each register might require a load instruction (LDR) and

one store instruction (STR) to store the code. Given all that, building and storing the default verification code takes roughly six assembly instructions.

To verify the code, another six assembly instructions are required, along with another three instructions to load the previously stored code, compare it, and branch to either the “okay” path or to the central error handling.

Chapter 6: PX5 RTOS Functional Overview

This chapter provides a complete functional overview of the PX5 RTOS. The following block diagram provides a high-level overview of the PX5 RTOS:



Each feature of the PX5 RTOS is discussed in the following paragraphs.

Composition

The PX5 RTOS comprises two main source files: *px5.c* and *px5_binding.s*. As shown in the block diagram above, almost all PX5 RTOS functionality is implemented in ANSI C and resides in *px5.c*. All development tool and processor-specific logic are contained in the assembly-language *px5_binding.s* file. The binding file is intentionally kept minimal to enhance PX5 RTOS portability. Detailed information regarding the binding layer assembly file for each unique processor and development tool support for the PX5 RTOS can be found in the accompanying *PX5_RTOS_Binding_User_Guide.pdf* document.

Namespace

All global symbols in the PX5 RTOS—both functions and global data—have names with *px5_* prepended. This makes identifying the PX5 RTOS symbols on a linker load map easy.

Native pthreads Implementation

The PX5 RTOS is a native implementation of the POSIX pthreads standards. By “native,” we mean that the pthreads API is not a layer on top of another RTOS API but instead is implemented directly. For example, the API *pthread_mutex_lock* is an actual C function implemented in *px5.c*, containing all the mutex lock code necessary for operation. There are no other layers or internal RTOS primitives that this code relies on or is built on top of. The result is the fastest and smallest possible implementation of *pthread* APIs.

pthreads Extensions

The POSIX pthreads standard provides services to create robust multithreaded applications, most commonly on Linux or embedded Linux platforms. That said, embedded development is often more demanding, requiring advanced real-time capabilities—in terms of performance and functionality. The PX5 RTOS implements the core pthreads APIs in the most real-time, deterministic way possible, satisfying the real-time demands of embedded systems. The PX5 RTOS also offers essential extensions to the pthreads standards in the form of new APIs and functionality. Collectively, these extensions are called pthreads+. All API and functionality extensions to pthreads are easily identifiable by the “*px5_*” prefix to the API name.

Internal Memory Management

The PX5 RTOS does not require dynamic memory allocation for its operation. However, each new system object (thread, mutex, semaphore, etc.) created by the application requires memory for its internal control structure. Some objects also have additional memory requirements, e.g., each thread requires a stack, and each



message queue requires memory to store messages. The PX5 RTOS object memory requirements can be satisfied in the following ways:

1. One-time, sequential allocation for each object creation/initialization from memory supplied to the *px5_thread_start* API. This is the easiest and fastest approach with the lowest overhead. This approach is often sufficient for systems not dynamically destroying and re-creating system objects.
2. The application explicitly provides memory for each object. Through attribute extensions, each object in the PX5 RTOS enables the application to supply the memory required for the object's creation explicitly. If this approach is used throughout, no additional memory is needed for the PX5 RTOS. Additionally, this approach allows the application to designate the location of every object's memory specifically. Please see the following APIs for more information:

pthread_attr_setstackaddr
px5_mq_extendattr_setcontroladdr
px5_mq_extendattr_setqueueaddr
px5_thread_attr_setcontroladdr
px5_thread_condattr_setcontroladdr
px5_thread_event_flagsattr_setcontroladdr
px5_thread_fastqueueattr_setcontroladdr
px5_thread_fastqueueattr_setqueueaddr
px5_thread_memorypoolattr_setcontroladdr
px5_thread_mutexattr_setcontroladdr
px5_thread_partitionpoolattr_setcontroladdr
px5_thread_ticktimerattr_setcontroladdr
px5_semattr_setcontroladdr

3. Dynamic memory management of the memory is supplied to the *px5_thread_start* API. This approach relies on the memory provided to *px5_thread_start*. However, it is fully managed underneath via a PX5 RTOS variable-length memory pool. This facilitates dynamic destruction and re-creation of objects. Please see the *px5_memory_manager_enable* API for more details.
4. Application overrides the internal memory management. This approach allows the application to override the internal PX5 RTOS memory management. This gives the application complete control of dynamic memory allocation within the PX5 RTOS. Please see the

`px5_memory_manager_get` and `px5_memory_manager_set` API for more details.

Multicore Support

The PX5 RTOS is designed for *Symmetric Multiprocessing (SMP)*, where threads are dynamically scheduled on available cores. The `PX5_MAXIMUM_CORES` constant defines how many cores are supported. By default, this is set to one for single-core operation. In SMP configurations, this constant is typically defined in the `px5_binding.h` file. Please see the *PX5_RTOS_Binding_User_Guide.pdf* for more information.

The default, single-core version of the PX5 RTOS can be used in multicore situations, where each core has its own copy of the PX5 RTOS and the associated application code. This multicore configuration is called *Asymmetric Multiprocessing (AMP)*.

Pointer/Data Verification (PDV)



The PX5 RTOS provides early detection and mitigation of memory corruption through its unique *Pointer/Data Verification (PDV)* functionality. During run-time, PDV can verify all internal function pointers, global data, object control structures, and stacks. Please see Chapter 5 for a complete description of PDV.

Run-time Stack Checking/Verification

Memory corruption via stack overflow is the leading cause of erroneous program execution in most embedded applications. In addition to taking advantage of hardware stack limit protections for overrun, the PX5 RTOS provides run-time stack checking. Simply build the PX5 RTOS source with `PX5_STACK_CHECK_ENABLE`, and each thread's stack size is checked throughout PX5 RTOS execution. The application may also utilize the `px5_pthread_stack_check` API to perform stack checking from the application's C code.

Stack integrity checking is also available. Building the PX5 RTOS source code with `PX5_STACK_VERIFY_ENABLE` defined enables run-time integrity checking of each thread's stack. This integrity checking—via the PX5 RTOS PDV technology—includes verification of the function caller's return address when accessible via the compiler.

If a stack corruption or overflow is detected, the central error handling function is called with a fatal error. If stack checking detects a stack at risk of overflow, the central error handling is called with the appropriate advisory error.

Central Error Handling

The PX5 RTOS provides central error handling, meaning all system errors are routed to a single internal handler. System errors are grouped into three basic categories as follows:

PX5_LEVEL_3_ERROR
PX5_LEVEL_2_ERROR
PX5_LEVEL_1_ERROR

Level 3 errors are considered fatal errors and are generally not recoverable. Level 2 errors are serious and could soon result in a fatal error. Level 1 errors are the least serious and often result from an invalid API parameter. The default processing inside of PX5 is to register the error information received and return. However, the developer can augment the error processing for each level by overriding these symbols (Typically, this would be done in the *px5_user_config.h*):

PX5_LEVEL_3_ERROR_PROCESSING
PX5_LEVEL_2_ERROR_PROCESSING
PX5_LEVEL_1_ERROR_PROCESSING

Setting a breakpoint on the *px5_internal_central_error_process* routine is also good practice during development.

Scheduling Policy

There are typically three scheduling policies offered in association with POSIX pthreads, as follows:

SCHED_FIFO

This is the basic priority-based, preemptive scheduling policy where higher-priority threads preempt lower-priority threads. Aside from preemption, each thread runs until it is blocked, completes, or is terminated.

SCHED_RR

This simply adds time-slicing to *SCHED_FIFO*.

SCHED_OTHER

This is the implementation-defined scheduling policy in pthreads, which is how the PX5 RTOS identifies its scheduling policy.

The PX5 RTOS employs a priority-based, preemptive scheduling policy that closely matches *SCHED_RR*. However, since the PX5 RTOS implements per-thread time-slicing and various pthread extensions, the *pthread_attr_getschedparam* API returns *SCHED_OTHER* to indicate it is implementation-defined. The PX5 RTOS does not support dynamic changes to scheduling policies.

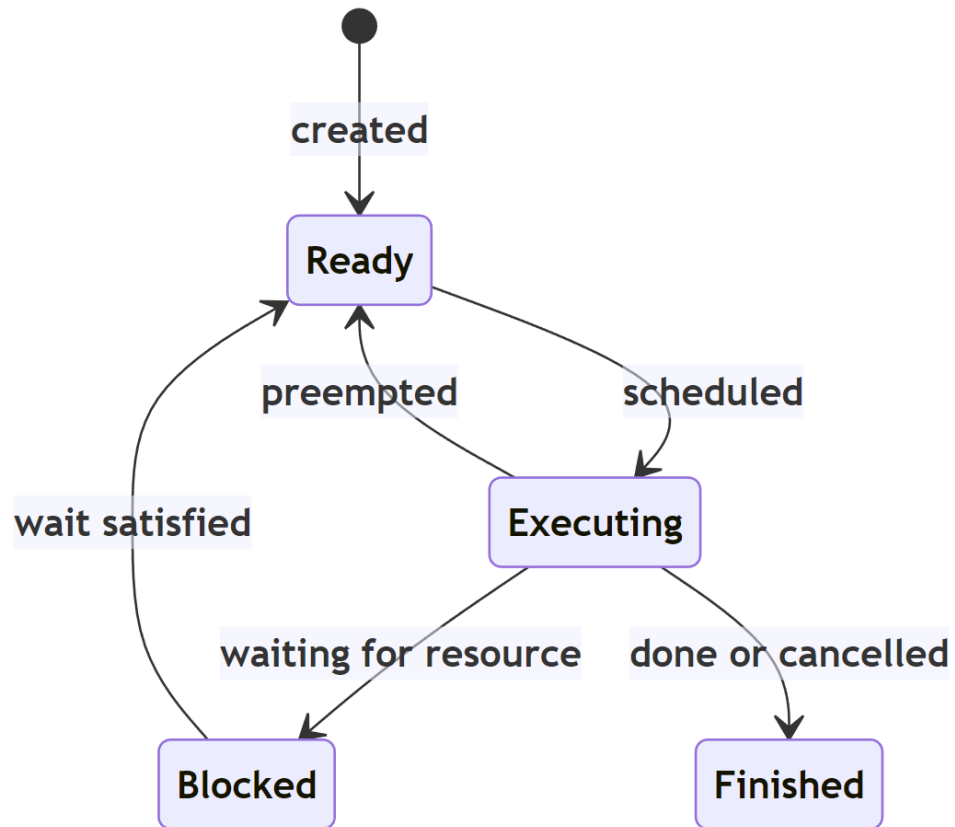
Thread Priorities

The PX5 RTOS supports 32 thread priority levels, ranging from 0 through 31. Priority level 0 is the lowest (least important). Conversely, priority level 31 is the highest (most important). Preemption occurs (thread context-switch) when a higher-priority thread becomes ready during the execution of a lower-priority thread.

Thread States

Each thread in the PX5 RTOS is in one of four high-level states—EXECUTING, READY, BLOCKED, or FINISHED. For single-core processors, there is only one thread in the EXECUTING state, which generally represents the highest-priority, ready thread. Threads in the READY state wait for their turn to enter the EXECUTING state. Threads in the BLOCKED state wait for an event that satisfies the previous request that caused them to enter the BLOCKED state. This may be a PX5 RTOS API call, a system event from another thread, or an Interrupt Service Routine (ISR). Once unblocked, these threads enter the READY state. Threads in the FINISHED state have called *pthread_exit*, have returned from their entry function, or have been canceled. The following diagram shows the typical thread state transitions in the PX5 RTOS:





Once in the FINISHED state, the thread can never become ready or execute again, i.e., there are no transitions from the FINISHED state.

System Objects

The PX5 RTOS supports the most popular POSIX objects, including *threads*, *condition variables*, *message queues*, *mutexes*, *semaphores*, *signals*, and *timers*. There is no compile-time limit on the number of system objects an application may have. The only limit is the amount of memory available in the application.

The PX5 RTOS does not create any objects itself, i.e., there are no hidden system threads or any other mutex. The only object created by the PX5 RTOS is the initial “*main*” thread, which ultimately is an application thread.

As mentioned, the application completely controls the memory required for object control blocks and associated memory areas.



Threads

The most fundamental object in PX5 RTOS is the thread, a semi-independent application program segment with its stack and assigned priority for execution. PX5 RTOS ensures that the highest-priority thread executes before lower-priority threads.

The following are the thread APIs supported by the PX5 RTOS:

```

pthread_attr_destroy
pthread_attr_getdetachstate
pthread_attr_getstackaddr
pthread_attr_getstacksize
pthread_attr_init
pthread_attr_setdetachstate
pthread_attr_setstackaddr
pthread_attr_setstacksize
pthread_cancel
pthread_cleanup_pop
pthread_cleanup_push
pthread_create
pthread_detach
pthread_equal
pthread_exit
pthread_getaffinity_np
pthread_getspecific
pthread_join
pthread_key_create
pthread_key_delete
pthread_once
pthread_self
pthread_setaffinity_np
pthread_setcancelstate
pthread_setcanceltype
pthread_setspecific
pthread_testcancel
px5_thread_attr_getaffinity
px5_thread_attr_getautostart
px5_thread_attr_getcontroladdr
px5_thread_attr_getcontrolsize
px5_thread_attr_getname
px5_thread_attr_getpriority
px5_thread_attr_gettimeslice

```

px5_thread_attr_setaffinity
px5_thread_attr_setautostart
px5_thread_attr_setcontroladdr
px5_thread_attr_setname
px5_thread_attr_setpriority
px5_thread_attr_settimeslice
*px5_thread_information_get**
px5_thread_priority_change
*px5_thread_resume**
px5_thread_stack_check
*px5_thread_suspend**
px5_thread_timeslice_change

**Denotes APIs available from ISRs*

Condition Variables



The POSIX pthreads condition variable is a communication object that provides built-in synchronization through direct association with a mutex object. Threads can block on a condition variable (even with timeout) to wait for data or some state to be reached. When the data or state is reached, another thread or threads can signal or broadcast this. The highest priority waiting thread is resumed if a condition variable is signaled. If the condition variable is broadcast, all threads waiting on the condition variable are resumed.

The following are the condition variable APIs supported by the PX5 RTOS:

pthread_cond_broadcast
pthread_cond_destroy
pthread_cond_init
pthread_cond_signal
pthread_cond_timedwait
pthread_cond_wait
pthread_condattr_destroy
pthread_condattr_getpshared
pthread_condattr_init
pthread_condattr_setpshared
*px5_thread_cond_information_get**
px5_thread_condattr_getcontroladdr
px5_thread_condattr_getcontrolsize
px5_thread_condattr_getname

px5_thread_condattr_setcontroladdr
px5_thread_condattr_setname

**Denotes APIs available from ISRs*



Message Queues

The POSIX message queues provide inter-thread communication. Messages passed between threads via message queues are passed by value, meaning messages are copied into the message queue when sending. Conversely, messages are copied from the message queue when received. The size of a message sent has an upper bound defined when the message queue was created (opened). The actual size of the message sent can be any size less than or equal to the maximum size—including a size of zero. When a message is received, the actual size of the message is provided as part of the receive API.

Messages also have priorities, ranging from 0 (lowest priority) through `_SC_MQ_PRIO_MAX` (highest priority, default 31). Messages are placed in the queue in priority order. Messages of the same priority are placed in the queue in FIFO order.

Threads can suspend either trying to receive from an empty queue or sending to a full queue, determined by a queue attribute. If the queue attribute `O_NONBLOCK` is present, no blocking is allowed on the message queue. This attribute must be set via the `mq_setattr` API call, i.e., it is not the default upon queue creation. Threads that do suspend on a message queue do so in FIFO order.

The following are the message queue APIs supported by the PX5 RTOS:

mq_close
*mq_getattr**
mq_open
mq_receive
*mq_send**
*mq_setattr**
mq_timedreceive
mq_timedsend
px5_mq_extendattr_destroy
px5_mq_extendattr_getcontroladdr
px5_mq_extendattr_getcontrolsize
px5_mq_extendattr_getqueueaddr
px5_mq_extendattr_getqueuesize


```

px5_mq_extendattr_init
px5_mq_extendattr_setcontroladdr
px5_mq_extendattr_setqueueaddr
px5_mq_information_get*
px5_mq_send_notify

```

*Denotes APIs available from ISRs



Mutexes

The POSIX pthreads mutex is a thread synchronization object typically used for mutual exclusion—often to protect a shared data structure from concurrent access by multiple threads. If a thread attempts to lock a mutex that is already owned, it suspends on the mutex, waiting for it to become available. When the owning thread unlocks the mutex, the thread with the highest priority waiting to lock the mutex is given the mutex and resumed. Both *priority inheritance* and *recursive* mutexes are supported; however, neither is the default.

Priority inheritance is a technique to help make priority inversion more deterministic. *Priority inversion* is when a lower-priority thread owns a mutex that a higher-priority thread is waiting for. This is an acceptable situation in many designs as long as the amount of time the lower-priority thread holds the mutex during priority inversion does not affect the real-time requirements of the system. However, if one or more threads have a priority between the lower-priority thread and the waiting higher-priority thread, the lower-priority thread may be preempted by the higher-priority thread, and the time in priority inversion may become undeterministic. This is where priority inheritance can help. When priority inheritance is enabled, the priority of the thread holding the mutex is automatically elevated to that of the highest-priority waiting thread on the mutex. When the thread releases the mutex, its original priority is restored. Priority inheritance is specified when the mutex is created with the protocol attribute specified as *PTHREAD_PRIO_INHERIT*.

Recursive mutexes allow the same owner to obtain the mutex multiple times (a nested call to lock the mutex). The mutex is not released until the same number of unlock calls have been made. Recursive mutex support is specified when the mutex is created with the *type* attribute set to *PTHREAD_MUTEX_RECURSIVE*.

When using mutexes, it's essential to avoid deadlock situations (each thread is waiting for a mutex owned by the other thread). Allowing each thread to lock only one mutex at a time can avert deadlocks. If the same thread must lock multiple mutexes, deadlock can be avoided if the mutexes are locked in the same order by

all threads. Finally, threads must not exit with a locked mutex. Doing so will permanently lock the mutex, and all threads attempting to lock it will wait indefinitely.

The following are the mutex APIs supported by the PX5 RTOS:

```
pthread_mutex_destroy
pthread_mutex_init
pthread_mutex_lock
pthread_mutex_trylock
pthread_mutex_unlock
pthread_mutexattr_destroy
pthread_mutexattr_getpshared
pthread_mutexattr_init
pthread_mutexattr_setprotocol
pthread_mutexattr_setpshared
pthread_mutexattr_settype
px5_pthread_mutex_information_get*
px5_pthread_mutexattr_getcontroladdr
px5_pthread_mutexattr_getcontrolsize
px5_pthread_mutexattr_getname
px5_pthread_mutexattr_setcontroladdr
px5_pthread_mutexattr_setname
```

**Denotes APIs available from ISRs*

Semaphores

POSIX semaphores are synchronization objects not part of the pthreads API per se but commonly found with pthread implementations. A semaphore is effectively a counter. Non-zero values indicate availability, while zero indicates the semaphore is unavailable. Threads may block when attempting to get a semaphore with a value of zero. When another thread releases the semaphore, the highest priority thread waiting on the semaphore is resumed.



A semaphore can be used in a producer-consumer fashion to synchronize thread execution. Semaphores can also be used for mutual exclusion. However, mutexes are generally a more robust object for mutual exclusion. (Semaphores don't have the concept of ownership and don't provide nesting or priority inheritance.)

The following are the semaphore APIs supported by the PX5 RTOS:

```

sem_destroy
sem_init
sem_post*
sem_timedwait
sem_trywait
sem_wait
px5_sem_extend_init
px5_sem_information_get*
px5_sem_post_notify
px5_sem_tickwait
px5_semattr_destroy
px5_semattr_getcontroladdr
px5_semattr_getcontrolsize
px5_semattr_getname
px5_semattr_init
px5_semattr_setcontroladdr
px5_semattr_setname

```

**Denotes APIs available from ISRs*

Signals

POSIX signals notify a thread of a system event. The notification can be done synchronously or asynchronously as determined by the receiving thread's signal mask setting. If a thread has a signal masked, the signal processing for that specific signal is synchronous and accomplished via the various signal wait API calls. However, if the thread has a signal unmasked, the signal processing for that signal is asynchronous, i.e., it happens immediately when the signal is raised.

For asynchronous signal processing, the application must have previously registered a signal handler for the signal via the *sigaction* API. This signal handler will execute immediately in the context of the selected thread when the corresponding signal is raised.

A single-bit position of a 32-bit word represents each signal. Multiple signals can be masked or unmasked or waited for simultaneously. Raising a signal via the *pthread_kill* API uses the signal's numeric value and the responsible thread's handle.

The following are the signal APIs supported by the PX5 RTOS:

```

pthread_kill
pthread_sigmask

```

sigaction
sigaddset
sigdelset
sigemptyset
sigfillset
sigismember
sigpending
sigtimedwait
sigwait
sigwaitinfo

Fastqueues

The PX5 RTOS provides Fastqueue thread communication objects, which are not part of the pthreads API but are useful in embedded real-time programming. Fastqueues have less overhead (are faster) than the standard POSIX message queue, since priority messages and variable length messages are not supported. Each Fastqueue can hold one or more fixed-sized messages. (Message sizes must be evenly divisible by the size of the *u_long* data type.) Threads may be suspended on queue full or queue empty requests. Suspended threads are resumed in FIFO order.



The following are the Fastqueue APIs supported by the PX5 RTOS:

px5_thread_fastqueue_create
px5_thread_fastqueue_destroy
*px5_thread_fastqueue_information_get**
px5_thread_fastqueue_receive
px5_thread_fastqueue_send
px5_thread_fastqueue_send_notify
px5_thread_fastqueue_timedreceive
px5_thread_fastqueue_timedsend
*px5_thread_fastqueue_tryreceive**
*px5_thread_fastqueue_trysend**
px5_thread_fastqueueattr_getcontroladdr
px5_thread_fastqueueattr_getcontrolsize
px5_thread_fastqueueattr_getname
px5_thread_fastqueueattr_getqueueaddr
px5_thread_fastqueueattr_getqueuesize
px5_thread_fastqueueattr_init
px5_thread_fastqueueattr_setcontroladdr
px5_thread_fastqueueattr_setname

px5_thread_fastqueueattr_setqueueaddr

**Denotes APIs available from ISRs*

Event Flags

The PX5 RTOS provides thread synchronization event flags objects, which are not part of the pthreads API but are useful in embedded real-time programming. Each event flag is represented by a single bit in a 32-bit word (maximum of 32 events per event flags group). Threads may wait for any one of the events they specify. Alternatively, threads may wait for the reception of all the specified events. When a thread sets an event, all threads whose event request is satisfied are resumed. Events that satisfy a suspended thread's request are automatically cleared.

The following are the event flags APIs supported by the PX5 RTOS:

*px5_thread_event_flags_clear**
*px5_thread_event_flags_clearspecific**
px5_thread_event_flags_create
px5_thread_event_flags_destroy
*px5_thread_event_flags_information_get**
*px5_thread_event_flags_set**
px5_thread_event_flags_set_notify
px5_thread_event_flags_timedwait
*px5_thread_event_flags_trywait**
px5_thread_event_flags_wait
px5_thread_event_flagsattr_getautoconsumemask
px5_thread_event_flagsattr_getcontroladdr
px5_thread_event_flagsattr_getcontrolsiz
px5_thread_event_flagsattr_getname
px5_thread_event_flagsattr_init
px5_thread_event_flagsattr_setautoconsumemask
px5_thread_event_flagsattr_setcontroladdr
px5_thread_event_flagsattr_setname

**Denotes APIs available from ISRs*



Memory Pools

The PX5 RTOS provides dynamic, variable-length memory allocation via memory pool objects, which are not part of the pthreads API but are often necessary for embedded real-time programming. Memory is allocated on a first-fit basis, i.e., the first memory encountered in the search that satisfies the request is used. The calling thread may be suspended if insufficient memory is available in the memory pool (depending on the exact API used). When memory becomes available, all suspended threads with a chance of a successful allocation are resumed.

The following are the memory pool APIs supported by the PX5 RTOS:

```

px5_thread_memorypool_allocate
px5_thread_memorypool_create
px5_thread_memorypool_destroy
px5_thread_memorypool_free
px5_thread_memorypool_information_get*
px5_thread_memorypool_timedallocate
px5_thread_memorypool_tryallocate
px5_thread_memorypoolattr_destroy
px5_thread_memorypoolattr_getcontroladdr
px5_thread_memorypoolattr_getcontrolsize
px5_thread_memorypoolattr_getname
px5_thread_memorypoolattr_init
px5_thread_memorypoolattr_setcontroladdr
px5_thread_memorypoolattr_setname

```

**Denotes APIs available from ISRs*



Partition Memory Pools

The PX5 RTOS provides fixed-length memory allocation via partition memory pool objects, which are not part of the pthreads API but are often necessary for embedded real-time programming. Partition memory is allocated as fixed-sized blocks defined when the partition memory pool is created. If an allocation request is made when no memory partitions are available in the pool, the calling thread may suspend (depending on the exact API used). When a memory partition becomes available, the first suspended thread is resumed.

The following are the partition memory pool APIs supported by the PX5 RTOS:

```

px5_thread_partitionpool_allocate
px5_thread_partitionpool_create
px5_thread_partitionpool_destroy
px5_thread_partitionpool_free*
px5_thread_partitionpool_information_get*
px5_thread_partitionpool_timedallocate
px5_thread_partitionpool_tryallocate*
px5_thread_partitionpoolattr_destroy
px5_thread_partitionpoolattr_getcontroladdr
px5_thread_partitionpoolattr_getcontrolsize
px5_thread_partitionpoolattr_getname
px5_thread_partitionpoolattr_init
px5_thread_partitionpoolattr_setcontroladdr
px5_thread_partitionpoolattr_setname

```

**Denotes APIs available from ISRs*

Ticktimer Services



The PX5 RTOS provides ticktimer services. (The tick interval is usually 1 ms but is entirely under application control.) Ticktimer services include retrieval of the current number of timer ticks, thread sleeping for a specified number of timer ticks, tick-based application one-shot, and periodic timers.

The following are the ticktimer APIs supported by the PX5 RTOS:

```

px5_thread_ticktimer_create
px5_thread_ticktimer_destroy
px5_thread_ticktimer_information_get*
px5_thread_ticktimer_start
px5_thread_ticktimer_stop
px5_thread_ticktimer_update
px5_thread_ticktimerattr_destroy
px5_thread_ticktimerattr_getcontroladdr
px5_thread_ticktimerattr_getcontrolsize
px5_thread_ticktimerattr_getname
px5_thread_ticktimerattr_init
px5_thread_ticktimerattr_setcontroladdr
px5_thread_ticktimerattr_setname

```

**Denotes APIs available from ISRs*

Chapter 7: PX5 RTOS API definitions

This chapter describes the PX5 RTOS APIs. Each API definition includes the API's C prototype, a description including the various real-time scenarios that can occur as a result of the API, where the API can be called, a definition of each parameter, return value(s), and a small C code example.



All APIs starting with "*px5_*" are PX5 RTOS pthreads+ extensions. These APIs are PX5-specific and are not generally available on non-PX5 platforms.

clock_getres

C Prototype:

```
#include <pthread.h>

int clock_getres(clockid_t clock_id, struct timespec *resolution);
```

Description:

This service returns the current time resolution in *resolution*, which is effectively the underlying ticktimer periodic interrupt frequency.

API Parameters:

clock_id	This parameter must be CLOCK_REALTIME since it is the only supported clock.
resolution	Destination for the clock resolution.

Return Codes:

PX5_SUCCESS (0)	Successful retrieval of clock resolution.
PX5_ERROR (-1)	Error attempting to get clock resolution. Please use <i>errno</i> to retrieve the exact error:
EINVAL	Invalid clock ID.
EFAULT	Invalid resolution pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location of where this API is called, there should be at least $(44 + \text{PX5_BINDING_STACK_FRAME_SIZE})$ bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

clock_gettime, clock_settime, time

Small Example:

```
#include <pthread.h>

Int          status;
struct timespec my_resolution;

/* Pickup the current clock resolution. */
status = clock_getres(CLOCK_REALTIME, &my_resolution);

/* If status contains PX5_SUCCESS (0), the clock resolution is
   in "my_resolution". */
```

clock_gettime

C Prototype:

```
#include <pthread.h>

int clock_gettime(clockid_t clock_id, struct timespec *current_time);
```

Description:

This service returns the current time in the destination specified by *current_time*.

API Parameters:

<code>clock_id</code>	This parameter must be <code>CLOCK_REALTIME</code> since it is the only supported clock.
<code>current_time</code>	Destination for the current time.

Return Codes:

PX5_SUCCESS (0)	Successful get time.
PX5_ERROR (-1)	Error attempting to get current time. Please use <i>errno</i> to retrieve the exact error:
EINVAL	Invalid clock ID.
EFAULT	Invalid current time pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location of where this API is called, there should be at least **(44 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

clock_getres, clock_settime, time

Small Example:

```
#include <pthread.h>

Int          status;
struct timespec my_current_time;

/* Pickup the current time. */
status = clock_gettime(CLOCK_REALTIME, &my_current_time);

/* If status contains PX5_SUCCESS (0), the current time is
   in "my_current_time". */
```

clock_gettime

C Prototype:

```
#include <pthread.h>

int  clock_gettime(clockid_t clock_id, struct timespec *new_time);
```

Description:

This service sets the current time to the value specified by *new_time*.

API Parameters:

<code>clock_id</code>	This parameter must be <code>CLOCK_REALTIME</code> since it is the only supported clock.
<code>new_time</code>	Developer supplied time value the system is set to.

Return Codes:

PX5_SUCCESS (0)	Successful set time.
PX5_ERROR (-1)	Error attempting to set time. Please use <i>errno</i> to retrieve the exact error:
EINVAL	Invalid clock ID or invalid time specification.
EFAULT	Invalid new time pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location of where this API is called, there should be at least **(44 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

clock_getres, clock_gettime, time

Small Example:

```
#include <pthread.h>

Int          status;
struct timespec my_new_time;

/* Get the current time. */
clock_gettime(CLOCK_REALTIME, &my_new_time);

/* Move one second forward. */
my_new_time.tv_sec++;

/* Set the new time. */
status = clock_settime(CLOCK_REALTIME, &my_new_time);

/* If status contains PX5_SUCCESS (0), the new time is one second
   later. */
```

mq_close

C Prototype:

```
#include <mqueue.h>

int  mq_close(mqd_t message_queue);
```

Description:

This service closes and destroys the specified message queue. An error is returned if any threads are suspended on the message queue.



To avoid unpredictable behavior, the application must ensure no further access to the message queue after it is closed.

API Parameters:

message_queue	Specifies the queue to close/destroy.
---------------	---------------------------------------

Return Codes:

PX5_SUCCESS (0)	Successful message queue close.
PX5_ERROR (-1)	Error attempting to close the message queue. Please use <i>errno</i> to retrieve the exact error:
EBADF	Invalid memory queue handle.
EBUSY	Specified message queue has threads suspended on it.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(76 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

mq_open, mq_receive, mq_send

Small Example:

```
#include <mqueue.h>

int          status;
mqd_t        my_queue_handle;

/* Close the previously opened queue "my_queue_handle". */
status = mq_close(my_queue_handle);

/* If status contains PX5_SUCCESS (0), the message queue
   "my_queue_handle" is closed. */
```


mq_getattr

C Prototype:

```
#include <mqueue.h>

int mq_getattr(mqd_t message_queue, struct mq_attr* queue_attributes);
```

Description:

This service retrieves the current attributes of the specified message queue.

API Parameters:

<code>message_queue</code>	Specifies the queue to retrieve attributes from.
<code>queue_attributes</code>	Specifies the destination for the queue attributes information.

Return Codes:

PX5_SUCCESS (0)	Successful message queue attributes retrieval.
PX5_ERROR (-1)	Error attempting to get the queue attributes. Please use <i>errno</i> to retrieve the exact error:
EBADF	Invalid memory queue handle or invalid attributes destination pointer.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(52 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable from the thread context and interrupt handlers (ISRs).

See Also:

mq_open, *mq_setattr*

Small Example:

```
#include <mqueue.h>

int          status;
struct mq_attr my_queue_attributes;
mqd_t        my_queue_handle;

/* Get the attributes of the previously opened queue
   "my_queue_handle" and return them in "my_queue_attributes". */
status = mq_getattr(my_queue_handle, &my_queue_attributes);

/* If status contains PX5_SUCCESS (0), "my_queue_attributes"
   contains the attributes of message queue "my_queue_handle". */
```

mq_open

C Prototype:

```
#include <mqueue.h>

mqd_t  mq_open(const char * queue_name, int operation, mode_t mode,
               struct mq_attr * queue_attributes);
```

Description:



If successful, this service opens (creates) the message queue specified and returns the message queue handle.

API Parameters:

queue_name
operation

Name of the queue to open/create.
Specifies the queue options. The supported options are:

O_CREAT
O_RDWR
O_NONBLOCK

O_CREAT and *O_RDWR* should be specified since the PX5 RTOS only supports creating queues *O_CREATE* with read/write capabilities *O_RDWR*. The *O_NONBLOCK* option that disables threads from suspending on the queue is optional.

mode
queue_attributes

Not currently used.
Attributes that specify the dimensions of the message queue, as defined by these structure members:

mq_maxmsg
mq_msgsize

Where *mq_maxmsg* defines the total number of messages the queue can hold. The maximum size of each message (in bytes) is

determined by *mq_msgsize*. Note the size and priority of each message must be stored along with the message content. In addition, there is one pointer type required for each message. On most architectures, this amounts to 12 bytes of additional overhead per message.

Return Codes:

queue handle	Positive value represents the successfully opened (created) queue handle.
PX5_ERROR (-1)	Error attempting to open/create the queue. Please use <i>errno</i> to retrieve the exact error:
ENOSPC	Not enough memory to create the specified queue.
EINVAL	Invalid operation or invalid attributes pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(180 + PX5_BINDING_STACK_FRAME_SIZE)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

mq_close, mq_getattr, mq_setattr, px5_mq_extend_open

Small Example:

```
#include <mqueue.h>

int          status;
struct mq_attr my_queue_attributes;
mqd_t        my_queue_handle;

/* Set up the attributes for 100 total messages, where each
   message is a maximum of 16 bytes. */
my_queue_attributes.mq_maxmsg = 100;
my_queue_attributes.mq_msgsize = 16;

/* Open (create) the queue "my_queue". */
my_queue_handle = mq_open("my_queue", (O_CREAT | O_RDWR), 0,
                           &my_queue_attributes);

/* If "my_queue_handle" is positive, the queue was successfully
   created. */
```

mq_receive

C Prototype:

```
#include <mqueue.h>

ssize_t mq_receive(mqd_t message_queue, char * message,
                  size_t message_size, unsigned int * message_priority);
```

Description:

This service receives the oldest message with the highest priority from the specified message queue. If the queue is empty and the *O_NONBLOCK* attribute is not set, the calling thread will suspend waiting for a message to arrive. If multiple threads are waiting on an empty queue, the first thread waiting is given the message.



This API is a cancellation point, meaning that if a cancellation is pending, it will be detected and executed by this API.

API Parameters:

message_queue	Message queue to get message from.
message	Pointer to the destination for the message.
message_size	Maximum size for the message. This must be equal to or greater than the queue's maximum message size.
message_priority	Priority of the message received.

Return Codes:

message size	A non-negative value represents the actual size of the successfully received message (messages of zero size are allowed).
PX5_ERROR (-1)	Error attempting to receive a message from the queue. Please use <i>errno</i> to retrieve the exact error:
EBADF	Invalid message queue handle or invalid message destination pointer.

EMSGSIZE	Invalid message size.
EAGAIN	Message queue is empty and <i>O_NONBLOCK</i> was specified to turn off thread suspension to wait for a message.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. No preemption is possible with this service if a message is in the message queue.



SUSPENSION. The calling thread is suspended until a message arrives in the queue.



PREEMPTION. If a higher-priority thread is waiting to place a message on the queue, it is resumed, and preemption will occur.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(188 + PX5_BINDING_STACK_FRAME_SIZE)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

mq_open, mq_send, mq_timedreceive

Small Example:

```
#include <mqueue.h>

ssize_t      received_size;
char         my_message[16];
unsigned int  my_priority;
mqd_t        my_queue_handle;

/* Receive message from "my_queue". */
received_size = mq_receive(my_queue_handle, &my_message[0],
                           sizeof(my_message), &my_priority);

/* If "received_size" is positive, the message was successfully
   received. */
```


mq_send

C Prototype:

```
#include <mqueue.h>

int  mq_send(mqd_t message_queue, char * message,
             size_t message_size, unsigned int message_priority);
```

Description:

This service sends the message to the specified message queue. If the queue is full and the O_NONBLOCK attribute is not set, the calling thread will suspend waiting for room in the queue.



This API is a cancellation point, meaning that if a cancellation is pending, it will be detected and executed by this API.

API Parameters:

message_queue	Message queue to send message to.
message	Pointer to the source of the message.
message_size	Size of the message. This must be equal to or less than the queue's maximum message size. Messages of zero size are allowed.
message_priority	Priority of the message to send.

Return Codes:

PX5_SUCCESS (0)	Successful send to the message queue.
PX5_ERROR (-1)	Error attempting to send a message to the queue. Please use <i>errno</i> to retrieve the exact error:
EBADF	Invalid message queue handle or invalid message pointer.
EMSGSIZE	Invalid message size.
EAGAIN	Message queue is full and O_NONBLOCK was specified to

EACCES	turn off thread suspension to wait for room in the queue. Invalid call from ISR or non-thread.
---------------	---

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. No preemption is possible with this service if there is room in the message queue.



SUSPENSION. The calling thread is suspended if the specified queue is full.



PREEMPTION. If a higher-priority thread is waiting for a message on an empty queue, it is resumed, and preemption will occur.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(172 + `PX5_BINDING_STACK_FRAME_SIZE`)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context and interrupt handlers (ISRs).



If this API is called from an ISR, the application must ensure that the queue is not full or suspension is not enabled. An easy way to prevent queue full suspension is to set the queue attribute `O_NONBLOCK` before this API is called. Otherwise, unpredictable behavior is possible.

See Also:

mq_open, mq_receive, mq_setattr, mq_timedreceive, mq_timedsend

Small Example:

```
#include <mqueue.h>

int          status;
char         my_message[16];
mqd_t        my_queue_handle;

/* Build the message. */
my_message[0] = 'm';
my_message[1] = 'y';
my_message[2] = ' ';
my_message[3] = 'm';
my_message[4] = 'e';
my_message[5] = 's';
my_message[6] = 's';
my_message[7] = 'a';
my_message[8] = 'g';
my_message[9] = 'e';

/* Send a priority 0 message to "my_queue". */
status = mq_send(my_queue_handle, &my_message[0], 10, 0);

/* If status is PX5_SUCCESS (0), the message was successfully
   sent. */
```

mq_setattr

C Prototype:

```
#include <mqueue.h>

int  mq_setattr(mqd_t message_queue, struct mq_attr * new_attributes,
                struct mq_attr * previous_attributes);
```

Description:

This service sets the attributes of the specified message queue. If the *previous_attributes* pointer is non-NULL, the attributes prior to this service are returned.

API Parameters:

message_queue	Specifies the queue to retrieve attributes from.
new_attributes	Specifies the new queue attributes to set, either <i>O_NONBLOCK</i> to turn off thread suspension on the queue or zero to enable thread suspension.
previous_attributes	If non-NULL, specifies the destination for the prior queue attribute information.

Return Codes:

PX5_SUCCESS (0)	Successful message queue attributes set.
PX5_ERROR (-1)	Error attempting to set the queue attributes. Please use <i>errno</i> to retrieve the exact error:
EBADF	Invalid memory queue handle or invalid attributes destination pointer.
EINVAL	Invalid attribute specification – must be either <i>O_NONBLOCK</i> or zero.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:

From the location where this API is called, there should be at least **(52 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable from the thread context and interrupt handlers (ISRs).

See Also:

mq_getattr, mq_open

Small Example:

```
#include <mqueue.h>

int          status;
struct mq_attr my_new_attributes;
struct mq_attr my_previous_attributes;
mqd_t        my_queue_handle;

/* Turn off thread suspension on "my_queue_handle" and return the
   previous attributes in "my_previous_attributes". */
my_new_attributes.mq_flags = O_NONBLOCK;
status = mq_setattr(my_queue_handle, &my_new_attributes,
                    &my_previous_attributes);

/* If status contains PX5_SUCCESS (0), the queue won't allow thread
   suspension and "my_previous_attributes" contains the attributes
   of message queue "my_queue_handle" before this call. */
```

mq_timedreceive

C Prototype:

```
#include <pthread.h>
#include <mqueue.h>

int  mq_timedreceive(mqd_t message_queue, char * message,
                    size_t message_size, unsigned int * message_priority,
                    const struct timespec * absolute_timeout);
```

Description:

This service receives the highest priority message from the specified message queue. If the queue is empty and the O_NONBLOCK attribute is not set, the calling thread will suspend waiting for a message to arrive. If the timeout is exceeded before a message arrives, the thread is resumed with an error.



This API is a cancellation point, meaning that if a cancellation is pending, it will be detected and executed by this API.

API Parameters:

message_queue	Message queue to get message from.
message	Pointer to the destination for the message.
message_size	Maximum size for the message. This must be equal to or greater than the queue's maximum message size.
message_priority	Priority of the message received.
absolute_timeout	Pointer to absolute time to wait for the message to arrive.

Return Codes:

message_size	A non-negative value represents the actual size of the successfully received message (messages of zero size are allowed).
PX5_ERROR (-1)	Error attempting to receive a message from the queue. Please use <i>errno</i> to retrieve the exact error:

EBADF	Invalid message queue handle or invalid message destination pointer.
EMSGSIZE	Invalid message size.
EAGAIN	Message queue is empty and <i>O_NONBLOCK</i> was specified to turn off thread suspension to wait for a message.
ETIMEDOUT	Timeout on thread suspension, waiting for a message.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. No preemption is possible with this service if a message is in the message queue.



SUSPENSION. The calling thread is suspended until a message arrives in the queue.



PREEMPTION. If a higher-priority thread is waiting to place a message on the queue, it is resumed, and preemption will occur.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(212 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

mq_open, mq_send, mq_receive, mq_timedreceive

Small Example:

```
#include <mqueue.h>

ssize_t      received_size;
char         my_message[16];
unsigned int  my_priority;
mqd_t        my_queue_handle;
struct timespec my_absolute_timeout;

/* Set up a timeout for one second in the future. */
clock_gettime(CLOCK_REALTIME, &my_absolute_timeout);
my_absolute_timeout.tv_sec += 1;

/* Receive message from "my_queue" but only wait for 1 second. */
received_size = mq_timedreceive(my_queue_handle, &my_message[0],
                                sizeof(my_message), &my_priority,
                                &my_absolute_timeout);

/* If "received_size" is positive, the message was successfully
   received. */
```

mq_timedsend

C Prototype:

```
#include <mqueue.h>

int  mq_timedsend(mqd_t message_queue, char * message,
                  size_t message_size, unsigned int message_priority,
                  const struct timespec * absolute_timeout);
```

Description:

This service sends the message to the specified message queue. If the queue is full and the *O_NONBLOCK* attribute is not set, the calling thread will suspend waiting for room in the queue. If the timeout is exceeded before room for the message is available in the queue, the thread is resumed with an error.



This API is a cancellation point, meaning that if a cancellation is pending, it will be detected and executed by this API.

API Parameters:

message_queue	Message queue to send message to.
message	Pointer to the source of the message.
message_size	Size of the message. This must be equal to or less than the queue's maximum message size. Messages with zero size are allowed.
message_priority	Priority of the message to send.
absolute_timeout	Pointer to absolute time to wait for room for the message in the queue.

Return Codes:

PX5_SUCCESS (0)	Successful send to the message queue.
PX5_ERROR (-1)	Error attempting to send a message to the queue. Please use <i>errno</i> to retrieve the exact error:

EBADF	Invalid message queue handle or invalid message pointer.
EMSGSIZE	Invalid message size.
EAGAIN	Message queue is full and O_NONBLOCK was specified to turn off thread suspension to wait for room in the queue.
ETIMEDOUT	Timeout on thread suspension waiting for room for the message in the queue.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. No preemption is possible with this service if there is room in the message queue.



SUSPENSION. The calling thread is suspended if the specified queue is full.



PREEMPTION. If a higher-priority thread is waiting for a message on an empty queue, it is resumed, and preemption will occur.

Stack Usage Estimate:



From the location where this API is called, there should be at least $(196 + \text{PX5_BINDING_STACK_FRAME_SIZE})$ bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

mq_open, mq_receive, mq_send, mq_timedreceive

Small Example:

```
#include <mqueue.h>

ssize_t      received_size;
char         my_message[16];
unsigned int  my_priority;
mqd_t        my_queue_handle;
struct timespec my_absolute_timeout;

/* Set up a timeout for one second in the future. */
clock_gettime(CLOCK_REALTIME, &my_absolute_timeout);
my_absolute_timeout.tv_sec += 1;

/* Send message to "my_queue" but only wait for 1 second. */
status = mq_timedsend(my_queue_handle, &my_message[0],
                      sizeof(my_message), my_priority,
                      &my_absolute_timeout);

/* If "status" is PX5_SUCCESS, the message was successfully
   sent. */
```

nanosleep

C Prototype:

```
#include <pthread.h>

int  nanosleep(struct timespec *request_time,
               struct timespec *remaining_time);
```

Description:

This service causes the calling thread to suspend for the time specified in *request_time*. If an unmasked signal is sent to the thread while sleeping, the thread is resumed, and the remaining time is returned in *remaining_time*.



Nanosleep requests are rounded up to the next time evenly divisible by the timer resolution.



This API is a cancellation point, meaning that if a cancellation is pending, it will be detected and executed by this API.

API Parameters:

<code>request_time</code>	The amount of time to sleep.
<code>remaining_time</code>	If non-NULL, the destination for the remaining time to sleep if nanosleep was interrupted by a signal.

Return Codes:

PX5_SUCCESS (0)	Successful sleep.
PX5_ERROR (-1)	Error attempting to sleep. Please use <i>errno</i> to retrieve the exact error:
EINVAL	Invalid request pointer or invalid request time.

EINTR	Nanosleep was interrupted by a signal. The time left to sleep is returned in <i>remaining_time</i> .
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



SUSPENSION. The calling thread is suspended until the specified time has lapsed or another thread sends a signal to this thread.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(104 + PX5_BINDING_STACK_FRAME_SIZE)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

px5_pthread_tick_sleep, sleep, usleep

Small Example:

```
#include <pthread.h>

int          status;
struct timespec my_sleep_time;
struct timespec my_remaining_time;

/* Sleep for 1 second. */
my_sleep_time.tv_sec = 1;
my_sleep_time.tv_nsec = 0;

status = nanosleep(&my_sleep_time, &my_remaining_time);

/* If status contains PX5_SUCCESS (0), the calling thread slept for
   1 second. */
```

pthread_attr_destroy

C Prototype:

```
#include <pthread.h>

int pthread_attr_destroy(pthread_attr_t *attributes);
```

Description:

This service destroys a previously initialized thread creation attributes structure. Once destroyed, the attributes structure can no longer be used.



The destruction of thread attributes doesn't affect threads previously created with the attributes. The only effect is that this attributes structure cannot be used to create additional threads.

API Parameters:

<code>attributes</code>	Pointer to a previously initialized attributes structure.
-------------------------	---

Return Codes:

PX5_SUCCESS (0)	Successful attributes structure destruction.
EINVAL	Attributes structure pointer is invalid.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(36 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*pthread_attr_**, *pthread_create*

Small Example:

```
#include <pthread.h>

Int          status;
pthread_attr_t my_thread_attributes;

/* Destroy the previously initialized attributes. */
status = pthread_attr_destroy(&my_thread_attributes);

/* If status contains PX5_SUCCESS, the attributes
   have been destroyed. */
```

pthread_attr_getdetachstate

C Prototype:

```
#include <pthread.h>

int pthread_attr_getdetachstate(pthread_attr_t *attributes,
                                int * detach_state);
```

Description:

This service returns the detach state stored in the thread attributes structure. Valid detach states are *PTHREAD_CREATE_JOINABLE* (default) and *PTHREAD_CREATE_DETACHED*.

API Parameters:

<code>attributes</code>	Pointer to a previously initialized attributes structure.
<code>detach_state</code>	Pointer to the destination of where to return the detach state setting in this attributes structure.

Return Codes:

PX5_SUCCESS (0)	Successful retrieval of the attributes detach state.
EINVAL	Attributes structure pointer or detach state destination pointer is invalid.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(44 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*pthread_attr_**, *pthread_create*

Small Example:

```
#include <pthread.h>

pthread_attr_t    my_thread_attributes;
int               detach_state;
int               status;

/* Get the detach state in the previously initialized
   attributes. */
status = pthread_attr_getdetachstate(&my_thread_attributes,
                                     &detach_state);

/* If status contains PX5_SUCCESS, the detach state value in this
   attribute structure is contained in "detach_state". */
```

pthread_attr_getstackaddr

C Prototype:

```
#include <pthread.h>

int pthread_attr_getstackaddr(pthread_attr_t *attributes,
                             void ** stack_address);
```

Description:

This service returns the stack address stored in the thread attributes structure. By default, this value is NULL unless specified by the application via a call to *pthread_attr_setstackaddr*.

API Parameters:

<code>attributes</code>	Pointer to a previously initialized attributes structure.
<code>stack_address</code>	Pointer to the destination of where to return the stack address of this attributes structure.

Return Codes:

PX5_SUCCESS (0)	Successful retrieval of the stack address.
EINVAL	Attributes structure pointer or stack address destination pointer is invalid.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(44 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*pthread_attr_**, *pthread_create*

Small Example:

```
#include <pthread.h>

pthread_attr_t    my_thread_attributes;
void *           stack_address;
int              status;

/* Get the stack address in the previously initialized
   attributes. */
status = pthread_attr_getstackaddr(&my_thread_attributes,
                                   &stack_address);

/* If status contains PX5_SUCCESS, the stack address value in this
   attribute structure is contained in "stack_address". */
```

pthread_attr_getstacksize

C Prototype:

```
#include <pthread.h>

int  pthread_attr_getstacksize(pthread_attr_t *attributes,
                               size_t *      stack_size);
```

Description:

This service returns the current stack size stored in the thread attributes structure. By default, this value is *PX5_DEFAULT_STACK_SIZE* unless specified by the application via a call to *pthread_attr_setstacksize*.

API Parameters:

<code>attributes</code>	Pointer to a previously initialized attributes structure.
<code>stack_size</code>	Pointer to the destination of where to return the stack size of this attributes structure.

Return Codes:

PX5_SUCCESS (0)	Successful retrieval of the stack size.
EINVAL	Attributes structure pointer or stack size destination pointer is invalid.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(44 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*pthread_attr_**, *pthread_create*

Small Example:

```
#include <pthread.h>

pthread_attr_t    my_thread_attributes;
size_t           stack_size;
int               status;

/* Get the stack size in the previously initialized
   attributes. */
status = pthread_attr_getstacksize(&my_thread_attributes,
                                   &stack_size);

/* If status contains PX5_SUCCESS, the stack size value in this
   attribute structure is contained in "stack_size". */
```

pthread_attr_init

C Prototype:

```
#include <pthread.h>

int pthread_attr_init(pthread_attr_t *attributes);
```

Description:

This service initializes the attributes structure with default thread creation values. These defaults are as follows:

Attribute	Default Setting
Detach State	Joinable – <i>PTHREAD_CREATE_JOINABLE</i>
Stack Address	NULL – Allocate dynamically
Stack Size	<i>PX5_DEFAULT_STACK_SIZE</i>
Priority	<i>PX5_DEFAULT_PRIORITY</i>
Thread Control Address	NULL – Allocate dynamically
Thread Control Size	Size of the internal thread control structure
Time Slice	No time-slice (0)

Once attributes are initialized via this service, the default settings above can be overridden by *pthread_attr_set** API calls.



Note that the attributes are only relevant when supplied to the pthread_create API. After the thread is created, the attributes can be changed for the next thread creation without affecting previously created threads.

API Parameters:

<code>attributes</code>	Pointer to a previously initialized attributes structure.
-------------------------	---

Return Codes:

PX5_SUCCESS (0)	Successful attribute structure initialization.
EINVAL	Attributes structure pointer is invalid.

EACCES

Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:

From the location where this API is called, there should be at least **(36 + PX5_BINDING_STACK_FRAME_SIZE)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

pthread_attr_, pthread_create*

Small Example:

```
#include <pthread.h>

pthread_attr_t    my_thread_attributes;
int              status;

/* Initialize the thread creation attributes. */
status = pthread_attr_init(&my_thread_attributes);

/* If status contains PX5_SUCCESS, the "my_thread_attributes"
   structure has been initialized with default values. */
```

pthread_attr_setdetachstate

C Prototype:

```
#include <pthread.h>

int pthread_attr_setdetachstate(pthread_attr_t *attributes,
                               int new_detach_state);
```

Description:

This service sets the detach state specified by *new_detach_state* in the thread attributes structure. Valid detach states are *PTHREAD_CREATE_JOINABLE* and *PTHREAD_CREATE_DETACHED*.



Note that threads created with PTHREAD_CREATE_DETACHED cannot be joined or canceled. When they exit, all their resources are released.

API Parameters:

<code>attributes</code>	Pointer to a previously initialized attributes structure.
<code>detach_state</code>	The new detach state setting for this attributes structure.

Return Codes:

PX5_SUCCESS (0)	Successful detach state set.
EINVAL	Attributes structure pointer or detach state specification.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(44 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*pthread_attr_**, *pthread_create*

Small Example:

```
#include <pthread.h>

pthread_attr_t    my_thread_attributes;
int               status;

/* Set the detach state in the previously initialized
   attributes such that threads are created as detached. */
status = pthread_attr_setdetachstate(&my_thread_attributes,
                                     PTHREAD_CREATE_DETACHED);

/* If status contains PX5_SUCCESS, the detach state in this
   attributes structure is set to PTHREAD_CREATE_DETACHED. */
```

pthread_attr_setstackaddr

C Prototype:

```
#include <pthread.h>

int pthread_attr_setstackaddr(pthread_attr_t *attributes,
                             void *      stack_address);
```

Description:

This service sets the stack address to the value specified by *stack_address*. The stack address must be at least *PTHREAD_STACK_MIN* number of bytes.



Note that each thread created must have its own unique stack memory. Hence, the stack address supplied here is valid for only one pthread_create call.

API Parameters:

<code>attributes</code>	Pointer to a previously initialized attributes structure.
<code>stack_address</code>	Stack address to use for the next thread creation.

Return Codes:

PX5_SUCCESS (0)	Successful setting of the stack address.
EINVAL	Attributes structure pointer or stack address pointer is invalid.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(44 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*pthread_attr_**, *pthread_create*

Small Example:

```
#include <pthread.h>

pthread_attr_t    my_thread_attributes;
int              status;

/* Set the stack address to 0x30000 in the previously initialized
   attributes. */
status = pthread_attr_setstackaddr(&my_thread_attributes,
                                   (void *) 0x30000);

/* If status contains PX5_SUCCESS, the stack address in this
   attributes structure is set to 0x30000. */
```

pthread_attr_setstacksize

C Prototype:

```
#include <pthread.h>

int pthread_attr_setstacksize(pthread_attr_t *attributes,
                             size_t          stack_size);
```

Description:

This service sets the specified stack size in the thread attributes structure. The stack address must be at least *PTHREAD_STACK_MIN* number of bytes.

API Parameters:

<code>attributes</code>	Pointer to a previously initialized attributes structure.
<code>stack_size</code>	Stack size to set in the attributes structure.

Return Codes:

PX5_SUCCESS (0)	Successful setting of the stack size.
EINVAL	Attributes structure pointer or stack size is invalid.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(44 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*pthread_attr_**, *pthread_create*

Small Example:

```
#include <pthread.h>

pthread_attr_t    my_thread_attributes;
int               status;

/* Set the stack size in the previously initialized
   attributes. */
status = pthread_attr_setstacksize(&my_thread_attributes,
                                   1024);

/* If status contains PX5_SUCCESS, the stack size value of 1024
   is placed in this attributes structure. */
```

pthread_cancel

C Prototype:

```
#include <pthread.h>

int pthread_cancel(pthread_t thread_handle);
```

Description:

This service cancels the specified thread. If the specified thread has cancellation disabled or deferred, this service marks the thread for cancellation at a later point determined by the thread itself. Otherwise, if the specified thread has asynchronous cancellation enabled, it is immediately canceled (terminated) by this service.



By default, threads are created with deferred cancellation enabled. This can be changed dynamically via the `pthread_setcancelstate` and `pthread_setcanceltype` APIs.

API Parameters:

<code>thread_handle</code>	Handle of thread to cancel.
----------------------------	-----------------------------

Return Codes:

PX5_SUCCESS (0)	Successful thread cancellation.
ESRCH	Specified thread handle is invalid.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. No preemption takes place as a result of this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(116 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

pthread_cleanup_pop, pthread_cleanup_push, pthread_create, pthread_setcancelstate, pthread_setcanceltype, pthread_testcancel

Small Example:

```
#include <pthread.h>

pthread_t  my_cancel_thread;
int        status;

/* Cancel "my_cancel_thread". */
status = pthread_cancel(my_cancel_thread);

/* If "status" is PX5_SUCCESS (0), "my_cancel_thread" was
   canceled. */
```

pthread_cleanup_pop

C Prototype:

```
#include <pthread.h>

void pthread_cleanup_pop(int execute);
```

Description:

This service pops the most recently pushed cleanup handler from the thread's cleanup handler stack. If the specified *execute* parameter is non-zero and the cleanup handler function pointer is non-NULL, the cleanup handler is executed before returning to the caller.

API Parameters:

<i>execute</i>	If non-zero and cleanup handler function pointer is non-NULL, the cleanup handler is executed.
----------------	--

Return Codes:

none

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. No preemption takes place as a result of this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(48 + *PX5_BINDING_STACK_FRAME_SIZE* + *cleanup handler stack usage*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

pthread_cancel, pthread_cleanup_push, pthread_create, pthread_setcancelstate, pthread_setcanceltype, pthread_testcancel

Small Example:

```
#include <pthread.h>

/* Pop and execute the most recently pushed cleanup
   handler for the calling thread. */
pthread_cleanup_pop(1);

/* At this point the cleanup handler was popped and executed. */
```

pthread_cleanup_push

C Prototype:

```
#include <pthread.h>

void pthread_cleanup_push(void (*cleanup_handler)(void *),
                          void *argument);
```

Description:

This service pushes the specified *cleanup_handler* on the calling thread's cleanup handler stack.



Note the number of cleanup handlers a thread can push is determined by the `PX5_MAXIMUM_CLEANUP_HANDLERS` define, which by default is 3. This can be changed via user configuration. If the limit is reached, the push request is silently discarded.

API Parameters:

<code>cleanup_handler</code>	Cleanup handler function pointer to push.
<code>argument</code>	Argument pointer to supply to the cleanup handler if it is called later.

Return Codes:

none

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. No preemption takes place as a result of this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(8 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

pthread_cancel, *pthread_cleanup_pop*, *pthread_create*,
pthread_setcancelstate, *pthread_setcanceltype*, *pthread_testcancel*

Small Example:

```
#include <pthread.h>

void my_cleanup_handler(void *argument)
{
    /* Cleanup handler processing goes here!  */
}

/* Push "my_cleanup_handler" to the thread's cleanup handler
   stack. */
pthread_cleanup_push(my_cleanup_handler, NULL);

/* At this point, the cleanup handler was pushed. */
```

pthread_cond_broadcast

C Prototype:

```
#include <pthread.h>

int pthread_cond_broadcast(pthread_cond_t * condition_var_handle);
```

Description:

This service resumes all threads currently waiting on this condition variable.



Note that the calling thread must own the mutex associated with this condition variable prior to making this call. Unpredictable behavior can result if the caller of this service does not own the associated mutex.

API Parameters:

<code>condition_var_handle</code>	Handle of the condition variable to broadcast on.
-----------------------------------	---

Return Codes:

PX5_SUCCESS (0)	Successful condition variable broadcast.
EINVAL	Invalid condition variable handle pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. No preemption occurs if no other threads are waiting for the condition variable.



PREEMPTION. If a higher-priority thread was waiting on the condition variable, when it is resumed, preemption will occur.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(100 + PX5_BINDING_STACK_FRAME_SIZE)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

pthread_cond_, pthread_cond_init, pthread_cond_signal, pthread_cond_wait*

Small Example:

```
#include <pthread.h>

/* Condition variable handle. */
pthread_cond_t    my_condition_var_handle;
int               status;

/* Broadcast to the condition variable
   "my_condition_var_handle". */
status = pthread_cond_broadcast(&my_condition_var_handle);

/* If status is PX5_SUCCESS, the broadcast has resumed all waiting
   threads on this condition variable. */
```

pthread_cond_destroy

C Prototype:

```
#include <pthread.h>

int pthread_cond_destroy(pthread_cond_t * condition_var_handle);
```

Description:

This service destroys the previously created condition variable specified by *condition_var_handle*. If the condition variable is still in use by another thread, an error is returned.



To avoid unpredictable behavior, the application must ensure there is no further access to the condition variable after it is destroyed.

API Parameters:

<code>condition_var_handle</code>	Handle of the condition variable to destroy.
-----------------------------------	--

Return Codes:

PX5_SUCCESS (0)	Successful condition variable destroy.
EINVAL	Invalid condition variable handle.
EBUSY	Another thread is currently using the condition variable.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(60 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*pthread_cond_**, *pthread_cond_init*

Small Example:

```
#include <pthread.h>

/* Condition variable handle. */
pthread_cond_t    my_condition_var_handle;
int               status;

/* Destroy the condition variable referenced by
   "my_condition_var_handle". */
status = pthread_cond_destroy(&my_condition_var_handle);

/* If status is PX5_SUCCESS, the condition variable was
   destroyed. */
```

pthread_cond_init

C Prototype:

```
#include <pthread.h>

int pthread_cond_init(pthread_cond_t * condition_var_handle,
                     pthread_condattr_t * condition_var_attributes);
```

Description:



This service initializes (creates) a condition variable with the optional condition variable attributes. If successful, the condition variable handle is set up for further use by the application.

API Parameters:

<code>condition_var_handle</code>	Handle of the condition variable to initialize.
<code>condition_var_attributes</code>	Optional condition variable attributes. This value is NULL if no condition variable attributes are specified.

Return Codes:

PX5_SUCCESS (0)	Successful condition variable initialization.
EINVAL	Invalid condition variable handle pointer or condition variable attributes.
EBUSY	Condition variable is already created.
ENOMEM	Insufficient memory to create condition variable.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(124 + PX5_BINDING_STACK_FRAME_SIZE)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

pthread_condattr_, pthread_cond_destroy*

Small Example:

```
#include <pthread.h>

/* Condition variable handle. */
pthread_cond_t    my_condition_var_handle;
int               status;

/* Create the condition variable and set up
   "my_condition_var_handle". */
status = pthread_cond_init(&my_condition_var_handle, NULL);

/* If status is PX5_SUCCESS, the condition variable was created and
   the condition variable handle is ready to use. */
```

pthread_cond_signal

C Prototype:

```
#include <pthread.h>

int pthread_cond_signal(pthread_cond_t * condition_var_handle);
```

Description:

This service resumes the highest priority thread currently waiting on this condition variable.



Note that the calling thread must own the mutex associated with this condition variable prior to making this call. Unpredictable behavior can result if the caller of this service does not own the associated mutex.

API Parameters:

`condition_var_handle` Handle of the condition variable to signal.

Return Codes:

PX5_SUCCESS (0)	Successful condition variable signal.
EINVAL	Invalid condition variable handle pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. No preemption occurs if no other threads are waiting for the condition variable.



PREEMPTION. If a higher-priority thread was waiting on the condition variable, when it is resumed, preemption will occur.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(116 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

pthread_cond_, pthread_cond_broadcast, pthread_cond_wait*

Small Example:

```
#include <pthread.h>

/* Condition variable handle. */
pthread_cond_t    my_condition_var_handle;
int               status;

/* Signal the condition variable "my_condition_var_handle". */
status = pthread_cond_signal(&my_condition_var_handle);

/* If status is PX5_SUCCESS, the highest priority waiting thread on
   this condition variable has been resumed. */
```

pthread_cond_timedwait

C Prototype:

```
#include <pthread.h>

int pthread_cond_timedwait(pthread_cond_t * condition_var_handle,
                           pthread_mutex_t * mutex_handle,
                           const struct timespec * absolute_time);
```

Description:

This service suspends on the condition variable specified by the *condition_var_handle* parameter. Before this service is called, the thread must have obtained the mutex specified by *mutex_handle*. Internally, PX5 releases the mutex atomically with the thread suspension. Once the calling thread is resumed via a signal or broadcast to the condition variable, the mutex is reobtained before this service returns to the caller.

This service waits until the absolute time specified by the *absolute_time* parameter.



Upon return, the calling thread will own the specified mutex – regardless of completion status. The one exception is the case where the mutex was not owned prior to calling this service.



This API is a cancellation point, meaning that if a cancellation is pending, it will be detected and executed by this API.

API Parameters:

<code>condition_var_handle</code>	Handle of the condition variable to suspend on.
<code>mutex_handle</code>	Handle of the mutex associated with this condition variable.
<code>absolute_time</code>	Pointer to absolute time to wait for. If the thread is still suspended on this condition variable when this time is reached, the service will timeout, and the thread will be resumed.

Return Codes:

PX5_SUCCESS (0)	Successful condition variable wait with timeout.
EINVAL	Invalid condition variable handle, mutex handle pointer, or absolute time. In addition, if the specified mutex is not owned or a different mutex is associated with this condition variable, this error is returned.
ETIMEDOUT	Maximum wait time was reached, and this service timed out.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



SUSPENSION. The calling thread is suspended until another thread signals or broadcasts to the condition variable. The calling thread times out and returns with an error if absolute time is reached.

Stack Usage Estimate:

From the location where this API is called, there should be at least **(260 + PX5_BINDING_STACK_FRAME_SIZE)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

pthread_cond_, pthread_cond_signal, pthread_cond_wait*

Small Example:

```
#include <pthread.h>

/* Condition variable and mutex handles. */
pthread_cond_t    my_condition_var_handle;
pthread_mutex_t    my_mutex_handle;
struct timespec    max_time_to_wait;
int               status;

/* Get current time. */
clock_gettime(CLOCK_REALTIME, &max_time_to_wait);

/* Only wait for 2 seconds on the condition variable. */
max_time_to_wait.tv_sec += 2;

/* Wait on this condition variable for a maximum of 2 seconds. */
status = pthread_cond_timedwait(&my_condition_var_handle,
                                &my_mutex_handle, &max_time_to_wait);

/* If status is PX5_SUCCESS, the condition variable was
   signed/broadcasted, and the mutex is again owned by the
   calling thread. */
```


pthread_cond_wait

C Prototype:

```
#include <pthread.h>

int pthread_cond_wait(pthread_cond_t * condition_var_handle,
                      pthread_mutex_t * mutex_handle);
```

Description:

This service suspends on the condition variable specified by the *condition_var_handle* parameter. Before this service is called, the thread must have obtained the mutex specified by *mutex_handle*. Internally, PX5 releases the mutex atomically with the thread suspension. Once the calling thread is resumed via a signal or broadcast to the condition variable, the mutex is reobtained before this service returns to the caller.



Upon return, the calling thread will again own the specified mutex – regardless of completion status. The one exception is the case where the mutex was not owned prior to calling this service.



This API is a cancellation point, meaning that if a cancellation is pending, it will be detected and executed by this API.

API Parameters:

<code>condition_var_handle</code>	Handle of the condition variable to suspend on.
<code>mutex_handle</code>	Handle of the mutex associated with this condition variable.

Return Codes:

PX5_SUCCESS (0)	Successful condition variable wait.
EINVAL	Invalid condition variable handle or mutex handle pointer. In addition, if the specified mutex is not owned or a different mutex is

associated with this condition variable, this error is returned.

EACCES Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



SUSPENSION. The calling thread is suspended until another thread signals or broadcasts to the condition variable.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(236 + PX5_BINDING_STACK_FRAME_SIZE)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

pthread_cond_, pthread_cond_timedwait, pthread_cond_signal*

Small Example:

```
#include <pthread.h>

/* Condition variable and mutex handles. */
pthread_cond_t    my_condition_var_handle;
pthread_mutex_t    my_mutex_handle;
int               status;

/* Wait on this condition variable. */
status = pthread_cond_wait(&my_condition_var_handle,
                          &my_mutex_handle);

/* If status is PX5_SUCCESS, the condition variable was
   signed/broadcasted, and mutex is again owned by the
   calling thread. */
```

pthread_condattr_destroy

C Prototype:

```
#include <pthread.h>

int pthread_condattr_destroy(pthread_condattr_t *
                             condition_var_attributes);
```

Description:

This service destroys the previously created condition variable attributes structure pointed to by *condition_var_attributes*. Once destroyed, the condition variable attributes structure cannot be used again unless it is recreated.

API Parameters:

condition_var_attributes Pointer to the condition variable attributes to destroy.

Return Codes:

PX5_SUCCESS (0)	Successful condition variable attributes destroy.
EINVAL	Invalid condition variable attributes pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(36 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*pthread_cond_**, *pthread_cond_init*

Small Example:

```
#include <pthread.h>

/* Condition variable attribute structure. */
pthread_condattr_t  my_condition_var_attributes;
int                 status;

/* Destroy the condition variable attributes referenced by
   "my_condition_var_attributes". */
status = pthread_condattr_destroy(&my_condition_var_attributes);

/* If status is PX5_SUCCESS, the condition variable attributes
   structure was destroyed. */
```

pthread_condattr_getpshared

C Prototype:

```
#include <pthread.h>

int pthread_condattr_getpshared(pthread_condattr_t *
                                condition_var_attributes, int * process_sharing_designation);
```

Description:

This service returns the current process sharing designation in the condition variable attribute structure. By default, the process sharing designation is *PTHREAD_PROCESS_PRIVATE*.

API Parameters:

<code>condition_var_attributes</code>	Pointer to the condition variable attributes.
<code>process_sharing_designation</code>	Pointer to the destination for the condition variable process sharing designation.

Return Codes:

PX5_SUCCESS (0)	Successful condition variable attributes process sharing designation retrieval.
EINVAL	Invalid condition variable attributes or process sharing destination pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(44 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*pthread_condattr_**, *pthread_cond_init*

Small Example:

```
#include <pthread.h>

/* Condition variable attribute structure. */
pthread_condattr_t  my_condition_var_attributes;
int                 process_sharing_designation;
int                 status;

/* Get the process-sharing designation found in the condition
   variable attributes structure "my_condition_var_attributes". */
status = pthread_condattr_getpshared(&my_condition_var_attributes,
                                     &process_sharing_designation);

/* If status is PX5_SUCCESS, the "process_sharing_designation"
   contains the current process sharing designation. */
```

pthread_condattr_init

C Prototype:

```
#include <pthread.h>

int pthread_condattr_init(pthread_condattr_t *
                           condition_var_attributes);
```

Description:

This service initializes the condition variable attributes structure with default condition variable creation values. These defaults are as follows:

Attribute	Default Setting
Process Sharing	Private – <i>PTHREAD_PROCESS_PRIVATE</i>

API Parameters:

`condition_var_attributes` Pointer to the condition variable attributes structure to create.

Return Codes:

PX5_SUCCESS (0)	Successful condition variable attributes structure creation.
EINVAL	Invalid condition variable attributes pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(36 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*pthread_cond_**, *pthread_cond_init*

Small Example:

```
#include <pthread.h>

/* Condition variable attribute structure. */
pthread_condattr_t    my_condition_var_attributes;
int                   status;

/* Create the condition variable attributes structure
   "my_condition_var_attributes". */
status = pthread_condattr_init(&my_condition_var_attributes);

/* If status is PX5_SUCCESS, the "my_condition_var_attributes"
   structure is ready for use. */
```

pthread_condattr_setpshared

C Prototype:

```
#include <pthread.h>

int pthread_condattr_setpshared(pthread_condattr_t *
                                condition_var_attributes, int process_sharing_designation);
```

Description:

This service sets the process sharing designation in the condition variable attribute structure to either *PTHREAD_PROCESS_PRIVATE* or *PTHREAD_PROCESS_SHARED*. By default, the process sharing designation is *PTHREAD_PROCESS_PRIVATE*.



These options have no effect in PX5 RTOS, as it is a single-process environment.

API Parameters:

`condition_var_attributes` Pointer to the condition variable attributes.

`process_sharing_designation` Process sharing designation, either *PTHREAD_PROCESS_PRIVATE* or *PTHREAD_PROCESS_SHARED*.

Return Codes:

PX5_SUCCESS (0)	Successful condition variable attributes process sharing designation selection.
EINVAL	Invalid condition variable attributes structure or invalid process sharing destination.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(44 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*pthread_cond_**, *pthread_cond_init*

Small Example:

```
#include <pthread.h>

/* Condition variable attribute structure. */
pthread_condattr_t  my_condition_var_attributes;
int                 status;

/* Set the process sharing designation in the condition variable
   attributes structure "my_condition_var_attributes". */
status = pthread_condattr_setpshared(&my_condition_var_attributes,
                                     PTHREAD_PROCESS_PRIVATE);

/* If status is PX5_SUCCESS, the "my_condition_var_attributes"
   structure contains the PTHREAD_PROCESS_PRIVATE designation. */
```

pthread_create

C Prototype:

```
#include <pthread.h>

int pthread_create(pthread_t * thread_handle,
                  const pthread_attr * attributes,
                  void * (*start_routine)(void *),
                  void *arguments);
```

Description:



This service creates a new thread that starts execution at the caller's *start_routine*, passing verbatim the supplied *arguments* parameter. Note that the optional *attributes* parameter can specify additional attributes for the thread, including explicitly setting its priority, stack location, stack size, etc. Please review the *pthread_attr_** APIs for more information on available thread creation attributes. The signal mask for the newly created thread is inherited from the calling thread.

If the thread creation is successful, the newly created thread's handle is returned in *thread_handle*. Otherwise, an error code is returned if the thread creation is unsuccessful (as defined below).



By default, threads are created as joinable, meaning that another thread can wait for the thread to complete. All joinable threads should use `pthread_exit` when their processing is complete and either be joined or detached via `pthread_join` or `pthread_detach`. Failure to do so can leave the thread in a perpetual terminated state with all its resources allocated (internal thread control structure and stack).



By default, threads are created with deferred cancellation enabled. This can be changed dynamically via the `pthread_setcancelstate` and `pthread_setcanceltype` APIs.

API Parameters:

`thread_handle`

Pointer supplied by the caller to place the new thread handle upon successful creation.

<code>attributes</code>	Optional pointer to the new thread attributes structure, <code>pthread_attr</code> . Please see the <code>pthread_attr_*</code> APIs for information on how to specify thread creation attributes.
<code>start_routine</code>	Specifies the new thread entry function. When the new thread is executed, this function will be called.
<code>arguments</code>	Specifies the value supplied to the <code>start_routine</code> when called. Note this is only for use by the application.

Return Codes:

PX5_SUCCESS (0)	Successful new thread creation.
EINVAL	Invalid argument(s).
EAGAIN	Not enough resources to create the thread.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption if the newly created thread has the same or less priority than the calling thread. This service returns immediately after the creation of the new thread.



PREEMPTION. If the newly created thread has a higher priority than the calling thread, the calling thread is *preempted*, and execution takes place in the newly created thread prior to returning from this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(196 + PX5_BINDING_STACK_FRAME_SIZE)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

pthread_attr_, pthread_exit, pthread_detach, pthread_join, pthread_cancel*

Small Example:

```

#include <pthread.h>

/* Define some memory for PX5. */
unsigned long  memory_area[1024];

pthread_t      new_thread_handle;

unsigned long  new_thread_counter;
unsigned long  main_thread_counter;

/* Define new thread's starting function. */
void *  new_thread_start(void *arguments)
{
    /* Loop forever incrementing a counter. */
    while (1)
    {
        new_thread_counter++;
    }
}

int main(void)
{
    int  status;

    /* Call the PX5 start function. */
    status =  px5_pthread_start(memory_area, sizeof(memory_area));

    /* Check completion status. */
    if (status != PX5_SUCCESS)
    {
        printf("Error starting PX5!\n");
        exit(1);
    }

    /* Create the new thread. */
    status =  pthread_create(&new_thread_handle, NULL,
                           new_thread_start, NULL);

    /* Check completion status. */
    if (status != PX5_SUCCESS)
        printf("Error creating new thread!\n");

    /* In any case, simply loop in the main program, which is the
       initial thread. */
    while(1)
    {
        /* Just increment the main thread's counter. */
        main_thread_counter++;
    }
}

```

pthread_detach

C Prototype:

```
#include <pthread.h>

int pthread_detach(pthread_t thread_handle);
```

Description:



This service places the specified thread in a detached state, such that it can't be joined in the future. It is equivalent to creating the thread with the `PTHREAD_CREATE_DETACHED` attribute. When the specified, detached thread exits or returns, all of its resources are immediately released.



A thread may call `pthread_detach` on itself.

API Parameters:

<code>thread_handle</code>	Handle of previously created thread.
----------------------------	--------------------------------------

Return Codes:

PX5_SUCCESS (0)	Successful detach of thread.
EINVAL	Specified thread was already detached or not in a joinable state.
ESRCH	Specified thread could not be found.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(44 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

pthread_create, pthread_exit, pthread_join, pthread_cancel

Small Example:

```
#include <pthread.h>

pthread_t      my_thread_handle;
int            status;

/* Set the detach state in the previously created thread. */
status = pthread_detach(my_thread_handle);

/* If status contains PX5_SUCCESS, the thread is now detached. */
```

pthread_equal

C Prototype:

```
#include <pthread.h>

int pthread_equal(pthread_t first_thread, pthread_t second_thread);
```

Description:

This service compares two thread handles. If the handles are the same, a non-zero value is returned. If they are not the same, a value of zero is returned.

API Parameters:

first_thread	Handle of the first thread.
second_thread	Handle of the second thread.

Return Codes:

0	A value of zero is returned if the handles are not the same.
1	A value of 1 is returned if the handles are the same.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(12 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

pthread_self

Small Example:

```
#include <pthread.h>

pthread_t    thread_to_exit;

void *  my_thread(void *  argument)
{
    /* Perform some processing...  */

    /* Determine if this is the thread designated to exit by
       the global thread handle "thread_to_exit". */
    if (pthread_equal(pthread_self(), thread_to_exit)
    {
        /* Yes, this is the thread that needs to exit.  */
        pthread_exit((void *) 7);
    }
}
```

pthread_exit

C Prototype:

```
#include <pthread.h>

void pthread_exit(void *exit_value);
```

Description:



This service terminates the currently executing thread. If this thread is detached, all of the thread's resources are released. Any cancellation cleanup handlers are popped and executed. The *exit_value* passed to this API will be sent to a thread that has joined this thread via the *pthread_join* API, assuming the value was requested in the join request (non-NULL destination value).



An implicit call to pthread_exit is made if the application returns from the thread's start routine.



If this function is called from the main thread (or the C main function returns), all multithreading stops, and execution is transferred to the C compiler's exit function.

API Parameters:

exit_value

Even though the type is "void *" this parameter is treated as a value and passed verbatim to a thread that has joined this thread. If no thread has joined, this parameter has no effect.

Return Codes:

none

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



PREEMPTION. If a higher-priority thread has joined this thread, the exit processing will resume its execution, and preemption will occur.



SUSPENSION. Once the currently executing thread exit processing is complete, the thread enters a permanent terminated state of suspension.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(84 + PX5_BINDING_STACK_FRAME_SIZE)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

pthread_detach, pthread_join, pthread_cancel

Small Example:

```
#include <pthread.h>

void * my_thread(void * argument)
{
    /* Perform some processing... */

    /* Processing is done, exit with a value of 7. */
    pthread_exit((void *) 7);

    /* Note: processing will never get here since exit does not
       return. */
}
```

pthread_getaffinity_np

C Prototype:

```
#include <pthread.h>

int pthread_getaffinity_np(pthread_t thread_handle,
                           size_t cpusetsize, cpu_set_t * cpuset);
```

Description:

When the PX5 RTOS is configured for SMP (*PX5_MAXIMUM_CORES* greater than one), this service retrieves the current set of cores this thread can execute in the parameter *cpuset*. If *PX5_MAXIMUM_CORES* is equal to one, this service is irrelevant and, therefore, not defined.

API Parameters:

<i>thread_handle</i>	Handle of thread to retrieve core affinity for.
<i>cpusetsize</i>	Size in bytes of the <i>cpuset</i> destination for the current core affinity selection.
<i>cpuset</i>	Bit map where each bit position represents a core, e.g., bit 0 represents core 0. A set bit indicates the thread is allowed to run on that core.

Return Codes:

PX5_SUCCESS (0)	Successful thread core affinity retrieval.
EINVAL	Invalid <i>cpuset</i> or <i>cpusetsize</i> parameters.
ESRCH	Thread specified is not a valid thread.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(52 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

pthread_setaffinity_np

Small Example:

```
#include <pthread.h>

cpu_set_t  my_cpuset;

/* Get the core affinity of the currently executing thread. */
pthread_getaffinity_np(pthread_self(), sizeof(my_cpuset), &my_cpuset);

/* Upon return, "my_cpuset" contains all the cores this thread is
   allowed to execute. */
```

pthread_getspecific

C Prototype:

```
#include <pthread.h>

void * pthread_getspecific(pthread_key_t key);
```

Description:

This service returns the *Thread-local Storage (TLS)* value of the calling thread associated with the specified key.

API Parameters:

key	Associated key for this TLS.
-----	------------------------------

Return Codes:

non-NULL	Successful retrieval of TLS pointer set previously with the <i>pthread_setspecific</i> API.
NULL	Invalid key, key modified, or TLS was not set using the <i>pthread_setspecific</i> API.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(52 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

pthread_key_create, *pthread_key_delete*, *pthread_setspecific*

Small Example:

```
#include <pthread.h>

void *   my_tls_pointer;

/* Get the Thread-local Storage (TLS) for the executing thread. */
my_tls_pointer = pthread_getspecific(pthread_self());

/* Upon return, "my_tls_poitner" contains the TLS pointer for the
   executing thread. */
```

pthread_join

C Prototype:

```
#include <pthread.h>

int pthread_join(pthread_t thread_to_join, void ** value_destination);
```

Description:



This service suspends the currently running thread until the specified thread completes its processing. When the specified thread exits, the value supplied in its exit call is returned in the destination specified by the caller—if a non-NULL exit value destination pointer is provided.



All joinable threads that complete, should be joined or detached via `pthread_join` and `pthread_detach`, respectively.



Only one thread is allowed to join another thread. Any additional join requests will return an error.



This API is a cancellation point, meaning that if a cancellation is pending, it will be detected and executed by this API.

API Parameters:

<code>thread_to_join</code>	Handle of thread to join, i.e., thread completion to wait for.
<code>value_desitination</code>	Destination for the value from the specified thread's exit call.

Return Codes:

PX5_SUCCESS (0)	Successful thread join.
EINVAL	Thread specified is not joinable.
ESRCH	Thread specified is not a valid thread.

EDEADLK	Thread is attempting to join with itself, which would create a deadlock.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



SUSPENSION. Once the currently executing thread is joined with the thread specified, it enters a suspended state. The highest priority ready thread will then start executing.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(104 + PX5_BINDING_STACK_FRAME_SIZE)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

pthread_exit, pthread_detach

Small Example:

```
#include <pthread.h>

pthread_t    my_thread2_handle;

void *  my_thread1(void *  argument)
{

    void *  exit_value;

    /* Join the my_thread2. The API will suspend until my_thread_2
       exits. */
    pthread_join(my_thread2_handle, &my_thread_2_exit_value);

    /* When processing returns here, my_thread_2 has exited with a
       value of "7", which has been stored in "exit_value". */
}

void *  my_thread1(void *  argument)
{

    pthread_exit((void *) 7);
}
```

pthread_key_create

C Prototype:

```
#include <pthread.h>

int  pthread_key_create(pthread_key_t  *key,
                        void  (*destructor)(void  *));
```

Description:

This service creates a key that can be used by threads that need *Thread-local Storage (TLS)*. The key is effectively a designation for a destructor that is called when the thread exits or is cancelled to release the TLS specified by the thread using the *pthread_setspecific* API.



The maximum number of keys is defined by the compile-time constant PTHREAD_KEYS_MAX, which is set to one by default. If more keys are needed, build PX5 RTOS with PTHREAD_KEYS_MAX defined to the desired amount.

API Parameters:

key	Destination for the newly created key, which is effectively an index from 0 through <i>PTHREAD_KEYS_MAX – 1</i> .
destructor	If non-NULL, the function to call to process (typically release) any non-NULL TLS pointers set up by the thread via <i>pthread_setspecific</i> when the thread exits or is cancelled.

Return Codes:

PX5_SUCCESS (0)	Successful key creation.
EINVAL	Invalid key destination pointer (NULL).
EAGAIN	No keys are available.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(52 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

pthread_getspecific, pthread_key_delete, pthread_set_specific

Small Example:

```
#include <pthread.h>

pthread_key_t  my_key;
int           status;

/* Create a key with "free" as the destructor, since it is an
   easier example than using memory pools. */
status = pthread_key_create(&my_key, free);

/* Is status is PX5_SUCCESS, the "my_key" is a valid key
   with "free" as the associated destructor. */
```

pthread_key_delete

C Prototype:

```
#include <pthread.h>

int pthread_key_delete(pthread_key_t key);
```

Description:

This service deletes a key that was previously created via *pthread_key_create*.



*Note that calling this API disables the previously associated destructor, which can lead to TLS values specified via *pthread_setspecific* not being released.*

API Parameters:

<code>key</code>	Previously created key, which is effectively an index from 0 through <i>PTHREAD_KEYS_MAX</i> – 1.
------------------	---

Return Codes:

PX5_SUCCESS (0)	Successful key deletion.
EINVAL	Invalid key.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(52 + PX5_BINDING_STACK_FRAME_SIZE)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

pthread_getspecific, pthread_key_create, pthread_set_specific

Small Example:

```
#include <pthread.h>

pthread_key_t  my_key;
int           status;

/* Delete a previously created key "my_key". */
status = pthread_key_delete(my_key);

/* Is status is PX5_SUCCESS, "my_key" has been deleted. */
```


pthread_kill

C Prototype:

```
#include <signal.h>

int pthread_kill(pthread_t thread, int signal);
```

Description:

This service raises a signal for the specified thread. If the signal is masked, it is added to the pending signals set. In addition, if the specified thread is suspended waiting for this signal, it is resumed. Otherwise, the registered signal handler is invoked if the specified thread does not mask the signal. If no signal handler has been registered, an error is returned.

API Parameters:

thread	Handle of thread to raise a signal for.
signal	Signal number to raise, valid values range from 0 through 31, where signal 0 does not cause any actual signal processing – only error checking is performed.

Return Codes:

PX5_SUCCESS (0)	Successful signal raise.
EINVAL	Invalid signal number or signal was unmasked, but no signal handler was registered.
ESRCH	Thread specified is not a valid thread.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. If the thread is not waiting for this signal, or there is no signal handler, or the signal handler does not unblock any higher-priority suspended thread, no preemption occurs.



PREEMPTION. If a higher-priority thread is waiting for the signal when it is delivered, the waiting thread is resumed, and preemption will occur.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(124 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

pthread_sigmask, sigaction, sigwait

Small Example:

```
#include <pthread.h>

pthread_t    my_thread2_handle;

void *  my_thread1(void *  argument)
{

    /* Raise SIGUSR1 signal for "my_thread2_hanlde". */
    pthread_kill(my_thread2_handle, SIGUSR1);

    /* If successful, the signal SIGUSR1 has been raised. */
}
```

pthread_mutex_destroy

C Prototype:

```
#include <pthread.h>

int pthread_mutex_destroy(pthread_mutex_t * mutex_handle);
```

Description:

This service destroys the previously created mutex specified by *mutex_handle*. If a thread still owns the mutex, an error is returned.



To avoid unpredictable behavior, the application must ensure there is no further access to the mutex after it is destroyed.

API Parameters:

<code>mutex_handle</code>	Handle of the mutex to destroy.
---------------------------	---------------------------------

Return Codes:

PX5_SUCCESS (0)	Successful mutex destroy.
EINVAL	Invalid mutex handle.
EBUSY	A thread currently owns the mutex.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(84 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*pthread_mutex_**, *pthread_mutex_create*

Small Example:

```
#include <pthread.h>

/* Mutex handle. */
pthread_mutex_t  my_mutex_handle;
int              status;

/* Destroy the mutex referenced by "my_mutex_handle". */
status = pthread_mutex_destroy(&my_mutex_handle);

/* If status is PX5_SUCCESS, the mutex was destroyed. */
```

pthread_mutex_init

C Prototype:

```
#include <pthread.h>

int pthread_mutex_init(pthread_mutex_t * mutex_handle,
                      pthread_mutexattr_t * mutex_attributes);
```

Description:



This service initializes (creates) a mutex with the optional mutex attributes. If no attributes are specified, the mutex is created with the type *PTHREAD_MUTEX_ERRORCHECK* (no nested locking) and *PTHREAD_PRIO_NONE* (no priority inheritance). If successful, the mutex handle is set up for further use by the application.

API Parameters:

<code>mutex_handle</code>	Handle of the mutex to initialize.
<code>mutex_attributes</code>	Optional mutex attributes. This value is NULL if no mutex attributes are specified.

Return Codes:

PX5_SUCCESS (0)	Successful mutex initialization.
EINVAL	Invalid mutex handle pointer or mutex attributes.
EBUSY	Mutex has already been created.
ENOMEM	Insufficient memory to create a mutex.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(124 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*pthread_mutexattr_**, *pthread_mutex_destroy*

Small Example:

```
#include <pthread.h>

/* Mutex handle. */
pthread_mutex_t  my_mutex_handle;
int              status;

/* Create the mutex and set up "my_mutex_handle". */
status = pthread_mutex_init(&my_mutex_handle, NULL);

/* If status is PX5_SUCCESS, the mutex was created, and the mutex
   handle is ready to use. */
```

pthread_mutex_lock

C Prototype:

```
#include <pthread.h>

int pthread_mutex_lock(pthread_mutex_t * mutex_handle);
```

Description:

This service assigns ownership to the calling thread if the mutex is available. Otherwise, if another thread already owns the mutex, the calling thread suspends until the other thread releases the mutex. If the mutex has priority inheritance enabled (created with `PTHREAD_PRIO_INHERIT`), the suspending thread's priority is elevated to the priority of the owning thread (if it has a higher priority).



If a thread terminates while owning a mutex, all other threads waiting for the mutex will be indefinitely suspended.

API Parameters:

<code>mutex_handle</code>	Handle of the mutex to lock.
---------------------------	------------------------------

Return Codes:

PX5_SUCCESS (0)	Successful mutex lock.
EDEADLK	Thread already owns mutex (non-recursive mutex).
EINVAL	Invalid mutex handle pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. No preemption occurs if the mutex is available and ownership is assigned to the calling thread.



SUSPENSION. If another thread already owns the mutex, the calling thread is suspended until the other thread releases ownership via *pthread_mutex_unlock*.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(140 + PX5_BINDING_STACK_FRAME_SIZE)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*pthread_mutex_**, *pthread_mutex_unlock*

Small Example:

```
#include <pthread.h>

/* Mutex handle. */
pthread_mutex_t  my_mutex_handle;
int              status;

/* Lock the mutex "my_mutex_handle". */
status = pthread_mutex_lock(&my_mutex_handle);

/* If status is PX5_SUCCESS, the mutex is now owned by the
   calling thread. Any other thread attempting to lock
   the mutex will suspend. */
```


pthread_mutex_trylock

C Prototype:

```
#include <pthread.h>

int pthread_mutex_trylock(pthread_mutex_t * mutex_handle);
```

Description:

This service assigns ownership to the calling thread if the mutex is available. Otherwise, if another thread already owns the mutex, this service immediately returns an error, i.e., there is no thread suspension like the *pthread_mutex_lock* service.



If a thread terminates while owning a mutex, all other threads waiting for the mutex will be indefinitely suspended.

API Parameters:

<code>mutex_handle</code>	Handle of the mutex to try to lock.
---------------------------	-------------------------------------

Return Codes:

PX5_SUCCESS (0)	Successful mutex lock.
EDEADLK	Thread already owns mutex (non-recursive mutex).
EBUSY	Another thread already owns the mutex.
EINVAL	Invalid mutex handle pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. No preemption occurs if the mutex is available and ownership is assigned to the calling thread.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(36 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*pthread_mutex_**, *pthread_mutex_lock*, *pthread_mutex_unlock*

Small Example:

```
#include <pthread.h>

/* Mutex handle. */
pthread_mutex_t  my_mutex_handle;
int              status;

/* Try to lock the mutex "my_mutex_handle". */
status = pthread_mutex_trylock(&my_mutex_handle);

/* If status is PX5_SUCCESS, the mutex is now owned by the
   calling thread. Any other thread attempting to lock
   the mutex will suspend. */
```

pthread_mutex_unlock

C Prototype:

```
#include <pthread.h>

int pthread_mutex_unlock(pthread_mutex_t * mutex_handle);
```

Description:

This service releases a previously owned mutex. If other threads are suspended waiting for the mutex, the first thread is given ownership and resumed.

API Parameters:

<code>mutex_handle</code>	Handle of the mutex to unlock.
---------------------------	--------------------------------

Return Codes:

PX5_SUCCESS (0)	Successful mutex unlock.
EINVAL	Invalid mutex handle pointer.
EPERM	Calling thread does not own the mutex.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. If no other threads are waiting for the mutex, no preemption occurs.



PREEMPTION. If a higher-priority thread is waiting for the mutex, when it is given the mutex, the waiting thread is resumed, and preemption will occur.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(172 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*pthread_mutex_**, *pthread_mutex_lock*, *pthread_mutex_trylock*

Small Example:

```
#include <pthread.h>

/* Mutex handle. */
pthread_mutex_t  my_mutex_handle;
int              status;

/* Unlock the mutex "my_mutex_handle". */
status = pthread_mutex_unlock(&my_mutex_handle);

/* If status is PX5_SUCCESS, the mutex is now unlocked, and no
   longer owned by the calling thread. */
```

pthread_mutexattr_destroy

C Prototype:

```
#include <pthread.h>

int pthread_mutexattr_destroy(pthread_mutexattr_t * mutex_attributes);
```

Description:

This service destroys the previously created mutex attributes structure pointed to by *mutex_attributes*. Once destroyed, the mutex attributes structure cannot be used again unless it is recreated.

API Parameters:

mutex_attributes Pointer to the mutex attributes to destroy.

Return Codes:

PX5_SUCCESS (0)	Successful mutex attributes destroy.
EINVAL	Invalid mutex attributes pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(36 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*pthread_mutex_**, *pthread_mutexattr_**, *pthread_mutexattr_init*

Small Example:

```
#include <pthread.h>

/* Mutex attribute structure. */
pthread_mutexattr_t my_mutex_attributes;
int status;

/* Destroy the mutex attributes referenced by
   "my_mutex_attributes". */
status = pthread_mutexattr_destroy(&my_mutex_attributes);

/* If status is PX5_SUCCESS, the mutex attributes structure
   was destroyed. */
```

pthread_mutexattr_getprotocol

C Prototype:

```
#include <pthread.h>

int pthread_mutexattr_getprotocol(pthread_mutexattr_t
                                *mutex_attributes, int * protocol);
```

Description:

This service returns the previously supplied mutex protocol. By default, the protocol is *PTHREAD_PRIO_NONE*.

API Parameters:

<code>mutex_attributes</code>	Pointer to the mutex attributes.
<code>protocol</code>	Pointer to the destination for the previously supplied mutex protocol.

Return Codes:

PX5_SUCCESS (0)	Successful mutex attributes mutex protocol retrieval.
EINVAL	Invalid mutex attributes or mutex protocol pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(44 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*pthread_mutex_**, *pthread_mutexattr_**, *pthread_mutexattr_setprotocol*

Small Example:

```
#include <pthread.h>

/* Mutex attribute structure. */
pthread_mutexattr_t  my_mutex_attributes;
char *               my_mutex_protocol;
int                  status;

/* Get the mutex protocol in the mutex attributes structure
   "my_mutex_attributes". */
status = pthread_mutexattr_getprotocol(&my_mutex_attributes,
                                       &my_mutex_protocol);

/* If status is PX5_SUCCESS, "my_mutex_protocol" contains the
   previously supplied protocol. */
```


pthread_mutexattr_getpshared

C Prototype:

```
#include <pthread.h>

int pthread_mutexattr_getpshared(pthread_mutexattr_t *mutex_attributes,
                                  int * process_sharing_designation);
```

Description:

This service returns the current process sharing designation in the mutex attribute structure. By default, the process sharing designation is *PTHREAD_PROCESS_PRIVATE*.

API Parameters:

<code>mutex_attributes</code>	Pointer to the mutex attributes.
<code>process_sharing_designation</code>	Pointer to the destination for the mutex process sharing designation.

Return Codes:

PX5_SUCCESS (0)	Successful mutex attributes process sharing designation retrieval.
EINVAL	Invalid mutex attributes or process sharing destination pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(44 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*pthread_mutex_**, *pthread_mutexattr_init*

Small Example:

```
#include <pthread.h>

/* Mutex attribute structure. */
pthread_mutexattr_t  my_mutex_attributes;
int                  process_sharing_designation;
int                  status;

/* Get the process sharing designation found in the mutex
   attributes structure "my_mutex_attributes". */
status = pthread_mutexattr_getpshared(&my_mutex_attributes,
                                       &process_sharing_designation);

/* If status is PX5_SUCCESS, "process_sharing_designation"
   contains the current process sharing designation. */
```

pthread_mutexattr_gettype

C Prototype:

```
#include <pthread.h>

int pthread_mutexattr_gettype(pthread_mutexattr_t
                               *mutex_attributes, int * type);
```

Description:

This service returns the previously supplied mutex type. By default, the mutex type is *PTHREAD_MUTEX_ERRORCHECK*.

API Parameters:

<code>mutex_attributes</code>	Pointer to the mutex attributes.
<code>type</code>	Pointer to the destination for the previously supplied mutex type.

Return Codes:

PX5_SUCCESS (0)	Successful mutex attributes mutex type retrieval.
EINVAL	Invalid mutex attributes or mutex type pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(44 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*pthread_mutex_**, *pthread_mutexattr_**, *pthread_mutexattr_settype*

Small Example:

```
#include <pthread.h>

/* Mutex attribute structure. */
pthread_mutexattr_t  my_mutex_attributes;
char *               my_mutex_type;
int                  status;

/* Get the mutex type in the mutex attributes structure
   "my_mutex_attributes". */
status = pthread_mutexattr_gettype(&my_mutex_attributes,
                                   &my_mutex_type);

/* If status is PX5_SUCCESS, "my_mutex_type" contains the
   previously supplied mutex type. */
```

pthread_mutexattr_init

C Prototype:

```
#include <pthread.h>

int pthread_mutexattr_init(pthread_mutexattr_t *mutex_attributes);
```

Description:

This service initializes the mutex attributes structure with default values for mutex creation, which include standard error checking, no nested mutex locking, and no priority inheritance. These defaults are as follows:

Attribute	Default Setting
Process Sharing	<i>PTHREAD_PROCESS_PRIVATE</i>
Type	<i>PTHREAD_MUTEX_ERRORCHECK</i>
Protocol	<i>PTHREAD_PRIO_NONE</i>

API Parameters:

<code>mutex_attributes</code>	Pointer to the mutex attributes structure to create.
-------------------------------	--

Return Codes:

PX5_SUCCESS (0)	Successful mutex attributes structure creation.
EINVAL	Invalid mutex attributes pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(36 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*pthread_mutex_**, *pthread_mutexattr_destroy*

Small Example:

```
#include <pthread.h>

/* Mutex attribute structure. */
pthread_mutexattr_t  my_mutex_attributes;
int                  status;

/* Create the mutex attributes structure "my_mutex_attributes". */
status = pthread_mutexattr_init(&my_mutex_attributes);

/* If status is PX5_SUCCESS, the "my_mutex_attributes" structure is
   ready for use. */
```

pthread_mutexattr_setprotocol

C Prototype:

```
#include <pthread.h>

int pthread_mutexattr_setprotocol(pthread_mutexattr_t
                                *mutex_attributes, int protocol);
```

Description:

This service sets the specified protocol in the previously created mutex attributes structure. By default, the protocol is *set to PTHREAD_PRIO_NONE, which means that priority inheritance is not enabled*. Setting the protocol to *PTHREAD_PRIO_INHERIT* enables priority inheritance when a mutex is created with this attribute structure. Please see the mutex section in Chapter 6 for more information on priority inheritance.

API Parameters:

mutex_attributes	Pointer to the mutex attributes.
protocol	Specified mutex protocol, as follows:

PTHREAD_PRIO_NONE
PTHREAD_PRIO_INHERIT

Return Codes:

PX5_SUCCESS (0)	Successful mutex attributes mutex protocol set.
EINVAL	Invalid mutex attributes or mutex protocol. Note that <i>PTHREAD_PRIO_PROTECT</i> is not currently supported.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(44 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

pthread_mutex_, pthread_mutexattr_*, pthread_mutexattr_getprotocol*

Small Example:

```
#include <pthread.h>

/* Mutex attribute structure. */
pthread_mutexattr_t  my_mutex_attributes;
char *               my_mutex_protocol;
int                  status;

/* Set the mutex protocol in the mutex attributes structure
   "my_mutex_attributes". */
status = pthread_mutexattr_setprotocol(&my_mutex_attributes,
                                       PTHREAD_PRIO_INHERIT);

/* If status is PX5_SUCCESS, "my_mutex_protocol" specifies that
   priority inheritance is enabled when the mutex using these
   attributes is created. */
```


pthread_mutexattr_setpshared

C Prototype:

```
#include <pthread.h>

int pthread_mutexattr_setpshared(pthread_mutexattr_t *mutex_attributes,
                                  int process_sharing_designation);
```

Description:

This service sets the process sharing designation in the mutex attribute structure to either *PTHREAD_PROCESS_PRIVATE* or *PTHREAD_PROCESS_SHARED*. By default, the process sharing designation is *PTHREAD_PROCESS_PRIVATE*.



These options have no effect in PX5 RTOS, as it is a single-process environment.

API Parameters:

<code>mutex_attributes</code>	Pointer to the mutex attributes.
<code>process_sharing_designation</code>	Process sharing designation, either <i>PTHREAD_PROCESS_PRIVATE</i> or <i>PTHREAD_PROCESS_SHARED</i> .

Return Codes:

PX5_SUCCESS (0)	Successful mutex attributes process sharing designation selection.
EINVAL	Invalid mutex attributes or invalid process sharing destination.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(44 + PX5_BINDING_STACK_FRAME_SIZE)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*pthread_mutex_**, *pthread_mutexattr_init*

Small Example:

```
#include <pthread.h>

/* Mutex attribute structure. */
pthread_mutexattr_t my_mutex_attributes;
int status;

/* Set the process sharing designation in the mutex
   attributes structure "my_mutex_attributes". */
status = pthread_mutexattr_setpshared(&my_mutex_attributes,
                                       PTHREAD_PROCESS_PRIVATE);

/* If status is PX5_SUCCESS, the "my_mutex_attributes" structure
   contains the PTHREAD_PROCESS_PRIVATE designation. */
```

pthread_mutexattr_settype

C Prototype:

```
#include <pthread.h>

int pthread_mutexattr_settype(pthread_mutexattr_t
                             * mutex_attributes, int type);
```

Description:

This service sets the specified mutex type in the previously created attributes structure. If *PTHREAD_MUTEX_RECURSIVE* is specified, a thread may lock the same mutex multiple times up to a maximum of *PTHREAD_MUTEX_MAX_NESTING*. By default, the type is *PTHREAD_MUTEX_ERRORCHECK*.



In PX5 RTOS, *PTHREAD_MUTEX_NORMAL* behaves the same as *PTHREAD_MUTEX_ERRORCHECK*, in the sense that standard error checking is applied to mutex locking and unlocking.

API Parameters:

<code>mutex_attributes</code>	Pointer to the mutex attributes.
<code>type</code>	Specified mutex type, as follows:

PTHREAD_MUTEX_NORMAL
PTHREAD_MUTEX_ERRORCHECK
PTHREAD_MUTEX_RECURSIVE

Return Codes:

PX5_SUCCESS (0)	Successful mutex attributes mutex type set.
EINVAL	Invalid mutex attributes or mutex type.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(44 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the context of the application.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*pthread_mutex_**, *pthread_mutexattr_**, *pthread_mutexattr_gettype*

Small Example:

```
#include <pthread.h>

/* Mutex attribute structure. */
pthread_mutexattr_t my_mutex_attributes;
int status;

/* Set the mutex type in the mutex attributes structure
   "my_mutex_attributes". */
status = pthread_mutexattr_settype(&my_mutex_attributes,
                                   PTHREAD_MUTEX_RECURSIVE);

/* If status is PX5_SUCCESS, "my_mutex_protocol" specifies that
   nested (recursive) mutex locking is enabled when the mutex using
   these attributes is created. */
```

pthread_once

C Prototype:

```
#include <pthread.h>

int  pthread_once(pthread_once_t  *once_control,
                  void  (*init_routine) (void));
```

Description:

This service executes the supplied `init_routine` one time, which is useful in situations where initialization needs be done within a library that is accessed by multiple threads.



If the `once_control` variable is not cleared automatically by the compiler startup, it may also be initialized via the macro `PTHREAD_ONCE_INIT(once_control)`.

API Parameters:

<code>once_control</code>	Control variable that is assumed to be a value of zero before the initialization routine is ever called.
<code>init_routine</code>	Initialization routine that is called only once (when <code>once_control</code> is zero).

Return Codes:

PX5_SUCCESS (0)	Successful once execution.
EINVAL	Invalid once control variable or initialization routine pointer (NULL).
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(52 + PX5_BINDING_STACK_FRAME_SIZE)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*pthread_**

Small Example:

```
#include <pthread.h>

pthread_once_t  my_once;
int            status;

/* Initialize once control variable. */
my_once = PTHREAD_ONCE_INIT;

/* Call "my_init" only one time. */
status = pthread_once(&my_once, my_init);

/* Is status is PX5_SUCCESS, the one-time initialization is
   completed. */
```

pthread_self

C Prototype:

```
#include <pthread.h>

pthread_t pthread_self(void);
```

Description:

This service returns the thread handle of the currently running thread.

API Parameters:

none

Return Codes:

pthread_t	Handle of the currently running thread.
-----------	---

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(12 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

pthread_equal

Small Example:

```
#include <pthread.h>

/* Thread handle. */
pthread_t  my_thread_handle;

void *  my_thread(void *)
{
    /* Pickup the thread handle for this thread. */
    my_thread_handle = pthread_self();

    /* my_thread handle can now be used in other
       pthread API calls. */
}
```


pthread_setaffinity_np

C Prototype:

```
#include <pthread.h>

int pthread_setaffinity_np(pthread_t thread_handle,
                           size_t cpusetsize, cpu_set_t * cpuset);
```

Description:

When the PX5 RTOS is configured for SMP (*PX5_MAXIMUM_CORES* greater than one), this service specifies the set of cores this thread can execute via the parameter *cpuset*. If *PX5_MAXIMUM_CORES* equals one, this service is irrelevant and, therefore, not defined.

API Parameters:

<code>thread_handle</code>	Handle of thread to specify core affinity for.
<code>cpusetsize</code>	Size in bytes of the <i>cpuset</i> source for the current core affinity selection.
<code>cpuset</code>	Bit map where each bit position represents a core, e.g., bit 0 represents core 0. A set bit indicates the thread is allowed to run on that core.

Return Codes:

PX5_SUCCESS (0)	Successful thread core affinity specification.
EINVAL	Invalid <i>cpuset</i> or <i>cpusetsize</i> parameters.
ESRCH	Thread specified is not a valid thread.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. If the new core affinity does not necessitate any changes in the assignment of threads to cores, no preemption occurs.



PREEMPTION. If the new core affinity selection eliminates available cores for execution, this thread is preempted until one of its allowed cores is available.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(312 + PX5_BINDING_STACK_FRAME_SIZE)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

pthread_getaffinity_np

Small Example:

```
#include <pthread.h>

cpu_set_t  my_cpuset;

/* Set the core affinity of the currently executing thread to only
   allow execution on core 0. */
my_cpuset = 1;
pthread_setaffinity_np(pthread_self(), sizeof(my_cpuset), &my_cpuset);

/* Upon return, the specified thread will only be allowed to execute
   on core 0. */
```

pthread_setcancelstate

C Prototype:

```
#include <pthread.h>

int pthread_setcancelstate(int new_state, int * old_state);
```

Description:



This service sets the calling thread's cancel state as specified by *new_state*. If *old_state* is non-NULL, the previous cancel state is stored in the specified destination.



By default, threads are created with the cancel state of PTHREAD_CANCEL_ENABLE.

API Parameters:

<code>new_state</code>	New cancellation state, which is one of the following: <i>PTHREAD_CANCEL_ENABLE</i> <i>PTHREAD_CANCEL_DISABLE</i>
<code>old_state</code>	If non-NULL, destination to store the previous cancel state of the calling thread.

Return Codes:

PX5_SUCCESS (0)	Successful change of calling thread's cancel state.
EINVAL	Invalid cancel state.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(124 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

pthread_cancel, pthread_cleanup_pop, pthread_cleanup_push, pthread_create, pthread_setcanceltype, pthread_testcancel

Small Example:

```
#include <pthread.h>

int          status;

/* Enable cancellation for the calling thread. */
status = pthread_setcancelstate(PTHREAD_CANCEL_ENABLE, NULL);

/* Cancellation is now available for the calling thread. */
```

pthread_setcanceltype

C Prototype:

```
#include <pthread.h>

int pthread_setcanceltype(int new_type, int * old_type);
```

Description:



This service sets the calling thread's cancel type as specified by *new_type*. If *old_type* is non-NULL, the previous cancel type is stored in the specified destination.



Threads making PX5 RTOS API calls should only use deferred cancellation (PTHREAD_CANCEL_DEFERRED) since asynchronous cancellation could terminate thread processing inside an API and leave the system in an unknown state.



By default, threads are created with the cancel type of PTHREAD_CANCEL_DEFERRED.

API Parameters:

new_type

New cancellation type, which is one of the following:

PTHREAD_CANCEL_DEFERRED
PTHREAD_CANCEL_ASYNCHRONOUS

old_type

If non-NULL, destination to store the previous cancel type of the calling thread.

Return Codes:

PX5_SUCCESS (0)
EINVAL
EACCES

Successful change of calling thread's cancel type.
Invalid cancel type.
Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(124 + PX5_BINDING_STACK_FRAME_SIZE)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

pthread_cancel, pthread_cleanup_pop, pthread_cleanup_push, pthread_create, pthread_setcancelstate, pthread_testcancel

Small Example:

```
#include <pthread.h>

int          status;

/* Enable asynchronous cancellation for the calling thread. */
status = pthread_setcanceltype(PTHREAD_CANCEL_ASYNCHRONOUS, NULL);

/* Asynchronous cancellation is now enabled for the calling
   thread. */
```

pthread_setspecific

C Prototype:

```
#include <pthread.h>

int pthread_setspecific(pthread_key_t key, const void * value);
```

Description:

This service assigns the previously created key with the *Thread-local Storage (TLS)* pointer for the executing thread.

API Parameters:

key	Previously created key.
value	TLS pointer associated with this key of the executing thread.

Return Codes:

PX5_SUCCESS (0)	Successful TLS for this thread and key.
EINVAL	Invalid key.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(52 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

pthread_getspecific, pthread_key_create, pthread_key_delete

Small Example:

```
#include <pthread.h>

void          *my_tls_pointer;
pthread_key_t my_key;
int           status;

/* Allocate TLS for this thread - should use memorypool, but
   easier to show with malloc/free. Assume "my_key" has already
   been successfully created. */
my_tls_pointer = malloc(1000);

/* Set the TLS of the executing thread. */
status = pthread_setspecific(my_key, my_tls_pointer);

/* Is status is PX5_SUCCESS, the "my_tls_pointer" has been
   associated with the specified key for the executing thread. */
```


pthread_sigmask

C Prototype:

```
#include <signal.h>

int pthread_sigmask(int operation, sigset_t * signal_set,
                    sigset_t * previous_mask);
```

Description:

This service sets or clears signals in the currently executing thread's signal mask. How the signal mask is modified depends on the *operation* parameter as described in the parameter section below. Note that each thread inherits its initial signal mask from the thread that created it.

If a pending signal is unmasked and there is a corresponding signal handler, the signal handler is executed prior to returning from this call.



Signals must be masked to synchronously wait for them via one of the signal wait APIs.

API Parameters:

operation	Specifies the operation to perform on the calling thread's signal mask. Valid operations are:
SIG_BLOCK	The supplied signal set is used to block (set) signals in the thread's signal mask.
SIG_UNBLOCK	The supplied signal set is used to unblock (clear) signals in the thread's signal mask.
SIG_SETMASK	The supplied signal set is used to update the thread's signal mask verbatim.

<code>signal_set</code>	The signal set bitmap used to modify the thread's signal mask based on the specified operation.
<code>previous_mask</code>	The thread's previous signal mask.

Return Codes:

PX5_SUCCESS (0)	Successful signal mask update.
EINVAL	Invalid operation or signal set pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.



PREEMPTION. If a pending signal is unmasked and the corresponding signal handler resumes a higher-priority thread, preemption will occur.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(92 + PX5_BINDING_STACK_FRAME_SIZE)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

pthread_kill, sigaction, sigwait

Small Example:

```
#include <pthread.h>

sigset_t  new_signal_mask;

void *  my_thread(void *)
{
    /* Set the signal mask to all ones - masking everything. */
    sigfillset(&new_signal_mask);

    /* Now mask all signals for this thread. */
    pthread_sigmask(SIG_SETMASK, &new_signal_mask, NULL);

    /* All signals are now masked for this thread. */
}
```

pthread_testcancel

C Prototype:

```
#include <pthread.h>

void pthread_testcancel(void);
```

Description:



This service checks for a pending cancel request for the calling thread. If detected, the calling thread cancels itself via *pthread_exit*.

API Parameters:

none

Return Codes:

none

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. If a cancel request is not pending for the calling thread, there is no preemption possible with this service.



SUSPENSION. If cancellation is pending, the calling thread is suspended due to termination.

Stack Usage Estimate:



From the location where this API is called, at least (**100 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

pthread_cancel, *pthread_cleanup_pop*, *pthread_cleanup_push*,
pthread_create, *pthread_setcancelstate*, *pthread_setcanceltype*

Small Example:

```
#include <pthread.h>

/* Test for pending cancellation for the calling thread. */
pthread_testcancel();

/* If we return, there was no pending cancellation request. */
```

px5_errno_get

C Prototype:

```
#include <px5_errno.h>

int  px5_errno_get(void);
```

Description:

This pthreads+ service retrieves the errno value of the currently executing thread.

API Parameters:

none

Return Codes:

errno

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. No preemption is possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(8 + PX5_BINDING_STACK_FRAME_SIZE)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable from the thread context and interrupt handlers (ISRs).

See Also:

px5_errno_set

Small Example:

```
#include <px5_errno.h>

int    my_errno;

/* Pickup the errno of the calling thread. */
my_errno = px5_errno_get();

/* Upon return, the thread's errno is stored in "my_errno". */
```

px5_errno_set

C Prototype:

```
#include <px5_errno.h>

void px5_errno_set(int new_error);
```

Description:

This pthreads+ service sets the errno value of the currently executing thread.

API Parameters:

<code>new_error</code>	New error value for the currently executing thread.
------------------------	---

Return Codes:

none

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. No preemption is possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(8 + PX5_BINDING_STACK_FRAME_SIZE)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable from the thread context and interrupt handlers (ISRs).

See Also:

px5_errno_get

Small Example:

```
#include <px5_errno.h>

/* Set the errno to EINVAL of the calling thread. */
px5_errno_set(EINVAL);

/* Upon return, the thread's errno is EINVAL. */
```

px5_mq_extend_open

C Prototype:

```
#include <fcntl.h>
#include <sys/stat.h>
#include <mqueue.h>

mqd_t px5_mq_extend_open(const char * queue_name, int operation,
                        mode_t mode, mq_attr * queue_attributes,
                        mq_extdatr * extend_attributes);
```

Description:



This pthreads+ service opens (creates) the message queue specified (with optional extended attributes) and returns the message queue handle if successful.

API Parameters:

queue_name	Name of the queue to open/create.
operation	Specifies how the queue will operate. The supported options are: <i>O_CREAT</i> <i>O_RDWR</i> <i>O_NONBLOCK</i> Both <i>O_CREAT</i> and <i>O_RDWR</i> should be specified. The <i>O_NONBLOCK</i> option that disables threads from suspending on the queue is optional.
mode	Not currently used.
queue_attributes	Attributes that specify the dimensions of the message queue, as defined by these structure members: <i>mq_maxmsg</i> <i>mq_msgsize</i> Where <i>mq_maxmsg</i> defines the total number of messages the queue can hold. The maximum size of each message (in bytes) is

defined by *mq_msgsize*. Note the size and priority of each message must be stored along with the message content. In addition, there is one pointer type required for each message. In most architectures, this amounts to 12 bytes of additional-per-message overhead.

`extend_attributes` Optional extended attributes that can specify memory for the queue control structure and queue memory area.

Return Codes:

queue handle

Positive value represents the successfully opened (created) queue handle.

PX5_ERROR (-1)

Error attempting to open/create the queue. Please use *errno* to retrieve the exact error:

ENOSPC	Not enough memory to create the specified queue.
EINVAL	Invalid operation or invalid attributes pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(188 + PX5_BINDING_STACK_FRAME_SIZE)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

mq_open, mq_close, mq_getattr, mq_setattr

Small Example:

```
#include <fcntl.h>
#include <sys/stat.h>
#include <mqueue.h>

int                status;
struct mq_attr     my_queue_attributes;
mq_extendattr_t    my_extended_queue_attributes;
mqd_t              my_queue_handle;

/* Set up the attributes for 100 total messages, where each
   message is a maximum of 16 bytes. */
my_queue_attributes.mq_maxmsg = 100;
my_queue_attributes.mq_msgsize = 16;

/* Open (create) the queue "my_queue". */
my_queue_handle = px5_mq_extend_open("my queue",
                                     (O_CREAT | O_RDWR), 0,
                                     &my_queue_attributes,
                                     &my_extended_queue_attribute);

/* If "my_queue_handle" is positive, the queue was successfully
   created. */
```

px5_mq_extendattr_destroy

C Prototype:

```
#include <mqueue.h>

int px5_mq_extendattr_destroy(mq_extendattr_t * queue_attributes);
```

Description:

This pthreads+ service destroys the previously created extended message queue attributes structure pointed to by *queue_attributes*. Once destroyed, the extended message queue attributes structure cannot be used again unless it is recreated.

API Parameters:

<i>queue_attributes</i>	Pointer to the extended message queue attributes to destroy.
-------------------------	--

Return Codes:

PX5_SUCCESS (0)	Successful extended message queue attributes destroy.
EINVAL	Invalid extended message queue attributes pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(36 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_mq_extend_open, px5_mq_extendattr_init, px5_mq_extendattr_**

Small Example:

```
#include <mqueue.h>

/* Message attribute structure. */
mq_extendattr_t    my_queue_attributes;
int                status;

/* Destroy the extended message queue attributes referenced by
   "my_queue_attributes". */
status = px5_mq_extendattr_destroy(&my_queue_attributes);

/* If status is PX5_SUCCESS, the extended message queue attributes
   structure was destroyed. */
```

px5_mq_extendattr_getcontroladdr

C Prototype:

```
#include <mqueue.h>

int px5_mq_extendattr_getcontroladdr(mq_extendattr_t* queue_attributes,
                                     void ** queue_control_address);
```

Description:

This pthreads+ service returns the previously supplied message queue control structure address.

API Parameters:

<code>queue_attributes</code>	Pointer to the extended message queue attributes.
<code>queue_control_address</code>	Pointer to the destination for the previously supplied message queue control address.

Return Codes:

PX5_SUCCESS (0)	Successful message queue control address retrieval.
EINVAL	Invalid extended message queue attributes or message queue control address destination pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(44 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_mq_extend_open, px5_mq_extendattr_init, px5_mq_extendattr_**

Small Example:

```
#include <mqueue.h>

/* Message queue attribute structure. */
mq_extendattr_t      my_queue_attributes;
void *               my_queue_control_address;
int                  status;

/* Get the message queue control structure address in the
   extended message queue attributes structure
   "my_queue_attributes". */
status = px5_mq_extendattr_getcontroladdr(&my_queue_attributes,
                                           &my_queue_control_address);

/* If status is PX5_SUCCESS, "my_queue_control_address"
   contains the address of the previously supplied message queue
   control memory. */
```


px5_mq_extendattr_getcontrolsize

C Prototype:

```
#include <mqueue.h>

int px5_mq_extendattr_getcontrolsize(mq_extendattr_t* queue_attributes,
                                     size_t * queue_control_size);
```

Description:

This pthreads+ service returns the size of the internal message queue control structure. The primary purpose of this API is to inform the application how much memory is required for the *px5_mq_extendattr_setcontroladdr* API.

API Parameters:

<code>queue_attributes</code>	Pointer to the attributes.
<code>queue_control_size</code>	Pointer to the destination for the internal message queue control structure size.

Return Codes:

PX5_SUCCESS (0)	Successful retrieval of internal message queue control structure size.
EINVAL	Invalid extended message queue attributes or invalid destination for message queue control structure size.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(44 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

px5_mq_extend_open, *px5_mq_extendattr_init*,
px5_mq_extendattr_setcontroladdr, *px5_mq_extendattr_**

Small Example:

```
#include <mqueue.h>

/* Message queue extended attribute structure. */
mq_extendattr_t    my_queue_attributes;
size_t             my_queue_control_size;
int                 status;

/* Get the internal message queue control structure memory size. */
status = px5_mq_extendattr_getcontrolsize(&my_queue_attributes,
                                           &my_queue_control_size);

/* If status is PX5_SUCCESS, "my_queue_control_size"
   contains the size of the internal message queue control
   structure. */
```

px5_mq_extendattr_getqueueaddr

C Prototype:

```
#include <mqueue.h>

int px5_mq_extendattr_getqueueaddr(mq_extendattr_t* queue_attributes,
                                   void ** queue_memory_address);
```

Description:

This pthreads+ service returns the previously supplied message queue memory address.

API Parameters:

<code>queue_attributes</code>	Pointer to the extended message queue attributes.
<code>queue_memory_address</code>	Pointer to the destination for the previously supplied message queue memory address.

Return Codes:

PX5_SUCCESS (0)	Successful message queue memory address retrieval.
EINVAL	Invalid extended message queue attributes or message queue memory address destination pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(44 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_mq_extend_open, px5_mq_extendattr_init,
px5_mq_extendattr_setqueueaddr, px5_mq_extendattr_**

Small Example:

```
#include <mqueue.h>

/* Message queue attribute structure. */
mq_extendattr_t    my_queue_attributes;
void *             my_queue_memory_address;
int                status;

/* Get the message queue memory address in the
   extended message queue attributes structure
   "my_queue_attributes". */
status = px5_mq_extendattr_getqueueaddr(&my_queue_attributes,
                                         &my_queue_memory_address);

/* If status is PX5_SUCCESS, "my_queue_memory_address"
   contains the address of the previously supplied message queue
   memory. */
```

px5_mq_extendattr_getqueuesize

C Prototype:

```
#include <mqueue.h>

int px5_mq_extendattr_getqueuesize(mq_extendattr_t* queue_attributes,
                                   size_t * queue_memory_size);
```

Description:

This pthreads+ service returns the size of the previously supplied message queue memory.

API Parameters:

queue_attributes	Pointer to the attributes.
queue_memory_size	Pointer to the destination for the previously supplied message queue memory size.

Return Codes:

PX5_SUCCESS (0)	Successful retrieval of message queue memory size.
EINVAL	Invalid extended message queue attributes or invalid destination for message queue memory size.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(44 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

px5_mq_extend_open, *px5_mq_extendattr_init*,
px5_mq_extendattr_setqueueaddr, *px5_mq_extendattr_**

Small Example:

```
#include <mqueue.h>

/* Message queue extended attribute structure. */
mq_extendattr_t    my_queue_attributes;
size_t             my_queue_memory_size;
int                 status;

/* Get the message queue memory size. */
status = px5_mq_extendattr_getqueuesize(&my_queue_attributes,
                                         &my_queue_memory_size);

/* If status is PX5_SUCCESS, "my_queue_memory_size"
   contains the size of the message queue memory. */
```

px5_mq_extendattr_init

C Prototype:

```
#include <mqueue.h>

int px5_mq_extendattr_init(mq_extendattr_t * queue_attributes);
```

Description:

This pthreads+ service initializes the extended message queue attributes structure with default condition variable creation values. Note that the extended message queue attributes are used only by the *px5_mq_extend_open* API.

API Parameters:

<code>queue_attributes</code>	Pointer to the extended message queue attributes structure to create.
-------------------------------	---

Return Codes:

PX5_SUCCESS (0)	Successful extended message queue attributes structure creation.
EINVAL	Invalid extended message queue attributes pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(36 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_mq_extend_open, px5_mq_extendattr_destroy, px5_mq_extendattr_**

Small Example:

```
#include <mqueue.h>

/* Extended message queue attribute structure. */
mq_extendattr_t    my_queue_attributes;
int                status;

/* Create the extended message queue attributes structure
   "my_queue_attributes". */
status = px5_mq_extendattr_init(&my_queue_attributes);

/* If status is PX5_SUCCESS, the "my_queue_attributes"
   structure is ready for use. */
```


px5_mq_extendattr_setcontroladdr

C Prototype:

```
#include <mqueue.h>

int px5_mq_extendattr_setcontroladdr(mq_extendattr_t* queue_attributes,
                                     void * queue_control_address,
                                     size_t queue_memory_size);
```

Description:

This pthreads+ service enables the developer to provide the memory for the internal PX5 RTOS message queue structure as specified by the address in the *queue_control_address* parameter. This memory will subsequently be used for the next message queue created with this attribute structure. The memory size required for the internal message queue control structure can be found via a call to the *px5_mq_extendattr_getcontrol_size* service.



Note that each message queue created must have its own unique message queue control structure memory. Hence, the message queue control memory supplied here is only valid for one px5_mq_extend_open call.



The exact amount of memory required for the internal queue control structure can be determined by including *px5.h* and using the *sizeof(px5_queue_control)* construct.

API Parameters:

<code>queue_attributes</code>	Pointer to the message queue attributes.
<code>queue_control_address</code>	Pointer to the internal message queue control structure memory.
<code>queue_control_size</code>	Size of the memory specified.

Return Codes:

PX5_SUCCESS (0)	Successful specification of message queue structure memory.
EINVAL	Invalid message queue attributes or invalid size of message queue control memory.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:

From the location where this API is called, there should be at least **(44 + PX5_BINDING_STACK_FRAME_SIZE)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

px5_mq_extend_open, px5_mq_extendattr_getcontrolsiz,
*px5_mq_extendattr_**

Small Example:

```
#include <mqueue.h>

/* Message queue attribute structure. */
mq_extendattr_t    my_queue_attributes;
int                status;

/* Set the message queue control structure memory address in the
   extended message queue attributes
   structure "my_queue_attributes". */
status = px5_mq_extendattr_setcontroladdr(&my_queue_attributes,
                                           0x10000, 100);
```

px5_mq_extendattr_setqueueaddr

C Prototype:

```
#include <mqueue.h>

int px5_mq_extendattr_setqueueaddr(mq_extendattr_t* queue_attributes,
                                   void * queue_memory_address,
                                   size_t queue_memory_size);
```

Description:

This pthreads+ service sets the internal message queue memory address to the address specified by *queue_memory_address*. This address will subsequently be used to supply the memory for the internal message queue memory on the next queue created with this attribute structure.



Note that each message queue created must have its own unique queue memory area. Hence, the message queue memory address supplied here is only valid for one px5_mq_extend_open call.

API Parameters:

<code>queue_attributes</code>	Pointer to the message queue attributes.
<code>queue_memory_address</code>	Pointer to the message queue memory address.
<code>queue_memory_size</code>	Size of specified message queue memory.

Return Codes:

PX5_SUCCESS (0)	Successful specification of message queue memory.
EINVAL	Invalid message queue attributes or invalid size of message queue memory.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(44 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_mq_extend_open, px5_mq_extendattr_init, px5_mq_extendattr_**

Small Example:

```
#include <mqueue.h>

/* Message queue attribute structure. */
mq_extendattr_t    my_queue_attributes;
int                status;

/* Set the message queue memory address in the
   extended message queue attributes
   structure "my_queue_attributes". */
status = px5_mq_extendattr_setqueueaddr(&my_queue_attributes,
                                         0x90000, 1024);
```

px5_mq_information_get

C Prototype:

```
#include <mqueue.h>

int px5_mq_information_get(mqd_t * queue_handle, char ** name,
    u_long * messages_queued, u_long * maximum_messages,
    u_long * maximum_message_size,
    u_long * threads_suspended, pthread_t *first_suspended_thread,
    mqd_t * next_queue_handle);
```

Description:

This pthreads+ service retrieves the specified information from the specified message queue.

API Parameters:

queue_handle	Handle of message queue to get information about.
name	If non-NULL, destination for the specified message queue's name.
messages_queued	If non-NULL, destination for the number of messages currently in the message queue.
maximum_messages	If non-NULL, destination for the maximum number of messages that can be queued.
maximum_message_size	If non-NULL, destination for the maximum message size for this queue.
threads_suspended	If non-NULL, destination for the number of threads currently suspended on this queue.
first_suspended_thread	If non-NULL, destination for the thread handle of the first thread suspended on the queue.
next_queue_handle	If non-NULL, destination for the handle of the next created message queue. This API can be called successively to examine all message queues using this parameter until the original message queue handle is detected.

Return Codes:

PX5_SUCCESS (0)	Successful information retrieval from the specified message queue.
ESRCH	Invalid message queue handle.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:

From the location where this API is called, there should be at least **(78 + PX5_BINDING_STACK_FRAME_SIZE)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable from the thread context and interrupt handlers (ISRs).

See Also:

*mq_open, px5_*_information_get*

Small Example:

```
#include <mqueue.h>

mqd_t      my_queue_handle;
char *     name;
u_long     messages_queued;
u_long     maximum_messages;
u_long     maximum_message_size;
u_long     threads_suspended;
pthread_t  first_suspended_thread;
mqd_t      next_queue;
int        status;

/* Get information about "my_queue_handle". */
status = px5_mq_information_get(&my_queue_handle,
                                &name, &messages_queued, &maximum_messages,
                                &maximum_message_size,
                                &threads_suspended, &first_suspended_thread,
                                &next_queue);

/* If status contains PX5_SUCCESS, the information about
   "my_queue_handle" is available for use. */
```


px5_mq_send_notify

C Prototype:

```
#include <mqueue.h>

int px5_mq_send_notify(mqd_t queue_handle,
                      void (*send_callback)(mqd_t));
```

Description:

This service registers an application callback that is executed whenever a message is sent to the specified message queue. If a NULL is supplied, notification for the specified message queue is disabled.



By default, application message queue notification is not enabled. To enable notification, PX5 RTOS and the application should be built with `PX5_NOTIFICATION_ENABLE` defined.

API Parameters:

<code>queue_handle</code>	Message queue handle.
<code>send_callback</code>	If non-NULL, application's notification function pointer to call whenever a message is sent to this queue.

Return Codes:

PX5_SUCCESS (0)	Successful registration of the notification callback.
ESRCH	Invalid message queue.
EACCES	Invalid call from ISR or non-thread. This error code is also returned if PX5 RTOS was not built with <code>PX5_NOTIFICATION_ENABLE</code> defined.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. No preemption takes place as a result of this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(8 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

px5_pthread_event_flags_set_notify, *px5_pthread_fastqueue_send_notify*, *px5_sem_post_notify*

Small Example:

```
#include <mqueue.h>

int      status;
mqd_t    my_queue;

void my_notification_callback(mqd_t queue_handle)
{
    /* Application processing goes here!  */
}

...

/* Register "my_notification_callback" to "my_queue". */
status = px5_mq_send_notify(my_queue, my_notification_callback);

/* If status is PX5_SUCCESS, the callback has been registered. */
```

px5_pthread_attr_getaffinity

C Prototype:

```
#include <pthread.h>

int  px5_pthread_attr_getaffinity(pthread_attr_t *attributes,
                                   size_t  cpusetsize, cpu_set_t *  core_affinity);
```

Description:

When the PX5 RTOS is configured for SMP (*PX5_MAXIMUM_CORES* greater than one), this pthreads+ service returns the core affinity specified in the supplied attribute structure. By default, the thread core affinity is set to the constant *PX5_DEFAULT_CORE_AFFINITY* when the attribute structure is initialized. If *PX5_MAXIMUM_CORES* equals one, this service is irrelevant and not defined.

API Parameters:

<code>attributes</code>	Pointer to a previously initialized attributes structure.
<code>cpusetsize</code>	Number of bytes in the <i>cpu_set_t</i> data type. This should always be <i>sizeof(cpu_set_t)</i> .
<code>core_affinity</code>	Pointer to the destination where the core affinity of this attribute structure is returned.

Return Codes:

PX5_SUCCESS (0)	Successful retrieval of the thread core affinity.
EINVAL	Attributes structure pointer or core affinity destination pointer is invalid.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(44 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*pthread_attr_**, *pthread_attr_setaffinity*

Small Example:

```
#include <pthread.h>

pthread_attr_t    my_thread_attributes;
cpu_set_t         my_core_affinity;
int               status;

/* Get the core affinity associated with this set of
   attributes. */
status =      px5_pthread_attr_getaffinity(&my_thread_attributes,
                                           sizeof(my_core_affinity), &my_core_affinity);

/* If status contains PX5_SUCCESS, the attribute core affinity
   is contained in "my_core_affinity". */
```

px5_pthread_attr_getautostart

C Prototype:

```
#include <pthread.h>

int  px5_pthread_attr_getautostart(pthread_attr_t *attributes,
                                   int *  automatic_start);
```

Description:

This pthreads+ service returns the automatic start option in the supplied attribute structure. By default, the thread automatic start option is set to *PX5_AUTO_START*, which means the thread will start automatically after it is created.

API Parameters:

<code>attributes</code>	Pointer to a previously initialized attributes structure.
<code>automatic_start</code>	Pointer to the destination where the automatic start option of this attribute structure is returned. The two valid values are <i>PX5_AUTO_START</i> and <i>PX5_DONT_START</i> .

Return Codes:

PX5_SUCCESS (0)	Successful retrieval of the thread automatic start option.
EINVAL	Attributes structure pointer or automatic start destination pointer is invalid.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(44 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*pthread_attr_**, *pthread_attr_setautostart*

Small Example:

```
#include <pthread.h>

pthread_attr_t    my_thread_attributes;
cpu_set_t        my_auto_start;
int              status;

/* Get the automatic start option associated with this set of
   attributes. */
status =      px5_pthread_attr_getautostart(&my_thread_attributes,
                                           &my_auto_start);

/* If status contains PX5_SUCCESS, the attribute automatic start
   option is contained in "my_auto_start." */
```

px5_pthread_attr_getcontroladdr

C Prototype:

```
#include <pthread.h>

int  px5_pthread_attr_getcontroladdr(pthread_attr_t *attributes,
                                     void ** thread_control_address);
```

Description:

This pthreads+ service returns the thread control structure address stored in the thread attributes structure. By default, this value is NULL unless specified by the application via a call to *px5_pthread_attr_setcontroladdr*.

API Parameters:

<code>attributes</code>	Pointer to a previously initialized attributes structure.
<code>thread_control_address</code>	Pointer to the destination of where to return the thread control address of this attributes structure.

Return Codes:

PX5_SUCCESS (0)	Successful retrieval of the thread control address.
EINVAL	Attributes structure pointer or thread control address destination pointer is invalid.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(44 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*pthread_attr_**, *pthread_attr_setcontroladdr*

Small Example:

```
#include <pthread.h>

pthread_attr_t    my_thread_attributes;
void *            thread_control_address;
int               status;

/* Get the thread control address in the previously initialized
   attributes. */
status =          px5_pthread_attr_getcontroladdr(&my_thread_attributes,
                                                  &thread_control_address);

/* If status contains PX5_SUCCESS, the thread control address value in
   this attribute set is contained in "thread_control_address". */
```


px5_pthread_attr_getcontrolsize

C Prototype:

```
#include <pthread.h>

int  px5_pthread_attr_getcontrolsize(pthread_attr_t *attributes,
                                     size_t *   thread_control_size);
```

Description:

This pthreads+ service returns the size of the internal thread control structure. The main purpose of this API is to inform the application how much memory is required for the *px5_pthread_attr_setcontroladdr* API.

API Parameters:

<code>attributes</code>	Pointer to a previously initialized attributes structure.
<code>thread_control_size</code>	Pointer to the destination of where to return the internal thread control structure size.

Return Codes:

PX5_SUCCESS (0)	Successful retrieval of the internal thread control size.
EINVAL	Attributes pointer or internal thread control size destination pointer is invalid.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(44 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*pthread_attr_**, *pthread_attr_getcontroladdr*, *pthread_attr_setcontroladdr*

Small Example:

```
#include <pthread.h>

pthread_attr_t    my_thread_attributes;
size_t           thread_control_size;
int               status;

/* Get the internal thread control size. */
status = px5_pthread_attr_getcontrolsize(&my_thread_attributes,
                                         &thread_control_size);

/* If status contains PX5_SUCCESS, the internal thread control
   structure size is found in "thread_control_size". */
```

px5_pthread_attr_getname

C Prototype:

```
#include <pthread.h>

int  px5_pthread_attr_getname(pthread_attr_t *attributes,
                               char **      return_name);
```

Description:

This pthreads+ service returns the current thread name in the thread attributes structure.

API Parameters:

attributes	Pointer to a previously initialized attributes structure.
return_name	Pointer to the destination of where to return the name in this attributes structure.

Return Codes:

PX5_SUCCESS (0)	Successful retrieval of the thread name.
EINVAL	Attributes structure pointer or name destination pointer is invalid.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(44 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*pthread_attr_**, *pthread_attr_setname*

Small Example:

```
#include <pthread.h>

pthread_attr_t    my_thread_attributes;
char *            my_name;
int               status;

/* Get the thread name in the previously initialized
   attributes. */
status = px5_pthread_attr_getname(&my_thread_attributes,
                                   &my_name);

/* If status contains PX5_SUCCESS, the thread name in this
   attribute set is stored in "my_name". */
```

px5_pthread_attr_getpriority

C Prototype:

```
#include <pthread.h>

int px5_pthread_attr_getpriority(pthread_attr_t *attributes,
                                int *priority);
```

Description:

This pthreads+ service returns the current thread priority in the thread attributes structure. By default, this value is *PX5_DEFAULT_PRIORITY* if the application does not specify it via a call to *px5_pthread_attr_setpriority*.

API Parameters:

<code>attributes</code>	Pointer to a previously initialized attributes structure.
<code>priority</code>	Pointer to the destination of where to return the priority of this attributes structure.

Return Codes:

PX5_SUCCESS (0)	Successful retrieval of the thread priority.
EINVAL	Attributes structure pointer or priority destination pointer is invalid.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(44 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

pthread_attr_t, *px5_pthread_attr_setpriority*

Small Example:

```
#include <pthread.h>

pthread_attr_t    my_thread_attributes;
int               priority;
int               status;

/* Get the priority in the previously initialized
   attributes. */
status = px5_pthread_attr_getpriority(&my_thread_attributes,
                                     &priority);

/* If status contains PX5_SUCCESS, the priority in this
   attribute set is stored in "priority". */
```

px5_pthread_attr_gettimeslice

C Prototype:

```
#include <pthread.h>

int  px5_pthread_attr_gettimeslice(pthread_attr_t *attributes,
                                   u_long *   thread_time_slice);
```

Description:

This pthreads+ service returns the current thread time-slice stored in the thread attributes structure. By default, this value is 0, which means time-slicing is disabled. The time-slice value for thread creation can be changed via the *px5_pthread_attr_settimeslice* service.



*Each thread may have its own unique time-slice value through the specific invocation of *px5_pthread_attr_settimeslice*.*

API Parameters:

<code>attributes</code>	Pointer to a previously initialized attributes structure.
<code>thread_time_slice</code>	Pointer to the destination of where to return the thread time-slice value of this attributes structure.

Return Codes:

PX5_SUCCESS (0)	Successful retrieval of the thread per time-slice.
EINVAL	Attributes structure pointer or thread time-slice destination pointer is invalid.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(44 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*pthread_attr_**, *pthread_attr_settimeslice*

Small Example:

```
#include <pthread.h>

pthread_attr_t    my_thread_attributes;
u_long           thread_time_slice;
int               status;

/* Get the thread time-slice in the previously initialized
   set of attributes. */
status = px5_pthread_attr_gettimeslice(&my_thread_attributes,
                                       &thread_time_slice);

/* If status contains PX5_SUCCESS, the thread time-slice value in this
   attribute set is stored in "thread_time_slice". */
```


px5_pthread_attr_setaffinity

C Prototype:

```
#include <pthread.h>

int  px5_pthread_attr_setaffinity(pthread_attr_t *  attributes,
                                   size_t  cpusetsize, cpu_set_t *  core_affinity);
```

Description:

When the PX5 RTOS is configured for SMP (*PX5_MAXIMUM_CORES* greater than one), this pthreads+ service sets the core affinity in the supplied attribute structure. When the attribute structure is initialized by default, the thread core affinity is set to the constant *PX5_DEFAULT_CORE_AFFINITY*. If *PX5_MAXIMUM_CORES* equals one, this service is irrelevant and not defined.

API Parameters:

<code>attributes</code>	Pointer to a previously initialized attributes structure.
<code>cpusetsize</code>	Number of bytes in the <i>cpu_set_t</i> data type. This should always be <i>sizeof(cpu_set_t)</i> .
<code>core_affinity</code>	Pointer to the source of the new core affinity for this attribute structure.

Return Codes:

PX5_SUCCESS (0)	Successful setup of the thread core affinity.
EINVAL	Attributes structure pointer or core affinity is invalid.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(44 + PX5_BINDING_STACK_FRAME_SIZE)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

pthread_attr_, pthread_attr_getaffinity*

Small Example:

```
#include <pthread.h>

pthread_attr_t    my_thread_attributes;
cpu_set_t         my_core_affinity;
int               status;

/* Designate that threads created with this attribute structure may
   execute on core 0 and core 1. */
my_core_affinity = 0x3; /* Bit 0 -> core 0, Bit 1 -> core 1 */

/* Set the core affinity. */
status = px5_pthread_attr_setaffinity(&my_thread_attributes,
                                       sizeof(my_core_affinity), &my_core_affinity);

/* If status contains PX5_SUCCESS, threads created with this attribute
   structure will execute on core 0 or core 1. */
```

px5_pthread_attr_setautostart

C Prototype:

```
#include <pthread.h>

int  px5_pthread_attr_setautostart(pthread_attr_t *attributes,
                                   int automatic_start);
```

Description:

This pthreads+ service sets the automatic start option in the supplied attribute structure. By default, the thread automatic start option is set to *PX5_AUTO_START*, which means the thread will start automatically after it is created. By supplying *PX5_DONT_START* to this API, threads subsequently created with this attribute structure will be created in a suspended state and not execute until *px5_pthread_resume* is called.

API Parameters:

<code>attributes</code>	Pointer to a previously initialized attributes structure.
<code>automatic_start</code>	Automatic start option, which is either <i>PX5_AUTO_START</i> or <i>PX5_DONT_START</i> .

Return Codes:

PX5_SUCCESS (0)	Successful set up of the thread automatic start option.
EINVAL	Attributes structure pointer or automatic start option is invalid.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(44 + PX5_BINDING_STACK_FRAME_SIZE)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

pthread_attr_, pthread_attr_getautostart, px5_pthread_resume*

Small Example:

```
#include <pthread.h>

pthread_attr_t    my_thread_attributes;
int              status;

/* Set the automatic start option associated with this set of
   attributes to create threads in a suspended state, which will
   require a call to px5_pthread_resume for them to execute. */
status =    px5_pthread_attr_setautostart(&my_thread_attributes,
                                           PX5_DONT_START);

/* If status contains PX5_SUCCESS, the attribute automatic start
   option has been set up. */
```

px5_pthread_attr_setcontroladdr

C Prototype:

```
#include <pthread.h>

int  px5_pthread_attr_setcontroladdr(pthread_attr_t *attributes,
                                     void *      thread_control_address,
                                     size_t      thread_control_size);
```

Description:

This pthreads+ service enables the developer to provide the memory for the internal PX5 RTOS thread control structure, as specified by the address in the *thread_control_address* parameter. This memory will subsequently be used for the next thread created with this attribute structure. The memory size required for the internal thread control structure can be found via a call to the *px5_pthread_attr_getcontrolsize* service.



Note that each thread created must have its own unique thread control structure memory. Hence, the thread control memory supplied here is only valid for one pthread_create call.



The exact amount of memory required for the internal thread control structure can be determined by including *px5.h* and using the *sizeof(px5_thread_control)* construct.

API Parameters:

<code>attributes</code>	Pointer to a previously initialized attributes structure.
<code>thread_control_address</code>	Thread control address to use for the next thread creation.
<code>thread_control_size</code>	Size of the memory specified.

Return Codes:

PX5_SUCCESS (0)	Successful setting of the thread control address.
EINVAL	Attributes pointer or thread control address pointer or size is invalid.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(44 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

pthread_attr_t, *px5_pthread_attr_getcontroladdr*,
px5_pthread_attr_getcontrolsize

Small Example:

```
#include <pthread.h>

pthread_attr_t    my_thread_attributes;
int               status;

/* Set the thread control address to the absolute address of
   0x20000 in the previously initialized attributes. */
status = px5_pthread_attr_setcontroladdr(&my_thread_attributes,
                                         (void *) 0x20000, 300);

/* If status contains PX5_SUCCESS, the thread control address in this
   attribute structure is 0x20000. */
```

px5_pthread_attr_setname

C Prototype:

```
#include <pthread.h>

int  px5_pthread_attr_setname(pthread_attr_t *attributes,
                               char *      name);
```

Description:

This pthreads+ service stores the specified thread name (ASCII string) in the thread attributes structure.

API Parameters:

<code>attributes</code>	Pointer to a previously initialized attributes structure.
<code>name</code>	Thread name in ASCII string format.

Return Codes:

PX5_SUCCESS (0)	Successful setting of the thread name.
EINVAL	Attributes pointer or specified thread name pointer is invalid.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(44 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*pthread_attr_**, *pthread_attr_getname*

Small Example:

```
#include <pthread.h>

pthread_attr_t    my_thread_attributes;
char              my_name[] = "my thread name";
int              status;

/* Set the thread name to "my thread name" in the previously
   initialized attributes. */
status = px5_pthread_attr_setname(&my_thread_attributes,
                                   my_name);

/* If status contains PX5_SUCCESS, the attributes structure now
   has the name of "my thread name". */
```


px5_pthread_attr_setpriority

C Prototype:

```
#include <pthread.h>

int px5_pthread_attr_setpriority(pthread_attr_t *attributes,
                                int priority);
```

Description:

This pthreads+ service stores the specified priority in the thread attributes structure. The thread priority must be between 0 and (`PX5_MAXIMUM_PRIORITIES – 1`), where larger numbers represent higher priority.

API Parameters:

<code>attributes</code>	Pointer to a previously initialized attributes structure.
<code>priority</code>	Priority to store in the attributes structure.

Return Codes:

PX5_SUCCESS (0)	Successful setting of the priority.
EINVAL	Attributes pointer or specified priority is invalid.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(44 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

pthread_attr_t, *px5_pthread_attr_getpriority*

Small Example:

```
#include <pthread.h>

pthread_attr_t    my_thread_attributes;
int               status;

/* Set the priority to 31 in the previously initialized
   attributes. */
status = px5_pthread_attr_setpriority(&my_thread_attributes, 31);

/* If status contains PX5_SUCCESS, the priority in the
   attributes structure is now 31. */
```

px5_pthread_attr_settimeslice

C Prototype:

```
#include <pthread.h>

int px5_pthread_attr_settimeslice(pthread_attr_t *attributes,
                                   u_long time_slice);
```

Description:

This pthreads+ service stores the specified time-slice in the thread attributes structure. A value of 0 turns off time-slicing (default), while positive values represent the number of timer ticks the thread can execute before giving other threads of the same priority a chance to execute.

API Parameters:

<code>attributes</code>	Pointer to a previously initialized attributes structure.
<code>time_slice</code>	Time-slice to store in the attributes structure.

Return Codes:

PX5_SUCCESS (0)	Successful setting of the thread time-slice.
EINVAL	Attributes structure pointer is invalid.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(44 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

pthread_attr_t, *px5_pthread_attr_gettimeslice*

Small Example:

```
#include <pthread.h>

pthread_attr_t    my_thread_attributes;
int              status;

/* Set the thread time-slice to 2 in the previously initialized
   attributes structure. */
status = px5_pthread_attr_settimeslice(&my_thread_attributes, 2);

/* If status contains PX5_SUCCESS, the time-slice in the
   attributes structure is now 2. */
```

px5_pthread_cond_information_get

C Prototype:

```
#include <pthread.h>

int px5_pthread_cond_information_get(pthread_cond_t * cond_var_handle,
    char ** name, u_long * signal_count,
    u_long * threads_suspended, pthread_t *first_suspended_thread,
    pthread_cond_t * next_cond_var_handle);
```

Description:

This pthreads+ service retrieves the specified information from the specified condition variable.

API Parameters:

cond_var_handle	Handle of condition variable to get information about.
name	If non-NULL, destination for the specified condition variable's name.
signal_count	If non-NULL, destination for the number of times this condition variable has been signaled.
threads_suspended	If non-NULL, destination for the number of threads currently suspended on this condition variable.
first_suspended_thread	If non-NULL, destination for the thread handle of the first thread suspended on this condition variable.
next_cond_var_handle	If non-NULL, destination for the handle of the next created condition variable. This API can be called successively to examine all condition variables using this parameter until the original condition variable handle is detected.

Return Codes:

PX5_SUCCESS (0)	Successful information retrieval from the specified condition variable.
ESRCH	Invalid condition variable handle.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:

From the location where this API is called, there should be at least **(78 + PX5_BINDING_STACK_FRAME_SIZE)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable from the thread context and interrupt handlers (ISRs).

See Also:

*pthread_cond_init, px5_*_information_get*

Small Example:

```
#include <pthread.h>

pthread_cond_t    my_cond_var_handle;
char *            name;
u_long            signal_count;
u_long            threads_suspended;
pthread_t         first_suspended_thread;
pthread_cond_t    next_cond_var;
int               status;

/* Get information about "my_cond_var_handle". */
status = px5_pthread_cond_information_get(&my_cond_var_handle,
&name, &signal_count,
&threads_suspended, &first_suspended_thread,
&next_cond_var);

/* If status contains PX5_SUCCESS, the information about
"my_cond_var_handle" is available for use. */
```

px5_pthread_condattr_getcontroladdr

C Prototype:

```
#include <pthread.h>

int px5_pthread_condattr_getcontroladdr(pthread_condattr_t *
                                         condition_var_attributes,
                                         void ** condition_var_control_address);
```

Description:

This pthreads+ service returns the previously supplied condition variable control structure address.

API Parameters:

`condition_var_attributes` Pointer to the condition variable attributes.

`condition_var_control_address` Pointer to the destination for the previously supplied condition variable control address.

Return Codes:

PX5_SUCCESS (0)	Successful condition variable attributes condition control address retrieval.
EINVAL	Invalid condition variable attributes or condition variable control address destination pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(44 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*pthread_cond_**, *pthread_condattr_**, *px5_pthread_condattr_setcontroladdr*, *px5_pthread_condattr_getcontrolsize*

Small Example:

```
#include <pthread.h>

/* Condition variable attribute structure. */
pthread_condattr_t    my_cond_var_attributes;
void *                 my_cond_var_control_address;
int                   status;

/* Get the condition variable control structure address in the
   condition variable attributes structure
   "my_cond_var_attributes". */
status = px5_pthread_condattr_getcontroladdr(&my_cond_var_attributes,
                                              &my_cond_var_control_address);

/* If status is PX5_SUCCESS, "my_cond_var_control_address"
   contains the address of the previously supplied condition
   variable control memory. */
```

px5_pthread_condattr_getcontrolsiz

C Prototype:

```
#include <pthread.h>

int px5_pthread_condattr_getcontrolsiz(pthread_condattr_t
    * condition_var_attributes,
    size_t * condition_var_control_size);
```

Description:

This pthreads+ service returns the size of the internal condition variable control structure. The primary purpose of this API is to inform the application how much memory is required for the *px5_pthread_condattr_setcontroladdr* API.

API Parameters:

<code>condition_var_attributes</code>	Pointer to the condition variable attributes.
<code>condition_var_control_size</code>	Pointer to the destination for the internal condition variable control structure size.

Return Codes:

PX5_SUCCESS (0)	Successful retrieval of internal condition variable control structure size.
EINVAL	Invalid condition variable attributes or invalid destination for condition variable control structure size.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(44 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*pthread_cond_**, *pthread_condattr_**, *px5_pthread_condattr_setcontroladdr*, *px5_pthread_condattr_getcontroladdr*

Small Example:

```
#include <pthread.h>

/* Condition variable attribute structure. */
pthread_condattr_t    my_cond_var_attributes;
size_t                my_cond_var_control_size;
int                   status;

/* Get the condition variable control structure memory size. */
status = px5_pthread_condattr_getcontrolsize(
    &my_cond_var_attributes, &my_cond_var_control_size);

/* If status is PX5_SUCCESS, "my_cond_var_control_size"
   contains the size of the internal condition variable control
   structure. */
```

px5_pthread_condattr_getname

C Prototype:

```
#include <pthread.h>

int px5_pthread_condattr_getname(pthread_condattr_t
                                * condition_var_attributes,
                                char ** condition_var_name);
```

Description:

This pthreads+ service returns the previously supplied condition variable name.

API Parameters:

<code>condition_var_attributes</code>	Pointer to the condition variable attributes.
<code>Condition_var_name</code>	Pointer to the destination for the previously supplied condition variable name.

Return Codes:

PX5_SUCCESS (0)	Successful retrieval of last supplied condition variable name.
EINVAL	Invalid condition variable attributes or name destination pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(44 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*pthread_cond_**, *pthread_condattr_**, *px5_pthread_condattr_setname*

Small Example:

```
#include <pthread.h>

/* Condition variable attribute structure. */
pthread_condattr_t  my_cond_var_attributes;
char *              my_cond_var_name;
int                 status;

/* Get the last supplied condition variable name. */
status = px5_pthread_condattr_getname(&my_cond_var_attributes,
                                       &my_cond_var_name);

/* If status is PX5_SUCCESS, "my_cond_var_name" contains the
   name previously supplied. */
```

px5_pthread_condattr_setcontroladdr

C Prototype:

```
#include <pthread.h>

int px5_pthread_condattr_setcontroladdr(pthread_condattr_t
    * condition_var_attributes,
    void * condition_var_control_address,
    size_t condition_var_control_size);
```

Description:

This pthreads+ service enables the developer to provide the memory for the internal PX5 RTOS condition variable structure as specified by the address in the *condition_var_control_address* parameter. This memory will subsequently be used for the next condition variable created with this attribute structure. The memory size required for the internal condition variable control structure can be found via a call to the *px5_pthread_condattr_getcontrolsize* service.



Note that each condition variable created must have its own unique condition variable control structure memory. Hence, the condition variable control memory supplied here is only valid for one pthread_cond_init call.



The exact amount of memory required for the internal condition variable control structure can be determined by including *px5.h* and using the *sizeof(px5_condition_var_control)* construct.

API Parameters:

condition_var_attributes Pointer to the condition variable attributes.

condition_var_control_address Pointer to the supplied condition variable control structure memory.

condition_var_control_size Size of the specified memory.

Return Codes:

PX5_SUCCESS (0)	Successful specification of condition variable structure memory.
EINVAL	Invalid condition variable attributes or invalid size of condition variable control memory.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:

From the location where this API is called, there should be at least **(44 + PX5_BINDING_STACK_FRAME_SIZE)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*pthread_cond_**, *pthread_condattr_**, *pthread_condattr_getcontroladdr*, *pthread_condattr_getcontrolsize*

Small Example:

```
#include <pthread.h>

/* Condition variable attribute structure. */
pthread_condattr_t  my_cond_var_attributes;
int                 status;

/* Set the condition variable control structure address in the
   condition variable attributes structure
   "my_cond_var_attributes". */
status = pthread_condattr_setcontroladdr(&my_cond_var_attributes,
                                         0x30000, 60);

/* If status is PX5_SUCCESS, the condition variable creation
   using these attributes will use address 0x30000 for the internal
   condition variable control structure. */
```


px5_pthread_condattr_setname

C Prototype:

```
#include <pthread.h>

int px5_pthread_condattr_setname(pthread_condattr_t
                                * condition_var_attributes,
                                char * cond_var_name);
```

Description:

This pthreads+ service sets the condition variable name in the specified attribute structure.

API Parameters:

<code>condition_var_attributes</code>	Pointer to the condition variable attributes.
<code>condition_var_name</code>	Pointer to the supplied condition variable name.

Return Codes:

PX5_SUCCESS (0)	Successful condition variable name set.
EINVAL	Invalid condition variable attributes.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(44 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

pthread_cond_, pthread_condattr_*, px5_pthread_condattr_getname*

Small Example:

```
#include <pthread.h>

/* Condition variable attribute structure. */
pthread_condattr_t  my_cond_var_attributes;
int                 status;

/* Set the condition variable name in the condition variable
   attributes structure "my_cond_var_attributes". */
status = px5_pthread_condattr_setname(&my_cond_var_attributes,
                                       "my_cond_var_name");

/* If status is PX5_SUCCESS, "my_cond_var_name" is set in the
   condition variable attribute structure. */
```

px5_pthread_event_flags_clear

C Prototype:

```
#include <pthread.h>

int px5_pthread_event_flags_clear(pthread_event_flags_t *
                                   event_flags_handle);
```

Description:

This pthreads+ service clears all event flags in the specified event flags group.

API Parameters:

event_flags_handle Handle of the event flags group.

Return Codes:

PX5_SUCCESS (0)	Successful event flags clear.
EINVAL	Invalid event flags handle pointer or any/all events option.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. No preemption is possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(32 + PX5_BINDING_STACK_FRAME_SIZE)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable from the thread context and interrupt handlers (ISRs).

See Also:

*px5_pthread_event_flags_**, *px5_pthread_eventflags_clearspecific*,
px5_pthread_event_flags_set

Small Example:

```
#include <pthread.h>

/* Event flags handle. */
pthread_event_flags_t my_event_flags_handle;
int status;

/* Clear all event flags in the event flags group
   "my_event_flags_handle". */
status = px5_pthread_event_flags_clear(&my_event_flags_handle);

/* If status is PX5_SUCCESS, all event flags are cleared. */
```

px5_pthread_event_flags_clearspecific

C Prototype:

```
#include <pthread.h>

int px5_pthread_event_flags_clearspecific(
    pthread_event_flags_t * event_flags_handle,
    u_long flags_to_clear);
```

Description:

This pthreads+ service clears the specified event flags in the event flags group.

API Parameters:

event_flags_handle Handle of the event flags group.

flags_to_clear Specific event flags to clear.

Return Codes:

PX5_SUCCESS (0)	Successful specific event flags clear.
EINVAL	Invalid event flags handle pointer or any/all events option.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. No preemption is possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(32 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable from the thread context and interrupt handlers (ISRs).

See Also:

*px5_pthread_event_flags_**, *px5_pthread_event_flags_clear*

Small Example:

```
#include <pthread.h>

/* Event flags handle. */
pthread_event_flags_t my_event_flags_handle;
int status;

/* Clear event flags 0 and 2, the event flags group
"my_event_flags_handle". */
status = px5_pthread_event_flags_clearspecific(
    &my_event_flags_handle, 0x5);

/* If status is PX5_SUCCESS, event flags 0 and 2 are cleared. */
```

px5_pthread_event_flags_create

C Prototype:

```
#include <pthread.h>

int px5_pthread_event_flags_create(pthread_event_flags_t *
                                   event_flags_handle,
                                   pthread_event_flagsattr_t * event_flags_attributes);
```

Description:

This pthreads+ service initializes (creates) an event flags group with the optional event flags attributes. If successful, the event flags handle is set up for further use by the application.

API Parameters:

event_flags_handle	Handle of the event flags to create.
event_flags_attributes	Optional event flags attributes. This value is NULL if no event flags attributes are specified.

Return Codes:

PX5_SUCCESS (0)	Successful event flags initialization.
EINVAL	Invalid event flags handle pointer or event flags attributes.
EBUSY	Event flags group is already created.
ENOMEM	Insufficient memory to create event flags.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(124 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_pthread_event_flagsattr_**, *px5_pthread_event_flags_destroy*

Small Example:

```
#include <pthread.h>

/* Event flags handle. */
pthread_event_flags_t my_event_flags_handle;
int status;

/* Create the event flags group, the handle of which is returned in
"my_event_flags_handle". */
status = px5_pthread_event_flags_create(&my_event_flags_handle,
                                         NULL);

/* If status is PX5_SUCCESS, the event flags group was created, and
the event flags handle is ready to use. */
```


px5_pthread_event_flags_destroy

C Prototype:

```
#include <pthread.h>

int px5_pthread_event_flags_destroy(pthread_event_flags_t *
                                   event_flags_handle);
```

Description:

This pthreads+ service destroys the previously created event flags group specified by *event_flags_handle*. An error is returned if the event flags group has any threads waiting for events.



To avoid unpredictable behavior, the application must ensure there is no further access to the event flags group after it is destroyed.

API Parameters:

`event_flags_handle` Handle of the event flags to destroy.

Return Codes:

PX5_SUCCESS (0)	Successful event flags group destroy.
EINVAL	Invalid event flags handle.
EBUSY	A thread currently is suspended on the event flags.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(60 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_pthread_event_flags_**, *px5_pthread_event_flags_create*

Small Example:

```
#include <pthread.h>

/* Event flags handle. */
pthread_event_flags_t    my_event_flags_handle;
int                      status;

/* Destroy the event flags referenced by
   "my_event_flags_handle". */
status = px5_pthread_event_flags_destroy(&my_event_flags_handle);

/* If status is PX5_SUCCESS, the event flags group was
   destroyed. */
```

px5_thread_event_flags_information_get

C Prototype:

```
#include <pthread.h>

int px5_thread_event_flags_information_get(
    pthread_event_flags_t * event_flags_handle, char **name,
    u_long *current_events, u_long *consumption_mask,
    u_long *threads_suspended, pthread_t *first_suspended_thread,
    pthread_event_flags_t * next_event_flags_handle);
```

Description:

This pthreads+ service retrieves the specified information from the specified event flags group.

API Parameters:

event_flags_handle	Handle of event flags group to get information about.
name	If non-NULL, destination for the specified event flags group name.
current_events	If non-NULL, destination for the current event flags of this group.
consumption_mask	If non-NULL, destination for the automatic consumption mask (if enabled) for this event flags group.
threads_suspended	If non-NULL, destination for the number of threads currently suspended on this event flags group.
first_suspended_thread	If non-NULL, destination for the thread handle of the first thread suspended on this event flags group.
next_event_flags_handle	If non-NULL, destination for the handle of the next created event flags group. This API can be called successively to examine all event flags groups using this parameter until the original event flags group handle is detected.

Return Codes:

PX5_SUCCESS (0)	Successful information retrieval from the specified event flags group.
ESRCH	Invalid event flags group handle.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:

From the location where this API is called, there should be at least **(78 + PX5_BINDING_STACK_FRAME_SIZE)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable from the thread context and interrupt handlers (ISRs).

See Also:

*px5_pthread_event_flags_create, px5_*_information_get*

Small Example:

```

#include <pthread.h>

pthread_event_flags_t    my_event_flags_handle;
char *                   name;
u_long                   current_events;
u_long                   consumption_mask;
u_long                   thread_suspended;
pthread_t                first_suspended_thread;
pthread_event_flags_t    next_event_flags;
int                      status;

/* Get information about "my_event_flags_handle". */
status =
    px5_pthread_event_flags_information_get(&my_event_flags_handle,
        &name, &current_events, &consumption_mask,
        &threads_suspended, &first_suspended_thread,
        &next_event_flags);

/* If status contains PX5_SUCCESS, the information about
   "my_event_flags_handle" is available for use. */

```

px5_pthread_event_flags_set

C Prototype:

```
#include <pthread.h>

int px5_pthread_event_flags_set(pthread_event_flags_t *
                                event_flags_handle, u_long events_to_set);
```

Description:

This pthreads+ service sets all the event flags specified in *events_to_set* in the event flags group identified by *event_flags_handle*. All threads suspended on this event flags group that have their event flags request satisfied are resumed.

API Parameters:

<i>event_flags_handle</i>	Handle of the event flags group.
<i>events_to_set</i>	Bit map of event flags to set.

Return Codes:

PX5_SUCCESS (0)	Successful event flags set.
EINVAL	Invalid event flags group handle pointer.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. If no other threads are waiting for event flags, no preemption occurs.



PREEMPTION. If a higher-priority thread is waiting for event flags and its request is satisfied, it is resumed, and preemption will occur.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(144 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable from the thread context and interrupt handlers (ISRs).

See Also:

*px5_pthread_event_flags_**, *px5_pthread_event_flags_wait*

Small Example:

```
#include <pthread.h>

/* Event flags handle. */
pthread_event_flags_t my_event_flags_handle;
int status;

/* Set event flags 7 and 3 (bits 7 and 3) in the event flags group
   "my_event_flags_handle". */
status = px5_pthread_event_flags_set(&my_event_flags_handle, 0x88);

/* If status is PX5_SUCCESS, event flags 7 and 3 are now set. */
```

px5_pthread_event_flags_set_notify

C Prototype:

```
#include <pthread.h>

int  px5_pthread_event_flags_set_notify(
        pthread_event_flags_t  event_flags_handle,
        void (*set_callback)(pthread_event_flags_t *));
```

Description:

This service registers an application callback that is executed whenever an event flag is set in the specified event flag group. If a NULL is supplied, notification for the specified event flag group is disabled.



By default, application event flags set notification is not enabled. To enable notification, PX5 RTOS and the application should be built with `PX5_NOTIFICATION_ENABLE` defined.

API Parameters:

<code>event_flags_handle</code>	Event flags group handle.
<code>set_callback</code>	If non-NULL, application's notification function pointer to call whenever an event flag is set in the event flag group.

Return Codes:

PX5_SUCCESS (0)	Successful registration of the notification callback.
ESRCH	Invalid event flags group.
EACCES	Invalid call from ISR or non-thread. This error code is also returned if PX5 RTOS was not built with <code>PX5_NOTIFICATION_ENABLE</code> defined.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. No preemption takes place as a result of this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(8 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

px5_mq_send_notify, *px5_thread_fastqueue_send_notify*,
px5_sem_post_notify

Small Example:

```
#include <pthread.h>

int                status;
pthread_event_flags_t  my_event_flags;

void my_notification_callback(pthread_event_flags_t *event_handle)
{
    /* Application processing goes here!  */
}

...

/* Register "my_notification_callback" to "my_event_flags". */
status = px5_thread_event_flags_set_notify(&my_event_flags,
                                           my_notification_callback);

/* If status is PX5_SUCCESS, the callback has been registered. */
```

px5_pthread_event_flags_timedwait

C Prototype:

```
#include <pthread.h>

int px5_pthread_event_flags_timedwait(pthread_event_flags_t *
                                     event_flags_handle, u_long requested_events,
                                     int all_or_any, u_long * received_events,
                                     ticks_t timeout_ticks);
```

Description:

This pthreads+ service retrieves the event flags specified by the *requested_events* parameter and in the manner specified by the *all_or_any* parameter from the specified event flags group. If the requested events are unavailable, this service suspends the calling thread until the specified timeout is reached.



This API is a cancellation point, meaning that if a cancellation is pending, it will be detected and executed by this API.

API Parameters:

<code>event_flags_handle</code>	Handle of the event flags group.
<code>requested_events</code>	Requested event flags.
<code>all_or_any</code>	This parameter specifies if all the event flags are required (<i>PTHREAD_ALL_EVENTS</i>), or if any of the events specified are required (<i>PTHREAD_ANY_EVENT</i>) to satisfy the request.
<code>received_events</code>	Optional destination of where to return the actual event flags that satisfied the request.
<code>timeout_ticks</code>	Maximum number of timer ticks to wait for the event flags. A value of zero represents no timeout.

Return Codes:

PX5_SUCCESS (0)	Successful event flags wait.
EINVAL	Invalid event flags handle pointer or any/all events option.
ETIMEDOUT	Events were not available within the specified timeout.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. If the event flags are available, no preemption takes place.



SUSPENSION. If the event flags are unavailable, the calling thread is suspended until the event flags become available or until the specified timeout is reached.

Stack Usage Estimate:

From the location where this API is called, there should be at least **(120 + PX5_BINDING_STACK_FRAME_SIZE)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

px5_pthread_event_flags_, px5_pthread_event_flags_set, px5_pthread_event_flags_trywait, px5_pthread_event_flags_wait*

Small Example:

```
#include <pthread.h>

/* Event flags handle. */
pthread_event_flags_t  my_event_flags_handle;
int                    status;

/* Get either flag 3 or flag 7 in the event flags group
   "my_event_flags_handle". */
status = px5_pthread_event_flags_timedwait(&my_event_flags_handle,
                                           0x88, PTHREAD_ANY_EVENT, NULL, 200);

/* If status is PX5_SUCCESS, the either or both event flag 3 and 7
   were set. */
```

px5_pthread_event_flags_trywait

C Prototype:

```
#include <pthread.h>

int px5_pthread_event_flags_trywait(pthread_event_flags_t *
                                   event_flags_handle, u_long requested_events,
                                   int all_or_any, u_long * received_events);
```

Description:

This pthreads+ service attempts to retrieve the event flags specified by the *requested_events* parameter as specified by the *all_or_any* parameter from the specified event flags group. If the requested events are not available, this service returns an error.

API Parameters:

<code>event_flags_handle</code>	Handle of the event flags group.
<code>requested_events</code>	Requested event flags.
<code>all_or_any</code>	This parameter specifies if all the event flags are required (<i>PTHREAD_ALL_EVENTS</i>), or if any of the events specified are required (<i>PTHREAD_ANY_EVENT</i>) to satisfy the request.
<code>received_events</code>	Optional destination of where to return the actual event flags that satisfied the request.

Return Codes:

PX5_SUCCESS (0)	Successful event flags retrieval.
EINVAL	Invalid event flags handle pointer or any/all events option.
EAGAIN	Requested event flags are not available.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. No preemption is possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(56 + PX5_BINDING_STACK_FRAME_SIZE)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable from the thread context and interrupt handlers (ISRs).

See Also:

px5_pthread_event_flags_, px5_pthread_event_flags_wait*

Small Example:

```
#include <pthread.h>

/* Event flags handle. */
pthread_event_flags_t my_event_flags_handle;
int status;

/* Attempt to get either flag 3 or flag 7 in the event flags group
   "my_event_flags_handle". */
status = px5_pthread_event_flags_trywait(&my_event_flags_handle,
                                         0x88, PTHREAD_ANY_EVENT, NULL);

/* If status is PX5_SUCCESS, the either or both event flag 3 and 7
   were set. */
```

px5_pthread_event_flags_wait

C Prototype:

```
#include <pthread.h>

int px5_pthread_event_flags_wait(pthread_event_flags_t *
                                event_flags_handle, u_long requested_events,
                                int all_or_any, u_long * received_events);
```

Description:

This pthreads+ service retrieves the event flags specified by the *requested_events* parameter as specified by the *all_or_any* parameter from the specified event flags group. If the requested events are not available, this service suspends the calling thread.



This API is a cancellation point, meaning that if a cancellation is pending, it will be detected and executed by this API.

API Parameters:

<code>event_flags_handle</code>	Handle of the event flags group.
<code>requested_events</code>	Requested event flags.
<code>all_or_any</code>	This parameter specifies if all the event flags are required (<i>PTHREAD_ALL_EVENTS</i>), or if any of the events specified are required (<i>PTHREAD_ANY_EVENT</i>) to satisfy the request.
<code>received_events</code>	Optional destination of where to return the actual event flags that satisfied the request.

Return Codes:

PX5_SUCCESS (0)	Successful event flags wait.
EINVAL	Invalid event flags handle pointer or any/all events option.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. If the event flags are available, no preemption takes place.



SUSPENSION. If the event flags are not present, the calling thread is suspended until the event flags become available.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(112 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_pthread_event_flags_**, *px5_pthread_event_flags_set*

Small Example:

```
#include <pthread.h>

/* Event flags handle. */
pthread_event_flags_t my_event_flags_handle;
int status;

/* Get either flag 3 or flag 7 in the event flags group
   "my_event_flags_handle". */
status = px5_pthread_event_flags_wait(&my_event_flags_handle,
                                       0x88, PTHREAD_ANY_EVENT, NULL);

/* If status is PX5_SUCCESS, the either or both event flag 3 and 7
   were set. */
```

px5_pthread_event_flagsattr_destroy

C Prototype:

```
#include <pthread.h>

int px5_pthread_event_flagsattr_destroy(pthread_event_flagsattr_t *
                                       event_flags_attributes);
```

Description:

This pthreads+ service destroys the previously created event flags attributes structure pointed to by *event_flags_attributes*. Once destroyed, the event flags attributes structure cannot be used again unless it is recreated.

API Parameters:

<code>event_flags_attributes</code>	Pointer to the event flags attributes to destroy.
-------------------------------------	---

Return Codes:

PX5_SUCCESS (0)	Successful event flags attributes destroy.
EINVAL	Invalid event flags attributes pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(12 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_pthread_event_flags_**, *px5_pthread_event_flagsattr_**,
px5_pthread_event_flagsattr_init

Small Example:

```
#include <pthread.h>

/* Event flags attribute structure. */
pthread_event_flagsattr_t  my_event_flags_attributes;
int                        status;

/* Destroy the event flags attributes referenced by
   "my_event_flags_attributes". */
status = px5_pthread_event_flagsattr_destroy(
        &my_event_flags_attributes);

/* If status is PX5_SUCCESS, the event flags attributes structure
   was destroyed. */
```

px5_pthread_event_flagsattr_getautoconsumemask

C Prototype:

```
#include <pthread.h>

int px5_pthread_event_flagsattr_getautoconsumemask(
    pthread_event_flagsattr_t * event_flags_attributes,
    u_long * auto_consumption_mask);
```

Description:

This pthreads+ service returns the previously supplied event flags automatic consumption mask, if PX5 RTOS is built with the *PX5_EVENT_FLAG_CONSUMPTION_MASK_ENABLE* option. Otherwise, an error is returned.

API Parameters:

<code>event_flags_attributes</code>	Pointer to the event flags attributes.
<code>auto_consumption_mask</code>	Pointer to the destination for the previously supplied event flags automatic consumption mask.

Return Codes:

PX5_SUCCESS (0)	Successful event flags attributes automatic event flag consumption mask retrieval.
EINVAL	Invalid event flags attributes or event flags destination pointer.
EACCES	Invalid call from ISR or non-thread. This error is also returned if the automatic consumption mask feature is not enabled.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(44 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_thread_event_flags_**, *px5_thread_event_flagsattr_**,
px5_thread_event_flagsattr_setautoconsumemask

Small Example:

```
#include <pthread.h>

/* Event flags attribute structure. */
pthread_event_flagsattr_t my_event_flags_attributes;
u_long                    my_auto_consumption_mask;
int                       status;

/* Get the event flags automatic consumption mask in the
   attributes structure "my_event_flags_attributes". */
status = px5_thread_event_flagsattr_getautoconsumemask(
        &my_event_flags_attributes,
        &my_auto_consumption_mask);

/* If status is PX5_SUCCESS, "my_auto_consumption_mask"
   contains the address of the previously supplied event flags
   auto consumption mask. */
```

px5_pthread_event_flagsattr_getcontroladdr

C Prototype:

```
#include <pthread.h>

int
px5_pthread_event_flagsattr_getcontroladdr(pthread_event_flagsattr_t
    * event_flags_attributes, void ** event_flags_control_address);
```

Description:

This pthreads+ service returns the previously supplied event flags control structure memory address.

API Parameters:

event_flags_attributes	Pointer to the event flags attributes.
event_flags_control_address	Pointer to the destination for the previously supplied event flags control address.

Return Codes:

PX5_SUCCESS (0)	Successful event flags attributes event flags control address retrieval.
EINVAL	Invalid event flags attributes or event flags control address destination pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(44 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_thread_event_flags_**, *px5_thread_event_flagsattr_**,
px5_thread_event_flagsattr_setcontroladdr,
px5_thread_event_flagsattr_getcontrolsize

Small Example:

```
#include <pthread.h>

/* Event flags attribute structure. */
pthread_event_flagsattr_t my_event_flags_attributes;
void * my_event_flags_control_address;
int status;

/* Get the event flags control structure address in the
   attributes structure "my_event_flags_attributes". */
status = px5_thread_event_flagsattr_getcontroladdr(
    &my_event_flags_attributes,
    &my_event_flags_control_address);

/* If status is PX5_SUCCESS, "my_event_flags_control_address"
   contains the address of the previously supplied event flags
   control memory. */
```

px5_pthread_event_flagsattr_getcontrolsize

C Prototype:

```
#include <pthread.h>

int
px5_pthread_event_flagsattr_getcontrolsize(pthread_event_flagsattr_t
    * event_flags_attributes, size_t * event_flags_control_size);
```

Description:

This pthreads+ service returns the size of the internal event flags control structure. The primary purpose of this API is to inform the application how much memory is required for the *px5_pthread_event_flagsattr_setcontroladdr* API.

API Parameters:

<code>event_flags_attributes</code>	Pointer to the event flags attributes.
<code>event_flags_control_size</code>	Pointer to the destination for the internal event flags control structure size.

Return Codes:

PX5_SUCCESS (0)	Successful retrieval of internal event flags control structure size.
EINVAL	Invalid event flags attributes or event flags control structure size destination pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(44 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_thread_event_flags_**, *px5_thread_event_flagsattr_**,
px5_thread_event_flagsattr_setcontroladdr

Small Example:

```
#include <pthread.h>

/* Event flags attribute structure. */
pthread_event_flagsattr_t my_event_flags_attributes;
size_t                    my_event_flags_control_size;
int                       status;

/* Get the event flags control structure memory size. */
status = px5_thread_event_flagsattr_getcontrolsize(
    &my_event_flags_attributes, &my_event_flags_control_size);

/* If status is PX5_SUCCESS, "my_event_flags_control_size"
   contains the size of the internal event flags
   control structure. */
```

px5_pthread_event_flagsattr_getname

C Prototype:

```
#include <pthread.h>

int px5_pthread_event_flagsattr_getname(pthread_event_flagsattr_t
    * event_flags_attributes, char ** event_flags_name);
```

Description:

This pthreads+ service returns the previously supplied event flags name.

API Parameters:

event_flags_attributes	Pointer to the event flags attributes.
event_flags_name	Pointer to the destination for the previous event flags name.

Return Codes:

PX5_SUCCESS (0)	Successful event flags attributes event flags name retrieval.
EINVAL	Invalid event flags attributes or event flags name pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(44 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_pthread_event_flags_**, *px5_pthread_event_flagsattr_**

Small Example:

```
#include <pthread.h>

/* Event flags attribute structure. */
pthread_event_flagsattr_t my_event_flags_attributes;
char * my_event_flags_name;
int status;

/* Get the previous event flags name. */
status = px5_pthread_event_flagsattr_getname(
    &my_event_flags_attributes, &my_event_flags_name);

/* If status is PX5_SUCCESS, "my_event_flags_name" contains the
   name previously supplied. */
```

px5_pthread_event_flagsattr_init

C Prototype:

```
#include <pthread.h>

int px5_pthread_event_flagsattr_init(pthread_event_flagsattr_t *
                                     event_flags_attributes);
```

Description:

This pthreads+ service initializes the event flags attributes structure with default event flags creation values.

API Parameters:

<code>event_flags_attributes</code>	Pointer to the event flags attributes structure to create.
-------------------------------------	--

Return Codes:

PX5_SUCCESS (0)	Successful event flags attributes structure creation.
EINVAL	Invalid event flags attributes pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(36 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_pthread_event_flags_**, *px5_pthread_event_flagsattr_**,
px5_pthread_event_flagsattr_destroy

Small Example:

```
#include <pthread.h>

/* Event flags attribute structure. */
pthread_event_flagsattr_t my_event_flags_attributes;
int status;

/* Create the event flags attributes structure
   "my_event_flags_attributes". */
status =
    px5_pthread_event_flagsattr_init(&my_event_flags_attributes);

/* If status is PX5_SUCCESS, the "my_event_flags_attributes"
   structure is ready for use. */
```

px5_pthread_event_flagsattr_setautoconsumemask

C Prototype:

```
#include <pthread.h>

int px5_pthread_event_flagsattr_setautoconsumemask(
    pthread_event_flagsattr_t * event_flags_attributes,
    u_long auto_consumption_mask);
```

Description:

This pthreads+ service sets the event flags automatic consumption mask. To disable automatic clearing of a specific event flag is accomplished by setting the corresponding bit to 0. Automatic consumption of all event flags is the default behavior (automatic consumption mask is all ones).

If PX5 RTOS is built without the *PX5_EVENT_FLAG_CONSUMPTION_MASK_ENABLE* option, an error is returned.

API Parameters:

<code>event_flags_attributes</code>	Pointer to the event flags attributes.
<code>auto_consumption_mask</code>	Bit map of the event flags that should be cleared automatically during consumption.

Return Codes:

PX5_SUCCESS (0)	Successful event flags attributes automatic event flag consumption mask set up.
EINVAL	Invalid event flags attributes.
EACCES	Invalid call from ISR or non-thread. This error is also returned if the automatic consumption mask feature is not enabled.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(44 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_pthread_event_flags_**, *px5_pthread_event_flagsattr_**,
px5_pthread_event_flagsattr_getautoconsumemask

n

Small Example:

```
#include <pthread.h>

/* Event flags attribute structure. */
pthread_event_flagsattr_t my_event_flags_attributes;
int status;

/* Set the event flags automatic consumption mask in the
   attributes structure "my_event_flags_attributes" to disable
   clearing of event flag zero (bit 0). */
status = px5_pthread_event_flagsattr_setautoconsumemask(
    &my_event_flags_attributes,
    (u_long) 0xFFFFFFFF);
```

```
/* If status is PX5_SUCCESS, "my_auto_consumption_mask"  
   now specifies that event flag 0 is not automatically  
   consumed. */
```


px5_pthread_event_flagsattr_setcontroladdr

C Prototype:

```
#include <pthread.h>

int
px5_pthread_event_flagsattr_setcontroladdr(pthread_event_flagsattr_t
    * event_flags_attributes, void * event_flags_control_address,
    size_t event_flags_control_size);
```

Description:

This pthreads+ service enables the developer to provide the memory for the internal PX5 RTOS event flags structure, as specified by the address in the *event_flags_control_address* parameter. This memory will subsequently be used for the next event flags created with this attribute structure. The memory size required for the internal event flags control structure can be found via a call to the *px5_pthread_event_flagsattr_getcontrolsize* service.



Note that each event flags created must have its own unique event flags control structure memory. Hence, the event flags control address supplied here is only valid for one px5_pthread_event_flags_create call.



The exact amount of memory required for the internal event flags control structure can be determined by including *px5.h* and using the *sizeof(px5_event_flags_control)* construct.

API Parameters:

<code>event_flags_attributes</code>	Pointer to the event flags attributes.
<code>event_flags_control_address</code>	Pointer to the supplied event flags control structure memory.
<code>event_flags_control_size</code>	Size of the memory specified.

Return Codes:

PX5_SUCCESS (0)	Successful internal event flags control address set.
EINVAL	Invalid event flags attributes or memory size.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:

From the location where this API is called, there should be at least **(44 + PX5_BINDING_STACK_FRAME_SIZE)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

px5_pthread_event_flags_, px5_pthread_event_flagsattr_*,
px5_pthread_event_flagsattr_getcontrolsizes*

Small Example:

```
#include <pthread.h>

/* Event flags attribute structure. */
pthread_event_flagsattr_t  my_event_flags_attributes;
int                        status;


/* Provide the memory for the event flags control structure in the
   next event flags create call by placing its address in the
   attributes structure "my_event_flags_attributes". */
status = px5_pthread_event_flagsattr_setcontroladdr(
    &my_event_flags_attributes, 0x40000, 60);


/* If status is PX5_SUCCESS, the next event flags creation using
   these attributes will use memory at address 0x40000 for the
   internal event flags control structure. */
```

px5_pthread_event_flagsattr_setname

C Prototype:

```
#include <pthread.h>

int px5_pthread_event_flagsattr_setname(pthread_event_flagsattr_t
    * event_flags_attributes, char * event_flags_name);
```

Description:

This pthreads+ service sets the event flags name in the specified attribute structure.

API Parameters:

event_flags_attributes	Pointer to the event flags attributes.
event_flags_name	Pointer to the supplied event flags name.

Return Codes:

PX5_SUCCESS (0)	Successful event flags attributes event flags name set.
EINVAL	Invalid event flags attributes.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(44 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_pthread_event_flags_**, *px5_pthread_event_flagsattr_**,
px5_pthread_event_flagsattr_getname

Small Example:

```
#include <pthread.h>

/* Event flags attribute structure. */
pthread_event_flagsattr_t my_event_flags_attributes;
int status;

/* Set the event flags name in the event flags attributes structure
   "my_event_flags_attributes". */
status = px5_pthread_event_flagsattr_setname(
    &my_event_flags_attributes, "my_event_flags_name");

/* If status is PX5_SUCCESS, "my_event_flags_name" is set in the
   event flags attribute structure. */
```

px5_pthread_fastqueue_create

C Prototype:

```
#include <pthread.h>

int px5_pthread_fastqueue_create(pthread_fastqueue_t *fastqueue_handle,
    pthread_fastqueueattr_t * fast_queue_attributes,
    size_t message_size, int max_messages);
```

Description:



This pthreads+ service initializes (creates) a fastqueue with the optional fast queue attributes. If successful, the fastqueue handle is set up for further use by the application.

API Parameters:

<code>fastqueue_handle</code>	Handle of the fastqueue to create.
<code>fastqueue_attributes</code>	Optional fastqueue attributes. This value is NULL if no fastqueue attributes are specified.
<code>message_size</code>	Fixed-size of queue message in bytes (must be evenly divisible by the size of the <i>u_long</i> type).
<code>max_messages</code>	Total number of messages the queue can hold.

Return Codes:

PX5_SUCCESS (0)	Successful fastqueue creation.
EINVAL	Invalid fastqueue handle pointer or attributes.
EBUSY	Fastqueue is already created.
ENOMEM	Insufficient memory to create fastqueue.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(164 + PX5_BINDING_STACK_FRAME_SIZE)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_pthread_fastqueueattr_**, *px5_pthread_fastqueue_destroy*

Small Example:

```
#include <pthread.h>

/* Fastqueue handle. */
pthread_fastqueue_t    my_fastqueue_handle;
int                    status;

/* Create a fastqueue that can hold 100 4-byte messages, the handle
   of which is returned in "my_fastqueue_handle". */
status = px5_pthread_fastqueue_create(&my_fastqueue_handle, NULL,
                                     4, 100);

/* If status is PX5_SUCCESS, the event flags group was created, and
   the event flags handle is ready to use. */
```

px5_pthread_fastqueue_destroy

C Prototype:

```
#include <pthread.h>

int px5_pthread_fastqueue_destroy(pthread_fastqueue_t *
                                fastqueue_handle);
```

Description:

This pthreads+ service destroys the previously created fastqueue specified by *fastqueue_handle*. An error is returned if the fastqueue has any threads waiting for a message.



To avoid unpredictable behavior, the application must ensure there is no further access to the fast queue after it is destroyed.

API Parameters:

fastqueue_handle Handle of the fastqueue to destroy.

Return Codes:

PX5_SUCCESS (0)	Successful fastqueue destroy.
EINVAL	Invalid fastqueue handle.
EBUSY	A thread currently is suspended on the fastqueue.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(68 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_pthread_fastqueue_**, *px5_pthread_fastqueue_create*

Small Example:

```
#include <pthread.h>

/* Fastqueue handle. */
pthread_fastqueue_t    my_fastqueue_handle;
int                    status;

/* Destroy the fastqueue referenced by
   "my_fastqueue_handle". */
status = px5_pthread_fastqueue_destroy(&my_fastqueue_handle);

/* If status is PX5_SUCCESS, the fastqueue was destroyed. */
```

px5_thread_fastqueue_information_get

C Prototype:

```
#include <pthread.h>

int  px5_thread_fastqueue_information_get(
    pthread_fastqueue_t * queue_handle, char ** name,
    u_long * messages_queued, u_long * message_size,
    u_long * maximum_messages,
    u_long * threads_suspended, pthread_t *first_suspended_thread,
    pthread_fastqueue_t * next_queue_handle);
```

Description:

This pthreads+ service retrieves the specified information from the specified fastqueue.

API Parameters:

queue_handle	Handle of fastqueue to get information about.
name	If non-NULL, destination for the specified fastqueue name.
messages_queued	If non-NULL, destination for the current number of messages in the fastqueue.
message_size	If non-NULL, destination for the size of each message in the fastqueue.
maximum_messages	If non-NULL, destination for the maximum number of messages this fastqueue can hold.
threads_suspended	If non-NULL, destination for the number of threads currently suspended on this fastqueue.
first_suspended_thread	If non-NULL, destination for the thread handle of the first thread suspended on this fastqueue.
next_fastqueue_handle	If non-NULL, destination for the handle of the next created fastqueue. This API can be called successively to examine all fastqueues using this parameter until the original fastqueue handle is detected.

Return Codes:

PX5_SUCCESS (0)	Successful information retrieval from the specified fastqueue.
ESRCH	Invalid fastqueue handle.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:

From the location where this API is called, there should be at least **(78 + PX5_BINDING_STACK_FRAME_SIZE)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable from the thread context and interrupt handlers (ISRs).

See Also:

*px5_pthread_fastqueue_create, px5_*_information_get*

Small Example:

```

#include <pthread.h>

pthread_fastqueue_t    my_queue_handle;
char *                 name;
u_long                 messages_queued;
u_long                 message_size;
u_long                 maximum_messages;
u_long                 thread_suspended;
pthread_t              first_suspended_thread;
pthread_fastqueue_t    next_fastqueue;
int                    status;

/* Get information about "my_fastqueue_handle". */
status =
    px5_pthread_fastqueue_information_get(&my_fastqueue_handle,
        &name, &messages_queued, &message_size, &maximum_messages,
        &threads_suspended, &first_suspended_thread,
        &next_fastqueue);

/* If status contains PX5_SUCCESS, the information about
   "my_fastqueue_handle" is available for use. */

```

px5_pthread_fastqueue_receive

C Prototype:

```
#include <pthread.h>

int px5_pthread_fastqueue_receive(pthread_fastqueue_t *
                                   fastqueue_handle, u_long * message_destination,
                                   size_t message_size);
```

Description:

This pthreads+ service receives a message from the specified fastqueue. If the fastqueue is empty, this thread suspends until a message is sent to the fastqueue. If a thread is waiting to send to the fastqueue, the thread's message is placed in the fastqueue and is resumed.

API Parameters:

fastqueue_handle	Handle of the fastqueue.
message_destination	Pointer to the destination for the message.
message_size	Size of the message to receive in bytes (must match the message size of the fastqueue).

Return Codes:

PX5_SUCCESS (0)	Successful fastqueue receive.
EINVAL	Invalid fastqueue handle pointer, or invalid message.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. If there are one or more messages in the fastqueue, no preemption occurs.



PREEMPTION. If a higher-priority thread is waiting to send a message to the fastqueue, it is resumed and preemption will occur.



SUSPENSION. If the fastqueue is empty, the calling thread is suspended until a message is sent to the fastqueue.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(132 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_pthread_fastqueue_**, *px5_pthread_fastqueue_send*

Small Example:

```
#include <pthread.h>

/* Fastqueue handle. */
pthread_fastqueue_t    my_fastqueue_handle;
u_long                 my_message;
int                    status;

/* Receive message from the fastqueue "my_fastqueue_handle". */
status = px5_pthread_fastqueue_receive(&my_fastqueue_handle,
                                         &my_message, 4);

/* If status is PX5_SUCCESS, "my_message" contains the message
   received. */
```

px5_pthread_fastqueue_send

C Prototype:

```
#include <pthread.h>

int px5_pthread_fastqueue_send(pthread_fastqueue_t *
                               fastqueue_handle, u_long * message_source,
                               size_t message_size);
```

Description:

This pthreads+ service sends the specified message to the specified fastqueue. If there is a thread waiting on the fastqueue, this message is delivered to the thread and it is resumed. If the fastqueue is full, this thread suspends until there is room in the fastqueue.

API Parameters:

fastqueue_handle	Handle of the fastqueue.
message_source	Pointer to the start of the message.
message_size	Size of the message in bytes (must match the message size of the fastqueue).

Return Codes:

PX5_SUCCESS (0)	Successful fastqueue send.
EINVAL	Invalid fastqueue handle pointer, invalid message source or invalid message size.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. If no other threads are waiting for a message on the fastqueue, no preemption occurs.



PREEMPTION. If a higher-priority thread is the first waiting for a message, it is resumed and preemption will occur.



SUSPENSION. If the fastqueue is full, the calling thread is suspended until there is room in the fastqueue.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(132 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_pthread_fastqueue_**, *px5_pthread_fastqueue_receive*

Small Example:

```
#include <pthread.h>

/* Fastqueue handle. */
pthread_fastqueue_t    my_fastqueue_handle;
u_long                 my_message;
int                    status;

/* Set the message to 0x12345678. */
my_message = 0x12345678;

/* Send the message to the fastqueue "my_fastqueue_handle". */
status = px5_pthread_fastqueue_send(&my_fastqueue_handle,
                                     &my_message, 4);

/* If status is PX5_SUCCESS, "my_message" was sent. */
```


px5_pthread_fastqueue_send_notify

C Prototype:

```
#include <pthread.h>

int px5_pthread_fastqueue_send_notify(
    pthread_fastqueue_t fastqueue_handle,
    void (*send_callback)(pthread_fastqueue_t *));
```

Description:

This service registers an application callback that is executed whenever a message is sent to the specified fastqueue. If a NULL is supplied, notification for the specified fastqueue is disabled.



By default, application fastqueue send notification is not enabled. To enable notification, PX5 RTOS and the application should be built with `PX5_NOTIFICATION_ENABLE` defined.

API Parameters:

<code>fastqueue_handle</code>	Fastqueue handle.
<code>send_callback</code>	If non-NULL, application's notification function pointer to call whenever a message is sent to the fastqueue.

Return Codes:

PX5_SUCCESS (0)	Successful registration of the notification callback.
ESRCH	Invalid fastqueue handle.
EACCES	Invalid call from ISR or non-thread. This error code is also returned if PX5 RTOS was not built with <code>PX5_NOTIFICATION_ENABLE</code> defined.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. No preemption takes place as a result of this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(8 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

px5_mq_send_notify, *px5_thread_event_flags_set_notify*,
px5_sem_post_notify

Small Example:

```
#include <pthread.h>

int                status;
pthread_fastqueue_t my_fastqueue;

void my_notification_callback(
    pthread_fastqueue_t *fastqueue_handle)
{
    /* Application processing goes here!  */
}

...

/* Register "my_notification_callback" to "my_fastqueue". */
status = px5_thread_fastqueue_send_notify(&my_fastqueue,
                                           my_notification_callback);

/* If status is PX5_SUCCESS, the callback has been registered. */
```

px5_pthread_fastqueue_timedreceive

C Prototype:

```
#include <pthread.h>

int px5_pthread_fastqueue_timedreceive(pthread_fastqueue_t *
                                       fastqueue_handle, u_long * message_destination,
                                       size_t message_size, tick_t timeout_ticks);
```

Description:

This pthreads+ service attempts to receive a message from the specified fastqueue. If the fastqueue is empty, this service suspends waiting for a message from the queue. An error is returned if no message is received before the specified timeout.

API Parameters:

fastqueue_handle	Handle of the fastqueue.
message_destination	Pointer to the destination for the message.
message_size	Size of the message in bytes (must match the message size of the fastqueue).
timeout_ticks	Maximum number of timer ticks to wait for a message. A value of zero represents no timeout.

Return Codes:

PX5_SUCCESS (0)	Successful fastqueue receive.
EINVAL	Invalid fastqueue handle pointer, invalid message destination, or invalid message size.
ETIMEDOUT	Message is unavailable within the specified timeout (fastqueue is empty).
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. If there are one or more messages in the fastqueue, no preemption occurs.



PREEMPTION. If a higher-priority thread is waiting to send a message to the fastqueue, it is resumed and preemption will occur.



SUSPENSION. If the fastqueue is empty, the calling thread is suspended until a message is sent to the fastqueue or until the timeout is reached.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(148 + PX5_BINDING_STACK_FRAME_SIZE)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

px5_pthread_fastqueue_, px5_pthread_fastqueue_receive, px5_pthread_fastqueue_tryreceive*

Small Example:

```
#include <pthread.h>

/* Fastqueue handle. */
pthread_fastqueue_t    my_fastqueue_handle;
u_long                 my_message;
int                     status;

/* Try to receive message from the fastqueue
   "my_fastqueue_handle" and wait for a maximum of 200 ticks. */
status = px5_thread_fastqueue_timedreceive(&my_fastqueue_handle,
                                           &my_message, 4, 200);

/* If status is PX5_SUCCESS, "my_message" contains the message
   received. */
```

px5_thread_fastqueue_timedsend

C Prototype:

```
#include <pthread.h>

int px5_thread_fastqueue_timedsend(pthread_fastqueue_t *
                                   fastqueue_handle, u_long * message_source,
                                   size_t message_size, ticks_t timeout_ticks);
```

Description:

This pthreads+ service tries to send the specified message to the specified fastqueue. If there is a thread waiting on the fastqueue, this message is delivered to the thread and it is resumed. If the fastqueue is full, this service suspends waiting for room in the queue. An error is returned if there is no room in the queue before the specified timeout.

API Parameters:

<code>fastqueue_handle</code>	Handle of the fastqueue.
<code>message_source</code>	Pointer to the start of the message.
<code>message_size</code>	Size of the message in bytes (must be evenly divisible by the size of the <code>u_long</code> type).
<code>timeout_ticks</code>	Maximum number of timer ticks to wait for room in the queue. A value of zero represents no timeout.

Return Codes:

PX5_SUCCESS (0)	Successful fastqueue send.
EINVAL	Invalid fastqueue handle pointer, invalid message source or invalid message size.
ETIMEDOUT	Fastqueue is full, no room for the message within the specified timeout.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. If no other threads are waiting for a message on the fastqueue, no preemption occurs.



PREEMPTION. If a higher-priority thread is the first waiting for a message, it is resumed and preemption will occur.



SUSPENSION. If the fastqueue is full, the calling thread is suspended until there is room in the fastqueue or until the timeout is reached.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(148 + PX5_BINDING_STACK_FRAME_SIZE)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

px5_pthread_fastqueue_, px5_pthread_fastqueue_send, px5_pthread_fastqueue_trysend*

Small Example:

```
#include <pthread.h>

/* Fastqueue handle. */
pthread_fastqueue_t    my_fastqueue_handle;
u_long                 my_message;
int                     status;

/* Set the message to 0x12345678. */
my_message = 0x12345678;

/* Try to send the message to the fastqueue
   "my_fastqueue_handle" waiting for a maximum of 200 ticks. */
status = px5_pthread_fastqueue_timedsend(&my_fastqueue_handle,
                                         &my_message, 4, 200);

/* If status is PX5_SUCCESS, "my_message" was sent. */
```


px5_pthread_fastqueue_tryreceive

C Prototype:

```
#include <pthread.h>

int px5_pthread_fastqueue_tryreceive(pthread_fastqueue_t *
                                     fastqueue_handle, u_long * message_destination,
                                     size_t message_size);
```

Description:

This pthreads+ service attempts to receive a message from the specified fastqueue. If the fastqueue is empty, this service returns an error.

API Parameters:

fastqueue_handle	Handle of the fastqueue.
message_destination	Pointer to the destination for the message.
message_size	Size of the message in bytes (must match the message size of the fastqueue).

Return Codes:

PX5_SUCCESS (0)	Successful fastqueue receive.
EINVAL	Invalid fastqueue handle pointer, invalid message destination or invalid message size.
EAGAIN	Message is not available (fastqueue is empty).

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. If there are one or more messages in the fastqueue, no preemption occurs.



PREEMPTION. If a higher-priority thread is waiting to send a message to the fastqueue, it is resumed and preemption will occur.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(128 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable from the thread context and interrupt handlers (ISRs).

See Also:

*px5_pthread_fastqueue_**, *px5_pthread_fastqueue_receive*

Small Example:

```
#include <pthread.h>

/* Fastqueue handle. */
pthread_fastqueue_t    my_fastqueue_handle;
u_long                 my_message;
int                    status;

/* Try to receive message from the fastqueue
   "my_fastqueue_handle". */
status = px5_pthread_fastqueue_tryreceive(&my_fastqueue_handle,
                                           &my_message, 4);

/* If status is PX5_SUCCESS, "my_message" contains the message
   received. */
```

px5_thread_fastqueue_trysend

C Prototype:

```
#include <pthread.h>

int px5_thread_fastqueue_trysend(pthread_fastqueue_t *
                                fastqueue_handle, u_long * message_source,
                                size_t message_size);
```

Description:

This pthreads+ service tries to send the specified message to the specified fastqueue. If there is a thread waiting on the fastqueue, this message is delivered to the thread and it is resumed. If the fastqueue is full, an error is returned.

API Parameters:

fastqueue_handle	Handle of the fastqueue.
message_source	Pointer to the start of the message.
message_size	Size of the message in bytes (must be evenly divisible by the size of the <i>u_long</i> type).

Return Codes:

PX5_SUCCESS (0)	Successful fastqueue send.
EINVAL	Invalid fastqueue handle pointer, invalid message source or invalid message size.
EAGAIN	Fastqueue is full, no room for the message.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. If no other threads are waiting for a message on the fastqueue, no preemption occurs.



PREEMPTION. If a higher-priority thread is the first waiting for a message, it is resumed and preemption will occur.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(128 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable from the thread context and interrupt handlers (ISRs).

See Also:

*px5_pthread_fastqueue_**, *px5_pthread_fastqueue_send*

Small Example:

```
#include <pthread.h>

/* Fastqueue handle. */
pthread_fastqueue_t    my_fastqueue_handle;
u_long                 my_message;
int                     status;

/* Set the message to 0x12345678. */
my_message = 0x12345678;

/* Try to send the message to the fastqueue
   "my_fastqueue_handle". */
status = px5_pthread_fastqueue_trysend(&my_fastqueue_handle,
                                         &my_message, 4);

/* If status is PX5_SUCCESS, "my_message" was sent. */
```

px5_pthread_fastqueueattr_destroy

C Prototype:

```
#include <pthread.h>

int px5_pthread_fastqueueattr_destroy(pthread_fastqueueattr_t *
                                     fastqueue_attributes);
```

Description:

This pthreads+ service destroys the previously created fastqueue attributes structure pointed to by *fastqueue_attributes*. Once destroyed, the fastqueue attributes structure cannot be used again unless it is recreated.

API Parameters:

<code>fastqueue_attributes</code>	Pointer to the fastqueue attributes to destroy.
-----------------------------------	---

Return Codes:

PX5_SUCCESS (0)	Successful fastqueue attributes destroy.
EINVAL	Invalid fastqueue attributes pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(36 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_thread_fastqueue_**, *px5_thread_fastqueueattr_**,
px5_thread_fastqueueattr_init

Small Example:

```
#include <pthread.h>

/* Fastqueue attribute structure. */
pthread_fastqueueattr_t my_fastqueue_attributes;
int status;

/* Destroy the fastqueue attributes referenced by
   "my_fastqueue_attributes". */
status = px5_thread_fastqueueattr_destroy(
    &my_fastqueue_attributes);

/* If status is PX5_SUCCESS, the fastqueue attributes structure
   was destroyed. */
```

px5_pthread_fastqueueattr_getcontroladdr

C Prototype:

```
#include <pthread.h>

int px5_pthread_fastqueueattr_getcontroladdr(pthread_fastqueueattr_t
      * fastqueue_attributes, void ** fastqueue_control_address);
```

Description:

This pthreads+ service returns the previously supplied fastqueue control structure memory address.

API Parameters:

fastqueue_attributes	Pointer to the fastqueue attributes.
fastqueue_control_address	Pointer to the destination for the previously supplied fastqueue control address.

Return Codes:

PX5_SUCCESS (0)	Successful fastqueue attributes fastqueue control address retrieval.
EINVAL	Invalid fastqueue attributes, or fastqueue control address destination pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(44 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_thread_fastqueue_**, *px5_thread_fastqueueattr_**,
px5_thread_fastqueueattr_setcontroladdr,
px5_thread_fastqueueattr_getcontrolsize

Small Example:

```
#include <pthread.h>

/* Fastqueue attribute structure. */
pthread_fastqueueattr_t  my_fastqueue_attributes;
void *                   my_fastqueue_control_address;
int                      status;

/* Get the fastqueue control structure address in the
   attributes structure "my_fastqueue_attributes". */
status = px5_thread_fastqueueattr_getcontroladdr(
    &my_fastqueue_attributes,
    &my_fastqueue_control_address);

/* If status is PX5_SUCCESS, "my_fastqueue_control_address"
   contains the address of the previously supplied fastqueue
   control memory. */
```


px5_pthread_fastqueueattr_getcontrolsize

C Prototype:

```
#include <pthread.h>

int px5_pthread_fastqueueattr_getcontrolsize(pthread_fastqueueattr_t
      * fastqueue_attributes, size_t * fastqueue_control_size);
```

Description:

This pthreads+ service returns the size of the internal fastqueue control structure. The primary purpose of this API is to inform the application how much memory is required for the *px5_pthread_fastqueueattr_setcontroladdr* API.

API Parameters:

<code>fastqueue_attributes</code>	Pointer to the fastqueue attributes.
<code>fastqueue_control_size</code>	Pointer to the destination for the internal fastqueue control structure size.

Return Codes:

PX5_SUCCESS (0)	Successful retrieval of internal fastqueue control structure size.
EINVAL	Invalid fastqueue attributes or fastqueue control structure size destination pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(44 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_pthread_fastqueue_**, *px5_pthread_fastqueueattr_**,
px5_pthread_fastqueueattr_setcontroladdr

Small Example:

```
#include <pthread.h>

/* Fastqueue attribute structure. */
pthread_fastqueueattr_t  my_fastqueue_attributes;
size_t                  my_fastqueue_control_size;
int                      status;

/* Get the fastqueue control structure memory size. */
status = px5_pthread_fastqueueattr_getcontrolsize(
    &my_fastqueue_attributes, &my_fastqueue_control_size);

/* If status is PX5_SUCCESS, "my_fastqueue_control_size"
   contains the size of the internal fastqueue
   control structure. */
```

px5_pthread_fastqueueattr_getname

C Prototype:

```
#include <pthread.h>

int px5_pthread_fastqueueattr_getname(pthread_fastqueueattr_t
                                     * fastqueue_attributes, char ** fastqueue_name);
```

Description:

This pthreads+ service returns the previously supplied fastqueue name.

API Parameters:

fastqueue_attributes	Pointer to the fastqueue attributes.
fastqueue_name	Pointer to the destination for the previous fastqueue name.

Return Codes:

PX5_SUCCESS (0)	Successful fastqueue attributes fastqueue name retrieval.
EINVAL	Invalid fastqueue attributes or fastqueue name pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(44 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_pthread_fastqueue_**, *px5_pthread_fastqueueattr_**

Small Example:

```
#include <pthread.h>

/* Fastqueue attribute structure. */
pthread_fastqueueattr_t  my_fastqueue_attributes;
char *                   my_fastqueue_name;
int                      status;

/* Get the previous fastqueue name. */
status = px5_pthread_fastqueueattr_getname(
    &my_fastqueue_attributes, &my_fastqueue_name);

/* If status is PX5_SUCCESS, "my_fastqueue_name" contains the
   name previously supplied. */
```

px5_pthread_fastqueueattr_getqueueaddr

C Prototype:

```
#include <pthead.h>

int px5_pthread_fastqueueattr_getqueueaddr(pthread_fastqueueattr_t*
                                           fastqueue_attributes,
                                           void ** fastqueue_memory_address);
```

Description:

This pthreads+ service returns the previously supplied fastqueue memory area address.

API Parameters:

<code>fastqueue_attributes</code>	Pointer to the fastqueue attributes.
<code>fastqueue_memory_address</code>	Pointer to the destination for the previously supplied fastqueue memory area address.

Return Codes:

PX5_SUCCESS (0)	Successful fastqueue memory area address retrieval.
EINVAL	Invalid fastqueue attributes or fastqueue memory area address destination pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(44 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_pthread_fastqueue_**, *px5_pthread_fastqueueattr_**

Small Example:

```
#include <mqueue.h>

/* Fastqueue queue attribute structure. */
pthread_fastqueueattr_t my_fastqueue_attributes;
void * my_fastqueue_memory_address;
int status;

/* Get the fastqueue memory area address in the
fastqueue attributes structure
"my_fastqueue_attributes". */
status = px5_pthread_fastqueueattr_getqueueaddr(
    &my_fastqueue_attributes,
    &my_queue_memory_address);

/* If status is PX5_SUCCESS, "my_fastqueue_memory_address"
contains the address of the previously supplied fastqueue
memory. */
```

px5_pthread_fastqueueattr_getqueuesize

C Prototype:

```
#include <pthread.h>

int px5_pthread_fastqueueattr_getqueuesize(pthread_fastqueueattr_t*
                                           fastqueue_attributes, size_t * fastqueue_memory_size);
```

Description:

This pthreads+ service returns the size of the previously supplied fastqueue memory area.

API Parameters:

fastqueue_attributes	Pointer to the attributes.
fastqueue_memory_size	Pointer to the destination for the previously supplied fastqueue memory area size.

Return Codes:

PX5_SUCCESS (0)	Successful retrieval of fastqueue memory area size.
EINVAL	Invalid extended fastqueue attributes or invalid destination for fastqueue memory area size.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(44 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_pthread_fastqueue_**, *px5_pthread_fastqueueattr_**

Small Example:

```
#include <pthread.h>

/* Fastqueue extended attribute structure. */
pthread_fastqueueattr_t my_fastqueue_attributes;
size_t my_fastqueue_memory_size;
int status;

/* Get the fastqueue memory area size. */
status = px5_pthread_fastqueueattr_getqueuesize(
    &my_fastqueue_attributes, &my_fastqueue_memory_size);

/* If status is PX5_SUCCESS, "my_fastqueue_memory_size"
   contains the size of the fastqueue memory area. */
```


px5_pthread_fastqueueattr_init

C Prototype:

```
#include <pthread.h>

int px5_pthread_fastqueueattr_init(pthread_fastqueueattr_t *
                                   fastqueue_attributes);
```

Description:

This pthreads+ service initializes the fastqueue attributes structure with default fastqueue creation values.

API Parameters:

<code>fastqueue_attributes</code>	Pointer to the fastqueue attributes structure to create.
-----------------------------------	--

Return Codes:

PX5_SUCCESS (0)	Successful fastqueue attributes structure creation.
EINVAL	Invalid fastqueue attributes pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(36 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_thread_fastqueue_**, *px5_thread_fastqueueattr_**,
px5_thread_fastqueueattr_destroy

Small Example:

```
#include <pthread.h>

/* Fastqueue attribute structure. */
pthread_fastqueueattr_t  my_fastqueue_attributes;
int                      status;

/* Create the fastqueue attributes structure
   "my_fastqueue_attributes". */
status = px5_thread_fastqueueattr_init(&my_fastqueue_attributes);

/* If status is PX5_SUCCESS, the "my_fastqueue_attributes"
   structure is ready for use. */
```

px5_pthread_fastqueueattr_setcontroladdr

C Prototype:

```
#include <pthread.h>

int px5_pthread_fastqueueattr_setcontroladdr(pthread_fastqueueattr_t
      * fastqueue_attributes, void * fastqueue_control_address,
      size_t fastqueue_control_size);
```

Description:

This pthreads+ service enables the developer to provide the memory for the internal PX5 RTOS fastqueue structure as specified by the address in the *fastqueue_control_address* parameter. This memory will subsequently be used for the fastqueue created with this attribute structure. The memory size required for the internal fastqueue control structure can be found via a call to the *px5_pthread_fastqueueattr_getcontrolsize* service.



Note that each fastqueue created must have its own unique fastqueue control structure memory. Hence, the fastqueue control address supplied here is only valid for one px5_pthread_fastqueue_create call.



The exact amount of memory required for the internal fastqueue control structure can be determined by including *px5.h* and using the *sizeof(px5_fastqueue_control)* construct.

API Parameters:

<code>fastqueue_attributes</code>	Pointer to the fastqueue attributes.
<code>fastqueue_control_address</code>	Pointer to the supplied fastqueue control structure memory.
<code>fastqueue_control_size</code>	Size of memory specified.

Return Codes:

PX5_SUCCESS (0)	Successful internal fastqueue control address set.
EINVAL	Invalid fastqueue attributes or memory size.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:

From the location where this API is called, there should be at least **(44 + PX5_BINDING_STACK_FRAME_SIZE)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

px5_pthread_fastqueue_, px5_pthread_fastqueueattr_*,
px5_pthread_fastqueueattr_getcontrolsiz*

Small Example:

```
#include <pthread.h>

/* Fastqueue attribute structure. */
pthread_fastqueueattr_t    my_fastqueue_attributes;
int                        status;

/* Provide the memory for the fastqueue control structure in the
   next fastqueue create call by placing it's address in the
   attributes structure "my_fastqueue_attributes". */
status = px5_pthread_fastqueueattr_setcontroladdr(
    &my_fastqueue_attributes, 0x50000, 100);

/* If status is PX5_SUCCESS, the next fastqueue creation using
   these attributes will use memory at address 0x50000 for the
   internal fastqueue control structure. */
```

px5_pthread_fastqueueattr_setname

C Prototype:

```
#include <pthread.h>

int px5_pthread_fastqueueattr_setname(pthread_fastqueueattr_t
                                     * fastqueue_attributes, char * fastqueue_name);
```

Description:

This pthreads+ service sets the fastqueue name in the specified attribute structure.

API Parameters:

<code>fastqueue_attributes</code>	Pointer to the fastqueue attributes.
<code>fastqueue_name</code>	Pointer to the supplied fastqueue name.

Return Codes:

PX5_SUCCESS (0)	Successful fastqueue attributes fastqueue name set.
EINVAL	Invalid fastqueue attributes.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(44 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_pthread_fastqueue_**, *px5_pthread_fastqueueattr_**,
px5_pthread_fastqueueattr_getname

Small Example:

```
#include <pthread.h>

/* Fastqueue attribute structure. */
pthread_fastqueueattr_t  my_fastqueue_attributes;
int                      status;

/* Set the fastqueue name in the fastqueue attributes structure
   "my_fastqueue_attributes". */
status = px5_pthread_fastqueueattr_setname(
        &my_fastqueue_attributes, "my_fastqueue_name");

/* If status is PX5_SUCCESS, "my_fastqueue_name" is set in the
   fastqueue attribute structure. */
```

px5_pthread_fastqueueattr_setqueueaddr

C Prototype:

```
#include <pthread.h>

int px5_pthread_fastqueueattr_setqueueaddr(pthread_fastqueueattr_t*
                                           fastqueue_attributes,
                                           void * fastqueue_memory_address,
                                           size_t fastqueue_memory_size);
```

Description:

This pthreads+ service sets the internal fastqueue message memory address to the address specified by *fastqueue_memory_address*. This address will subsequently be used to supply the memory for the message area on the next fastqueue created with this attribute structure.



Note that each fastqueue created must have its own unique fastqueue memory area. Hence, the memory address supplied here is only valid for one px5_pthread_fastqueue_create call.

API Parameters:

<code>fastqueue_attributes</code>	Pointer to the fastqueue attributes.
<code>fastqueue_memory_address</code>	Pointer to the fastqueue message memory area address.
<code>fastqueue_memory_size</code>	Size of specified fastqueue memory area.

Return Codes:

PX5_SUCCESS (0)	Successful specification of fastqueue memory.
EINVAL	Invalid fastqueue attributes or invalid size of fastqueue message memory area.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(44 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_pthread_fastqueue_**, *px5_pthread_fastqueueattr_**,

Small Example:

```
#include <pthread.h>

/* Fastqueue attribute structure. */
pthread_fastqueueattr_t my_fastqueue_attributes;
int status;

/* Set the fastqueue message memory area address in the
   fastqueue attributes structure "my_fastqueue_attributes". */
status = px5_pthread_fastqueueattr_setqueueaddr(
    &my_fastqueue_attributes, 0x90000, 1024);
```

px5_pthread_information_get

C Prototype:

```
#include <pthread.h>

int  px5_pthread_information_get(pthread_t thread_handle,
                                char **name, int * state, int * priority,
                                void ** stack_limit, void ** stack_pointer,
                                u_long * minimum_stack, pthread_t * next_thread);
```

Description:

This pthreads+ service retrieves the specified information from the specified thread.

API Parameters:

thread_handle	Handle of thread to get information about.
name	If non-NULL, destination for the specified thread's name.
state	If non-NULL, destination for the specified thread's current state. A value of zero indicates the thread is currently ready for execution. All other values indicate the thread is blocked, finished, or canceled.
priority	If non-NULL, destination for the specified thread's priority.
stack_limit	If non-NULL, destination for the specified thread's stack limit.
stack_pointer	If non-NULL, destination for the specified thread's current stack pointer.
minimum_stack	If non-NULL, destination for the specified thread's minimum stack value.
next_thread	If non-NULL, destination for the next created thread handle. This API can be called successively to examine all threads using this parameter until the original thread handle is detected.

Return Codes:

PX5_SUCCESS (0)	Successful information retrieval from the specified thread.
ESRCH	Invalid thread handle.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:

From the location where this API is called, there should be at least **(78 + PX5_BINDING_STACK_FRAME_SIZE)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable from the thread context and interrupt handlers (ISRs).

See Also:

*pthread_attr_**, *pthread_create*

Small Example:

```
#include <pthread.h>

pthread_t      my_thread_handle;
char *         name;
int            state;
int            priority;
void *         stack_limit;
void *         stack_pointer;
u_long         minimum_stack;
pthread_t      next_thread;
int            status;

/* Get information about "my_thread_handle". */
status = px5_pthread_information_get(&my_thread_handle,
                                     &name, &state, &priority, &stack_limit, &stack_pointer,
                                     &minimum_stack, &next_thread);

/* If status contains PX5_SUCCESS, the information about
   "my_thread_handle" is available for use. */
```

px5_pthread_memory_manager_enable

C Prototype:

```
#include <pthread.h>

int px5_pthread_memory_manager_enable(void);
```

Description:



This pthreads+ service enables full management of the remaining memory supplied by the application when the PX5 RTOS is started via *px5_pthread_start*. The memory allocated with all subsequent system object creation and destruction will be fully managed.



This API creates a PX5 RTOS variable-length memory pool to manage the memory. The application should not destroy this memory pool.



When this API is used, the stack requirements for all PX5 RTOS object creation APIs increase by approximately 360 bytes, and all object destruction APIs increase by approximately 230 bytes. These estimates are based on typical 32-bit architectures.

API Parameters:

none

Return Codes:

PX5_SUCCESS (0)	Successful enablement of internal PX5 RTOS memory management.
EINVAL	Invalid (NULL) pointer supplied to the <i>px5_pthread_start</i> API.
ENOMEM	Not enough memory remaining.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. This service does not result in preemption.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(246 + PX5_BINDING_STACK_FRAME_SIZE)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

px5_pthread_memory_manager_get, *px5_pthread_memory_manager_set*,
px5_pthread_start

Small Example:

```
#include <pthread.h>

/* Enable full management of PX5 RTOS memory. */
status = px5_pthread_memory_manager_enable();

/* If status is PX5_SUCCESS, all default memory
   allocation/deallocations are managed by the PX5 RTOS. */
```

px5_pthread_memory_manager_get

C Prototype:

```
#include <pthread.h>

int px5_pthread_memory_manager_get(
    void ** memory_allocate)(u_int type, u_long size),
    void (** memory_release)(u_int type, void *memory_to_release));
```

Description:

This pthreads+ service retrieves the internal memory manager allocate and release function pointers. These function pointers can be used to restore the previous memory manager allocation and deallocation selection.



The simple sequential memory allocation manager is set up by default when PX5 begins operation via px5_pthread_start. This memory manager does not support releasing memory. Hence, the memory_release function pointer is NULL.

API Parameters:

memory_allocate	Destination for current memory allocation function pointer.
memory_release	Destination for current memory release function pointer.

Return Codes:

PX5_SUCCESS (0)	Successful retrieval of memory manager function pointers.
EINVAL	Invalid (NULL) destination pointer(s) supplied.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. This service does not result in preemption.

Stack Usage Estimate:

From the location where this API is called, there should be at least **(44 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

px5_pthread_memory_manager_set, px5_pthread_start

Small Example:

```
#include <pthread.h>

void *(* my_memory_allocate)(u_int type, u_long size);
void (* my_memory_release)(u_int type, void *memory_to_release);
int status;

/* Retrieve the current memory manager function pointers. */
status = px5_pthread_memory_manager_get(&my_memory_allocate,
                                         &my_memory_release);

/* If status is PX5_SUCCESS, the "my_memory_allocate" and
   "my_memory_release" contain the current default memory manager
   allocate and release function pointers. */
```


px5_pthread_memory_manager_set

C Prototype:

```
#include <pthread.h>

int px5_pthread_memory_manager_set(
    void *(* memory_allocate)(u_int type, u_long size),
    void (* memory_release)(u_int type, void *memory_to_release));
```

Description:

This pthreads+ service installs the specified memory manager allocate and release function pointers.

API Parameters:

memory_allocate	Memory manager's allocation function pointer.
memory_release	Memory manager's memory release function pointer.

Return Codes:

PX5_SUCCESS (0)	Successful set up of memory manager function pointers.
EINVAL	Invalid (NULL) allocation function pointer supplied.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. This service does not result in preemption.

Stack Usage Estimate:

From the location where this API is called, there should be at least **(36 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

px5_pthread_memory_manager_get, *px5_pthread_start*

Small Example:

```

#include <pthread.h>

void *my_memory_allocate(u_int type, u_long size)
{
    void *memory;

    /* Assuming thread-safe malloc, simply malloc the memory. If not,
       the malloc and free must be protected by mutex. */
    memory = malloc(size);

    /* Return the memory. */
    return(memory);
}

void my_memory_release(u_int type, void *memory_to_release)
{
    /* Release the memory. */
    free(memory);
}

/* Set up the new memory manager allocate and release memory
   routines. */
status = px5_pthread_memory_manager_set(my_memory_allocate,
                                         my_memory_release);

/* If status is PX5_SUCCESS, the "my_memory_allocate" and
   "my_memory_release" memory manager routines will be used for default
   internal memory allocation and deallocation. */

```

px5_pthread_memorypool_allocate

C Prototype:

```
#include <pthread.h>

int px5_pthread_memorypool_allocate(pthread_memorypool_t *
                                   memorypool_handle, void ** allocated_memory,
                                   size_t request_size);
```

Description:

This pthreads+ service attempts to allocate memory from the specified memory pool. If there is insufficient memory in the pool, the calling thread is suspended until the request can be satisfied.



The memory in the memory pool is managed as a linked-list. The number of fragments in this linked-list is unknown and can vary over time. Hence, the allocation processing time is undeterministic, so this API should not be used in hard real-time situations.



This API is a cancellation point, meaning that if a cancellation is pending, it will be detected and executed by this API.

API Parameters:

<code>memorypool_handle</code>	Handle of the memory pool.
<code>allocated_memory</code>	Destination pointer to return the allocated memory.
<code>request_size</code>	Number of bytes requested.

Return Codes:

PX5_SUCCESS (0)	Successful memory allocation.
EINVAL	Invalid memory pool handle pointer, allocated memory destination pointer, or requested size.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. If the requested memory is available, no preemption takes place.



SUSPENSION. If the requested memory is unavailable, the calling thread is suspended until the memory becomes available.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(268 + PX5_BINDING_STACK_FRAME_SIZE)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the context of the application.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

px5_pthread_memorypool_, px5_pthread_memorypool_timedallocate, px5_pthread_memorypool_tryallocate, px5_pthread_memorypool_free*

Small Example:

```
#include <pthread.h>

/* Memory pool handle and pointer for allocated memory. */
pthread_memorypool_t    my_memorypool_handle;
void *                  allocated_memory;
int                     status;


/* Attempt to allocate 100 bytes from "my_memorypool_handle." */
status = px5_thread_memorypool_allocate(&my_memorypool_handle,
                                         &allocated_memory, 100);


/* If status is PX5_SUCCESS, a pointer to the allocated memory is
   in "allocated_memory." */
```

px5_pthread_memorypool_create

C Prototype:

```
#include <pthread.h>

int px5_pthread_memorypool_create(pthread_memorypool_t *
    memorypool_handle,
    pthread_memorypoolattr_t * memorypool_attributes,
    void *pool_start, size_t pool_size);
```

Description:



This pthreads+ service creates a variable-length memory pool with the optional memory pool attributes. If successful, the memory pool handle is set up for further use by the application.

API Parameters:

<code>memorypool_handle</code>	Handle of the memory pool.
<code>memorypool_attributes</code>	Optional memory pool attributes. This value is NULL if no memory pool attributes are specified.
<code>pool_start</code>	Starting address of the memory area for the pool. This address must be at least 4-byte aligned.
<code>pool_size</code>	Number of bytes in the memory area for the pool.

Return Codes:

PX5_SUCCESS (0)	Successful memory pool creation.
EINVAL	Invalid memory pool handle or pool starting address.
EBUSY	Memory pool is already created.
ENOMEM	Insufficient memory to create the memory pool.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:

From the location where this API is called, there should be at least **(244 + PX5_BINDING_STACK_FRAME_SIZE)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the context of the application.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_pthread_memorypoolattr_**, *px5_pthread_memorypool_destroy*

Small Example:

```
#include <pthread.h>

/* Memory pool handle. */
pthread_memorypool_t my_memorypool_handle;
int status;

/* Create a memory pool, the handle of which is returned in
   "my_memorypool_handle". */
status = px5_pthread_memorypool_create(&my_memorypool_handle,
                                       NULL, 0x40000, 8196);

/* If status is PX5_SUCCESS, the memory pool was created at address
   0x40000 with a size of 8,196 bytes. */
```


px5_pthread_memorypool_destroy

C Prototype:

```
#include <pthread.h>

int px5_pthread_memorypool_destroy(pthread_memorypool_t *
                                   memorypool_handle);
```

Description:

This pthreads+ service destroys the previously created memory pool specified by *memorypool_handle*. An error is returned if the memory pool has suspended threads waiting for memory.



To avoid unpredictable behavior, the application must ensure there is no further access to the memory pool (or memory allocated from the pool) after it is destroyed.

API Parameters:

`memorypool_handle` Handle of the memory pool to destroy.

Return Codes:

PX5_SUCCESS (0)	Successful memory pool destruction.
EINVAL	Invalid memory pool handle.
EBUSY	A thread is currently suspended on the memory pool.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(124 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the context of the application.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_pthread_memorypool_**, *px5_pthread_memorypool_create*

Small Example:

```
#include <pthread.h>

/* Memory pool handle. */
pthread_memorypool_t    my_memorypool_handle;
int                      status;

/* Destroy the memory pool referenced by
   "my_memorypool_handle". */
status = px5_pthread_memorypool_destroy(&my_memorypool_handle);

/* If status is PX5_SUCCESS, the memory pool was destroyed. */
```

px5_pthread_memorypool_free

C Prototype:

```
#include <pthread.h>

int px5_pthread_memorypool_free(void * memory_to_release);
```

Description:

This pthreads+ service releases the previously allocated memory back to the memory pool it was allocated from. Any threads that are suspended on the memory pool that are waiting for this memory are resumed.

API Parameters:

`memory_to_release` Pointer to previously allocated memory.

Return Codes:

PX5_SUCCESS (0)	Successful memory release.
EINVAL	Invalid memory pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. No preemption occurs if no other threads are waiting for memory from the associated memory pool.



PREEMPTION. If a higher-priority thread is waiting for this amount of memory, it is resumed, and preemption will occur.

Stack Usage Estimate:



From the location where this API is called, at least (**236 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_pthread_memorypool_**, *px5_pthread_memorypool_allocate*

Small Example:

```
#include <pthread.h>

/* Allocated memory pointer. */
void *      my_memory_pointer;
int         staus;

/* Release the memory pointed to by "my_memory_pointer." */
status = px5_pthread_memorypool_free(my_memory_pointer);

/* If status is PX5_SUCCESS, the memory was released. */
```

px5_pthread_memorypool_information_get

C Prototype:

```
#include <pthread.h>

int px5_pthread_memorypool_information_get(
    pthread_memorypool_t * memorypool_handle, char ** name,
    u_long * maximum_available, u_long * total_size,
    u_long * fragments,
    u_long * threads_suspended, pthread_t *first_suspended_thread,
    pthread_memorypool_t * next_memorypool_handle);
```

Description:

This pthreads+ service retrieves the specified information from the specified memory pool.

API Parameters:

memorypool_handle	Handle of memory pool to get information about.
name	If non-NULL, destination for the specified memory pool name.
maximum_available	If non-NULL, destination for the current maximum available number of bytes in the memory pool (not necessarily contiguous).
total_size	If non-NULL, destination for the total size of the memory pool.
fragments	If non-NULL, destination for the current number of fragments in the memory pool.
threads_suspended	If non-NULL, destination for the number of threads currently suspended on this memory pool.
first_suspended_thread	If non-NULL, destination for the thread handle of the first thread suspended on this memory pool.
next_memorypool_handle	If non-NULL, destination for the handle of the next created memory pool. This API can be called successively to examine all memory pool using

this parameter until the original memory pool handle is detected.

Return Codes:

PX5_SUCCESS (0)	Successful information retrieval from the specified memory pool.
ESRCH	Invalid memory pool handle.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(78 + PX5_BINDING_STACK_FRAME_SIZE)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable from the thread context and interrupt handlers (ISRs).

See Also:

*px5_pthread_memorypool_create, px5_*_information_get*

Small Example:

```

#include <pthread.h>

pthread_memorypool_t    my_memorypool_handle;
char *                  name;
u_long                  maximum_available;
u_long                  total_size;
u_long                  fragments;
u_long                  thread_suspended;
pthread_t               first_suspended_thread;
pthread_memorypool_t    next_memorypool;
int                     status;

/* Get information about "my_memorypool_handle". */
status =
    px5_pthread_memorypool_information_get(&my_memorypool_handle,
        &name, &maximum_available, &total_size, &fragments,
        &threads_suspended, &first_suspended_thread,
        &next_memorypool);

/* If status contains PX5_SUCCESS, the information about
   "my_memorypool_handle" is available for use. */

```

px5_pthread_memorypool_timedallocate

C Prototype:

```
#include <pthread.h>

int px5_pthread_memorypool_timedallocate(pthread_memorypool_t *
                                         memorypool_handle, void ** allocated_memory,
                                         size_t request_size, ticks_t timeout_ticks);
```

Description:

This pthreads+ service attempts to allocate memory from the specified memory pool. If there is insufficient memory in the pool, the calling thread is suspended until the request can be satisfied or the specified timeout expires.



The memory in the memory pool is managed as a linked-list. The number of fragments in this linked-list is unknown and can vary over time. Hence, the allocation processing time is undeterministic; therefore, this API should not be used in hard real-time situations.



This API is a cancellation point, meaning that if a cancellation is pending, it will be detected and executed by this API.

API Parameters:

memorypool_handle	Handle of the memory pool.
allocated_memory	Destination pointer to return the allocated memory.
request_size	Number of bytes requested.
timeout_ticks	Maximum number of timer ticks to wait for memory. A value of zero represents no timeout.

Return Codes:

PX5_SUCCESS (0)	Successful memory allocation.
-----------------	-------------------------------

EINVAL	Invalid memory pool handle pointer, allocated memory destination pointer, or requested size.
ETIMEDOUT	Timeout on waiting for a memory to become available.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. If the requested memory is available, no preemption takes place.



SUSPENSION. If the requested memory is unavailable, the calling thread is suspended until the memory becomes available or until the timeout expires.

Stack Usage Estimate:



From the location where this API is called, at least (**284 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_pthread_memorypool_**, *px5_pthread_memorypool_allocate*,
px5_pthread_memorypool_tryallocate, *px5_pthread_memorypool_free*

Small Example:

```
#include <pthread.h>

/* Memory pool handle and pointer for allocated memory. */
pthread_memorypool_t    my_memorypool_handle;
void *                  allocated_memory;
int                     status;


/* Attempt to allocate 100 bytes from "my_memorypool_handle." */
status =
    px5_pthread_memorypool_timedallocate(&my_memorypool_handle,
                                         &allocated_memory, 100, 200);


/* If status is PX5_SUCCESS, a pointer to the allocated memory is
   in "allocated_memory." */
```

px5_pthread_memorypool_tryallocate

C Prototype:

```
#include <pthread.h>

int px5_pthread_memorypool_tryallocate(pthread_memorypool_t *
                                       memorypool_handle, void ** allocated_memory,
                                       size_t request_size);
```

Description:

This pthreads+ service attempts to allocate memory from the specified memory pool. An error is returned if there is not enough memory in the pool.



The memory in the memory pool is managed as a linked-list. The number of fragments in this linked-list is unknown and can vary over time. Hence, the allocation processing time is undeterministic, so this API should not be used in hard real-time situations.

API Parameters:

memorypool_handle	Handle of the memory pool.
allocated_memory	Destination pointer to return the allocated memory.
request_size	Number of bytes requested.

Return Codes:

PX5_SUCCESS (0)	Successful memory allocation.
EINVAL	Invalid memory pool handle pointer, allocated memory destination pointer, or requested size.
ENOMEM	Not enough free memory to satisfy the request.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. No preemption takes place with this API.

Stack Usage Estimate:



From the location where this API is called, at least **(260 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_pthread_memorypool_**, *px5_pthread_memorypool_allocate*,
px5_pthread_memorypool_timedallocate

Small Example:

```
#include <pthread.h>

/* Memory pool handle and pointer for allocated memory. */
pthread_memorypool_t    my_memorypool_handle;
void *                  allocated_memory;
int                     status;


/* Attempt to allocate 100 bytes from "my_memorypool_handle." */
status = px5_thread_memorypool_tryallocate(&my_memorypool_handle,
                                           &allocated_memory, 100);


/* If status is PX5_SUCCESS, a pointer to the allocated memory is
   in "allocated_memory." */
```

px5_pthread_memorypoolattr_destroy

C Prototype:

```
#include <pthread.h>

int px5_pthread_memorypoolattr_destroy(pthread_memorypoolattr_t *
                                     memorypool_attributes);
```

Description:

This pthreads+ service destroys the previously created memory pool attributes structure pointed to by *memorypool_attributes*. Once destroyed, the memory pool attributes structure cannot be used again unless it is recreated.

API Parameters:

<code>memorypool_attributes</code>	Pointer to the memory pool attributes to destroy.
------------------------------------	---

Return Codes:

PX5_SUCCESS (0)	Successful memory pool attributes destroy.
EINVAL	Invalid memory pool attributes pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, at least (**36 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_pthread_memorypool_**, *px5_pthread_memorypoolattr_**,
px5_pthread_memorypoolattr_init

Small Example:

```
#include <pthread.h>

/* Memory pool attribute structure. */
pthread_memorypoolattr_t my_memorypool_attributes;
int status;

/* Destroy the memory pool attributes referenced by
   "my_memorypool_attributes". */
status = px5_pthread_memorypoolattr_destroy(
    &my_memorypool_attributes);

/* If status is PX5_SUCCESS, the memory pool attributes structure
   was destroyed. */
```

px5_pthread_memorypoolattr_getcontroladdr

C Prototype:

```
#include <pthread.h>

int px5_pthread_memorypoolattr_getcontroladdr(pthread_memorypoolattr_t
      * memorypool_attributes, void ** memorypool_control_address);
```

Description:

This pthreads+ service returns the previously supplied memory pool control structure memory address.

API Parameters:

memorypool_attributes	Pointer to the memory pool attributes.
memorypool_control_address	Pointer to the destination for the previously supplied memory pool control address.

Return Codes:

PX5_SUCCESS (0)	Successful memory pool attributes control address retrieval.
EINVAL	Invalid memory pool attributes or memory pool control address destination pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, at least (**44 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_pthread_memorypool_**, *px5_pthread_memorypoolattr_**,
px5_pthread_memorypoolattr_setcontroladdr,
px5_pthread_memorypoolattr_getcontrolsiz

Small Example:

```
#include <pthread.h>

/* Memory pool attribute structure. */
pthread_memorypoolattr_t  my_memorypool_attributes;
void *                    my_memorypool_control_address;
int                        status;

/* Get the memory pool control structure address in the
   attributes structure "my_memorypool_attributes". */
status = px5_pthread_memorypoolattr_getcontroladdr(
    &my_memorypool_attributes,
    &my_memorypool_control_address);

/* If status is PX5_SUCCESS, "my_memorypool_control_address"
   contains the address of the previously supplied memory pool
   control memory. */
```

px5_pthread_memorypoolattr_getcontrolsize

C Prototype:

```
#include <pthread.h>

int px5_pthread_memorypoolattr_getcontrolsize(pthread_memorypoolattr_t
        * memorypool_attributes, size_t * memorypool_control_size);
```

Description:

This pthreads+ service returns the size of the internal memory pool control structure. The primary purpose of this API is to inform the application how much memory is required for the *px5_pthread_memorypoolattr_setcontroladdr* API.

API Parameters:

<code>memorypool_attributes</code>	Pointer to the memory pool attributes.
<code>memorypool_control_size</code>	Pointer to the destination for the internal memory pool control structure size.

Return Codes:

PX5_SUCCESS (0)	Successful retrieval of internal memory pool control structure size.
EINVAL	Invalid memory pool attributes or memory pool control structure size destination pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, at least (**44 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_pthread_memorypool_**, *px5_pthread_memorypoolattr_**,
px5_pthread_memorypoolattr_setcontroladdr

Small Example:

```
#include <pthread.h>

/* Memory pool attribute structure. */
pthread_memorypoolattr_t  my_memorypool_attributes;
size_t                   my_memorypool_control_size;
int                       status;

/* Get the memory pool control structure memory size. */
status = px5_pthread_memorypoolattr_getcontrolsize(
    &my_memorypool_attributes, &my_memorypool_control_size);

/* If status is PX5_SUCCESS, "my_memorypool_control_size"
   contains the size of the internal memory pool
   control structure. */
```

px5_pthread_memorypoolattr_getname

C Prototype:

```
#include <pthread.h>

int px5_pthread_memorypoolattr_getname(pthread_memorypoolattr_t
    * memorypool_attributes, char ** memorypool_name);
```

Description:

This pthreads+ service returns the previously supplied memory pool name.

API Parameters:

memorypool_attributes	Pointer to the memory pool attributes.
memorypool_name	Pointer to the destination for the previous memory pool name.

Return Codes:

PX5_SUCCESS (0)	Successful memory pool attributes name retrieval.
EINVAL	Invalid memory pool attributes or memory pool name pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, at least (**44 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_pthread_memorypool_**, *px5_pthread_memorypoolattr_**

Small Example:

```
#include <pthread.h>

/* Memory pool attribute structure. */
pthread_memorypoolattr_t my_memorypool_attributes;
char * my_memorypool_name;
int status;

/* Get the previous memory pool name. */
status = px5_pthread_memorypoolattr_getname(
    &my_memorypool_attributes, &my_memorypool_name);

/* If status is PX5_SUCCESS, "my_memorypool_name" contains the
   name previously supplied. */
```

px5_pthread_memorypoolattr_init

C Prototype:

```
#include <pthread.h>

int px5_pthread_memorypoolattr_init(pthread_memorypoolattr_t *
                                   memorypool_attributes);
```

Description:

This pthreads+ service initializes the memory pool attributes structure with default memory pool creation values.

API Parameters:

<code>memorypool_attributes</code>	Pointer to the memory pool attributes structure to create.
------------------------------------	--

Return Codes:

PX5_SUCCESS (0)	Successful memory pool attributes structure creation.
EINVAL	Invalid memory pool attributes pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, at least (**36 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_pthread_memorypool_**, *px5_pthread_memorypoolattr_**,
px5_pthread_memorypoolattr_destroy

Small Example:

```
#include <pthread.h>

/* Memory pool attribute structure. */
pthread_memorypoolattr_t  my_memorypool_attributes;
int                        status;

/* Create the memory pool attributes structure
   "my_memorypool_attributes". */
status =
    px5_pthread_memorypoolattr_init(&my_memorypool_attributes);

/* If status is PX5_SUCCESS, the "my_memorypool_attributes"
   structure is ready for use. */
```

px5_thread_memorypoolattr_setcontroladdr

C Prototype:

```
#include <pthread.h>

int px5_thread_memorypoolattr_setcontroladdr(pthread_memorypoolattr_t
      * memorypool_attributes, void * memorypool_control_address,
      size_t memorypool_control_size);
```

Description:

This pthreads+ service enables the developer to provide the memory for the internal PX5 RTOS memory pool structure as specified by the address in the *memorypool_control_address* parameter. This memory will subsequently be used for the next memory pool created with this attribute structure. The memory size required for the internal memory pool control structure can be found via a call to the *px5_thread_memorypoolattr_getcontrolsize* service.



Note that each memory pool created must have its own unique control structure memory. Hence, the memory pool control address supplied here is only valid for one px5_thread_memorypool_create call.



The exact amount of memory required for the internal memory pool control structure can be determined by including *px5.h* and using the *sizeof(px5_memorypool_control)* construct.

API Parameters:

<code>memorypool_attributes</code>	Pointer to the memory pool attributes.
<code>memorypool_control_address</code>	Pointer to the supplied memory pool control structure memory.
<code>memorypool_control_size</code>	Size of the memory specified.

Return Codes:

PX5_SUCCESS (0)	Successful internal memory pool control address set.
EINVAL	Invalid memory pool attributes or memory size.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:

From the location where this API is called, at least (**44 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_pthread_memorypool_**, *px5_pthread_memorypoolattr_**,
px5_pthread_memorypoolattr_getcontrolsiz

Small Example:

```
#include <pthread.h>

/* Memory pool attribute structure. */
pthread_memorypoolattr_t  my_memorypool_attributes;
int                        status;

/* Provide the memory for the memory pool control structure in the
   next memory pool create call by placing its address in the
   attributes structure "my_memorypool_attributes". */
status = pthread_memorypoolattr_setcontroladdr(
    &my_memorypool_attributes, 0x60000, 170);

/* If status is PX5_SUCCESS, the next memory pool creation using
   these attributes will use memory at address 0x60000 for the
   internal memory pool control structure. */
```

px5_pthread_memorypoolattr_setname

C Prototype:

```
#include <pthread.h>

int px5_pthread_memorypoolattr_setname(pthread_memorypoolattr_t
                                     * memorypool_attributes, char * memorypool_name);
```

Description:

This pthreads+ service sets the memory pool name in the specified attribute structure.

API Parameters:

memorypool_attributes	Pointer to the memory pool attributes.
memorypool_name	Pointer to the supplied memory pool name.

Return Codes:

PX5_SUCCESS (0)	Successful memory pool attributes name set.
EINVAL	Invalid memory pool attributes.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, at least (**44 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_pthread_memorypool_**, *px5_pthread_memorypoolattr_**,
px5_pthread_memorypoolattr_getname

Small Example:

```
#include <pthread.h>

/* Memory pool attribute structure. */
pthread_memorypoolattr_t  my_memorypool_attributes;
int                        status;

/* Set the memory pool name in the memory pool attributes structure
   "my_memorypool_attributes". */
status = px5_pthread_memorypoolattr_setname(
        &my_memorypool_attributes, "my_memorypool_name");

/* If status is PX5_SUCCESS, "my_memorypool_name" is set in the
   memory pool attribute structure. */
```

px5_pthread_mutex_information_get

C Prototype:

```
#include <pthread.h>

int px5_pthread_mutex_information_get(pthread_mutex_t * mutex_handle,
    char ** name, u_long * nested_count, u_long * maximum_nesting,
    pthread_t * owner,
    u_long * threads_suspended, pthread_t *first_suspended_thread,
    pthread_mutex_t * next_mutex_handle);
```

Description:

This pthreads+ service retrieves the specified information from the specified mutex.

API Parameters:

mutex_handle	Handle of mutex to get information about.
name	If non-NULL, destination for the specified mutex name.
nested_count	If non-NULL, destination for the current nested ownership count of the mutex.
maximum_nesting	If non-NULL, destination for the maximum nesting of the mutex. This will be one unless the mutex was created with the type of <i>PTHREAD_MUTEX_RECURSIVE</i> .
owner	If non-NULL, destination for the thread handle of the current owner of the mutex.
threads_suspended	If non-NULL, destination for the number of threads currently suspended on this mutex.
first_suspended_thread	If non-NULL, destination for the thread handle of the first thread suspended on this mutex.
next_mutex_handle	If non-NULL, destination for the handle of the next created mutex. This API can be called successively to examine all mutexes using this parameter until the original mutex handle is detected.

Return Codes:

PX5_SUCCESS (0)	Successful information retrieval from the specified mutex.
ESRCH	Invalid mutex handle.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:

From the location where this API is called, there should be at least **(78 + PX5_BINDING_STACK_FRAME_SIZE)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable from the thread context and interrupt handlers (ISRs).

See Also:

*pthread_mutex_init, px5_*_information_get*

Small Example:

```

#include <pthread.h>

pthread_mutex_t      my_mutex_handle;
char *               name;
u_long               nested_count;
u_long               maximum_nesting;
pthread_t            owner;
u_long               thread_suspended;
pthread_t            first_suspended_thread;
pthread_mutex_t      next_mutex;
int                  status;

/* Get information about "my_mutex_handle". */
status =
    px5_pthread_mutex_information_get(&my_mutex_handle,
    &name, &nested_count, &maximum_nesting, &owner,
    &threads_suspended, &first_suspended_thread,
    &next_mutex);

/* If status contains PX5_SUCCESS, the information about
   "my_mutex_handle" is available for use. */

```

px5_pthread_mutexattr_getcontroladdr

C Prototype:

```
#include <pthread.h>

int px5_pthread_mutexattr_getcontroladdr(pthread_mutexattr_t
                                         *mutex_attributes, void ** mutex_control_address);
```

Description:

This pthreads+ service returns the previously supplied mutex control structure address.

API Parameters:

<code>mutex_attributes</code>	Pointer to the mutex attributes.
<code>mutex_control_address</code>	Pointer to the destination for the previously supplied mutex control address.

Return Codes:

PX5_SUCCESS (0)	Successful mutex attributes mutex control address retrieval.
EINVAL	Invalid mutex attributes or mutex control address destination pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, at least (**44 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*pthread_mutex_**, *pthread_mutexattr_**,
px5_pthread_mutexattr_setcontroladdr,
px5_pthread_mutexattr_getcontrolsize

Small Example:

```
#include <pthread.h>

/* Mutex attribute structure. */
pthread_mutexattr_t  my_mutex_attributes;
void *               my_mutex_control_address;
int                  status;

/* Get the mutex control structure address in the mutex
   attributes structure "my_mutex_attributes". */
status = px5_pthread_mutexattr_getcontroladdr(&my_mutex_attributes,
                                              &my_mutex_control_address);

/* If status is PX5_SUCCESS, "my_mutex_control_address"
   contains the address of the previously supplied mutex
   control memory. */
```

px5_pthread_mutexattr_getcontrolsize

C Prototype:

```
#include <pthread.h>

int px5_pthread_mutexattr_getcontrolsize(pthread_mutexattr_t
                                         *mutex_attributes, size_t * mutex_control_size);
```

Description:

This pthreads+ service returns the size of the internal mutex control structure. The primary purpose of this API is to inform the application how much memory is required for the *px5_pthread_mutexattr_setcontroladdr* API.

API Parameters:

<code>mutex_attributes</code>	Pointer to the mutex attributes.
<code>mutex_control_size</code>	Pointer to the destination for the internal mutex control structure size.

Return Codes:

PX5_SUCCESS (0)	Successful retrieval of internal mutex control structure size.
EINVAL	Invalid mutex attributes or mutex control structure size destination pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, at least (**44 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*pthread_mutex_**, *pthread_mutexattr_**,
px5_pthread_mutexattr_setcontroladdr,
px5_pthread_mutexattr_getcontroladdr

Small Example:

```
#include <pthread.h>

/* Mutex attribute structure. */
pthread_mutexattr_t  my_mutex_attributes;
size_t               my_mutex_control_size;
int                  status;

/* Get the mutex control structure memory size. */
status = px5_pthread_mutexattr_getcontrolsize(&my_mutex_attributes,
                                              &my_mutex_control_size);

/* If status is PX5_SUCCESS, "my_mutex_control_size"
   contains the size of the internal mutex
   control structure. */
```

px5_pthread_mutexattr_getname

C Prototype:

```
#include <pthread.h>

int px5_pthread_mutexattr_getname(pthread_mutexattr_t
                                *mutex_attributes, char ** mutex_name);
```

Description:

This pthreads+ service returns the previously supplied mutex name.

API Parameters:

mutex_attributes	Pointer to the mutex attributes.
mutex_name	Pointer to the destination for the previous mutex name.

Return Codes:

PX5_SUCCESS (0)	Successful mutex attributes mutex name retrieval.
EINVAL	Invalid mutex attributes or mutex name pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, at least (**44 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*pthread_mutex_**, *pthread_mutexattr_**, *px5_pthread_mutexattr_setname*

Small Example:

```
#include <pthread.h>

/* Mutex attribute structure. */
pthread_mutexattr_t  my_mutex_attributes;
char *               my_mutex_name;
int                  status;

/* Get the previous mutex name. */
status = px5_pthread_mutexattr_getname(&my_mutex_attributes,
                                       &my_mutex_name);

/* If status is PX5_SUCCESS, "my_mutex_name" contains the
   name previously supplied. */
```

px5_pthread_mutexattr_setcontroladdr

C Prototype:

```
#include <pthread.h>

int px5_pthread_mutexattr_setcontroladdr(pthread_mutexattr_t
                                         *mutex_attributes, void * mutex_control_address,
                                         size_t mutex_control_size);
```

Description:

This pthreads+ service enables the developer to provide the memory for the internal PX5 RTOS mutex structure, as specified by the address in the *mutex_control_address* parameter. This memory will subsequently be used for the next mutex created with this attribute structure. The memory size required for the internal mutex control structure can be found via a call to the *px5_pthread_mutexattr_getcontrolsize* service.



Note that each mutex created must have its own unique mutex control structure memory. Hence, the mutex control memory supplied here is only valid for one pthread_mutex_init call.



The exact amount of memory required for the internal mutex control structure can be determined by including *px5.h* and using the *sizeof(px5_mutex_control)* construct.

API Parameters:

<code>mutex_attributes</code>	Pointer to the mutex attributes.
<code>mutex_control_address</code>	Pointer to the supplied mutex control structure memory.
<code>mutex_control_size</code>	Size of the specified mutex control structure memory.

Return Codes:

PX5_SUCCESS (0)	Successful internal mutex control address set.
EINVAL	Invalid mutex attributes or internal mutex control memory size.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:

From the location where this API is called, at least (**44 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*pthread_mutex_**, *pthread_mutexattr_**, *pthread_mutexattr_getcontroladdr*, *pthread_mutexattr_getcontrolsiz*

Small Example:

```
#include <pthread.h>

/* Mutex attribute structure. */
pthread_mutexattr_t    my_mutex_attributes;
int                    status;

/* Set the mutex control structure address in the mutex
   attributes structure "my_mutex_attributes". */
status = pthread_mutexattr_setcontroladdr(&my_mutex_attributes,
                                           0x70000, 90);

/* If status is PX5_SUCCESS, the mutex creation using these
   attributes will use address 0x70000 for the internal mutex
   control structure. */
```


px5_pthread_mutexattr_setname

C Prototype:

```
#include <pthread.h>

int px5_pthread_mutexattr_setname(pthread_mutexattr_t
                                  *mutex_attributes, char * mutex_name);
```

Description:

This pthreads+ service sets the mutex name in the specified attribute structure.

API Parameters:

mutex_attributes	Pointer to the mutex attributes.
mutex_name	Pointer to the supplied mutex name.

Return Codes:

PX5_SUCCESS (0)	Successful mutex attributes mutex name set.
EINVAL	Invalid mutex attributes.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, at least (**44 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*pthread_mutex_**, *pthread_mutexattr_**, *px5_pthread_mutexattr_getname*

Small Example:

```
#include <pthread.h>

/* Mutex attribute structure. */
pthread_mutexattr_t my_mutex_attributes;
int status;

/* Set the mutex name in the mutex attributes structure
   "my_mutex_attributes". */
status = px5_pthread_mutexattr_setname(&my_mutex_attributes,
                                       "my_mutex_name");

/* If status is PX5_SUCCESS, "my_mutex_name" is set in the
   mutex attribute structure. */
```

px5_pthread_partitionpool_allocate

C Prototype:

```
#include <pthread.h>

int px5_pthread_partitionpool_allocate(pthread_partitionpool_t *
                                     partitionpool_handle, void ** allocated_memory,
                                     size_t request_size);
```

Description:

This pthreads+ service attempts to allocate a memory partition from the specified partition memory pool. If there is no partition memory in the pool, the calling thread is suspended until the request can be satisfied.



This API is a cancellation point, meaning that if a cancellation is pending, it will be detected and executed by this API.

API Parameters:

partitionmemorypool_handle	Handle of the partition memory pool.
allocated_memory	Destination pointer to return the allocated memory partition.
request_size	Number of bytes requested (must be less than the partition size of the specified partition pool).

Return Codes:

PX5_SUCCESS (0)	Successful memory partition allocation.
EINVAL	Invalid partition memory pool handle pointer, allocated memory destination pointer, or requested size.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. If the requested partition memory is available, no preemption takes place.



SUSPENSION. If the requested partition memory is unavailable, the calling thread is suspended until a memory partition becomes available.

Stack Usage Estimate:



From the location where this API is called, at least (**104 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_pthread_partitionpool_**, *px5_pthread_partitionpool_timedallocate*,
px5_pthread_partitionpool_tryallocate, *px5_pthread_partitionpool_free*

Small Example:

```
#include <pthread.h>

/* Partition memory pool handle and pointer for allocated memory. */
pthread_partitionpool_t my_partitionpool_handle;
void * allocated_memory;
int status;

/* Attempt to allocate a memory partition from
   "my_partitionpool_handle." */
status = px5_thread_partitionpool_allocate(
        &my_partitionpool_handle,
        &allocated_memory, 100);

/* If status is PX5_SUCCESS, a pointer to the allocated memory
   partition is in "allocated_memory." */
```

px5_pthread_partitionpool_create

C Prototype:

```
#include <pthread.h>

int px5_pthread_partitionpool_create(pthread_partitionpool_t *
    partitionpool_handle,
    pthread_partitionpoolattr_t * partitionpool_attributes,
    void *pool_start, size_t pool_size, size_t partition_size);
```

Description:



This pthreads+ service creates a fixed-length partition memory pool with the specified partition size and optional partition memory pool attributes. If successful, the partition memory pool handle is set up for further use by the application.

API Parameters:

<code>partitionpool_handle</code>	Handle of the partition memory pool.
<code>partitionpool_attributes</code>	Optional partition memory pool attributes. This value is NULL if no partition memory pool attributes are specified.
<code>pool_start</code>	Starting address of the memory area for the pool. This address must be at least 4-byte aligned.
<code>pool_size</code>	Number of bytes in the memory area for the pool.
<code>partition_size</code>	Size in bytes of each memory partition in the pool.

Return Codes:

PX5_SUCCESS (0)	Successful partition memory pool creation.
EINVAL	Invalid partition memory pool handle or pool starting address.

EBUSY	Partition memory pool is already created.
ENOMEM	Insufficient memory to create the partition memory pool.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, at least (**164 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_pthread_partitionpoolattr_**, *px5_pthread_partitionpool_destroy*

Small Example:

```
#include <pthread.h>

/* Partition memory pool handle. */
pthread_partitionpool_t my_partitionpool_handle;
int status;

/* Create a partition memory pool with 100-byte partitions, the
   handle of which is returned in "my_partitionpool_handle". */
status = px5_thread_partitionpool_create(
    &my_partitionpool_handle, NULL, 0x50000, 8196, 100);

/* If status is PX5_SUCCESS, the memory pool was created at address
   0x50000 with 100-byte fixed-sized partitions. */
```


px5_pthread_partitionpool_destroy

C Prototype:

```
#include <pthread.h>

int px5_pthread_partitionpool_destroy(pthread_partitionpool_t *
                                     partitionpool_handle);
```

Description:

This pthreads+ service destroys the previously created partition memory pool specified by *partitionpool_handle*. An error is returned if the partition memory pool has any suspended threads waiting for memory.



To avoid unpredictable behavior, the application must ensure there is no further access to the partition memory pool (or previously allocated memory partitions) after it is destroyed.

API Parameters:

<code>partitionpool_handle</code>	Handle of the partition memory pool to destroy.
-----------------------------------	---

Return Codes:

PX5_SUCCESS (0)	Successful partition memory pool destruction.
EINVAL	Invalid partition memory pool handle.
EBUSY	A thread is currently suspended on the partition memory pool.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, at least (**60 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_pthread_partitionpool_**, *px5_pthread_partitionpool_create*

Small Example:

```
#include <pthread.h>

/* Partition memory pool handle. */
pthread_partitionpool_t my_partitionpool_handle;
int status;

/* Destroy the partition memory pool referenced by
   "my_partitionpool_handle". */
status = px5_pthread_partitionpool_destroy(
    &my_partitionpool_handle);

/* If status is PX5_SUCCESS, the partition memory pool was
   destroyed. */
```

px5_pthread_partitionpool_free

C Prototype:

```
#include <pthread.h>

int px5_pthread_partitionpool_free(void * memory_to_release);
```

Description:

This pthreads+ service releases the previously allocated partition memory back to the partition memory pool from which it was allocated. If a thread is suspended on this partition memory pool, the memory partition is given to the suspended thread, and it is resumed.

API Parameters:

<code>memory_to_release</code>	Pointer to previously allocated partition memory.
--------------------------------	---

Return Codes:

<code>PX5_SUCCESS (0)</code>	Successful memory partition release.
<code>EINVAL</code>	Invalid memory partition pointer.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. No preemption occurs if no other threads are waiting for partition memory from the associated partition memory pool.



PREEMPTION. If a higher-priority thread is waiting for a memory partition, it is resumed, and preemption will occur.

Stack Usage Estimate:



From the location where this API is called, at least (**112 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable from the thread context and interrupt handlers (ISRs).

See Also:

*px5_pthread_partitionpool_**, *px5_pthread_partitionpool_allocate*

Small Example:

```
#include <pthread.h>

/* Allocated memory partition pointer. */
void *      my_memory_pointer;
int         staus;

/* Release the memory partition pointed to by
   "my_memory_partition_pointer." */
status = px5_pthread_partitionpool_free(
        my_memory_partition_pointer);

/* If status is PX5_SUCCESS, the memory partition was released. */
```

px5_pthread_partitionpool_information_get

C Prototype:

```
#include <pthread.h>

int px5_pthread_partitionpool_information_get(
    pthread_partitionpool_t * partitionpool_handle, char ** name,
    u_long * available_partitions, u_long * total_partitions,
    u_long * partition_size,
    u_long * threads_suspended, pthread_t *first_suspended_thread,
    pthread_partitionpool_t * next_partitionpool_handle);
```

Description:

This pthreads+ service retrieves the specified information from the specified partition memory pool.

API Parameters:

partitionpool_handle	Handle of the partition memory pool to get information about.
name	If non-NULL, destination for the specified partition memory pool name.
available_partitions	If non-NULL, destination for the current number of available partitions in the partition memory pool.
total_partitions	If non-NULL, destination for the total number of memory partitions in the pool.
partition_size	If non-NULL, destination for the size of each memory partition in the pool.
threads_suspended	If non-NULL, destination for the number of threads currently suspended on this partition memory pool.
first_suspended_thread	If non-NULL, destination for the thread handle of the first thread suspended on this partition memory pool.
next_partitionpool_handle	If non-NULL, destination for the handle of the

next created partition memory pool. This API can be called successively to examine all partition memory pools using this parameter until the original handle is detected.

Return Codes:

PX5_SUCCESS (0)	Successful information retrieval from the specified partition memory pool.
ESRCH	Invalid partition memory handle.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(78 + PX5_BINDING_STACK_FRAME_SIZE)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable from the thread context and interrupt handlers (ISRs).

See Also:

*px5_pthread_partitionpool_create, px5_*_information_get*

Small Example:

```

#include <pthread.h>

pthread_partitionpool_t    my_partitionpool_handle;
char *                     name;
u_long                    available_partitions;
u_long                    total_partitions;
u_long                    partition_size;
u_long                    thread_suspended;
pthread_t                 first_suspended_thread;
pthread_partitionpool_t    next_partitionpool;
int                        status;

/* Get information about "my_partitionpool_handle". */
status =
    px5_pthread_partitionpool_information_get(&my_partitionpool_handle,
        &name, &available_partitions, &total_partitions,
        &partition_size,
        &threads_suspended, &first_suspended_thread,
        &next_partitionpool);

/* If status contains PX5_SUCCESS, the information about
   "my_partitionpool_handle" is available for use. */

```

px5_pthread_partitionpool_timedallocate

C Prototype:

```
#include <pthread.h>

int px5_pthread_partitionpool_timedallocate(pthread_partitionpool_t *
                                             partitionpool_handle, void ** allocated_memory,
                                             size_t request_size, ticks_t timeout_ticks);
```

Description:

This pthreads+ service attempts to allocate a memory partition from the specified partition memory pool. If there are no memory partitions in the pool, the calling thread is suspended until the request can be satisfied or the specified timeout expires.



This API is a cancellation point, meaning that if a cancellation is pending, it will be detected and executed by this API.

API Parameters:

partitionpool_handle	Handle of the partition memory pool.
allocated_memory	Destination pointer to return the allocated memory partition.
request_size	Number of bytes requested (this must be less than or equal to the partition size of the specified pool).
timeout_ticks	Maximum number of timer ticks to wait for a memory partition. A value of zero represents no timeout.

Return Codes:

PX5_SUCCESS (0)	Successful memory partition allocation.
EINVAL	Invalid partition memory pool handle pointer, allocated memory partition destination pointer, or requested size.

ETIMEDOUT	Timeout on waiting for a memory partition to become available.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. If the requested memory partition is available, no preemption takes place.



SUSPENSION. If the requested memory partition is unavailable, the calling thread is suspended until a memory partition becomes available or until the timeout expires.

Stack Usage Estimate:



From the location where this API is called, at least (**120 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_pthread_partitionpool_**, *px5_pthread_partitionpool_allocate*,
px5_pthread_partitionpool_tryallocate, *px5_pthread_partitionpool_free*

Small Example:

```
#include <pthread.h>

/* Partition memory pool handle and pointer for allocated memory
   partition. */
pthread_partitionpool_t my_partitionpool_handle;
void * allocated_memory;
int status;

/* Attempt to allocate a partition at least 100 bytes in size from
   "my_partitionpool_handle" and wait for a maximum of 200 ticks.
*/
status = px5_pthread_partitionpool_timedallocate(
    &my_partitionpool_handle,
    &allocated_memory, 100, 200);

/* If status is PX5_SUCCESS, a pointer to the allocated memory
   partition is in "allocated_memory." */
```

px5_pthread_partitionpool_tryallocate

C Prototype:

```
#include <pthread.h>

int px5_pthread_partitionpool_tryallocate(pthread_partitionpool_t *
                                         partitionpool_handle, void ** allocated_memory,
                                         size_t request_size);
```

Description:

This pthreads+ service attempts to allocate a memory partition from the specified partition memory pool. An error is returned if there are no memory partitions in the pool.

API Parameters:

partitionpool_handle	Handle of the partition memory pool.
allocated_memory	Destination pointer to return the allocated memory partition.
request_size	Number of bytes requested (this must be less than or equal to the partition size of the specified pool).

Return Codes:

PX5_SUCCESS (0)	Successful memory partition allocation.
EINVAL	Invalid partition memory pool handle pointer, allocated memory partition destination pointer, or requested size.
ENOMEM	No memory partition available to satisfy the request.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. No preemption takes place with this API.

Stack Usage Estimate:



From the location where this API is called, at least (**40 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable from the thread context and interrupt handlers (ISRs).

See Also:

*px5_pthread_partitionpool_**, *px5_pthread_partitionpool_allocate*,
px5_pthread_partitionpool_timedallocate

Small Example:

```
#include <pthread.h>

/* Partition memory pool handle and pointer for allocated memory. */
pthread_partitionpool_t my_partitionpool_handle;
void * allocated_memory;
int status;

/* Attempt to allocate a memory partition of at least 100 bytes
   in size from "my_partitionpool_handle." */
status = px5_pthread_partitionpool_tryallocate(
    &my_partitionpool_handle,
    &allocated_memory, 100);

/* If status is PX5_SUCCESS, a pointer to the allocated memory
   partition is in "allocated_memory." */
```

px5_pthread_partitionpoolattr_destroy

C Prototype:

```
#include <pthread.h>

int px5_pthread_partitionpoolattr_destroy(pthread_partitionpoolattr_t *
                                         partitionpool_attributes);
```

Description:

This pthreads+ service destroys the previously created partition memory pool attributes structure pointed to by *partitionpool_attributes*. Once destroyed, the partition memory pool attributes structure cannot be used again unless it is recreated.

API Parameters:

partitionpool_attributes Pointer to the partition memory pool attributes to destroy.

Return Codes:

PX5_SUCCESS (0)	Successful partition memory pool attributes destroy.
EINVAL	Invalid partition memory pool attributes pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, at least (**44 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_pthread_partitionpool_**, *px5_pthread_partitionpoolattr_**,
px5_pthread_partitionpoolattr_init

Small Example:

```
#include <pthread.h>

/* Partition memory pool attribute structure. */
pthread_partitionpoolattr_t  my_partitionpool_attributes;
int                           status;

/* Destroy the partition memory pool attributes referenced by
   "my_partitionpool_attributes". */
status = px5_pthread_partitionpoolattr_destroy(
        &my_partitionpool_attributes);

/* If status is PX5_SUCCESS, the partition memory pool attributes
   structure was destroyed. */
```

px5_pthread_partitionpoolattr_getcontroladdr

C Prototype:

```
#include <pthread.h>

int px5_pthread_partitionpoolattr_getcontroladdr(
    pthread_partitionpoolattr_t * memorypool_attributes,
    void ** partitionpool_control_address);
```

Description:

This pthreads+ service returns the previously supplied partition memory pool control structure memory address.

API Parameters:

partitionpool_attributes	Pointer to the partition memory pool attributes.
partitionpool_control_address	Pointer to the destination for the previously supplied partition memory pool control address.

Return Codes:

PX5_SUCCESS (0)	Successful partition memory pool attributes control address retrieval.
EINVAL	Invalid partition memory pool attributes, or partition memory pool control address destination pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, at least (**44 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_thread_partitionpool_**, *px5_thread_partitionpoolattr_**,
px5_thread_partitionpoolattr_setcontroladdr,
px5_thread_partitionpoolattr_getcontrolsize

Small Example:

```
#include <pthread.h>

/* Partition memory pool attribute structure. */
pthread_partitionpoolattr_t    my_partitionpool_attributes;
void *                         my_partitionpool_control_address;
int                             status;

/* Get the partition memory pool control structure address in the
   attributes structure "my_partitionpool_attributes". */
status = px5_thread_partitionpoolattr_getcontroladdr(
    &my_partitionpool_attributes,
    &my_partitionpool_control_address);

/* If status is PX5_SUCCESS, "my_partitionpool_control_address"
   contains the address of the previously supplied partition
   memory pool control memory. */
```

px5_pthread_partitionpoolattr_getcontrolsize

C Prototype:

```
#include <pthread.h>

int px5_pthread_partitionpoolattr_getcontrolsize(
    pthread_partitionpoolattr_t * memorypool_attributes,
    size_t * memorypool_control_size);
```

Description:

This pthreads+ service returns the size of the internal partition memory pool control structure. The primary purpose of this API is to inform the application how much memory is required for the *px5_pthread_partitionpoolattr_setcontroladdr* API.

API Parameters:

<code>partitionpool_attributes</code>	Pointer to the partition memory pool attributes.
<code>partitionpool_control_size</code>	Pointer to the destination for the internal partition memory pool control structure size.

Return Codes:

PX5_SUCCESS (0)	Successful retrieval of internal partition memory pool control structure size.
EINVAL	Invalid partition memory pool attributes or memory pool control structure size destination pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, at least (**44 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_pthread_partitionpool_**, *px5_pthread_partitionpoolattr_**,
px5_pthread_partitionpoolattr_setcontroladdr

Small Example:

```
#include <pthread.h>

/* Partition memory pool attribute structure. */
pthread_partitionpoolattr_t    my_partitionpool_attributes;
size_t                        my_partitionpool_control_size;
int                             status;

/* Get the partition memory pool control structure memory size. */
status = px5_pthread_partitionpoolattr_getcontrolsize(
    &my_partitionpool_attributes, &my_partitionpool_control_size);

/* If status is PX5_SUCCESS, "my_partitionpool_control_size"
   contains the size of the internal partition memory pool
   control structure. */
```

px5_pthread_partitionpoolattr_getname

C Prototype:

```
#include <pthread.h>

int px5_pthread_partitionpoolattr_getname(pthread_partitionpoolattr_t
                                           * partitionpool_attributes,
                                           char ** partitionpool_name);
```

Description:

This pthreads+ service returns the previously supplied partition memory pool name.

API Parameters:

partitionpool_attributes	Pointer to the partition memory pool attributes.
partitionpool_name	Pointer to the destination for the previous partition memory pool name.

Return Codes:

PX5_SUCCESS (0)	Successful partition memory pool attributes name retrieval.
EINVAL	Invalid partition memory pool attributes or partition memory pool name pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, at least (**44 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_pthread_partitionpool_**, *px5_pthread_partitionpoolattr_**

Small Example:

```
#include <pthread.h>

/* Partition memory pool attribute structure. */
pthread_partitionpoolattr_t my_partitionpool_attributes;
char * my_partitionpool_name;
int status;

/* Get the previous partition memory pool name. */
status = px5_pthread_partitionpoolattr_getname(
    &my_partitionpool_attributes, &my_partitionpool_name);

/* If status is PX5_SUCCESS, "my_partitionpool_name" contains the
   name previously supplied. */
```

px5_pthread_partitionpoolattr_init

C Prototype:

```
#include <pthread.h>

int px5_pthread_partitionpoolattr_init(pthread_partitionpoolattr_t *
                                       partitionpool_attributes);
```

Description:

This pthreads+ service initializes the partition memory pool attributes structure with default partition memory pool creation values.

API Parameters:

`partitionpool_attributes` Pointer to the partition memory pool attributes structure to create.

Return Codes:

PX5_SUCCESS (0)	Successful partition memory pool attributes structure creation.
EINVAL	Invalid partition memory pool attributes pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, at least (**36 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_pthread_partitionpool_**, *px5_pthread_partitionpoolattr_**,
px5_pthread_partitionpoolattr_destroy

Small Example:

```
#include <pthread.h>

/* Partition memory pool attribute structure. */
pthread_partitionpoolattr_t    my_partitionpool_attributes;
int                             status;

/* Create the partition memory pool attributes structure
   "my_partitionpool_attributes". */
status =
px5_pthread_partitionpoolattr_init(&my_partitionpool_attributes);

/* If status is PX5_SUCCESS, the "my_partitionpool_attributes"
   structure is ready for use. */
```

px5_pthread_partitionpoolattr_setcontroladdr

C Prototype:

```
#include <pthread.h>

int px5_pthread_partitionpoolattr_setcontroladdr(
    pthread_partitionpoolattr_t * partitionpool_attributes,
    void * partitionpool_control_address,
    size_t partitionpool_control_size);
```

Description:

This pthreads+ service enables the developer to provide the memory for the internal PX5 RTOS partition memory pool structure as specified by the address in the *partitionpool_control_address* parameter. This memory will subsequently be used for the next partition memory pool created with this attribute structure. The memory size required for the internal memory pool control structure can be found via a call to the *px5_pthread_partitionpoolattr_getcontrolsize* service.



*Note that each partition memory pool created must have its own unique control structure memory. Hence, the partition memory pool control address supplied here is only valid for one *px5_pthread_partitionpool_create* call.*



The exact amount of memory required for the internal partition memory pool control structure can be determined by including *px5.h* and using the *sizeof(px5_partitionpool_control)* construct.

API Parameters:

<code>partitionpool_attributes</code>	Pointer to the partition memory pool attributes.
<code>partitionpool_control_address</code>	Pointer to the supplied partition memory pool control structure memory.
<code>partitionpool_control_size</code>	Size of the memory specified.

Return Codes:

PX5_SUCCESS (0)	Successful internal partition memory pool control address set.
EINVAL	Invalid partition memory pool attributes or memory size.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:

From the location where this API is called, at least (**44 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_pthread_partitionpool_**, *px5_pthread_partitionpoolattr_**,
px5_pthread_partitionpoolattr_getcontrolsiz

Small Example:

```
#include <pthread.h>

/* Partition memory pool attribute structure. */
pthread_partitionpoolattr_t    my_partitionpool_attributes;
int                             status;

/* Provide the memory for the partition memory pool control
   structure in the next partition memory pool create call by
   placing it's address in the attributes structure
   "my_partitionpool_attributes". */
status = pthread_partitionpoolattr_setcontroladdr(
    &my_partitionpool_attributes, 0x80000, 70);

/* If status is PX5_SUCCESS, the next partition memory pool
   creation using these attributes will use memory at address
   0x80000 for the internal partition memory pool control
   structure. */
```

px5_pthread_partitionpoolattr_setname

C Prototype:

```
#include <pthread.h>

int px5_pthread_memorypoolattr_setname(pthread_memorypoolattr_t
                                     * memorypool_attributes, char * memorypool_name);
```

Description:

This pthreads+ service sets the partition memory pool name in the specified attribute structure.

API Parameters:

partitionpool_attributes	Pointer to the partition memory pool attributes.
partitionpool_name	Pointer to the supplied partition memory pool name.

Return Codes:

PX5_SUCCESS (0)	Successful partition memory pool attributes name set.
EINVAL	Invalid partition memory pool attributes.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, at least (**44 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_pthread_partitionpool_**, *px5_pthread_partitionpoolattr_**,
px5_pthread_partitionpoolattr_getname

Small Example:

```
#include <pthread.h>

/* Partition memory pool attribute structure. */
pthread_partitionpoolattr_t    my_partitionpool_attributes;
int                             status;

/* Set the partition memory pool name in the memory pool attributes
   structure "my_partitionpool_attributes". */
status = px5_pthread_partitionpoolattr_setname(
    &my_partitionpool_attributes,
    "my_partitionpool_name");

/* If status is PX5_SUCCESS, "my_partitionpool_name" is set in the
   partition memory pool attribute structure. */
```

px5_pthread_priority_change

C Prototype:

```
#include <pthread.h>

int px5_pthread_priority_change(pthread_t thread_handle,
                                int new_priority, int * old_priority);
```

Description:

This pthread+ service changes the priority of the specified thread. If the *old_priority* parameter is non-NULL, the previous priority is placed in the specified destination.

API Parameters:

<code>thread_handle</code>	Handle of thread to change priority.
<code>new_priority</code>	New priority for the specified thread, where larger numeric values represent higher priority. By default, there are 32 priority levels, from 0 through 31.
<code>old_priority</code>	If non-NULL, destination pointer to return the thread's previous priority.

Return Codes:

PX5_SUCCESS (0)	Successful thread priority change.
ESRCH	Thread handle is not valid.
EINVAL	Specified priority is invalid.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. If the new priority is not higher than the calling thread, there is no preemption.



PREEMPTION. If the new priority is higher than the calling thread's and the specified thread is in a state ready for execution, preemption will occur.

Stack Usage Estimate:



From the location where this API is called, at least **(124 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

pthread_create, px5_pthread_attr_setpriority

Small Example:

```
#include <pthread.h>

pthread_t      my_thread_handle;
int            status;

/* Change the priority of "my_thread_handle" to 16. */
status = px5_pthread_priority_change(my_thread_handle, 16, NULL);

/* If status contains PX5_SUCCESS, "my_thread_handle" now has a
   priority of 16. */
```

px5_thread_resume

C Prototype:

```
#include <pthread.h>

int px5_thread_resume(pthread_t thread_handle);
```

Description:

This pthread+ service resumes a previously suspended thread. If the specified thread is not suspended, an error is returned.

API Parameters:

thread_handle	Handle of previously suspended thread.
---------------	--

Return Codes:

PX5_SUCCESS (0)	Successful thread resumption.
ESRCH	Thread handle is not valid.
EINVAL	Specified thread is not suspended.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption if the thread resumed is not a higher priority.



PREEMPTION. If a higher-priority thread is resumed, preemption will occur.

Stack Usage Estimate:



From the location where this API is called, at least (**80 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable from the thread context and interrupt handlers (ISRs).

See Also:

px5_pthread_suspend

Small Example:

```
#include <pthread.h>

pthread_t      my_thread_handle;
int            status;

/* Resume the previously suspended thread. */
status = px5_pthread_resume(my_thread_handle);

/* If status contains PX5_SUCCESS, the thread is now resumed. */
```


px5_pthread_stack_check

C Prototype:

```
#include <pthread.h>

int  px5_pthread_stack_check(pthread_t thread_handle,
                             u_long *  minimum_available_stack);
```

Description:

This pthread+ service calculates the minimum available stack of the specified thread. If the size is less than the constant `PX5_BINDING_STACK_FRAME_SIZE`, the central error handler is called by this service.

API Parameters:

<code>thread_handle</code>	Handle of thread to check.
<code>minimum_available_stack</code>	Destination of where to return the minimum amount of stack for the specified thread.

Return Codes:

PX5_SUCCESS (0)	Successful thread stack check.
ESRCH	Thread handle is not valid.
EINVAL	Specified destination for the minimum available stack is invalid (NULL).
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, at least (**60 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

px5_pthread_resume

Small Example:

```
#include <pthread.h>

int          status;
u_long      my_minimum_stack;

/* Calculate the minimum available stack for the currently
   executing thread. */
status= px5_pthread_stack_check(pthread_self(), &my_minimum_stack);

/* If status contains PX5_SUCCESS, the minimum available stack for
   the current thread is in "my_minimum_stack". */
```

px5_pthread_start

C Prototype:

```
#include <pthread.h>

int px5_pthread_start(u_long run_time_id, void * memory_start,
                    u_long memory_size);
```

Description:

This pthreads+ service starts the PX5 RTOS. It should be called early in C *main*. Once invoked, all internal data structures are initialized and prepared for operation. The initialization processing also includes verification of the binding layer – the processor/compiler-specific layer of PX5. If the start process is successful, the C *main* function is upscaled into the first thread in the system.



The optional *memory_start* and *memory_size* parameters provide the PX5 RTOS with memory to sequentially allocate from. For simple applications without object deletion, this memory allocation scheme is sufficient.

API Parameters:

`run_time_id`

This is a developer-supplied run time identification of this particular execution instance. Ideally, the value supplied here is generated from a True Random Number Generator (TRNG) such that it is unique for each execution of an application via PX5. Note that this value is an integral component of the Pointer/Data Verification (PDV) of PX5.

`memory_start`

Optional pointer to the memory provided by the application for internal memory allocation by PX5. This pointer should be aligned to at least the word size of the underlying processor architecture. If the application calls the `px5_user_memory_manager_override` service, the *memory_start* pointer may be NULL.

<code>memory_size</code>	Optional memory size supplied by the application.
--------------------------	---

Return Codes:

PX5_SUCCESS (0)	Successful PX5 start—C <i>main</i> is now a thread!
EMVSERR	System error—invalid processor binding.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. Even though C *main* is upscaled into the first thread, there is no actual preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, at least (**64 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only once from the C *main* entry function or sub-functions.

See Also:

*pthread_**, *pthread_create*

Small Example:

```
#include <pthread.h>

/* Define some memory for PX5. */
unsigned long  memory_area[1024];

unsigned long  main_thread_counter;

int main(void)
{
    int  status;

    /* Call the PX5 start function. */
    status = px5_pthread_start(memory_area, sizeof(memory_area));

    /* Check completion status. */
    if (status != PX5_SUCCESS)
    {
        printf("Error starting PX5!\n");
        exit(1);
    }

    /* When we return, "main" is now the first PX5 thread.
       Simply loop incrementing the counter. */
    while(1)
    {
        /* Increment the main thread's counter. */
        main_thread_counter++;
    }
}
```

px5_pthread_suspend

C Prototype:

```
#include <pthread.h>

int px5_pthread_suspend(pthread_t thread_handle);
```

Description:

This pthread+ service suspends the specified thread. If the specified thread is already suspended, an error is returned.



Care should be taken to avoid suspending threads in critical execution paths, such as holding needed system resources or making PX5 RTOS API calls. This API should generally be used by the executing thread or on free-running threads.

API Parameters:

thread_handle	Handle of thread to suspend.
---------------	------------------------------

Return Codes:

PX5_SUCCESS (0)	Successful thread suspension.
ESRCH	Thread handle is not valid.
EINVAL	Specified thread is already suspended.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. If the thread suspended is not the executing thread, there is no preemption.



SUSPENSION. If the currently executing thread calls this service, it is suspended until another thread calls *px5_pthread_resume*.

Stack Usage Estimate:



From the location where this API is called, at least (**60 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable from the thread context and interrupt handlers (ISRs).

See Also:

px5_pthread_resume

Small Example:

```
#include <pthread.h>

int                                status;

/* Suspend the the currently executing thread. */
status = px5_pthread_suspend(pthread_self());

/* If status contains PX5_SUCCESS, the current thread was
   suspended. */
```

px5_pthread_tick_sleep

C Prototype:

```
#include <pthread.h>

int px5_pthread_tick_sleep(tick_t ticks_to_sleep);
```

Description:

This pthreads+ service suspends the calling thread for the number of timer ticks specified by *ticks_to_sleep*. If a value of 0 ticks is supplied, this service returns immediately, i.e., the calling thread is not suspended.



This API is a cancellation point, meaning that if a cancellation is pending, it will be detected and executed by this API.

API Parameters:

ticks_to_sleep

This specifies the number of ticks the calling thread will be suspended for. If a value of zero is supplied, this service returns immediately.

Return Codes:

PX5_SUCCESS (0)
EACCES

Successful thread tick sleep.
Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. If the number of ticks to sleep is zero, no preemption occurs.



SUSPENSION. If the number of ticks to sleep is non-zero, the calling thread is suspended for that amount of ticks.



PREEMPTION. After the number of ticks has expired, the calling thread is resumed and will preempt the currently running thread if it is a higher priority.

Stack Usage Estimate:



From the location where this API is called, at least (**60 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

sleep, nanosleep, usleep

Small Example:

```
#include <pthread.h>

unsigned long    my_example_thread_counter;

/* Define example thread. */
void *  my_example_thread_start(void *arguments)
{
    /* Loop forever incrementing a counter and sleeping for 100
       ticks. */
    while (1)
    {
        /* Increment my example thread's counter. */
        my_example_thread_counter++;

        /* Sleep for 1000 ticks, which is 1 second @1ms tick rate. */
        px5_pthread_tick_sleep(100);
    }
}
```

px5_pthread_ticks_get

C Prototype:

```
#include <pthread.h>

tick_t px5_pthread_ticks_get(void);
```

Description:

This pthreads+ service returns the internal tick count, representing the number of timer interrupts.

API Parameters:

none

Return Codes:

current tick count

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. No preemption takes place as a result of this service.

Stack Usage Estimate:



From the location where this API is called, at least (**8 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

px5_pthread_ticks_set

Small Example:

```
#include <pthread.h>

unsigned long    current_tick_count;

/* Pickup the current tick count. */
current_tick_count = px5_pthread_ticks_get();
```

px5_thread_ticktimer_create

C Prototype:

```
#include <pthread.h>

int px5_thread_ticktimer_create(pthread_ticktimer_t *ticktimer_handle,
    pthread_ticktimerattr_t * attributes,
    void (*expiration_routine)(pthread_ticktimer_t *, void *),
    void *argument, tick_t initial_ticks, tick_t reload_ticks);
```

Description:



This pthreads+ service creates a ticktimer that calls the specified expiration routine after the specified number of initial ticks. If a non-zero value is supplied for reload, this ticktimer will operate periodically.



The ticktimer is created in a stopped state. To start the ticktimer, please call the `px5_thread_ticktimer_start` API after successful creation.

API Parameters:

<code>ticktimer_handle</code>	Upon successful completion, returned handle of the ticktimer.
<code>attributes</code>	Optional attributes for the ticktimer creation.
<code>expiration_routine</code>	Application routine to call when the ticktimer expires.
<code>argument</code>	Argument pointer that is passed verbatim when the application expiration routine is called.
<code>initial_ticks</code>	Number of initial ticks before expiration. This value must be non-zero.
<code>reload_ticks</code>	Number of ticks for periodic expiration. If this value is zero, this ticktimer is a one-shot ticktimer.

Return Codes:

PX5_SUCCESS (0)	Successful ticktimer creation.
EINVAL	Invalid ticktimer handle pointer, expiration routine, or initial ticks value.
ENOMEM	Insufficient memory to create ticktimer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:

From the location where this API is called, at least (**156 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_ticktimer_**, *px5_ticktimer_destroy*

Small Example:

```
#include <pthread.h>

/* Ticktimer handle. */
pthread_ticktimer_t  my_ticktimer_handle;
int                  status;

/* Create a ticktimer that first expires after 10 ticks and then
   every 5 ticks thereafter. Upon each expiration, the
   "my_expiration_routine" will be called. */
status = px5_pthread_ticktimer_create(&my_ticktimer_handle, NULL,
                                       my_expiration_routine, 10, 5);

/* If status is PX5_SUCCESS, the ticktimer was created and is ready
   to be started via px5_pthread_ticktimer_start. */
```

px5_pthread_ticktimer_destroy

C Prototype:

```
#include <pthread.h>

int px5_pthread_ticktimer_destroy(pthread_ticktimer_t
                                *ticktimer_handle);
```

Description:

This pthreads+ service destroys a previously created but stopped ticktimer.



To avoid unpredictable behavior, the application must ensure there is no further access to the ticktimer after it is destroyed.

API Parameters:

`ticktimer_handle` Handle of the ticktimer to destroy.

Return Codes:

PX5_SUCCESS (0)	Successful ticktimer destroy.
EINVAL	Invalid ticktimer handle.
EBUSY	Ticktimer is not stopped, i.e., it is still active.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, at least (**60 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_ticktimer_**, *px5_ticktimer_create*

Small Example:

```
#include <pthread.h>

/* Ticktimer handle. */
pthread_ticktimer_t my_ticktimer_handle;
int status;

/* Destroy the previously created ticktimer "my_ticktimer_handle". */
status = px5_pthread_ticktimer_destroy(&my_ticktimer_handle);

/* If status is PX5_SUCCESS, the ticktimer was destroyed. */
```


px5_pthread_ticktimer_information_get

C Prototype:

```
#include <pthread.h>

int px5_pthread_ticktimer_information_get(
    pthread_ticktimer_t * ticktimer_handle, char ** name,
    int * active, tick_t * remaining_ticks, tick_t * reload_ticks,
    pthread_ticktimer_t * next_ticktimer_handle);
```

Description:

This pthreads+ service retrieves the specified information from the specified ticktimer.

API Parameters:

ticktimer_handle	Handle of the ticktimer to get information about.
name	If non-NULL, destination for the specified ticktimer name.
active	If non-NULL, destination for the active flag of the ticktimer (<i>PX5_TRUE</i> indicates the timer is active, while <i>PX5_FALSE</i> indicates the timer is not active).
remaining_ticks	If non-NULL, destination for the number of remaining ticks left before this ticktimer expires.
reload_ticks	If non-NULL, destination for the number of ticks this ticktimer is reloaded with after it expires. For one-shot timers, this value is always zero.
next_ticktimer_handle	If non-NULL, destination for the handle of the next created ticktimer. This API can be called successively to examine all ticktimers using this parameter until the original handle is detected.

Return Codes:

PX5_SUCCESS (0)	Successful information retrieval from the specified ticktimer.
------------------------	--

ESRCH

Invalid ticktimer handle.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:

From the location where this API is called, there should be at least **(78 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable from the thread context and interrupt handlers (ISRs).

See Also:

px5_pthread_ticktimer_create, *px5_*_information_get*

Small Example:

```
#include <pthread.h>

pthread_t my_ticktimer_handle;
char * name;
int active;
u_long remaining_ticks;
u_long reload_ticks;
pthread_t next_ticktimer;
int status;

/* Get information about "my_ticktimer_handle". */
status =
    px5_thread_ticktimer_information_get(&my_ticktimer_handle,
        &name, &active, &remaining_ticks, &reload_ticks,
        &next_ticktimer);

/* If status contains PX5_SUCCESS, the information about
   "my_ticktimer_handle" is available for use. */
```

px5_pthread_ticktimer_start

C Prototype:

```
#include <pthread.h>

int px5_pthread_ticktimer_start(pthread_ticktimer_t *ticktimer_handle);
```

Description:

This pthreads+ service starts a previously created ticktimer.

API Parameters:

ticktimer_handle	Handle of the ticktimer to start.
------------------	-----------------------------------

Return Codes:

PX5_SUCCESS (0)	Successful ticktimer start.
EINVAL	Invalid ticktimer handle.
EBUSY	Ticktimer was already started.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, at least (**52 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_ticktimer_**, *px5_ticktimer_stop*

Small Example:

```
#include <pthread.h>

/* Ticktimer handle. */
pthread_ticktimer_t my_ticktimer_handle;
int status;

/* Start the previously created ticktimer "my_ticktimer_handle". */
status = px5_pthread_ticktimer_start(&my_ticktimer_handle);

/* If status is PX5_SUCCESS, the ticktimer was started. */
```

px5_pthread_ticktimer_stop

C Prototype:

```
#include <pthread.h>

int px5_pthread_ticktimer_stop(pthread_ticktimer_t *ticktimer_handle);
```

Description:

This pthreads+ service stops a previously created ticktimer. When stopped, the remaining number of ticks before expiration is saved. If the ticktimer is restarted, it is restarted with the previous remaining number of ticks.

API Parameters:

<code>ticktimer_handle</code>	Handle of the ticktimer to stop.
-------------------------------	----------------------------------

Return Codes:

PX5_SUCCESS (0)	Successful ticktimer stop.
EINVAL	Invalid ticktimer handle, or the ticktimer was already stopped.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, at least (**52 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_ticktimer_**, *px5_ticktimer_start*

Small Example:

```
#include <pthread.h>

/* Ticktimer handle. */
pthread_ticktimer_t my_ticktimer_handle;
int status;

/* Stop the previously created ticktimer "my_ticktimer_handle". */
status = px5_pthread_ticktimer_stop(&my_ticktimer_handle);

/* If status is PX5_SUCCESS, the ticktimer was stopped. */
```

px5_pthread_ticktimer_update

C Prototype:

```
#include <pthread.h>

int px5_pthread_ticktimer_update(pthread_ticktimer_t *ticktimer_handle,
                                tick_t initial_ticks, tick_t reload_ticks);
```

Description:

This pthreads+ service updates the initial ticks and reload ticks of a previously created but stopped ticktimer.

API Parameters:

<code>ticktimer_handle</code>	Handle of ticktimer to update.
<code>initial_ticks</code>	Updated number of initial ticks before expiration. This value must be non-zero.
<code>reload_ticks</code>	Updated number of ticks for periodic expiration. If this value is zero, this ticktimer is a one-shot ticktimer.

Return Codes:

PX5_SUCCESS (0)	Successful ticktimer update.
EINVAL	Invalid ticktimer handle pointer or new initial ticks value.
EBUSY	Ticktimer is running. It must be stopped to use this service.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, at least (**40 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_ticktimer_**, *px5_ticktimer_stop*

Small Example:

```
#include <pthread.h>

/* Ticktimer handle. */
pthread_ticktimer_t  my_ticktimer_handle;
int                  status

/* Update a ticktimer to first expire after 20 ticks and then every 2
   ticks thereafter. */
status = px5_pthread_ticktimer_update(&my_ticktimer_handle, 20, 2);

/* If status is PX5_SUCCESS, the ticktimer was updated with the new
   initial and reload values. */
```

px5_pthread_ticktimerattr_destroy

C Prototype:

```
#include <pthread.h>

int px5_pthread_ticktimerattr_destroy(pthread_ticktimerattr_t *
                                     ticktimer_attributes);
```

Description:

This pthreads+ service destroys the previously created ticktimer attributes structure pointed to by *ticktimer_attributes*. Once destroyed, the ticktimer attributes structure cannot be used again unless it is recreated.

API Parameters:

<i>ticktimer_attributes</i>	Pointer to the ticktimer attributes to destroy.
-----------------------------	---

Return Codes:

PX5_SUCCESS (0)	Successful ticktimer attributes destroy.
EINVAL	Invalid ticktimer attributes pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, at least (**36 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_pthread_ticktimer_**, *px5_pthread_ticktimerattr_**,
px5_pthread_ticktimerattr_init

Small Example:

```
#include <pthread.h>

/* Ticktimer attribute structure. */
pthread_ticktimerattr_t  my_ticktimer_attributes;
int                       status;

/* Destroy the ticktimer attributes referenced by
   "my_ticktimer_attributes". */
status = px5_pthread_ticktimerattr_destroy(
                                               &my_ticktimer_attributes);

/* If status is PX5_SUCCESS, the ticktimer attributes structure
   was destroyed. */
```

px5_pthread_ticktimerattr_getcontroladdr

C Prototype:

```
#include <pthread.h>

int px5_pthread_ticktimerattr_getcontroladdr(pthread_ticktimerattr_t
      * ticktimer_attributes, void ** ticktimer_control_address);
```

Description:

This pthreads+ service returns the previously supplied ticktimer control structure memory address.

API Parameters:

ticktimer_attributes	Pointer to the ticktimer attributes.
ticktimer_control_address	Pointer to the destination for the previously supplied ticktimer control address.

Return Codes:

PX5_SUCCESS (0)	Successful ticktimer attributes ticktimer control address retrieval.
EINVAL	Invalid ticktimer attributes or ticktimer control address destination pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, at least (**44 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_pthread_ticktimer_**, *px5_pthread_ticktimerattr_**,
px5_pthread_ticktimerattr_setcontroladdr,
px5_pthread_ticktimerattr_getcontrolsize

Small Example:

```
#include <pthread.h>

/* Ticktimer attribute structure. */
pthread_ticktimerattr_t  my_ticktimer_attributes;
void *                   my_ticktimer_control_address;
int                      status;

/* Get the ticktimer control structure address in the ticktimer
   attributes structure "my_ticktimer_attributes". */
status = px5_pthread_ticktimerattr_getcontroladdr(
    &my_ticktimer_attributes,
    &my_ticktimer_control_address);

/* If status is PX5_SUCCESS, "my_ticktimer_control_address"
   contains the address of the previously supplied ticktimer
   control memory. */
```

px5_pthread_ticktimerattr_getcontrolsize

C Prototype:

```
#include <pthread.h>

int px5_pthread_ticktimerattr_getcontrolsize(pthread_ticktimerattr_t
      *ticktimer_attributes, size_t * ticktimer_control_size);
```

Description:

This pthreads+ service returns the size of the internal ticktimer control structure. The primary purpose of this API is to inform the application how much memory is required for the *px5_pthread_ticktimerattr_setcontroladdr* API.

API Parameters:

<code>ticktimer_attributes</code>	Pointer to the ticktimer attributes.
<code>ticktimer_control_size</code>	Pointer to the destination for the internal ticktimer control structure size.

Return Codes:

PX5_SUCCESS (0)	Successful retrieval of internal ticktimer control structure size.
EINVAL	Invalid ticktimer attributes or ticktimer control structure size destination pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, at least (**44 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_pthread_ticktimer_**, *px5_pthread_ticktimerattr_**,
px5_pthread_ticktimerattr_setcontroladdr

Small Example:

```
#include <pthread.h>

/* Ticktimer attribute structure. */
pthread_ticktimerattr_t  my_ticktimer_attributes;
size_t                   my_ticktimer_control_size;
int                       status;

/* Get the ticktimer control structure memory size. */
status = px5_pthread_ticktimerattr_getcontrolsize(
    &my_ticktimer_attributes, &my_ticktimer_control_size);

/* If status is PX5_SUCCESS, "my_ticktimer_control_size"
   contains the size of the internal ticktimer
   control structure. */
```

px5_pthread_ticktimerattr_getname

C Prototype:

```
#include <pthread.h>

int px5_pthread_ticktimerattr_getname(pthread_ticktimerattr_t
                                     *ticktimer_attributes, char ** ticktimer_name);
```

Description:

This pthreads+ service returns the previously supplied ticktimer name.

API Parameters:

ticktimer_attributes	Pointer to the ticktimer attributes.
ticktimer_name	Pointer to the destination for the previous ticktimer name.

Return Codes:

PX5_SUCCESS (0)	Successful ticktimer attributes ticktimer name retrieval.
EINVAL	Invalid ticktimer attributes or ticktimer name pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, at least (**44 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_pthread_ticktimer_**, *px5_pthread_ticktimerattr_**

Small Example:

```
#include <pthread.h>

/* Ticktimer attribute structure. */
pthread_ticktimerattr_t  my_ticktimer_attributes;
char *                   my_ticktimer_name;
int                      status;

/* Get the previous ticktimer name. */
status = px5_pthread_ticktimerattr_getname(
    &my_ticktimer_attributes, &my_ticktimer_name);

/* If status is PX5_SUCCESS, "my_ticktimer_name" contains the
   name previously supplied. */
```

px5_pthread_ticktimerattr_init

C Prototype:

```
#include <pthread.h>

int px5_pthread_ticktimerattr_init(pthread_ticktimerattr_t *
                                   ticktimer_attributes);
```

Description:

This pthreads+ service initializes the ticktimer attributes structure with default ticktimer creation values.

API Parameters:

<code>ticktimer_attributes</code>	Pointer to the ticktimer attributes structure to create.
-----------------------------------	--

Return Codes:

PX5_SUCCESS (0)	Successful ticktimer attributes structure creation.
EINVAL	Invalid ticktimer attributes pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, at least (**36 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_thread_ticktimer_**, *px5_thread_ticktimerattr_**,
px5_thread_ticktimerattr_destroy

Small Example:

```
#include <pthread.h>

/* Ticktimer attribute structure. */
pthread_ticktimerattr_t  my_ticktimer_attributes;
int                      status;

/* Create the ticktimer attributes structure
   "my_ticktimer_attributes". */
status = px5_thread_ticktimerattr_init(&my_ticktimer_attributes);

/* If status is PX5_SUCCESS, the "my_ticktimer_attributes"
   structure is ready for use. */
```

px5_pthread_ticktimerattr_setcontroladdr

C Prototype:

```
#include <pthread.h>

int px5_pthread_ticktimerattr_setcontroladdr(pthread_ticktimerattr_t
      * ticktimer_attributes, void * ticktimer_control_address,
      size_t ticktimer_control_size);
```

Description:

This pthreads+ service enables the developer to provide the memory for the internal PX5 RTOS ticktimer structure as specified by the address in the *ticktimer_control_address* parameter. This memory will subsequently be used for the next ticktimer created with this attribute structure. The memory size required for the internal ticktimer control structure can be found via a call to the *px5_pthread_ticktimerattr_getcontrolsize* service.



*Note that each ticktimer created must have its own unique ticktimer control structure memory. Hence, the ticktimer control address supplied here is only valid for one *px5_pthread_ticktimer_create* call.*



The exact amount of memory required for the internal ticktimer control structure can be determined by including *px5.h* and using the *sizeof(px5_ticktimer_control)* construct.

API Parameters:

<code>ticktimer_attributes</code>	Pointer to the ticktimer attributes.
<code>ticktimer_control_address</code>	Pointer to the supplied ticktimer control structure memory.
<code>ticktimer_control_size</code>	Size of the memory specified.

Return Codes:

PX5_SUCCESS (0)	Successful internal ticktimer control address set.
EINVAL	Invalid ticktimer attributes or memory size.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:

From the location where this API is called, at least (**44 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_pthread_ticktimer_**, *px5_pthread_ticktimerattr_**,
px5_pthread_ticktimerattr_getcontrolsiz

Small Example:

```
#include <pthread.h>

/* Ticktimer attribute structure. */
pthread_ticktimerattr_t  my_ticktimer_attributes;
int                      status;

/* Provide the memory for the ticktimer control structure in the
   next ticktimer create call by placing it's address in the
   attributes structure "my_ticktimer_attributes". */
status = pthread_ticktimerattr_setcontroladdr(
    &my_ticktimer_attributes, 0x90000, 90);

/* If status is PX5_SUCCESS, the next ticktimer creation using
   these attributes will use memory at address 0x90000 for the
   internal ticktimer control structure. */
```

px5_pthread_ticktimerattr_setname

C Prototype:

```
#include <pthread.h>

int px5_pthread_ticktimerattr_setname(pthread_ticktimerattr_t
                                     * ticktimer_attributes, char * ticktimer_name);
```

Description:

This pthreads+ service sets the ticktimer name in the specified attribute structure.

API Parameters:

<code>ticktimer_attributes</code>	Pointer to the ticktimer attributes.
<code>ticktimer_name</code>	Pointer to the supplied ticktimer name.

Return Codes:

PX5_SUCCESS (0)	Successful ticktimer attributes ticktimer name set.
EINVAL	Invalid ticktimer attributes.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, at least (**44 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_pthread_ticktimer_**, *px5_pthread_ticktimerattr_**,
px5_pthread_ticktimerattr_getname

Small Example:

```
#include <pthread.h>

/* Ticktimer attribute structure. */
pthread_ticktimerattr_t  my_ticktimer_attributes;
int                      status;

/* Set the ticktimer name in the ticktimer attributes structure
   "my_ticktimer_attributes". */
status = px5_pthread_ticktimerattr_setname(
        &my_ticktimer_attributes, "my_ticktimer_name");

/* If status is PX5_SUCCESS, "my_ticktimer_name" is set in the
   ticktimer attribute structure. */
```


px5_pthread_timeslice_change

C Prototype:

```
#include <pthread.h>

int  px5_pthread_timeslice_change(pthread_t thread_handle,
                                   u_long new_timeslice, u_long * previous_timeslice);
```

Description:

This pthread+ service changes the timeslice of the specified thread. If the *old_timeslice* parameter is non-NULL, the previous priority is placed in the specified destination.

API Parameters:

<code>thread_handle</code>	Handle of thread to change timeslice.
<code>new_timeslice</code>	New timeslice for the specified thread, where a value of 0 disables timeslicing for the thread.
<code>old_timeslice</code>	If non-NULL, destination pointer to return the thread's previous timeslice.

Return Codes:

PX5_SUCCESS (0)	Successful thread timeslice change.
ESRCH	Thread handle is not valid.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. If the new priority is not higher than the calling thread, there is no preemption.



PREEMPTION. If the new priority is higher than the calling thread's and the specified thread is in a state ready for execution, preemption will occur.

Stack Usage Estimate:



From the location where this API is called, at least (**124 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

pthread_create, px5_pthread_attr_settimeslice

Small Example:

```
#include <pthread.h>

pthread_t      my_thread_handle;
int            status;

/* Change the timeslice of "my_thread_handle" to 10. */
status = px5_pthread_timeslice_change(my_thread_handle, 10, NULL);

/* If status contains PX5_SUCCESS, "my_thread_handle" now has a
   timeslice of 10. */
```

px5_sem_extend_init

C Prototype:

```
#include <semaphore.h>

int px5_sem_extend_init(sem_t * semaphore_handle, int pshared,
                        unsigned int value, semattr_t * semaphore_attributes);
```

Description:



This pthreads+ service extension initializes (creates) a semaphore with the specified initial value and with optional PX5 semaphore attributes. If successful, the semaphore handle is available for use by the application. This service is an alternative to the *sem_init* API, i.e., all semaphore APIs can be used on the semaphore created with this extended API.

API Parameters:

<code>semaphore_handle</code>	Handle of the semaphore to set up.
<code>pshared</code>	Process sharing selection - not used by the PX5 RTOS.
<code>value</code>	Initial value of the semaphore.
<code>semaphore_attributes</code>	Optional semaphore attributes.

Return Codes:

PX5_SUCCESS (0)	Successful semaphore initialization.
PX5_ERROR (-1)	Error attempting to initialize the semaphore. Please use <i>errno</i> to retrieve the exact error:
EINVAL	Invalid semaphore handle or value exceeds <i>SEM_VALUE_MAX</i> .
ENOSPC	Insufficient memory to create the semaphore.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, at least (**148 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*sem**, *sem_destroy*, *sem_init*, *px5_semattr_**

Small Example:

```
#include <semaphore.h>

/* Semaphore handle. */
sem_t          my_semaphore_handle;

/* Semaphore attributes. */
semattr_t      my_semaphore_attributes;

int            status;

/* Create the semaphore and set up "my_semaphore_handle" using
   the already initialized semaphore attributes
   "my_semaphore_attributes". */
status = px5_sem_extend_init(&my_semaphore_handle, 0, 1,
                             &my_semaphore_attributes);

/* If status is PX5_SUCCESS, the semaphore was created with a value
   of 1 and is ready to use! */
```

px5_sem_information_get

C Prototype:

```
#include <semaphore.h>

int px5_sem_information_get(sem_t * semaphore_handle, char ** name,
    u_long * current_count,
    u_long * threads_suspended, pthread_t *first_suspended_thread,
    sem_t * next_semaphore_handle);
```

Description:

This pthreads+ service retrieves the specified information from the specified semaphore.

API Parameters:

semaphore_handle	Handle of the semaphore to get information about.
name	If non-NULL, destination for the specified semaphore name.
current_count	If non-NULL, destination for the current semaphore count.
threads_suspended	If non-NULL, destination for the number of threads currently suspended on this semaphore.
first_suspended_thread	If non-NULL, destination for the thread handle of the first thread suspended on this semaphore.
next_semaphore_handle	If non-NULL, destination for the handle of the next created semaphore. This API can be called successively to examine all semaphores using this parameter until the original handle is detected.

Return Codes:

PX5_SUCCESS (0)	Successful information retrieval from the specified semaphore.
ESRCH	Invalid semaphore handle.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:

From the location where this API is called, there should be at least **(78 + PX5_BINDING_STACK_FRAME_SIZE)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable from the thread context and interrupt handlers (ISRs).

See Also:

*px5_sem_extend_init, sem_init, px5_*_information_get*

Small Example:

```
#include <semaphore.h>

sem_t          my_semaphore_handle;
char *         name;
u_long         current_count;
u_long         thread_suspended;
pthread_t      first_suspended_thread;
sem_t          next_semaphore;
int            status;

/* Get information about "my_semaphore_handle". */
status =
    px5_sem_information_get(&my_semaphore_handle,
                           &name, &current_count,
                           &threads_suspended, &first_suspended_thread,
                           &next_semaphore);

/* If status contains PX5_SUCCESS, the information about
   "my_semaphore_handle" is available for use. */
```

px5_sem_post_notify

C Prototype:

```
#include <semaphore.h>

int px5_sem_post_notify(sem_t semaphore_handle,
                        void (*post_callback)(sem_t *));
```

Description:

This service registers an application callback that is executed whenever a post is made to the specified semaphore. If a NULL is supplied, notification for the specified semaphore is disabled.



By default, application semaphore post notification is not enabled. To enable notification, PX5 RTOS and the application should be built with `PX5_NOTIFICATION_ENABLE` defined.

API Parameters:

<code>semaphore_handle</code>	Semaphore handle.
<code>post_callback</code>	If non-NULL, application's notification function pointer to call whenever a post is made to the semaphore.

Return Codes:

PX5_SUCCESS (0)	Successful registration of the notification callback.
ESRCH	Invalid semaphore handle.
EACCES	Invalid call from ISR or non-thread. This error code is also returned if PX5 RTOS was not built with <code>PX5_NOTIFICATION_ENABLE</code> defined.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



NO PREEMPTION. No preemption takes place as a result of this service.

Stack Usage Estimate:



From the location where this API is called, there should be at least **(8 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

px5_mq_send_notify, *px5_pthread_event_flags_set_notify*,
px5_pthread_fastqueue_send_notify

Small Example:

```
#include <semaphore.h>

int                status;
sem_t              my_semaphore;

void my_notification_callback(sem_t *semaphore_handle)
{
    /* Application processing goes here!  */
}

...

/* Register "my_notification_callback" to "my_semaphore". */
status = px5_sem_post_notify(&my_semaphore,
                             my_notification_callback);

/* If status is PX5_SUCCESS, the callback has been registered. */
```

px5_sem_tickwait

C Prototype:

```
#include <semaphore.h>

int px5_sem_tickwait(sem_t * semaphore_handle, tick_t timeout_ticks);
```

Description:

If the semaphore is available (count greater than zero), this service decrements the semaphore count by one and returns success to the caller. Otherwise, if the semaphore is zero, the calling thread suspends until the semaphore is available or until the timeout expires.



This API is a cancellation point, meaning that if a cancellation is pending, it will be detected and executed by this API.

API Parameters:

<code>semaphore_handle</code>	Handle of the semaphore to get.
<code>timeout_ticks</code>	Maximum number of timer ticks to wait for the semaphore. A value of zero represents no timeout.

Return Codes:

PX5_SUCCESS (0)	Successful semaphore get.
PX5_ERROR (-1)	Error attempting to get the semaphore. Please use <i>errno</i> to retrieve the exact error:
EINVAL	Invalid semaphore handle.
ETIMEDOUT	Timeout on waiting for a semaphore to become available.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. If the semaphore is available, no preemption takes place.



SUSPENSION. If the semaphore is unavailable (count is zero), the calling thread is suspended until the semaphore becomes available via *sem_post* or until the timeout expires.

Stack Usage Estimate:



From the location where this API is called, at least (**104 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*sem**, *sem_timedwait*, *sem_trywait*, *sem_trywait*, *sem_wait*, *sem_post*

Small Example:

```
#include <semaphore.h>

/* Semaphore handle. */
sem_t      my_semaphore_handle;
int        status;

/* Get the semaphore "my_semaphore_handle" and wait a maximum of
   200 ticks. */
status = px5_sem_tickwait(&my_semaphore_handle, 200);

/* If status is PX5_SUCCESS, the semaphore was retrieved. */
```

px5_semattr_destroy

C Prototype:

```
#include <semaphore.h>

int px5_semattr_destroy(semattr_t * semaphore_attributes);
```

Description:

This pthreads+ service destroys the previously created semaphore attributes structure pointed to by *semaphore_attributes*. Once destroyed, the semaphore attributes structure cannot be used again unless recreated.

API Parameters:

<code>semaphore_attributes</code>	Pointer to the semaphore attributes to destroy.
-----------------------------------	---

Return Codes:

PX5_SUCCESS (0)	Successful semaphore attributes destroy.
EINVAL	Invalid semaphore attributes pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, at least (**36 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

px5_sem_extend_init, *px5_semattr_init*, *px5_semattr_**

Small Example:

```
#include <semaphore.h>

/* Semaphore attribute structure. */
semattr_t      my_semaphore_attributes;
int            status;

/* Destroy the semaphore attributes referenced by
   "my_semaphore_attributes". */
status = px5_semattr_destroy(&my_semaphore_attributes);

/* If status is PX5_SUCCESS, the semaphore attributes
   structure was destroyed. */
```

px5_semattr_getcontroladdr

C Prototype:

```
#include <semaphore.h>

int px5_semattr_getcontroladdr(semattr_t * semaphore_attributes,
                               void ** semaphore_control_address);
```

Description:

This pthreads+ service returns the previously supplied semaphore control structure address.

API Parameters:

<code>semaphore_attributes</code>	Pointer to the semaphore attributes.
<code>semaphore_control_address</code>	Pointer to the destination for the previously supplied semaphore control address.

Return Codes:

PX5_SUCCESS (0)	Successful semaphore attributes control address retrieval.
EINVAL	Invalid semaphore attributes or semaphore control address destination pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, at least (**44 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

px5_sem_extend_init, *px5_semattr_init*, *px5_semattr_setcontroladdr*,
*px5_semattr_**

Small Example:

```
#include <semaphore.h>

/* Semaphore attribute structure. */
semattr_t      my_semaphore_attributes;
void *         my_semaphore_control_address;
int            status;

/* Get the semaphore control structure address in the
   semaphore attributes structure "my_semaphore_attributes". */
status = px5_semattr_getcontroladdr(&my_semaphore_attributes,
                                     &my_semaphore_control_address);

/* If status is PX5_SUCCESS, "my_semaphore_control_address"
   contains the address of the previously supplied semaphore
   control memory. */
```

px5_semattr_getcontrolsize

C Prototype:

```
#include <semaphore.h>

int px5_semattr_getcontrolsize(semattr_t * semaphore_attributes,
                               size_t * semaphore_control_size);
```

Description:

This pthreads+ service returns the size of the internal semaphore control structure. The primary purpose of this API is to inform the application how much memory is required for the *px5_semattr_setcontroladdr* API.

API Parameters:

<code>semaphore_attributes</code>	Pointer to the attributes.
<code>semaphore_control_size</code>	Pointer to the destination for the internal semaphore control structure size.

Return Codes:

PX5_SUCCESS (0)	Successful retrieval of internal semaphore control structure size.
EINVAL	Invalid semaphore attributes or invalid destination for semaphore control structure size.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, at least (**44 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

px5_sem_extend_init, *px5_semattr_init*, *px5_semattr_setcontroladdr*, *px5_semattr_**

Small Example:

```
#include <semaphore.h>

/* Semaphore variable attribute structure. */
semattr_t      my_semaphore_attributes;
size_t         my_semaphore_control_size;
int            status;

/* Get the internal semaphore control structure memory size. */
status = px5_semattr_getcontrolsize(
        &my_semaphore_attributes,
        &my_semaphore_control_size);

/* If status is PX5_SUCCESS, "my_semaphore_control_size"
   contains the size of the internal semaphore control
   structure. */
```

px5_semattr_getname

C Prototype:

```
#include <semaphore.h>

int px5_semattr_getname(semattr_t* semaphore_attributes,
                        char** semaphore_name);
```

Description:

This pthreads+ service returns the previously supplied semaphore name.

API Parameters:

<code>semaphore_attributes</code>	Pointer to the semaphore attributes.
<code>semaphore_name</code>	Pointer to the destination for the previously supplied semaphore name.

Return Codes:

PX5_SUCCESS (0)	Successful retrieval of last supplied semaphore name.
EINVAL	Invalid semaphore attributes or name destination pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, at least (**44 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

px5_sem_extend_init, *px5_semattr_init*, *px5_semattr_setname*,
*px5_semattr_**

Small Example:

```
#include <semaphore.h>

/* Semaphore attribute structure. */
semattr_t      my_semaphore_attributes;
char *         my_semaphore_name;
int            status;

/* Get the last supplied semaphore name. */
status = px5_semattr_getname(&my_semaphore_attributes,
                             &my_semaphore_name);

/* If status is PX5_SUCCESS, "my_semaphore_name" contains the
   name previously supplied. */
```

px5_semattr_init

C Prototype:

```
#include <semaphore.h>

int px5_semattr_init(semattr_t * semaphore_attributes);
```

Description:

This pthreads+ service initializes the semaphore attributes structure with default semaphore variable creation values. Note that semaphore attributes are used only by the *px5_sem_extend_init* API.

API Parameters:

<code>semaphore_attributes</code>	Pointer to the semaphore attributes structure to create.
-----------------------------------	--

Return Codes:

PX5_SUCCESS (0)	Successful semaphore attributes structure creation.
EINVAL	Invalid semaphore attributes pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, at least **(36 + *PX5_BINDING_STACK_FRAME_SIZE*)** bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

px5_sem_extend_init, *px5_semattr_destroy*, *px5_semattr_**

Small Example:

```
#include <semaphore.h>

/* Semaphore attribute structure. */
semattr_t      my_semaphore_attributes;
int            status;

/* Create the semaphore attributes structure
   "my_semaphore_attributes". */
status = px5_semattr_init(&my_semaphore_attributes);

/* If status is PX5_SUCCESS, the "my_semaphore_attributes"
   structure is ready for use. */
```

px5_semattr_setcontroladdr

C Prototype:

```
#include <semaphore.h>

int px5_semattr_setcontroladdr(semattr_t * semaphore_attributes,
                               void * semaphore_control_address,
                               size_t semaphore_control_size);
```

Description:

This pthreads+ service enables the developer to provide the memory for the internal PX5 RTOS semaphore structure, as specified by the address in the *semaphore_control_address* parameter. This memory will subsequently be used for the next semaphore created with this attribute structure. The memory size required for the internal semaphore control structure can be found via a call to the *px5_semattr_getcontrolsize* service.



Note that each semaphore created must have its own unique semaphore control structure memory. Hence, the semaphore control memory supplied here is only valid for one px5_sem_extend_init call.



The exact amount of memory required for the internal semaphore control structure can be determined by including *px5.h* and using the *sizeof(px5_semaphore_control)* construct.

API Parameters:

<code>semaphore_attributes</code>	Pointer to the semaphore attributes.
<code>semaphore_control_address</code>	Pointer to the internal semaphore control structure memory.
<code>semaphore_control_size</code>	Size of the specified memory.

Return Codes:

PX5_SUCCESS (0)	Successful specification of semaphore structure memory.
EINVAL	Invalid semaphore attributes or invalid size of semaphore control memory.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:

From the location where this API is called, at least (**40 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*px5_sem_extend_init, px5_semattr_init, px5_semattr_getcontroladdr, px5_semattr_**

Small Example:

```
#include <semaphore.h>

/* Semaphore attribute structure. */
semattr_t      my_semaphore_attributes;
int            status;

/* Set the semaphore control structure memory address in the
   semaphore attributes structure "my_semaphore_attributes". */
status = px5_semattr_setcontroladdr(&my_semaphore_attributes,
                                     0xA0000, 60);

/* If status is PX5_SUCCESS, the next semaphore creation using
   these attributes will use memory at address 0xA0000 for the
   internal semaphore control structure. */
```


px5_semattr_setname

C Prototype:

```
#include <semaphore.h>

int px5_semattr_setname(semattr_t * semaphore_attributes,
                        char * semaphore_name);
```

Description:

This pthreads+ service sets the semaphore name in the specified attribute structure.

API Parameters:

semaphore_attributes	Pointer to the semaphore attributes.
semaphore_name	Pointer to the supplied semaphore name.

Return Codes:

PX5_SUCCESS (0)	Successful semaphore name set.
EINVAL	Invalid semaphore attributes.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, at least (**44 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

px5_sem_extend_init, *px5_semattr_init*, *px5_semattr_getname*,
*px5_semattr_**

Small Example:

```
#include <semaphore.h>

/* Semaphore attribute structure. */
semattr_t      my_semaphore_attributes;
int             status;

/* Set the semaphore name in the semaphore attributes
   structure "my_semaphore_attributes". */
status = px5_semattr_setname(&my_semaphore_attributes,
                             "my_semaphore_name");

/* If status is PX5_SUCCESS, "my_semaphore_name" is set in the
   semaphore attribute structure. */
```

sched_yield

C Prototype:

```
#include <sched.h>

int sched_yield(void);
```

Description:

This service relinquishes control to the next thread of the same priority currently ready for execution. If there is no other thread of the same priority ready for execution, this service simply returns a successful status.

API Parameters:

none

Return Codes:

PX5_SUCCESS (0)	Successful thread yield.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. If no other threads of the same priority are ready for execution, this service returns without preemption.



PREEMPTION. If other threads are ready for execution at the same priority level, the other thread(s) will execute before this service returns.

Stack Usage Estimate:



From the location where this API is called, at least (**12 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

px5_ticktimer_sleep, sleep, nanosleep

Small Example:

```

#include <sched.h>

unsigned long  my_first_thread_counter;
unsigned long  my_second_thread_counter;

/* Define first thread. */
void * my_first_thread_start(void *arguments)
{
    /* Loop forever incrementing a counter and yielding
       the processor to thread 1. */
    while (1)
    {
        /* Increment my first thread's counter. */
        my_first_thread_counter++;

        /* Yield to my second thread. */
        sched_yield();

        /* Once sched_yield returns, my second thread has executed. */
    }
}

/* Define my second thread. */
void * my_second_thread_start(void *arguments)
{
    /* Loop forever incrementing a counter and yielding
       the processor to my first thread. */
    while (1)
    {
        /* Increment my second thread's counter. */
        my_second_thread_counter++;

        /* Yield to my first thread. */
        sched_yield();

        /* Once sched_yield returns, my first thread has executed. */
    }
}

```

sem_destroy

C Prototype:

```
#include <semaphore.h>

int sem_destroy(sem_t * semaphore_handle);
```

Description:

This service destroys the previously created semaphore specified by *semaphore_handle*. An error is returned if one or more threads are suspended on this semaphore.



To avoid unpredictable behavior, the application must ensure there is no further access to the semaphore after it is destroyed.

API Parameters:

<code>semaphore_handle</code>	Handle of the semaphore to destroy.
-------------------------------	-------------------------------------

Return Codes:

PX5_SUCCESS (0)

Successful semaphore destroy.

PX5_ERROR (-1)

Error attempting to destroy semaphore. Please use *errno* to retrieve the exact error:

EINVAL

Invalid semaphore handle.

EBUSY

One or more threads are currently suspended on this semaphore.

EACCES

Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, at least (**68 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*sem**, *sem_init*

Small Example:

```
#include <semaphore.h>

/* Semaphore handle. */
sem_t      my_semaphore_handle;
int         status;

/* Destroy the semaphore referenced by "my_semaphore_handle". */
status = sem_destroy(&my_semaphore_handle);

/* If status is PX5_SUCCESS, the semaphore was destroyed. */
```

sem_init

C Prototype:

```
#include <semaphore.h>

int sem_init(sem_t * semaphore_handle, int pshared,
              unsigned int value);
```

Description:



This service initializes (creates) a semaphore with the specified initial value. If successful, the semaphore handle is available for use by the application.

API Parameters:

<code>semaphore_handle</code>	Handle of the semaphore to set up.
<code>pshared</code>	Process sharing selection - not used by the PX5 RTOS.
<code>value</code>	Initial value of the semaphore.

Return Codes:

PX5_SUCCESS (0)	Successful semaphore initialization.
PX5_ERROR (-1)	Error attempting to initialize the semaphore. Please use <i>errno</i> to retrieve the exact error:
EINVAL	Invalid semaphore handle or value exceeds SEM_VALUE_MAX.
ENOSPC	Insufficient memory to create a semaphore.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. There is no preemption possible with this service.

Stack Usage Estimate:



From the location where this API is called, at least (**132 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*sem**, *sem_destroy*, *px5_sem_extend_init*, *px5_semattr_**

Small Example:

```
#include <semaphore.h>

/* Semaphore handle. */
sem_t      my_semaphore_handle;
int         status;

/* Create the semaphore and set up "my_semaphore_handle". */
status = sem_init(&my_semaphore_handle, 0, 1);

/* If status is PX5_SUCCESS, the semaphore was created with a value
   of 1 and is ready to use! */
```

sem_post

C Prototype:

```
#include <semaphore.h>

int sem_post(sem_t * semaphore_handle);
```

Description:

This service posts to the specified semaphore. If one or more threads are waiting, the thread with the highest priority is resumed. Otherwise, if no threads are waiting, the internal semaphore count is incremented by one.

API Parameters:

`semaphore_handle` Handle of the semaphore to post.

Return Codes:

PX5_SUCCESS (0)	Successful semaphore post.
PX5_ERROR (-1)	Error attempting to post to the semaphore. Please use <i>errno</i> to retrieve the exact error:
	EINVAL Invalid semaphore handle or value exceeds SEM_VALUE_MAX.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. If no other threads are waiting for the semaphore, no preemption occurs.



PREEMPTION. If a higher-priority thread was waiting for the semaphore when given the semaphore, the waiting thread is resumed, and preemption will occur.

Stack Usage Estimate:



From the location where this API is called, at least (**120 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable from the thread context and interrupt handlers (ISRs).

See Also:

*sem**, *sem_wait*, *sem_trywait*

Small Example:

```
#include <semaphore.h>

/* Semaphore handle. */
sem_t      my_semaphore_handle;
int        status;

/* Post to the semaphore "my_semaphore_handle". */
status = sem_post(&my_semaphore_handle);

/* If status is PX5_SUCCESS, the post was made to the semaphore. */
```

sem_timedwait

C Prototype:

```
#include <semaphore.h>

int sem_timedwait(sem_t * semaphore_handle,
                  const struct timespec * absolute_time);
```

Description:

If the semaphore is available (count greater than zero), this service decrements the semaphore count by one and returns success to the caller. Otherwise, if the semaphore is zero, the calling thread suspends until the semaphore is available or until the absolute time is reached.



This API is a cancellation point, meaning that if a cancellation is pending, it will be detected and executed by this API.

API Parameters:

<code>semaphore_handle</code>	Handle of the semaphore to get.
<code>absolute_time</code>	Pointer to absolute time to wait for. If the semaphore is unavailable when this time is reached, the service will timeout, and the thread will be resumed.

Return Codes:

PX5_SUCCESS (0)	Successful semaphore get.
PX5_ERROR (-1)	Error attempting to get the semaphore. Please use <i>errno</i> to retrieve the exact error:
EINVAL	Invalid semaphore handle.
ETIMEDOUT	Timeout on waiting for a semaphore to become available.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. If the semaphore is available, no preemption takes place.



SUSPENSION. If the semaphore is unavailable (count is zero), the calling thread is suspended until the semaphore becomes available via *sem_post* or until the timeout expires.

Stack Usage Estimate:



From the location where this API is called, at least (**120 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*sem**, *px5_sem_tickwait*, *sem_trywait*, *sem_trywait*, *sem_wait*, *sem_post*

Small Example:

```
#include <semaphore.h>

/* Semaphore handle. */
sem_t      my_semaphore_handle;
struct timespec  max_time_to_wait;
int        status;

/* Get current time. */
clock_gettime(CLOCK_REALTIME, &max_time_to_wait);

/* Get the semaphore "my_semaphore_handle" and wait a maximum of
 2 seconds. */
max_time_to_wait.tv_sec += 2;
status = sem_timedwait(&my_semaphore_handle, &max_time_to_wait);

/* If status is PX5_SUCCESS, the semaphore was retrieved. */
```

sem_trywait

C Prototype:

```
#include <semaphore.h>

int sem_trywait(sem_t * semaphore_handle);
```

Description:

If the semaphore is available (count greater than zero), this service decrements the count and returns success. Otherwise, an error is returned.

API Parameters:

`semaphore_handle` Handle of the semaphore to try to get.

Return Codes:

PX5_SUCCESS (0)	Successful semaphore get.
PX5_ERROR (-1)	Error attempting to get the semaphore. Please use <i>errno</i> to retrieve the exact error:
EINVAL	Invalid semaphore handle.
EAGAIN	Semaphore is not available.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. No preemption occurs if the semaphore is available.

Stack Usage Estimate:



From the location where this API is called, at least (**44 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*sem**, *sem_wait*

Small Example:

```
#include <semaphore.h>

/* Semaphore handle. */
sem_t      my_semaphore_handle;
int        status;

/* Try to get the semaphore "my_semaphore_handle". */
status = sem_trywait(&my_semaphore_handle);

/* If status is PX5_SUCCESS, the semaphore was retrieved. */
```


sem_wait

C Prototype:

```
#include <semaphore.h>

int sem_wait(sem_t * semaphore_handle);
```

Description:

If the semaphore is available (count greater than zero), this service decrements the semaphore count by one and returns success to the caller. Otherwise, if the semaphore is zero, the calling thread suspends until the semaphore is available.



This API is a cancellation point, meaning that if a cancellation is pending, it will be detected and executed by this API.

API Parameters:

<code>semaphore_handle</code>	Handle of the semaphore to get.
-------------------------------	---------------------------------

Return Codes:

PX5_SUCCESS (0)	Successful semaphore get.
PX5_ERROR (-1)	Error attempting to get the semaphore. Please use <i>errno</i> to retrieve the exact error:
EINVAL	Invalid semaphore handle.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. If the semaphore is available, no preemption takes place.



SUSPENSION. If the semaphore is unavailable (count is zero), the calling thread is suspended until the semaphore becomes available via *sem_post*.

Stack Usage Estimate:



From the location where this API is called, at least (**96 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*sem**, *sem_trywait*, *sem_post*

Small Example:

```
#include <semaphore.h>

/* Semaphore handle. */
sem_t      my_semaphore_handle;
int         status;

/* Get the semaphore "my_semaphore_handle". */
status = sem_wait(&my_semaphore_handle);

/* If status is PX5_SUCCESS, the semaphore was retrieved. */
```

sigaction

C Prototype:

```
#include <signal.h>

int sigaction(int signal_number, sigaction_t * new_handler,
               sigaction_t * previous_handler);
```

Description:

This service sets up the signal handler for the specified signal number. If the previous handler pointer is non-NULL, the information for the previous signal handler is also returned.



Signal handlers are invoked only if the corresponding signal is unmasked by the thread receiving the raised signal. A signal handler should be set up via `sigaction` before enabling any signal.

API Parameters:

<code>signal_number</code>	Signal number to set up a handler for.
<code>new_handler</code>	Pointer to signal action structure that contains signal information, including the handler. This pointer is NULL if the handler is being removed.
<code>previous_handler</code>	Optional pointer to structure to store the previous handler information. If the previous handler information is not wanted, this value is NULL.

Return Codes:

PX5_SUCCESS (0)	Successful signal handler set up.
PX5_ERROR (-1)	Error attempting to set up signal handler. Please use <i>errno</i> to retrieve the exact error:
EINVAL	Invalid signal number or invalid new signal handler.

EACCES

Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. This service only sets up the signal handler, so no preemption is possible.

Stack Usage Estimate:

From the location where this API is called, at least (**52 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*sig**, *pthread_kill*, *pthread_sigmask*, *sigwait*

Small Example:

```
#include <signal.h>

struct sigaction  my_signal_handler_request;
struct sigaction  previous_signal_handler;
int               my_signal_handler_count;

/* My signal handler. */
void my_signal_handler(int signal)
{

    /* Signal will be equal to SIGUSR1. */

    /* Increment signal count. */
    my_signal_handler_count++;

}

/* Later in the thread processing.. */

/* Set up the signal handler for SIGUSR1. */
my_signal_handler_request.signal_handler = my_signal_handler;
status = sigaction(SIGUSR1, &my_signal_handler_request,
                  &previous_signal_handler);

/* If status is PX5_SUCCESS (0), "my_signal_handler" is set up for
   SIGUSR1. Any pthread_kill request to a thread with SIGUSR1
   enabled will result in a call to "my_signal_handler". */
```

sigaddset

C Prototype:

```
#include <signal.h>

int sigaddset(sigset_t * signal_set, int signal_number);
```

Description:

This service adds the specified *signal_number* to the set of signals.

API Parameters:

<code>signal_set</code>	Bit map of signals (signal set).
<code>signal_number</code>	Number of signal to add to the signal set.

Return Codes:

PX5_SUCCESS (0)	Successful signal added to the set.
PX5_ERROR (-1)	Error attempting to add signal to the set. Please use <i>errno</i> to retrieve the exact error:
EINVAL	Invalid signal number or invalid signal set pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. This service only adds the signal number in the signal set, so no preemption is possible.

Stack Usage Estimate:



From the location where this API is called, at least (**52 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*sig**, *sigdelset*, *sigemptyset*, *sigfillset*

Small Example:

```
#include <signal.h>

sigset_t          my_signal_set;

/* Add the SIGUSR1 signal to "my_signal_set". */
status = sigaddset(&my_signal_set, SIGUSR1);

/* If status is PX5_SUCCESS (0), SIGUSR1 has been added to the signal
   set in "my_signal_set". */
```

sigdelset

C Prototype:

```
#include <signal.h>

int sigdelset(sigset_t * signal_set, int signal_number);
```

Description:

This service deletes the specified *signal_number* from the set of signals.

API Parameters:

<code>signal_set</code>	Bit map of signals (signal set).
<code>signal_number</code>	Number of signal to remove from the signal set.

Return Codes:

PX5_SUCCESS (0)	Successful signal removal from set.
PX5_ERROR (-1)	Error attempting to remove the signal from the set. Please use <i>errno</i> to retrieve the exact error:
EINVAL	Invalid signal number or invalid signal set pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. This service only deletes the signal number in the signal set, so no preemption is possible.

Stack Usage Estimate:



From the location where this API is called, at least (**52 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*sig**, *sigaddset*, *sigemptyset*, *sigfillset*

Small Example:

```
#include <signal.h>

sigset_t          my_signal_set;

/* Remove the SIGUSR1 signal from "my_signal_set". */
status = sigdelset(&my_signal_set, SIGUSR1);

/* If status is PX5_SUCCESS, SIGUSR1 has been removed from the signal
   set "my_signal_set". */
```

sigemptyset

C Prototype:

```
#include <signal.h>

int sigemptyset(sigset_t * signal_set);
```

Description:

This service removes all signals from the specified set of signals.

API Parameters:

<code>signal_set</code>	Bit map of signals (signal set).
-------------------------	----------------------------------

Return Codes:

PX5_SUCCESS (0)	Successful signal removal of all signals from the set.
PX5_ERROR (-1)	Error attempting to remove all signals from the set. Please use <i>errno</i> to retrieve the exact error:
EINVAL	Invalid signal set pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. This service removes all signals from the signal set, making no preemption possible.

Stack Usage Estimate:



From the location where this API is called, at least (**36 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*sig**, *sigdelset*, *sigaddset*, *sigfillset*

Small Example:

```
#include <signal.h>

sigset_t          my_signal_set;

/* Remove all signals from "my_signal_set". */
status = sigemptyset(&my_signal_set);

/* If status is PX5_SUCCESS (0), all signals have been removed from
the signal set "my_signal_set". */
```

sigfillset

C Prototype:

```
#include <signal.h>

int sigfillset(sigset_t * signal_set);
```

Description:

This service sets all signals in the specified set of signals.

API Parameters:

<code>signal_set</code>	Bit map of signals (signal set).
-------------------------	----------------------------------

Return Codes:

PX5_SUCCESS (0)	Successful signal setting of all signals in the set.
PX5_ERROR (-1)	Error attempting to set all signals in the set . Please use <i>errno</i> to retrieve the exact error:
EINVAL	Invalid signal set pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. This service only sets signals in the signal set, so no preemption is possible.

Stack Usage Estimate:



From the location where this API is called, at least (**36 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*sig**, *sigdelset*, *sigemptyset*, *sigaddset*

Small Example:

```
#include <signal.h>

sigset_t          my_signal_set;

/* Set all signals in "my_signal_set". */
status = sigfillset(&my_signal_set);

/* If status is PX5_SUCCESS (0), all signals have been set in
"my_signal_set". */
```

sigismember

C Prototype:

```
#include <signal.h>

int sigismember(sigset_t * signal_set, int signal_number);
```

Description:

This service determines if *signal_number* is part of the specified signal set.

API Parameters:

<code>signal_set</code>	Bit map of signals (signal set).
<code>signal_number</code>	Signal number to check for in the set.

Return Codes:

1	Signal is part of the set
0	Signal is not part of the set.
PX5_ERROR (-1)	Error attempting to set all signals in the set. Please use <i>errno</i> to retrieve the exact error:
EINVAL	Invalid signal set pointer or an invalid signal number.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. This service only checks for a signal in the signal set, so no preemption is possible.

Stack Usage Estimate:



From the location where this API is called, at least (**52 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*sig**, *sigaddset*, *sigdelset*, *sigemptyset*, *sigfillset*, *sigpending*

Small Example:

```
#include <signal.h>

sigset_t          my_signal_set;

/* Check if SIGUSR1 is in the "my_signal_set" signal set. */
status = sigismember(&my_signal_set, SIGUSR1);

/* If status is 0, SIGUSR1 is not in the "my_signal_set" signal
   set. */
```

sigpending

C Prototype:

```
#include <signal.h>

int sigpending(sigset_t * pending_signals);
```

Description:

This service retrieves the pending signals for the calling thread.

API Parameters:

<code>pending_signals</code>	Pointer to destination for the pending signals of the calling thread.
------------------------------	---

Return Codes:

PX5_SUCCESS (0)	Successful pending signal retrieval.
PX5_ERROR (-1)	Error attempting to get the pending signals. Please use <i>errno</i> to retrieve the exact error:
EINVAL	Invalid pending signal pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. This service only retrieves the pending signals of the calling thread, so no preemption is possible.

Stack Usage Estimate:



From the location where this API is called, at least (**36 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*sig**, *sigaddset*, *sigdelset*, *sigemptyset*, *sigfillset*

Small Example:

```
#include <signal.h>

sigset_t          my_pending_signals;

/* Retrieve the pending signals for the calling thread. */
status = sigpending(&my_pending_signal);

/* If status is PX5_SUCCESS (0), the pending signals for the calling
   thread can be found in "my_pending_signals". */
```

sigtimedwait

C Prototype:

```
#include <signal.h>

int sigtimedwait(const sigset_t * signals, siginfo_t * signal_info,
                  const struct timespec * timeout);
```

Description:

This service suspends the calling thread until a signal in the specified signal set is raised. If the signal is already pending, this service returns immediately with the corresponding signal number. If none of the signals in the set are pending, this service suspends the calling thread until a signal in the set is raised or until the specified timeout occurs.



All signals specified in the signal set must be masked to wait for them synchronously.

API Parameters:

<code>signals</code>	Bit map of signals (signal set) to wait for.
<code>signal_info</code>	Optional pointer to a structure containing signal information.
<code>timeout</code>	Maximum time this service will wait before returning.

Return Codes:

Positive Number	Signal raised.
PX5_ERROR (-1)	Error attempting to wait for signal. Please use <i>errno</i> to retrieve the exact error:
EINVAL	Invalid signal set or timeout value.
EAGAIN:	Timeout occurred before the signal was raised.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. A signal is already pending in the specified signal set, and this service returns immediately. No preemption takes place.



SUSPENSION. If no signal in the specified signal set is pending, this service suspends the calling thread until a signal is raised via *pthread_kill* or the timeout occurs.

Stack Usage Estimate:



From the location where this API is called, at least (**136 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*sig**, *sigwait*, *sigwaitinfo*, *pthread_kill*

Small Example:

```
#include <signal.h>

int          signal_received;
struct timespec my_timeout;
sigset_t      my_signals;

/* Set up the signal set to specify SIGUSR1 signal. */
sigemptyset(&my_signals);
sigaddset(&my_signals, SIGUSR1);

/* Set up timeout for 1 second. */
my_timeout.tv_sec = 1;
my_timeout.tv_nsec = 0;

/* Wait for SIGUSR1 signals. */
signal_received = sigtimedwait(&my_signal, NULL, &my_timeout);

/* If signal_received is SIGUSR1 the service was successful. */
```

sigwait

C Prototype:

```
#include <signal.h>

int sigwait(const sigset_t * signals, int * signal_number);
```

Description:

This service suspends the calling thread until a signal in the specified signal set is raised. If the signal is already pending, this service returns immediately with the corresponding signal number. If none of the signals in the specified set are pending, this service suspends the calling thread until a signal in the set is raised.



All signals specified in the signal set must be masked to wait for them synchronously.



This API is a cancellation point, meaning that if a cancellation is pending, it will be detected and executed by this API.

API Parameters:

<code>signals</code>	Bit map of signals (signal set) to wait for.
<code>signal_number</code>	Pointer to return the signal within the specified set that was raised.

Return Codes:

PX5_SUCCESS (0)	Successful completion.
EINVAL	Error, invalid signal set, or signal number return pointer.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. A signal is already pending in the specified signal set, and this service returns immediately. No preemption takes place.



SUSPENSION. If no signal in the specified signal set is pending, this service suspends the calling thread until a signal is raised via *pthread_kill*.

Stack Usage Estimate:



From the location where this API is called, at least (**96 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*sig**, *sigtimedwait*, *sigwaitinfo*, *pthread_kill*

Small Example:

```
#include <signal.h>

int          signal_raised;
int          status;
sigset_t     my_signals;

/* Set up the signal set to specify SIGUSR1 signal. */
sigemptyset(&my_signals);
sigaddset(&my_signals, SIGUSR1);

/* Wait for SIGUSR1 signals. */
status = sigwait(&my_signal, &my_signal_number);

/* If status is PX5_SUCCESS (0), my_signal_number contains
   SIGUSR1. */
```

sigwaitinfo

C Prototype:

```
#include <signal.h>

int sigwaitinfo(const sigset_t * signals, siginfo_t * signal_info);
```

Description:

This service suspends the calling thread until a signal in the specified signal set is raised. If the signal is already pending, this service returns immediately with the corresponding signal number (and additional signal information in *signal_info*). If none of the signals in the specified set are pending, this service suspends the calling thread until a signal in the set is raised.



All signals specified in the signal set must be masked to wait for them synchronously.



This API is a cancellation point, meaning that if a cancellation is pending, it will be detected and executed by this API.

API Parameters:

<code>signals</code>	Bit map of signals (signal set) to wait for.
<code>signal_info</code>	Optional pointer to a structure containing signal information.

Return Codes:

Positive Number	Signal raised.
PX5_ERROR (-1)	Error attempting to wait for signal. Please use <i>errno</i> to retrieve the exact error:
EINVAL	Invalid signal set.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. A signal is already pending in the specified signal set, and this service returns immediately. No preemption takes place.



SUSPENSION. If no signal in the specified signal set is pending, this service suspends the calling thread until a signal is raised via *pthread_kill*.

Stack Usage Estimate:



From the location where this API is called, at least (**112 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

*sig**, *sigwait*, *sigtimedwait*, *pthread_kill*

Small Example:

```
#include <signal.h>

int          signal_received;
siginfo_t    my_signal_info;
sigset_t     my_signals;

/* Set up the signal set to specify SIGUSR1 signal. */
sigemptyset(&my_signals);
sigaddset(&my_signals, SIGUSR1);

/* Wait for SIGUSR1 signals. */
signal_received = sigwaitinfo(&my_signal, &my_signal_info);

/* If signal_received is SIGUSR1 the service was successful and
   my_signal_info.si_signo also contains SIGUSR1. */
```

sleep

C Prototype:

```
#include <unistd.h>

unsigned int sleep(unsigned int seconds);
```

Description:

This service causes the calling thread to suspend for the number of seconds specified in *seconds*. If an unmasked signal is sent to the thread while sleeping, the thread is resumed, and the remaining seconds are returned.



Sleep requests are rounded up to the next second.



This API is a cancellation point, meaning that if a cancellation is pending, it will be detected and executed by this API.

API Parameters:

<code>seconds</code>	The number of seconds to sleep.
----------------------	---------------------------------

Return Codes:

<code>0</code>	Successful sleep – no remaining seconds.
<code>positive number</code>	Sleep was interrupted by a signal sent to this thread. This value represents the number of seconds left to sleep.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



SUSPENSION. The calling thread is suspended until the specified time has lapsed or another thread sends a signal to this thread.

Stack Usage Estimate:



From the location where this API is called, at least (**136 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

nanosleep

Small Example:

```
#include "unistd.h"

unsigned int    remaining_seconds;

/* Sleep for 5 seconds. */
remaining_seconds = sleep(5);

/* If "remaining_seconds" is 0, the calling thread slept for
   5 seconds. */
```

time

C Prototype:

```
#include <pthread.h>

time_t time(time_t * return_seconds);
```

Description:

This service returns the current number of seconds. If a non-null value for *return_seconds* is provided, the current number of seconds is also placed in the destination pointed to by *return_seconds*.

API Parameters:

<code>return_seconds</code>	If non-null, pointer to the destination of where to store the current seconds (in addition to current seconds returned by the function).
-----------------------------	--

Return Value:

<code>current seconds</code>	Current seconds.
------------------------------	------------------

Real-time Scenarios:

Upon the successful completion of this service, the following real-time scenarios are possible:



NO PREEMPTION. No preemption takes place as a result of this service.

Stack Usage Estimate:



From the location where this API is called, at least (**8 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

pthread_ticks_get

Small Example:

```
#include <pthread.h>

/* Variable to store the current seconds. */
time_t      my_current_seconds;

/* Get the current seconds. */
my_current_seconds = time(NULL);

/* At this point, my_current_seconds contains the current
seconds. */
```

usleep

C Prototype:

```
#include <unistd.h>

int  usleep(useconds_t microseconds);
```

Description:

This service causes the calling thread to suspend for the time specified in *microseconds*. If an unmasked signal is sent to the thread while sleeping, the thread is resumed, and an error is returned.



usleep requests are rounded up to the next timer tick that is evenly divisible by the timer resolution.



This API is a cancellation point, meaning that if a cancellation is pending, it will be detected and executed by this API.

API Parameters:

<code>microsecond</code>	The amount of time in microseconds to sleep.
--------------------------	--

Return Codes:

PX5_SUCCESS (0)	Successful <i>usleep</i> .
PX5_ERROR (-1)	Error attempting to <i>usleep</i> . Please use <i>errno</i> to retrieve the exact error:
EINTR	<i>usleep</i> was interrupted by a signal.
EACCES	Invalid call from ISR or non-thread.

Real-time Scenarios:

Upon the successful execution of this service, the following real-time scenarios are possible:



SUSPENSION. The calling thread is suspended until the specified time has lapsed or another thread sends a signal to this thread.

Stack Usage Estimate:



From the location where this API is called, at least (**80 + *PX5_BINDING_STACK_FRAME_SIZE***) bytes of stack should be available. This estimate is based on a typical 32-bit architecture. Stack usage should be verified in the application context.

Callable From:

This service is callable only from the thread context, i.e., it may not be called from an interrupt handler.

See Also:

nanosleep, px5_pthread_tick_sleep, sleep

Small Example:

```
#include <unistd.h>

int                status;

/* Sleep for 1 second. */
status = usleep(1000000);

/* If status contains PX5_SUCCESS (0), the calling thread slept for
   1 second. */
```

Index

A

AMP (Asymmetric Multiprocessing), [30](#)
 Anti-glitch, [19](#)
 Anti-tampering, [18](#)
 Application Best Practices, [i](#), [21](#)
 Asymmetric Multiprocessing, [30](#)

C

Central Error Handling, [ii](#), [31](#)
 Clock API Definitions
 clock_getres, [ii](#), [45](#), [46](#), [48](#), [50](#)
 clock_gettime, [ii](#), [46](#), [47](#), [48](#), [50](#), [69](#), [72](#), [108](#), [514](#)
 clock_settime, [ii](#), [46](#), [48](#), [49](#), [50](#)
 Composition, [ii](#), [28](#)
 Condition Variable API Definitions
 pthread_cond_broadcast, [iii](#), [35](#), [98](#), [99](#), [105](#)
 pthread_cond_destroy, [iii](#), [35](#), [100](#), [101](#), [103](#)
 pthread_cond_init, [iii](#), [35](#), [99](#), [101](#), [102](#), [103](#), [113](#), [115](#), [117](#), [119](#), [242](#), [250](#)
 pthread_cond_signal, [iii](#), [35](#), [99](#), [104](#), [105](#), [107](#), [110](#)
 pthread_cond_timedwait, [iii](#), [35](#), [106](#), [108](#), [110](#)
 pthread_cond_wait, [iii](#), [35](#), [99](#), [105](#), [107](#), [109](#), [111](#)
 pthread_condattr_destroy, [iii](#), [35](#), [112](#), [113](#)
 pthread_condattr_getcontrolsiz, [251](#)
 pthread_condattr_getname, [36](#)
 pthread_condattr_getpshared, [iii](#), [35](#), [114](#), [115](#)
 pthread_condattr_init, [iii](#), [35](#), [116](#), [117](#)
 pthread_condattr_setpshared, [iii](#), [35](#), [118](#), [119](#)
 px5_pthread_condattr_getcontroladdr, [iv](#), [35](#), [244](#), [245](#), [247](#)
 px5_pthread_condattr_getcontrolsiz, [iv](#), [35](#), [245](#), [246](#), [247](#), [250](#)
 px5_pthread_condattr_getname, [v](#), [36](#), [248](#), [249](#), [254](#)
 px5_pthread_condattr_setcontroladdr, [v](#), [29](#), [36](#), [245](#), [246](#), [247](#), [250](#)
 px5_pthread_condattr_setname, [v](#), [36](#), [249](#), [253](#), [254](#)
 Condition Variables, [ii](#), [35](#)

Configuration Build Options

PTHREAD_KEYS_MAX, [9](#), [137](#), [139](#)
 PX5_DEFAULT_PRIORITY, [9](#), [10](#), [84](#), [137](#), [139](#), [225](#)
 PX5_DEFAULT_SIGNAL_MASK, [10](#)
 PX5_DEFAULT_STACK_SIZE, [11](#), [82](#), [84](#)
 PX5_EVENT_FLAG_CONSUMPTION_MASK_ENABLE, [11](#), [280](#), [290](#)
 PX5_FUNCTION_POINTER_VERIFY_ENABLE, [11](#), [24](#)
 PX5_MEMORYPOOL_VERIFY_ENABLE, [11](#), [24](#)
 PX5_MISRA_ENABLE, [11](#)
 PX5_NOTIFICATION_ENABLE, [12](#), [213](#), [268](#), [309](#), [484](#)
 PX5_OBJECT_VERIFY_ENABLE, [12](#), [24](#)
 PX5_PARAMETER_CHECKING_DISABLE, [12](#)

D

deadlock, [37](#), [135](#)
 Default Verification Code, [ii](#), [23](#)

E

Enabling PDV, [ii](#), [24](#)
 Event Flags, [ii](#), [41](#)
 Event Flags API Definitions
 px5_pthread_event_flags_clear, [v](#), [41](#), [255](#), [256](#), [257](#), [258](#)
 px5_pthread_event_flags_create, [v](#), [41](#), [259](#), [260](#), [262](#), [264](#), [293](#)
 px5_pthread_event_flags_destroy, [v](#), [41](#), [260](#), [261](#), [262](#)
 px5_pthread_event_flags_set, [v](#), [41](#), [214](#), [256](#), [266](#), [267](#), [268](#), [269](#), [271](#), [276](#), [310](#), [485](#)
 px5_pthread_event_flags_timedwait, [v](#), [41](#), [270](#), [272](#)
 px5_pthread_event_flags_trywait, [v](#), [41](#), [271](#), [273](#), [274](#)
 px5_pthread_event_flags_wait, [v](#), [41](#), [267](#), [271](#), [274](#), [275](#), [277](#)
 px5_pthread_event_flagsattr_destroy, [v](#), [278](#), [279](#), [289](#)
 px5_pthread_event_flagsattr_getcontroladdr, [v](#), [41](#), [282](#), [283](#)
 px5_pthread_event_flagsattr_getcontrolsiz, [v](#), [41](#), [283](#), [284](#), [285](#), [293](#), [294](#)

[px5_pthread_event_flagsattr_getname](#), [v](#),
[41](#), [286](#), [287](#), [297](#)
[px5_pthread_event_flagsattr_init](#), [v](#), [41](#), [279](#),
[288](#), [289](#)
[px5_pthread_event_flagsattr_setcontroladdr](#)
[, v](#), [29](#), [41](#), [283](#), [284](#), [285](#), [293](#), [295](#)
[px5_pthread_event_flagsattr_setname](#), [v](#),
[41](#), [296](#), [297](#)

F

Fastqueue API Definitions

[px5_pthread_fastqueueattr_setcontroladdr](#),
[41](#)
[px5_pthread_fastqueue_create](#), [v](#), [40](#), [298](#),
[299](#), [301](#), [303](#), [335](#), [340](#)
[px5_pthread_fastqueue_destroy](#), [v](#), [40](#), [299](#),
[300](#), [301](#)
[px5_pthread_fastqueue_receive](#), [v](#), [40](#), [305](#),
[306](#), [308](#), [312](#), [318](#)
[px5_pthread_fastqueue_send](#), [v](#), [40](#), [214](#),
[269](#), [306](#), [307](#), [308](#), [309](#), [310](#), [315](#), [320](#), [485](#)
[px5_pthread_fastqueue_timedreceive](#), [v](#), [40](#),
[311](#), [313](#)
[px5_pthread_fastqueue_timsend](#), [v](#), [40](#),
[314](#), [316](#)
[px5_pthread_fastqueue_tryreceive](#), [v](#), [40](#),
[312](#), [317](#), [318](#)
[px5_pthread_fastqueue_trysend](#), [v](#), [40](#), [315](#),
[319](#), [320](#)
[px5_pthread_fastqueueattr_destroy](#), [v](#), [321](#),
[322](#), [334](#)
[px5_pthread_fastqueueattr_getcontroladdr](#),
[v](#), [40](#), [323](#), [324](#)
[px5_pthread_fastqueueattr_getcontrolsize](#),
[v](#), [40](#), [324](#), [325](#), [326](#), [335](#), [336](#)
[px5_pthread_fastqueueattr_getname](#), [v](#), [40](#),
[327](#), [328](#), [339](#)
[px5_pthread_fastqueueattr_getqueueaddr](#),
[v](#), [40](#), [329](#), [330](#)
[px5_pthread_fastqueueattr_getqueuesize](#), [v](#),
[40](#), [331](#), [332](#)
[px5_pthread_fastqueueattr_init](#), [v](#), [41](#), [322](#),
[333](#), [334](#)
[px5_pthread_fastqueueattr_setcontroladdr](#),
[v](#), [29](#), [324](#), [325](#), [326](#), [335](#), [337](#)
[px5_pthread_fastqueueattr_setname](#), [v](#), [41](#),
[338](#), [339](#)
[px5_pthread_fastqueueattr_setqueueaddr](#),
[vi](#), [29](#), [41](#), [340](#), [341](#)

Fastqueues, [ij](#), [40](#)

flash, [19](#)

H

Hardware stack limit, [19](#)

Hardware watchdog timer, [19](#)

I

Important Information Verified, [ij](#), [24](#)

Internal Memory Management, [ij](#), [28](#)

interrupt, [9](#), [15](#), [19](#)

Interrupt Service Routine (ISR), [32](#)

L

Lock-step, [18](#)

M

Memory Management Unit (MMU), [19](#)

Memory Manager API Definitions

[px5_pthread_memory_manager_enable](#), [vi](#),
[345](#), [346](#)
[px5_pthread_memory_manager_get](#), [vi](#), [346](#),
[347](#), [348](#), [350](#)
[px5_pthread_memory_manager_set](#), [vi](#), [346](#),
[348](#), [349](#), [351](#)

Memory Partition Pool API Definitions

[px5_pthread_partitionpool_allocate](#), [vi](#), [43](#),
[399](#), [401](#), [408](#), [413](#), [416](#)
[px5_pthread_partitionpool_create](#), [vi](#), [43](#),
[402](#), [404](#), [406](#), [410](#), [428](#)
[px5_pthread_partitionpool_destroy](#), [vi](#), [43](#),
[403](#), [405](#), [406](#)
[px5_pthread_partitionpool_free](#), [vi](#), [43](#), [400](#),
[407](#), [408](#), [413](#)
[px5_pthread_partitionpool_timedallocate](#),
[vi](#), [43](#), [400](#), [412](#), [414](#), [416](#)
[px5_pthread_partitionpool_tryallocate](#), [vi](#),
[43](#), [400](#), [413](#), [415](#), [417](#)
[px5_pthread_partitionpoolattr_destroy](#), [vi](#),
[43](#), [418](#), [419](#), [427](#)
[px5_pthread_partitionpoolattr_getcontrolad](#)
[dr](#), [vi](#), [43](#), [420](#), [421](#)
[px5_pthread_partitionpoolattr_getcontrolsiz](#)
[e](#), [vi](#), [43](#), [421](#), [422](#), [423](#), [428](#), [429](#)
[px5_pthread_partitionpoolattr_getname](#), [vi](#),
[43](#), [424](#), [425](#), [432](#)
[px5_pthread_partitionpoolattr_init](#), [vi](#), [43](#),
[419](#), [426](#), [427](#)
[px5_pthread_partitionpoolattr_setcontrolad](#)
[dr](#), [vi](#), [29](#), [43](#), [421](#), [422](#), [423](#), [428](#)
[px5_pthread_partitionpoolattr_setname](#), [vi](#),
[43](#), [431](#), [432](#)

Memory Pool API Definitions

[px5_pthread_memorypool_allocate](#), [vi](#), [42](#), [352](#), [354](#), [360](#), [365](#), [368](#)
[px5_pthread_memorypool_create](#), [vi](#), [42](#), [355](#), [356](#), [358](#), [362](#), [380](#)
[px5_pthread_memorypool_destroy](#), [vi](#), [42](#), [356](#), [357](#), [358](#)
[px5_pthread_memorypool_free](#), [vi](#), [42](#), [353](#), [359](#), [360](#), [365](#)
[px5_pthread_memorypool_timedallocate](#), [vi](#), [42](#), [353](#), [364](#), [366](#), [368](#)
[px5_pthread_memorypool_tryallocate](#), [vi](#), [42](#), [353](#), [365](#), [367](#), [369](#)
[px5_pthread_memorypoolattr_destroy](#), [vi](#), [42](#), [370](#), [371](#), [379](#)
[px5_pthread_memorypoolattr_getcontroladdr](#), [vi](#), [42](#), [372](#), [373](#)
[px5_pthread_memorypoolattr_getcontrolsiz](#)
[e](#), [vi](#), [42](#), [373](#), [374](#), [375](#), [380](#), [381](#)
[px5_pthread_memorypoolattr_getname](#), [vi](#), [42](#), [376](#), [377](#), [384](#)
[px5_pthread_memorypoolattr_init](#), [vi](#), [42](#), [371](#), [378](#), [379](#)
[px5_pthread_memorypoolattr_setcontroladdr](#), [vi](#), [29](#), [42](#), [373](#), [374](#), [375](#), [380](#)
[px5_pthread_memorypoolattr_setname](#), [vi](#), [42](#), [383](#), [384](#), [431](#)
Memory Pools, [ii](#), [42](#)
Memory Protection Unit (MPU), [19](#)
Message Queue API Definitions
[mq_close](#), [ii](#), [36](#), [51](#), [52](#), [57](#), [192](#)
[mq_getattr](#), [ii](#), [36](#), [53](#), [54](#), [57](#), [65](#), [192](#)
[mq_open](#), [ii](#), [36](#), [52](#), [54](#), [55](#), [57](#), [59](#), [63](#), [65](#), [69](#), [72](#), [192](#), [211](#)
[mq_receive](#), [ii](#), [36](#), [52](#), [58](#), [60](#), [63](#), [69](#), [72](#)
[mq_send](#), [ii](#), [36](#), [52](#), [59](#), [61](#), [63](#), [69](#), [72](#)
[mq_setattr](#), [ii](#), [36](#), [54](#), [57](#), [63](#), [64](#), [66](#), [192](#)
[mq_timedreceive](#), [ii](#), [36](#), [59](#), [63](#), [67](#), [69](#), [72](#)
[mq_timedsend](#), [ii](#), [36](#), [63](#), [70](#), [72](#)
[px5_mq_extend_open](#), [iv](#), [57](#), [190](#), [192](#), [194](#), [196](#), [198](#), [200](#), [202](#), [203](#), [204](#), [205](#), [206](#), [208](#), [209](#)
[px5_mq_extendattr_destroy](#), [iv](#), [36](#), [193](#), [194](#), [204](#)
[px5_mq_extendattr_getcontroladdr](#), [iv](#), [36](#), [195](#), [196](#)
[px5_mq_extendattr_getcontrolsiz](#)
[e](#), [iv](#), [36](#), [197](#), [198](#), [206](#)
[px5_mq_extendattr_getqueueaddr](#), [iv](#), [37](#), [199](#), [200](#)
[px5_mq_extendattr_getqueuesiz](#)
[e](#), [iv](#), [37](#), [201](#), [202](#)
[px5_mq_extendattr_init](#), [iv](#), [37](#), [194](#), [196](#), [198](#), [200](#), [202](#), [203](#), [204](#), [209](#)
[px5_mq_extendattr_setcontroladdr](#), [iv](#), [29](#), [37](#), [197](#), [198](#), [205](#), [207](#)
[px5_mq_extendattr_setqueueaddr](#), [iv](#), [29](#), [37](#), [200](#), [202](#), [208](#), [209](#)

[px5_mq_information_get](#), [iv](#), [37](#), [210](#), [212](#)
[px5_mq_send_notify](#), [iv](#), [37](#), [213](#), [214](#), [269](#), [310](#), [485](#)

Message Queues, [ii](#), [36](#)

Mutex API Definitions

[pthread_mutex_destroy](#), [iii](#), [38](#), [143](#), [144](#), [146](#)
[pthread_mutex_init](#), [iii](#), [38](#), [145](#), [146](#), [386](#), [394](#)
[pthread_mutex_lock](#), [iii](#), [28](#), [38](#), [147](#), [148](#), [149](#), [150](#), [152](#)
[pthread_mutex_trylock](#), [iii](#), [38](#), [149](#), [150](#), [152](#)
[pthread_mutex_unlock](#), [iii](#), [38](#), [148](#), [150](#), [151](#), [152](#)
[pthread_mutexattr_destroy](#), [iii](#), [38](#), [153](#), [154](#), [162](#)
[pthread_mutexattr_getprotocol](#), [iii](#), [155](#), [156](#), [164](#)
[pthread_mutexattr_getpshared](#), [iii](#), [38](#), [157](#), [158](#)
[pthread_mutexattr_gettype](#), [iii](#), [159](#), [160](#), [168](#)
[pthread_mutexattr_init](#), [iv](#), [38](#), [154](#), [158](#), [161](#), [162](#), [166](#)
[pthread_mutexattr_setprotocol](#), [iv](#), [38](#), [156](#), [163](#), [164](#)
[pthread_mutexattr_setpshared](#), [iv](#), [38](#), [165](#), [166](#)
[pthread_mutexattr_settype](#), [iv](#), [38](#), [160](#), [167](#), [168](#)
[px5_pthread_mutexattr_getcontroladdr](#), [vi](#), [38](#), [388](#), [389](#), [391](#)
[px5_pthread_mutexattr_getcontrolsiz](#)
[e](#), [vi](#), [38](#), [389](#), [390](#), [391](#), [394](#)
[px5_pthread_mutexattr_getname](#), [vi](#), [38](#), [392](#), [393](#), [398](#)
[px5_pthread_mutexattr_setcontroladdr](#), [vi](#), [29](#), [38](#), [389](#), [390](#), [391](#), [394](#)
[px5_pthread_mutexattr_setname](#), [vi](#), [38](#), [393](#), [397](#), [398](#)

Mutexes, [ii](#), [37](#)

N

Namespace, [ii](#), [28](#)

Native pthreads Implementation, [ii](#), [28](#)

O

[O_CREAT](#), [55](#), [57](#), [190](#), [192](#)

[O_NONBLOCK](#), [36](#), [55](#), [58](#), [59](#), [61](#), [62](#), [64](#), [66](#), [67](#), [68](#), [70](#), [71](#), [190](#)

[O_RDWR](#), [55](#), [57](#), [190](#), [192](#)

P

Partition Memory Pool API Definitions

[px5_pthread_partitionpool_allocate](#), [vi](#), [43](#), [399](#), [401](#), [408](#), [413](#), [416](#)
[px5_pthread_partitionpool_create](#), [vi](#), [43](#), [402](#), [404](#), [406](#), [410](#), [428](#)
[px5_pthread_partitionpool_destroy](#), [vi](#), [43](#), [403](#), [405](#), [406](#)
[px5_pthread_partitionpool_free](#), [vi](#), [43](#), [400](#), [407](#), [408](#), [413](#)
[px5_pthread_partitionpool_timedallocate](#), [vi](#), [43](#), [400](#), [412](#), [414](#), [416](#)
[px5_pthread_partitionpool_tryallocate](#), [vi](#), [43](#), [400](#), [413](#), [415](#), [417](#)
[px5_pthread_partitionpoolattr_destroy](#), [vi](#), [43](#), [418](#), [419](#), [427](#)
[px5_pthread_partitionpoolattr_getcontroladdr](#), [vi](#), [43](#), [420](#), [421](#)
[px5_pthread_partitionpoolattr_getcontrolsiz](#)
[e](#), [vi](#), [43](#), [421](#), [422](#), [423](#), [428](#), [429](#)
[px5_pthread_partitionpoolattr_getname](#), [vi](#), [43](#), [424](#), [425](#), [432](#)
[px5_pthread_partitionpoolattr_init](#), [vi](#), [43](#), [419](#), [426](#), [427](#)
[px5_pthread_partitionpoolattr_setcontroladdr](#), [vi](#), [29](#), [43](#), [421](#), [422](#), [423](#), [428](#)
[px5_pthread_partitionpoolattr_setname](#), [vi](#), [43](#), [431](#), [432](#)
Partition Memory Pools, [ii](#), [42](#)
PDV Overhead, [ii](#), [25](#)
Pointer/Data Verification (PDV), [i](#), [ii](#), [1](#), [19](#), [20](#), [21](#), [30](#), [439](#)
priority inheritance, [37](#), [38](#), [145](#), [147](#), [161](#), [163](#), [164](#)
priority inversion, [37](#)
producer-consumer, [38](#)
PTHREAD_MUTEX_RECURSIVE, [37](#), [167](#), [168](#), [385](#)
PTHREAD_PRIO_INHERIT, [37](#), [147](#), [163](#), [164](#)
PX5_DEFAULT_CORE_AFFINITY, [10](#), [215](#), [229](#)
PX5_DEFAULT_MAIN_CORE_AFFINITY, [10](#)
px5_internal_central_error_process, [15](#), [31](#)
PX5_SMP_SAME_PRIORITY_RESTRICT, [13](#)
PX5_STACK_CHECK_ENABLE, [13](#), [30](#)
px5_timer_interrupt_process, [8](#), [15](#)

R

recursive mutex, [37](#), [147](#), [149](#)
Run-Time Stack Checking & Verification, [i](#), [ii](#), [20](#), [30](#)

S

SCHED_FIFO, [31](#)
SCHED_OTHER, [4](#), [32](#)
SCHED_RR, [31](#), [32](#)
Scheduling Policy, [ii](#), [31](#), [32](#)
Secure Element (SE), [20](#)
Semaphore API Definitions
[px5_sem_extend_init](#), [vii](#), [39](#), [479](#), [480](#), [482](#), [489](#), [491](#), [493](#), [495](#), [496](#), [497](#), [498](#), [499](#), [502](#), [509](#)
[px5_sem_information_get](#), [vii](#), [39](#), [481](#), [483](#)
[px5_sem_post_notify](#), [vii](#), [39](#), [214](#), [269](#), [310](#), [484](#), [485](#)
[px5_sem_tickwait](#), [vii](#), [39](#), [486](#), [487](#), [513](#)
[px5_semattr_destroy](#), [vii](#), [39](#), [488](#), [489](#), [497](#)
[px5_semattr_getcontroladdr](#), [vii](#), [39](#), [490](#), [491](#), [499](#)
[px5_semattr_getcontrolsiz](#), [vii](#), [39](#), [492](#), [493](#), [498](#)
[px5_semattr_getname](#), [vii](#), [39](#), [494](#), [495](#), [502](#)
[px5_semattr_init](#), [vii](#), [39](#), [489](#), [491](#), [493](#), [495](#), [496](#), [497](#), [499](#), [502](#)
[px5_semattr_setcontroladdr](#), [vii](#), [29](#), [39](#), [491](#), [492](#), [493](#), [498](#), [500](#)
[px5_semattr_setname](#), [vii](#), [39](#), [495](#), [501](#), [502](#)
sem_destroy, [vii](#), [39](#), [480](#), [506](#), [507](#), [509](#)
sem_init, [vii](#), [39](#), [479](#), [480](#), [482](#), [507](#), [508](#), [509](#)
sem_post, [vii](#), [39](#), [487](#), [510](#), [511](#), [513](#), [518](#)
sem_timedwait, [vii](#), [39](#), [487](#), [512](#), [514](#)
sem_trywait, [vii](#), [39](#), [487](#), [511](#), [513](#), [515](#), [516](#), [518](#)
sem_wait, [vii](#), [39](#), [487](#), [511](#), [513](#), [516](#), [517](#), [518](#)
Semaphores, [ii](#), [38](#)
Signal API Definitions
[pthread_kill](#), [iii](#), [39](#), [40](#), [141](#), [142](#), [183](#), [520](#), [521](#), [535](#), [537](#), [540](#)
[pthread_sigmask](#), [iv](#), [40](#), [142](#), [181](#), [183](#), [520](#)
sigaction, [vii](#), [13](#), [39](#), [40](#), [142](#), [183](#), [519](#), [521](#)
sigaddset, [vii](#), [40](#), [522](#), [523](#), [525](#), [527](#), [529](#), [531](#), [533](#), [535](#), [538](#), [541](#)
sigdelset, [vii](#), [40](#), [523](#), [524](#), [525](#), [527](#), [529](#), [531](#), [533](#)
sigemptyset, [vii](#), [40](#), [523](#), [525](#), [526](#), [527](#), [529](#), [531](#), [533](#), [535](#), [538](#), [541](#)
sigfillset, [viii](#), [40](#), [183](#), [523](#), [525](#), [527](#), [528](#), [529](#), [531](#), [533](#)
sigismember, [viii](#), [40](#), [530](#), [531](#)
sigpending, [viii](#), [40](#), [531](#), [532](#), [533](#)
sigtimedwait, [viii](#), [40](#), [534](#), [535](#), [537](#), [540](#)
sigwait, [viii](#), [40](#), [142](#), [183](#), [520](#), [535](#), [536](#), [538](#), [540](#)
sigwaitinfo, [viii](#), [40](#), [535](#), [537](#), [539](#), [541](#)
Signals, [ii](#), [39](#), [181](#)

SMP (Symmetric Multiprocessing), [13](#), [30](#), [130](#),
[173](#), [215](#), [229](#)
 Software Safety & Security Features, [i](#), [20](#)
 Symmetric Multiprocessing, [5](#), [30](#)
 System Objects, [ii](#), [33](#)

T

Thread API Definitions

nanosleep, [iii](#), [73](#), [75](#), [445](#), [504](#), [543](#), [547](#)
 pthread_attr_destroy, [iii](#), [34](#), [76](#), [77](#)
 pthread_attr_getdetachstate, [iii](#), [34](#), [78](#), [79](#)
 pthread_attr_getstackaddr, [iii](#), [34](#), [80](#), [81](#)
 pthread_attr_getstacksize, [iii](#), [34](#), [82](#), [83](#)
 pthread_attr_init, [iii](#), [34](#), [84](#), [85](#)
 pthread_attr_setdetachstate, [iii](#), [34](#), [86](#), [87](#)
 pthread_attr_setstackaddr, [iii](#), [29](#), [34](#), [80](#), [88](#),
[89](#)
 pthread_attr_setstacksize, [iii](#), [34](#), [82](#), [90](#), [91](#)
 pthread_cancel, [iii](#), [34](#), [92](#), [93](#), [95](#), [97](#), [122](#),
[125](#), [129](#), [176](#), [178](#), [185](#)
 pthread_cleanup_pop, [iii](#), [34](#), [93](#), [94](#), [95](#), [97](#),
[176](#), [178](#), [185](#)
 pthread_cleanup_push, [iii](#), [34](#), [93](#), [95](#), [96](#), [97](#),
[176](#), [178](#), [185](#)
 pthread_create, [iii](#), [4](#), [6](#), [34](#), [77](#), [79](#), [81](#), [83](#),
[84](#), [85](#), [87](#), [88](#), [89](#), [91](#), [93](#), [95](#), [97](#), [120](#), [123](#),
[125](#), [176](#), [178](#), [185](#), [233](#), [343](#), [434](#), [440](#), [478](#)
 pthread_detach, [iii](#), [34](#), [120](#), [122](#), [124](#), [125](#),
[129](#), [134](#), [135](#)
 pthread_equal, [iii](#), [34](#), [126](#), [127](#), [172](#)
 pthread_exit, [iii](#), [32](#), [34](#), [120](#), [122](#), [125](#), [127](#),
[128](#), [129](#), [135](#), [136](#), [184](#)
 pthread_getspecific, [iii](#), [34](#), [132](#), [133](#), [138](#),
[140](#), [180](#)
 pthread_join, [iii](#), [34](#), [120](#), [122](#), [125](#), [128](#), [129](#),
[134](#), [136](#)
 pthread_key_create, [iii](#), [34](#), [133](#), [137](#), [138](#),
[139](#), [140](#), [180](#)
 pthread_key_delete, [iii](#), [34](#), [133](#), [138](#), [139](#),
[140](#), [180](#)
 pthread_once, [iv](#), [34](#), [169](#), [170](#)
 pthread_self, [iv](#), [4](#), [34](#), [127](#), [131](#), [133](#), [171](#),
[172](#), [174](#), [438](#), [443](#)
 pthread_setcancelstate, [iv](#), [34](#), [92](#), [93](#), [95](#), [97](#),
[120](#), [175](#), [176](#), [178](#), [185](#)
 pthread_setcanceltype, [iv](#), [34](#), [92](#), [93](#), [95](#), [97](#),
[120](#), [176](#), [177](#), [178](#), [185](#)
 pthread_setspecific, [iv](#), [34](#), [132](#), [133](#), [137](#),
[139](#), [179](#), [180](#)
 pthread_testcancel, [iv](#), [34](#), [93](#), [95](#), [97](#), [176](#),
[178](#), [184](#), [185](#)
 px5_errno_get, [iv](#), [13](#), [186](#), [187](#), [189](#)
 px5_errno_set, [iv](#), [187](#), [188](#), [189](#)
 px5_pthread_attr_getcontroladdr, [iv](#), [34](#),
[219](#), [220](#), [234](#)
 px5_pthread_attr_getcontrolsize, [iv](#), [34](#), [221](#),
[222](#), [233](#), [234](#)
 px5_pthread_attr_getname, [iv](#), [35](#), [223](#), [224](#)
 px5_pthread_attr_getpriority, [iv](#), [35](#), [225](#),
[226](#), [238](#)
 px5_pthread_attr_gettimeslice, [iv](#), [35](#), [227](#),
[228](#), [240](#)
 px5_pthread_attr_setcontroladdr, [iv](#), [29](#), [35](#),
[219](#), [221](#), [233](#), [234](#)
 px5_pthread_attr_setname, [iv](#), [35](#), [235](#), [236](#)
 px5_pthread_attr_setpriority, [iv](#), [35](#), [225](#),
[226](#), [237](#), [238](#), [434](#)
 px5_pthread_attr_settimeslice, [iv](#), [35](#), [227](#),
[239](#), [240](#), [478](#)
 px5_pthread_cond_information_get, [iv](#), [35](#),
[241](#), [243](#)
 px5_pthread_event_flags_clearspecific, [v](#),
[41](#), [257](#), [258](#)
 px5_pthread_event_flags_information_get,
[v](#), [41](#), [263](#), [265](#)
 px5_pthread_event_flags_set_notify, [v](#), [41](#),
[214](#), [268](#), [269](#), [310](#), [485](#)
 px5_pthread_event_flagsattr_getautoconsu
 memask, [v](#), [41](#), [280](#), [281](#), [291](#)
 px5_pthread_event_flagsattr_setautoconsu
 memask, [v](#), [41](#), [281](#), [290](#), [291](#)
 px5_pthread_fastqueue_information_get, [v](#),
[40](#), [302](#), [304](#)
 px5_pthread_fastqueue_send_notify, [v](#), [40](#),
[214](#), [269](#), [309](#), [310](#), [485](#)
 px5_pthread_information_get, [vi](#), [34](#), [35](#),
[342](#), [344](#)
 px5_pthread_memorypool_information_get,
[vi](#), [42](#), [361](#), [363](#)
 px5_pthread_mutex_information_get, [vi](#), [38](#),
[385](#), [387](#)
 px5_pthread_partitionpool_information_get,
[vi](#), [43](#), [409](#), [411](#)
 px5_pthread_priority_change, [vi](#), [35](#), [433](#),
[434](#)
 px5_pthread_resume, [vi](#), [35](#), [231](#), [232](#), [435](#),
[436](#), [438](#), [442](#), [443](#)
 px5_pthread_stack_check, [vii](#), [20](#), [30](#), [35](#),
[437](#), [438](#)
 px5_pthread_start, [vii](#), [8](#), [9](#), [20](#), [23](#), [29](#), [123](#),
[345](#), [346](#), [347](#), [348](#), [350](#), [439](#), [441](#)
 px5_pthread_suspend, [vii](#), [35](#), [436](#), [442](#), [443](#)
 px5_pthread_ticktimer_information_get, [vii](#),
[43](#), [453](#), [455](#)
 px5_pthread_timeslice_change, [vii](#), [35](#), [477](#),
[478](#)
 sched_yield, [vii](#), [4](#), [503](#), [505](#)
 sleep, [viii](#), [73](#), [74](#), [75](#), [444](#), [445](#), [504](#), [542](#), [543](#),
[546](#), [547](#)
 usleep, [viii](#), [74](#), [445](#), [546](#), [547](#)
 Thread Priorities, [ii](#), [32](#)
 Thread States, [ii](#), [32](#)

Thread-local Storage (TLS), [132](#), [133](#), [137](#), [179](#)

Ticktimer API Definitions

px5_thread_tick_sleep, [vii](#), [74](#), [444](#), [445](#), [547](#)

px5_thread_ticks_get, [vii](#), [446](#), [447](#)

px5_thread_ticktimer_create, [vii](#), [43](#), [448](#), [450](#), [454](#), [472](#)

px5_thread_ticktimer_destroy, [vii](#), [43](#), [451](#), [452](#)

px5_thread_ticktimer_start, [vii](#), [43](#), [448](#), [450](#), [456](#), [457](#)

px5_thread_ticktimer_stop, [vii](#), [43](#), [458](#), [459](#)

px5_thread_ticktimer_update, [vii](#), [43](#), [460](#), [461](#)

px5_thread_ticktimerattr_destroy, [vii](#), [43](#), [462](#), [463](#), [471](#)

px5_thread_ticktimerattr_getcontroladdr, [vii](#), [43](#), [464](#), [465](#)

px5_thread_ticktimerattr_getcontrolsiz, [vii](#), [43](#), [465](#), [466](#), [467](#), [472](#), [473](#)

px5_thread_ticktimerattr_getname, [vii](#), [43](#), [468](#), [469](#), [476](#)

px5_thread_ticktimerattr_init, [vii](#), [43](#), [463](#), [470](#), [471](#)

px5_thread_ticktimerattr_setcontroladdr, [vii](#), [29](#), [43](#), [465](#), [466](#), [467](#), [472](#)

px5_thread_ticktimerattr_setname, [vii](#), [43](#), [475](#), [476](#)

Ticktimer Services, [ij](#), [43](#)

True Random Number Generator (TRNG), [19](#), [23](#), [439](#)

Trusted Platform Module (TPM), [20](#)



Enhance • Simplify • Unite

11440 West Bernardo Court • Suite 300
San Diego, CA 92127, USA

Phone: +1 (858) 753-1715
Website: px5rtos.com

© PX5 • All Rights Reserved